MACOS Manual

Modeling and Analysis for Controlled Optical Systems

Version 4.00 (beta) — October 2026

Jet Propulsion Laboratory, California Institute of Technology

Contact

For information on MACOS contact:

Scott Basinger Scott.A.Basinger@jpl.nasa.gov

Jet Propulsion Laboratory 306-438, 4800 Oak Grove Drive, Pasadena CA 91109

Authorship

David C. Redding, Scott Basinger, John Lou, Luis Marchen, Laura Needels, Carl Nissly, Norbert Sigrist and Sam Sirlin

Acknowledgment

The MACOS program was developed at the Jet Propulsion Laboratory, California Institute of Technology.

MACOS software and documentation are copyright 1992-2026 by the California Institute of Technology, Jet Propulsion Laboratory.

U.S. Government sponsorship of parts of MACOS under NASA Contract NAS7-918 is acknowledged.

Installation

MACOS is distributed as two git repositories on GitHub: the engine and this manual in `macos`, and the MATLAB and Python bindings, the templates and the design tools in `MACOS_resources`. Clone them side by side (the build scripts expect `MACOS_resources` next to `macos`):

```
git clone git@github.com:nasa-jpl/macos.git
git clone git@github.com:nasa-jpl/MACOS_resources.git
cd macos
source ./makems.sh release            # the engine and the macos command-line program (ifx)
source ./makems.sh release gfortran   # the same with gfortran (the build the MATLAB interface links)
```

`makems.sh` builds the engine library and the interactive program into `build_release[_gfortran]/bin/macos`; `makeall.sh` adds the SMACOS driver and the GMI MATLAB interface. The requirements (CMake, a Fortran compiler – Intel ifx or gfortran – and gcc), the Linux, macOS and Windows notes and the debug builds are in `BUILD.md` at the top of the `macos` repository; the branch model is in `BRANCHES.md`. The MATLAB interface (`MACOS_resources/mmacos`, built by its `run_mmacos_tests.sh`) and the Python package (`MACOS_resources/pymacos`) each carry their own README. Access to the repositories is by invitation; write to the contact above.

Contents

SECTION 1: Introduction....6
1.1 Modeling and Analysis for Controlled Optical Systems....6
1.2 Overview of this Manual....7
1.3 MACOS and AI Agents....7
1.4 New Features of Version 4.00....8
SECTION 2: Technical Overview....10
2.1 Geometry and Coordinate Systems....10
2.2 Geometric Optics: Tracing Rays....13
2.3 Physical Optics: Diffraction....14
2.4 Polarized Light....20
SECTION 3: User Interface....21
3.1 Starting and Stopping MACOS....21
3.2 Help....21
3.3 MACOS Model Size Specification....22
3.4 Entering Commands....23
3.5 Graphics Display....25
3.6 Off-Line Graphics Display....25
3.7 PostScript and GIF Graphics Output....26
3.8 MACOS Files....27
3.9 Using Macros....28
3.10 Memory Requirements....29
SECTION 4: Describing Optical Systems....30
4.1 Specifying the Light Source....30
4.2 Example Light Source....34
4.3 Specifying Optics....36
4.4 Element Types....38
4.5 Element Surface Types....62
4.6 Coatings and Polarization Data....73

4.7 Obscurations and Apertures....74
4.8 Specifying Coatings....80
4.9 Example Telescope....80
4.10 Editing .In-Files....85
4.11 Commands....86
4.12 Summary of Prescription Variables....89
4.13 Design Optimization (CALIB) Keywords....93
SECTION 5: Ray-Trace Analysis....98
5.1 Background....98
5.2 Ray-Trace Commands....99
5.3 Beam Set-Up Commands....111
5.4 Polarized Ray Tracing....121
SECTION 6: Diffraction Analysis....122
6.1 Single- versus Multi-plane Diffraction....122
6.2 Setting up Diffraction Propagations....124
6.3 Commands....130
SECTION 7: Beam Propagation and Image Simulation....135
7.1 Background....135
7.2 Beam Propagation Commands....136
7.3 Image polarity and stretch-aware labels....151
7.4 Plot Type Commands....152
SECTION 8: Differential Ray-Tracing and Linear Optical Models....157
8.1 Background....157
8.2 Commands for Building Models....161
8.3 Commands for Changing Models....162
8.4 Commands for Exercisizing Models....163
8.5 Commands for Exporting Models....165
SECTION 9: Subroutine MACOS and User Extensions....166
9.1 Background....166

9.2 Call Line Variables....167
9.3 Specification of Model Size in SMACOS....170
9.4 Example....170
9.5 Commands....171
9.6 GMI: MATLAB MEX interface (FreeForm updates)....179
SECTION 10: Worked Examples....180
10.1 10.1 A sensitivity workflow and an error budget (e5hex1_budget)....180
10.2 10.2 A telescope design: the long-slit TMA front end (tma_longslit)....180
10.3 10.3 A spectrometer design: the Dyson challenge (dyson5)....181
10.4 10.4 A gauge bench: COPHI on the TG96 interferometer (cophi_dm96)....181
SECTION 11: Polarization....183
11.1 Turning polarization on....183
11.2 Conventions....183
11.3 What the trace does with the field....184
11.4 Polarizing elements....185
11.5 Vector diffraction....185
11.6 Polarization analysis....186
11.7 Design rules the measurements taught....187
11.8 Validation....187
11.9 Idealizations and gaps....188
REFERENCES....189
APPENDIX A: Examples and Tutorial....190
A.1 Cassegrain Telescope Examples....190
A.2 Diffractive Elements Examples....193
A.3 Non-Sequential Surface Examples....196
A.4 Adaptive Optics Example....201
A.5 Segmented Telescope Example....216
A.6 Near-Field Propagation Example....221
A.7 Image Simulation Example....225

A.8 S-MACOS Example Files....229
APPENDIX C: Complete Command Reference....231
APPENDIX D: Working with AI Agents....234
D.1 Where the facts live....234
D.2 Driving the program....234
D.3 Conventions that bite....235
D.4 Working rules....235

SECTION 1: Introduction

This section introduces the features of MACOS and gives an overview of the manual. New features introduced with this version of the code are summarized.

1.1 Modeling and Analysis for Controlled Optical Systems

The Modeling and Analysis for Controlled Optical Systems software provides tools for modeling and analyzing optical systems. It consists of a stand-alone application program called MACOS and a Fortran subroutine package called SMACOS. SMACOS provides the full functionality of the MACOS application in a form that can be directly integrated with other codes.

MACOS is an optical analysis and model generation program, focused on generating and exercising fast, accurate, and detailed models combining ray-trace, differential ray-trace, and diffraction techniques. It is not intended as a replacement for a traditional optical design code, though it does include optimization features that can be used for design.

MACOS provides unique modeling capability for *system-level* design and analysis tasks. SMACOS functions can be directly integrated into simulation programs, to provide optical outputs driven by structural and/or thermal dynamics, feedback controls, atmospheric effects, or other system perturbations – the GMI Matlab MEX function provides an example of how to do this using the SMACOS API. More commonly, MACOS is used to generate Linear Optical Models, either directly in MACOS, or driving SMACOS from Matlab or Python. This release provides *mmacos* and *pymacos*, providing utilities and runners to generate LOMs, as well as telescope design templates and other useful functions. MACOS LOMs are easily integrated with standard structures, controls, and user written analysis tools to create end-to-end system models.

MACOS is commonly used in conjunction with a design code such as CodeV. The design code produces an initial optical prescription, which is converted into a MACOS prescription. Further features are added in MACOS, including mechanical and control system interfaces, optical figure aberrations, segmented mirrors, actuated mirrors, deformable mirrors, and reference and return surfaces for diffraction calculations. Using MACOS linear optical models (or SMACOS subroutines), models can be integrated with structures, dynamics, and control codes. These models are exercised in end-to-end dynamic simulations, Monte Carlo analyses, covariance analyses, or optimization studies to determine integrated system-level performance.

MACOS (and SMACOS) provide computationally efficient general ray-trace capabilities. These support standard analyses of optical performance such as ray-traces, spot-diagrams, and wavefront (OPD) plots. They also support MACOS diffraction calculations.

MACOS has a differential ray-trace capability which generates linear matrix models of optical systems. These models are accurate for systems that undergo only small perturbations and are essential for tasks such as covariance analysis and control design. Models can be exported to linear analysis programs such as Matlab where they can be exercised alone or in combination with structural, thermal, and control models.

For diffraction calculations, MACOS uses Fresnel propagators driven by exact phases computed by its ray-trace engine. Simulated images (point-spread functions) can be generated using single- or multi-plane diffraction. MACOS supports simulation of multiple source and wavelength images. Detector pixilation can be set to model detector resolution limits.

1.2 Overview of this Manual

This manual provides two services to the user:

- A technical orientation outlining MACOS ray-trace, differential ray-trace, and diffraction modeling capabilities.
- Detailed description of MACOS commands including reference material and numerous examples.

Section 2 of this manual briefly summarizes the technical areas covered by MACOS: geometric optics and physical optics. The purpose is to establish basic definitions and to introduce certain MACOS features. More detailed technical discussion is provided at the beginning of each subsequent section and in the references. Section 3 discusses the basic procedures of starting MACOS, getting help, entering commands, and using files. Sections 4 through 9 proceed in the sequence that a typical MACOS session might:

- Entering the optical prescription from a drawing of the system (Section 4)
- Ray-tracing the system (Section 5)
- Setting up the model for diffraction analysis (Section 6)
- Simulating images (Section 7)
- Building and exporting linear models (Section 8)
- Generating and using SMACOS models (Section 9)

Several example problems are worked in the manual to illustrate specific features of the code. Appendix A contains the prescriptions and journal files for the examples. Appendix B contains selected technical papers.

This manual is the *learning* document; its companion, the **MACOS Command Reference** (`macosCmdRef.pdf`, source in `docs/macos-manual/cmdref/`), is the *lookup* document: every interactive command, and every function of the MATLAB (mmacos) and Python (pymacos) programming interfaces, cataloged one at a time with its syntax. Appendix C of this manual lists the commands in summary form; the Command Reference is the full per-command catalog and the place to start if you are driving MACOS from MATLAB or Python.

Two companion tools are worth knowing from the outset. **SegMirMaker** (`MACOS_resources/segmirmaker/`) converts a monolithic primary mirror into a many-segment one, generating the segmented-mirror prescription and segment-influence files from a parent-mirror description (see §4.4.11). **GMI** (`MACOS_resources/GMI/`) is a MATLAB MEX front end built directly on the SMACOS subroutine interface — both a complete forward simulator and the reference example of a real SMACOS application (see Section 9).

1.3 MACOS and AI Agents

MACOS is increasingly driven by AI coding agents as well as by people, and the parts of the system an agent uses are the documented ones: the interactive program, which an agent runs on a pseudo-terminal and feeds command by command exactly as a user types them (the command-line test suite in `cli_tests/` is built that way, one journal of commands and expected output per command); the MATLAB and Python bindings (`mmacos`, `pymacos`), whose functions are the programmatic surface; and the Command Reference, which states every command's syntax, prompts and defaults and is the authority when prose and transcript

disagree. Appendix D collects what an agent needs to drive MACOS reliably: where the facts live, how to run and read the program, the conventions that trip newcomers, and the working rules the project holds its own agents to. The repositories also carry agent-facing notes beside the code (`CLAUDE.md` at the top of each repository and in each subsystem directory), which are maintained with the code and are part of the release.

1.4 New Features of Version 4.00

MACOS 4.00 (April 2026) is a feature release built on the Fortran 90 / SMACOS infrastructure introduced in v3.2. It adds a new general-purpose composite surface (FreeForm), adds multiple different forms of Zernike polynomials describing for optical surfaces, modernizes the graphics layer, expands the SMACOS / GMI MATLAB interface, and adds new utility commands plus a separate segmented-mirror prescription generator.

A summary of changes since MACOS 3.2:

- FreeForm composite surface (SrfType = 14). Single surface type combining a conic base with a Cartesian monomial sag (Mon), a free-form Zernike sag (FF), and an interpolated grid sag, all on independent coordinate frames. See §4.5.11.
- Per-element Zernike inputs. Sparse “modes + coefs” form MonZernCoef / MonZernModes and FFZernCoef / FFZernModes is accepted directly in .in files; conversion to monomial form happens internally at trace time. See §4.5.4.
- Per-ray status tracking. Every ray records why it failed (Miss / Obscured / Bracket / MaxIter) and at which element. The end-of-trace WARN summary breaks down failures by category and throttles bracket / iter messages after the first 20.
- Graphics modernization. Built-in giza graphics backend replacing legacy PGPLOT (still selectable at build time). New IMGMODE command toggles between astronomical (negative, large->dark) and conventional (positive, large->light) display polarity. Colorbar wedge labels reflect the active STRETCH (LIN / LOG10 / SQRT). See §7.x.
- SEGRAYTRACE command. Trace a single ray through the geometric center of a chosen segment element; print ray state at every downstream surface. Useful for verifying segmented-mirror prescriptions. See §5.2.
- SegMirMaker external tool. Standalone utility, in MACOS_resources/segmirmaker/, that generates segmented-mirror .presc and Hx.m files. Replaces SMPGe (1992) and adds support for FreeForm parents. See §4.4.11.
- GMI MATLAB interface upgrades. New 14th MEX argument pmonzern for FreeForm MonZernCoef perturbations (parallel to pzern). New param.monzernSrf element list. Nominal-state save/restore extended to cover FreeForm coordinate frames so per-element rigid-body perturbations no longer leak across GMI calls. See §9.
- Graceful Ctrl-C. SIGINT installs a clean-exit handler at start-up; pressing Ctrl-C at the MACOS> prompt prints ‘MACOS: interrupted’ and exits cleanly, replacing the Intel-runtime ‘forrtl: severe (69)’ traceback.
- Design optimization (CALIB). The native multi-field optimizer, never described in the 3.2 manual, is documented: the CALib command and its companions (SETC, AVAR, MVAR, DVAR, VARS, AFOV, DFOV, SFOV, FOVS, WLENS) run a Levenberg-Marquardt least-

squares solve over the rigid-body, radius, conic, aspheric and Zernike degrees of freedom of the variable elements, scored at up to 12 fields and 6 wavelengths against a wavefront, spot, Zernike-mode or beam target, with bounds through the SLSQP driver. Section 4 (Table 13) is the keyword reference; the Command Reference carries the dialogs; the setup is written by SAVE and announced at every load.

- Metrology (MET). The metrology model – per-element metrology points and target surfaces (`nMetPos=`, `tMetElt=`, `EdgeSensors=`) and the METCALC command that computes every metrology beam length and the edge-sensor readings – is in the Command Reference (METcalc, which lists the prescription keys); the MATLAB design package's `add_met`, `edge_sensors` and `dmet_dx` build and differentiate metrology layouts on a prescription.

- Design-optimization keyword reference. The `Opt*` / `VarElt=` prescription keywords that set up CALib are tabulated with their defaults in Section 4 (Table 13), together with the solver's stopping rule.

- HELP command rebuilt. All ~140 dispatched commands are now listed (vs. ~75 in v3.2), grouped into 15 categories with one-line effect descriptions and prerequisite tags ([Rx] / [BLD] / [DIFF]). See §3.2 and Appendix C.

SECTION 2: Technical Overview

This section provides a technical orientation to Modeling and Analysis for Controlled Optical Systems, providing brief reviews of geometric and physical optics. Though in the form of a tutorial, it is intended mainly as a summary of definitions and approach.

2.1 Geometry and Coordinate Systems

Optical systems are composed of *elements* — mirrors, lenses, detectors, gratings, holographic optical elements, and others — that have optical properties. These optical ele-ments are distributed in 3-space, which is to say, they are placed at different locations. *Where* they are located is of critical importance. Telescopes, cameras, optical systems in general depend on precise element positioning to achieve good optical performance.

Equally important is element *orientation*—where each element is pointed. Angles of reflection or refraction of a light beam depend on the orientation in 3-space of each surface in the system.

2.1.1 Global Coordinates

MACOS requires the user to specify the locations and orientations of elements using vectors in a single *global coordinate system*, as opposed to the usual approach of specifying each optical element relative to the preceding element. A single user-defined coordinate system is often more convenient when preparing models to be integrated with structural models: the structural coordinates can be used for the optical model as well. It facilitates checking results, especially for decentered and tilted systems. A global coordinate system also eliminates unnecessary calculations from the ray-trace. All MACOS calculations are actually done in the user-specified global coordinate system (Ref. Redding and Breckenridege). Specifying the system in those coordinates eliminates coordinate transformations as well as ray calculations at dummy surfaces. The result is significantly faster execution times.

To illustrate the choice of a global coordinate system, consider the simple 2-mirror Cassegrain telescope drawn in Figure 1.

In this optical system there are three optical elements: the primary mirror (PM); the secondary mirror (SM); and a focal plane (FP). Two possible global coordinate system are

- The structural coordinate frame (*Frame A)*, which would be shared by structural models, simplifying integration of optics and structures models.
- A frame centered at an optical element, such as the primary mirror, and aligned with the optical axis of the system (*Frame B*). This frame is more natural for optical layout. It can also be convenient when comparing results with optical design programs.

The location of each element is defined in MACOS using a 3-vector called VptElt (for “vertex point”). For the primary mirror of our example, VptElt is drawn from the origin of the global coordinate system to the vertex of the element, which is shown as the small circle. VptElt is indexed by 1=x, 2=y, 3=z coordinate values. For planes and

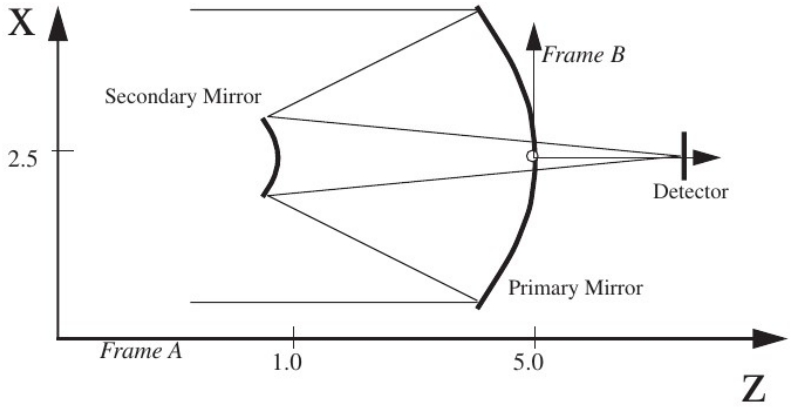


FIGURE 1 Coordinate systems for the Cassegrain telescope example.

spheres, which have no unique vertex, VptElt can be any point on the surface. For the two example frames, VptElt takes values as listed in Tables 1 and 2

TABLE 1. Primary mirror location specification

Location	Frame A	Frame B
VptElt(1)	2.5	0.0
VptElt(2)	0.0	0.0
VptElt(3)	5.0	0.0

TABLE 2. Secondary mirror location specification

Location	Frame A	Frame B
VptElt(1)	2.5	0.0
VptElt(2)	0.0	0.0
VptElt(3)	1.0	-4.0

Comparing the intervertex spacings of the two mirrors (the difference of the PM and SM VptElt), you can see that they are the same in either frame. It does not matter what frame is used, provided the relative location of each element is correct.

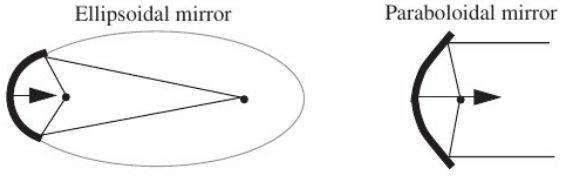


FIGURE 2 Principal axis directions

The orientation of each element is specified by a *unit-vector* (a 3-vector of length 1) called PsiElt. For conicoid elements PsiElt is the *principal axis direction* of the element surface: the direction of a line

drawn from the vertex of the element towards the nearest geometric focus. Figure 2 shows the principal axis for elliptical and paraboloidal mirrors. The principal axes for spheres or flats, which have no unique vertex, are defined from VptElt.

For the primary and secondary mirrors in our example, the principal axes are pointed in the negative z direction (Table 3). Since Frame A and Frame B have the same orientation, (they differ only in translation) the values of PsiElt are the same in both coordinate frames.

TABLE 3. Mirror orientations

Orientation	Primary Mirror	Secondary Mirror
PsiElt(1)	0.0	0.0
PsiElt(2)	0.0	0.0
PsiElt(3)	-1.0	-1.0

Detailed discussion of how elements are specified is provided in Section 4.

2.1.2 Local coordinates

MACOS allows the user to explicitly specify three types of local coordinates: *element coordinates*, *output coordinates*, and *surface coordinates*. Also important, but not explicitly set at each surface, are *beam coordinates*.

Element coordinates allow the user to simulate optical *actuators*, such as steering mirrors or translation stages, by defining coordinates local to the controlled element and aligned with the actuator axes. Actions of the steering mirror are rotations about, or translations along, that axis, and are easily implemented via the MACOS PERturb command (see Section 5.3). These *element coordinates* can also be imbedded into a linear model of the system using the MACOS BUIld and EXPort commands (see Sections 8.2.1 and 3.8.5). This enables creation of optical models directly in terms of actuator actions.

Output coordinates are used when calculating certain graphical outputs, such as spot diagrams. They help define sensing axes for simulating *detectors* (e.g., rows and columns of CCDs). When embedded in a linear model, they define the optical outputs of that model by selecting and transforming the optical states into combinations specified by the user.

Element coordinates are centered at a *rotation point*, specified as a 3-vector RptElt, which is the (0,0,0) point of the element coordinate system expressed in global coordinates. All rotations imparted to the element through the PERturb function are about RptElt.

The element coordinate axes are 6-vectors which describe rotational and translational perturbation to the element. Each 6-vector consists of two 3-vectors designating the rotation and translation, respectively. Usually an element will be actuated in rotation or translation only, but the option exists to use coupled rotation/translation coordinates.

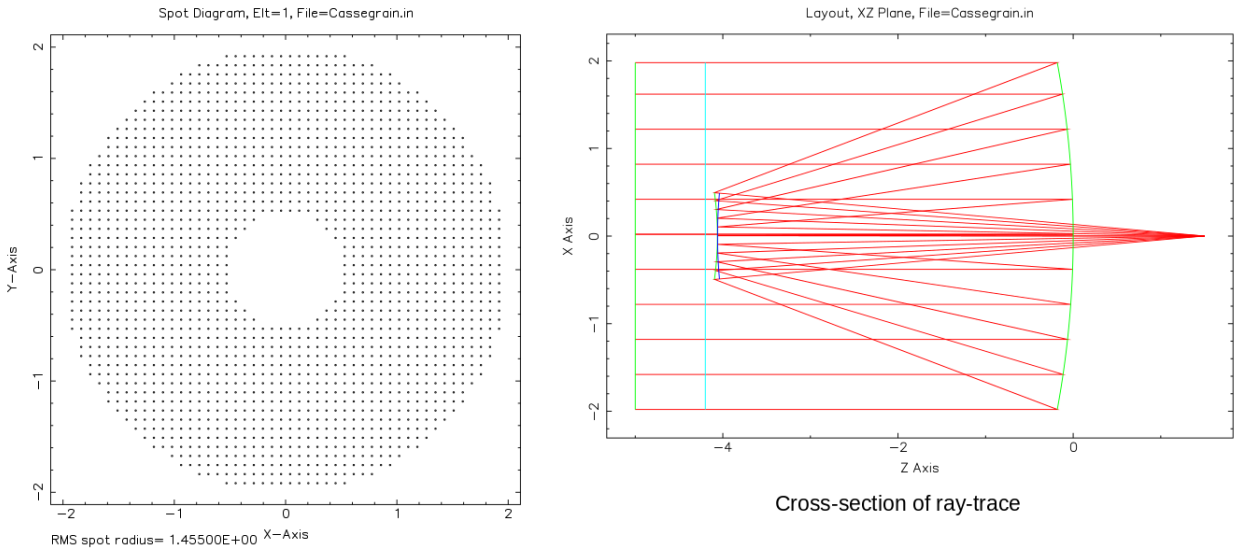
Surface coordinates are required to define propertires of certain surface types, such as Zernike surfaces, and are described in Section 4.

Beam coordinates define the orientation of the ray and diffraction grids at particular elements. They are initially defined at the source, and propagated to each subsequent optic by the ray tracing process. In general they are not the same at each element (due to tilting, rotating and inverting properties of the optical system). Beam coordinates can be computed at any surface using the MACOS COORD command.

2.2 Geometric Optics: Tracing Rays

Geometric optics capture the *particle nature* of light. In geometric optics, light beams are represented as bundles of *rays*, which are the trajectories of individual photons. Rays are generally composed of numerous straight-line segments, with direction changes at the reflective or refractive surfaces of optical elements (curved segments occur in regions of continuously changing refractive index). *Ray-tracing* is the process of propagating a bundle of rays through an entire optical system.

Ray-tracing is an extremely valuable tool for determining the performance of optical systems. For example, characteristics of a well-designed imaging system include an extremely small *spot diagram* at the detector focal plane. Spot diagrams are plots of the pierce points of a ray bundle on a surface, and a small spot indicates a tight focus. An example is provided by the Cassegrain telescope of Figure 3. Two spot diagrams are shown in Figure 4. The unaberrated telescope indeed has a focus at a single spot (diameter is approximately 10-10 in arbitrary units). Misaligning the primary mirror causes the spot to spread as shown in Figure 4 to approximately 2x10-8.



Spot diagram at entrance pupil

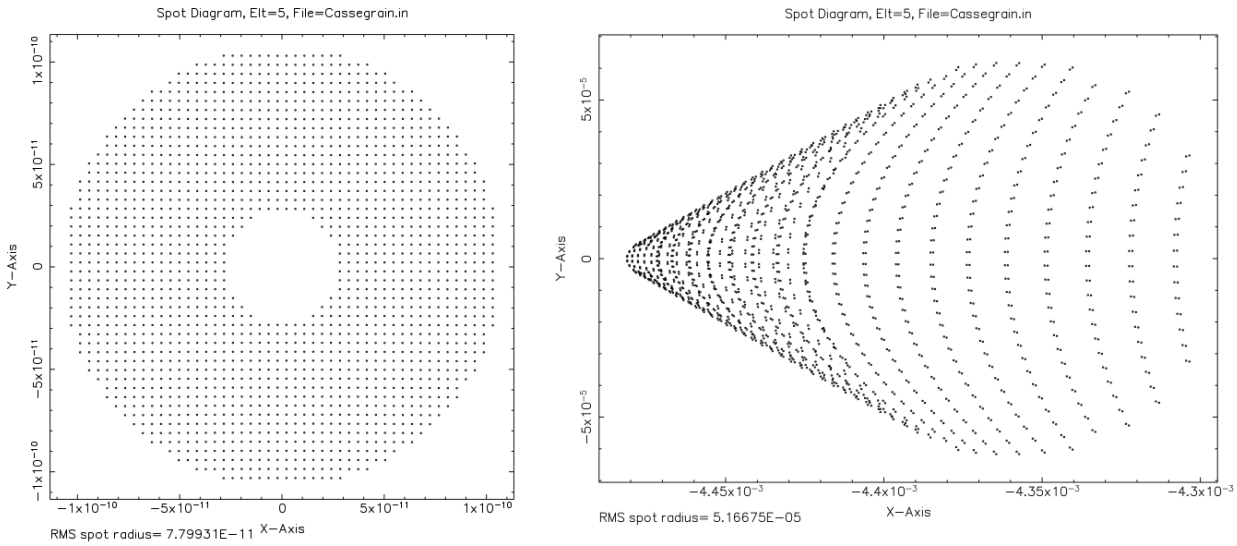
Cross-section of ray-trace

FIGURE 3 Ray trace of a Cassegrain telescope

As another example, interferometer systems can be evaluated by calculating the flatness of two interfering wavefronts at a pupil of a system. The wavefront of an extended beam is accurately represented by an *Optical Path Difference* (OPD) map at a particular surface, such as the beam-splitter where the interference is to take place.

The predictions of geometric optics are exact only in cases where the wavelength of the light is infinitely small. Of course, no such system exists. Geometric optics do provide extremely good predictions in a wide range of cases, where wavelengths are small compared to the dimensions of the optical system. Even in systems with long wavelengths and large diffraction effects, geometric optics are useful for determining the best design conditions.

One thing that ray-tracing cannot do is predict the intensity pattern of a focused spot. For that, use the image simulations features decribed in Section 7.



Focal plane spot diagram
(unaberrated system)

Focal plane spot diagram
(primary mirror tilted 1e-4 radians)

FIGURE 4 Spot diagrams for Cassegrain.in.

Near-focus is a region where diffraction effects dominate, as a perfectly-focused spot is itself infinitely small. For this case as well as others, *physical optics* approaches must be employed. Physical optics capture the wave nature of light and can accurately account for diffraction around obstructions, or in tightly focused beams, or at diffraction gratings. The MACOS approach to physical optics is discussed in the Section 6.

The exit pupil is a conjugate (an image) of the aperture stop of the system. For the Cassegrain telescope example, the entrance pupil is located at the primary mirror. A point source at the primary produces a virtual image behind the secondary mirror (see Figure 5). The exit pupil is a spherical “reference” surface at the virtual image point, with center of curvature on the focal plane. This surface coincides with the nominal spherical wavefront converging on the geometric focus.The OPD distribution at the exit pupil determines the image quality. Perfect optical systems have zero OPD; the wavefront leaving the optical system is spherical. In an aberrated optical system the OPD is non-zero as shown in Figure 6.

2.3 Physical Optics: Diffraction

Physical optics capture the *wave nature* of light. In physical optics, Huygen’s Principle states that each point within a beam of light radiates a spherical wave of light. A *wave-front* is the envelope of all the waves emitted at a particular instant of time. Waves of light propagate through an optical train by bouncing off of mirrors or bending through lenses, much as predicted by geometric optics. Unlike geometric optics, however, physical optics predicts that light can leak back into areas of shadow, or areas outside of the edge of the geometric beam. This process is *diffraction* and is important when modelling images, the propagation of beams over long distances, and other phenomena.

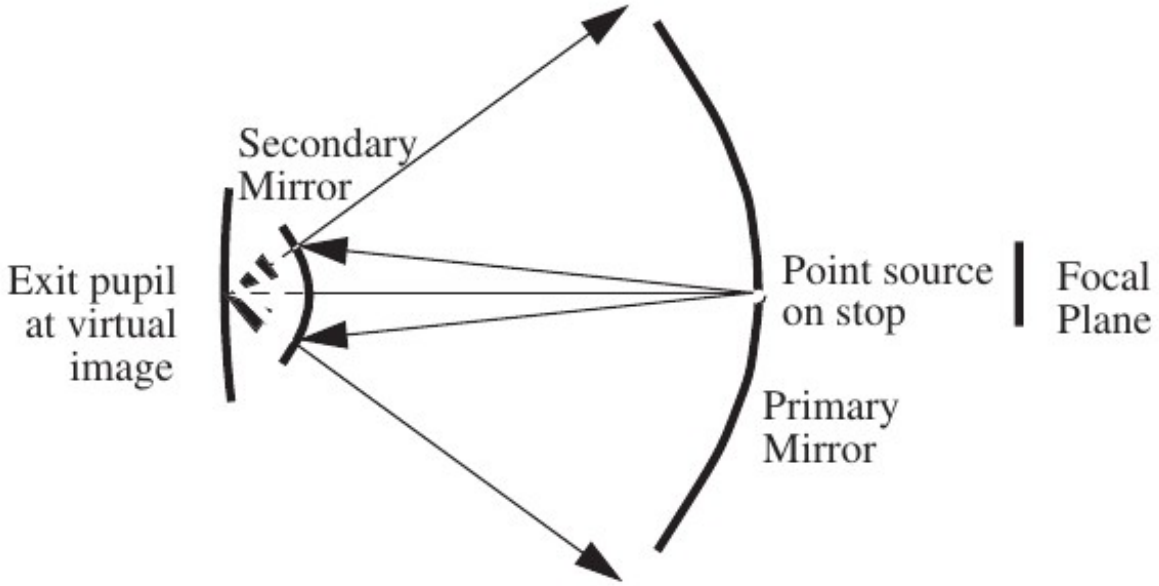


FIGURE 5 Exit pupil for Cassegrain.in

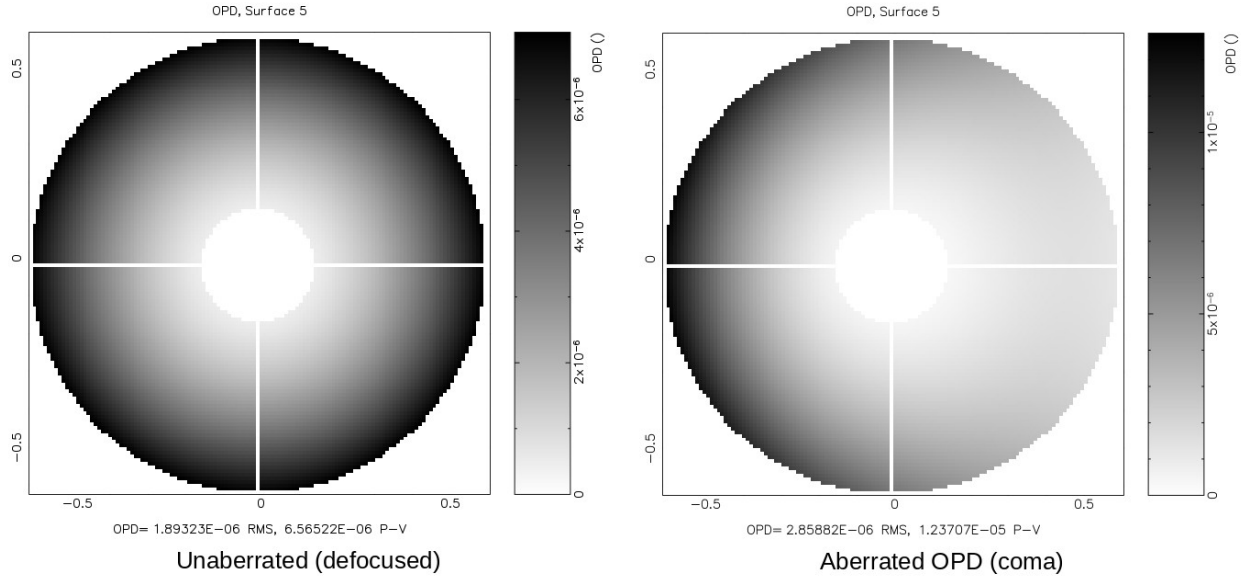


FIGURE 6 OPD at the exit pupil

The physics underlying diffraction and the free-space propagation of light are captured in Maxwell's equations. Physical optics theories have simplified these equations, enabling practical solutions to optical problems. *Scalar diffraction theory* assumes time invariance and ignores polarization effects (Ref. Goodman). *Fresnel propagation theory* further assumes that the beam is paraxial, or a small beam divergence/convergence. This allows the expression of the propagation of light from one planar surface to another planar surface in terms of the *paraxial wave equation*.

Geometric optics is the solution of Maxwell's equations in the limit of an infinitesimally small wavelength. This is a good approximation for many practical optics problems.

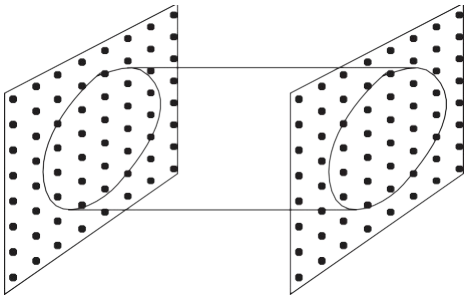
Diffraction effects start to dominate the propagation of a beam when the beam size approaches its wavelength (i.e., near focus), or when the dimensions of a surface approaches the beam wavelength (i.e., diffraction grating grooves), or when the beam is projected over distances exceeding its Rayleigh length (Ref. Goodman, Siegman).

The paraxial wave equation can be solved using two-dimensional Fourier transformations for free-space propagation between planes. MACOS implements these using computationally efficient fast Fourier transforms (FFTs). Care must be taken when using Fourier optics because of the approximations inherent in the Fresnel assumptions. Using FFTs brings additional concerns of *sampling density*, *aliasing* and *wrap-around*. Violating the Fresnel conditions, or sampling incorrectly, can affect the accuracy of computed results. MACOS provides several features to help you set problems up correctly.

In a physical optics computer model, the light beam is evaluated on *reference surfaces*. These are "dummy" surfaces located in the beam and aligned to the beam. They have no optical effect and are essentially transparent screens on which the beam is sampled.

The light on the reference surface is represented as a *complex matrix*, each element of which is the complex amplitude of the scalar field at a particular location. These locations are distributed as a regular rectangular grid of points across each reference surface (Fig. 7). Light is *propagated* between these reference surfaces using algorithms based (in the case of Fresnel diffraction) on the solutions to the paraxial wave equation.

The complex amplitudes can be interpreted as having *magnitude* and *phase*. Viewed geometrically, the phases describe the OPD (see Fig. 8). If a reference surface is not aligned with a particular wavefront, the phases will vary "rapidly" across the reference surface grid. On the other hand, if the reference surface is nearly aligned with the wave-front, the phases vary only slowly. The magnitude describes the amplitude of the electromagnetic field across the exit pupil.



First reference surface

Second reference surface

FIGURE 7 Sampling grid for a collimated beam

Sampling refers to the density of samples per phase crossing. The FFT algorithm works quite well when there are many (at least 4 to 8) sampling points per phase crossing.

When there are fewer, the ability of the FFT to capture higher spatial frequency effects is diminished. This difficulty can become acute when trying to sample diverging or converging waves using planar reference surfaces. Fortunately, this is not necessary. A coordinate transformation (Ref. Sziklas and Siegman) allows the paraxial wave equation to be applied to converging or diverging beams, by removing the spherical component of phase from the light. This is equivalent to sampling the light on a *spherical reference surface*.

Sampling on a sphere takes out the need to sample the spherical part of the wavefront.

Sampling on a plane must be higher to capture the spherical part of the wavefront.

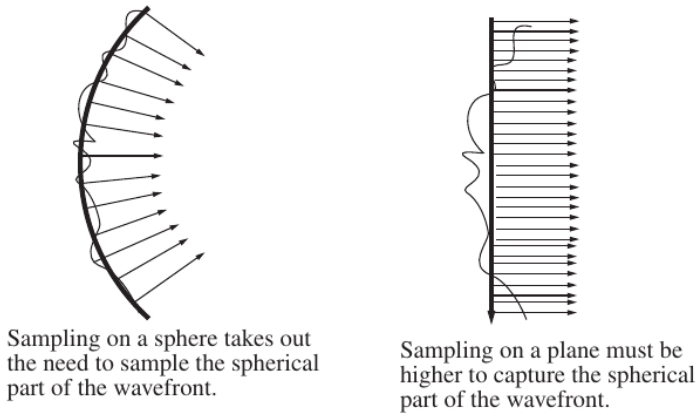


FIGURE 8 Sampling a converging beam

Using a spherical reference surface in a converging or diverging beam allows the sampling to be determined by the aberrations in the beam, which are the phase distortions, or the deviations from the nominal spherical wave. This vastly reduces the sampling requirement for most optics problems.

Whether planar or spherical reference surfaces are to be used to sample a particular light beam depends on the curvature of the wavefronts that make up that beam, and also on the width of the beam. MACOS has functions that allow the geometric determination of the curvature. These functions fit pairs of reference surfaces into the beam at specified points, setting up near-field propagations.

The width of the beam grid relative to the amplitude grid is set to avoid aliasing. *Aliasing* is an artifact of the FFT algorithm, which transforms amplitudes distributed in space to frequencies distributed in the spatial frequency domain. Because the FFT is a periodic calculation, the frequencies computed for the amplitudes are replicated at higher frequencies. If the beam is too wide compared to the extent of the amplitude matrix, some of the replicated “beams” can leak back into the main beam. This effect is aliasing.

The main symptom of aliasing in a diffraction computation is a “noisy looking” beam intensity pattern. Aliasing causes spikes to appear in the beam. These are usually most pronounced in regions of high intensity. Aliasing can be minimized by *padding* the amplitude matrix so that the gridpoints that coincide with the beam occupy about half of the width, or about a quarter of the area of the grid (Figure 9). The padding has the effect of restricting aliasing to very high-frequency effects which are usually of quite low amplitude. Additional padding can be specified by using smaller values of the nGridPts parameter, which governs the relative size of the ray and diffraction grid.

MACOS does not currently provide shaped windowing features.

The width of the beam in regions far from focus is determined geometrically. When the beam is small compared to the wavelength, the width is determined largely by diffraction considerations. Near focus, diffraction effects dominate. MACOS is set up to automatically require adequate padding in diffraction calculations for imaging systems.

Nonetheless, the user is urged to be aware of the rules for constructing diffraction models, as discussed in Section 6. Good discussions of these issues are in Goodman, *Introduction to Fourier Optics* (sampling

and the discrete Fourier transform), and in Sziklas and Siegman's paper on the fast-Fourier-transform propagation method that MACOS uses (References 2 and 3).

The shape and quality of the *point-spread function* (PSF) are a good indicator of the performance of the optical system. The PSF is the image or intensity pattern due to a point

1/4 L 1/2 L 1/4 L

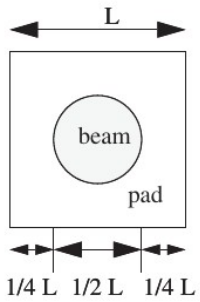


FIGURE 9 Padding the amplitude matrix.

source. Without aberrations, geometrical optics predicts infinitely small images: a delta function of infinite amplitude and zero width. Alas due to diffraction from finite sized apertures of the optical system, the PSF is never a delta function. If the aberrations are small the imaging performance of a system is not degraded, but the performance is *diffraction limited*. As the aberrations in the optical system are increased, the PSF degrades further: its peak value decreases, the side-lobes grow, and the pattern spread out.

2.3.1 Single Plane Diffraction

The simplest and most common PSF calculation uses a ray trace to compute the OPD in the exit pupil of the optical system followed by a single *far-field diffraction propagation* to the focal plane. Figure 10 shows the PSFs computed by MACOS for the Cassegrain telescope calculated in this fashion (see Figure 6 for the OPD maps).

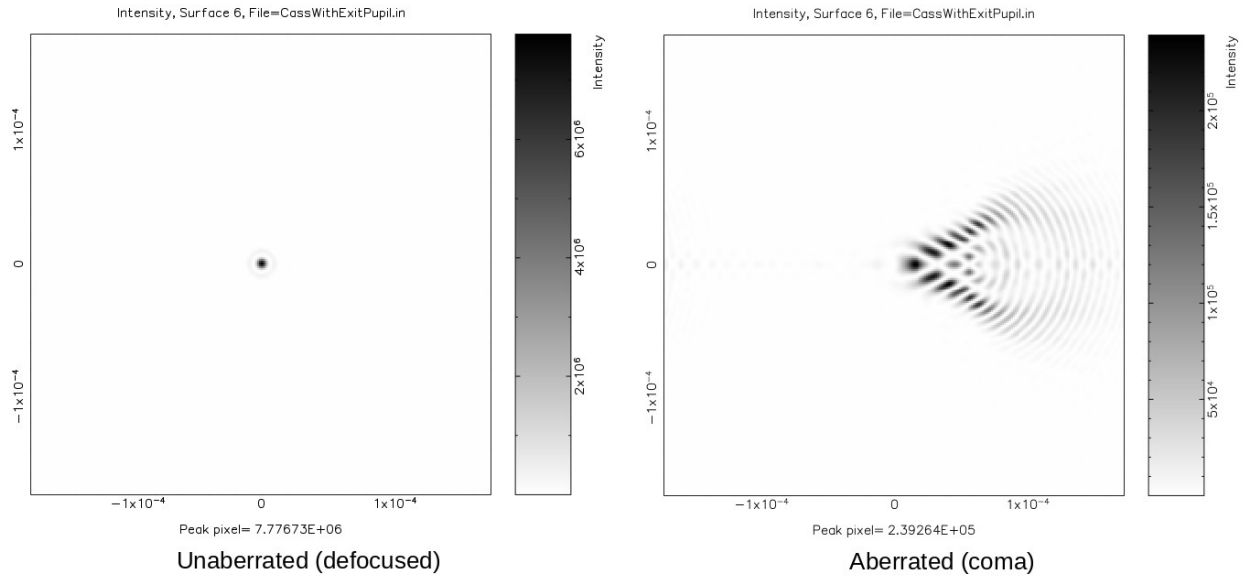


FIGURE 10 Point spread functions for Cassegrain.in

2.3.2 Conventional Multi-plane Diffraction

Conventional physical optics modeling of multi-plane diffraction phenomena *unfolds* the optical beam train into an equivalent set of *thin lenses* arranged in linear fashion along the optical axis.2,4 The effect of the lenses is modeled using the *thin-phase approximation*. This approach assumes that:

- The beam is paraxial, so that phase changes occur as a quadratic function of distance from the center of the array.
- The grid does not change shape or dimension across the lens; the beam out of the lens is the same size as the beam into the lens.
- The lens has no thickness, so all phase changes occur in a plane.

MACOS does not make these approximations. MACOS does not unfold the system nor use the thin-phase approximation.

2.3.3 Multi-plane Diffraction in MACOS

MACOS multi-plane diffraction models are set up by placing reference surfaces immediately before and after the optical elements of a system, or at the start and end of free-space propagations. These reference surfaces are optimized in pairs using MACOS ORS, SRS and FEX functions, discussed in Sections 5 and 6.

MACOS diffraction calculations use four basic propagators:

1. Geometric propagator.
2. Fresnel near-field sphere-to-sphere propagator for converging or diverging beams.
3. Fresnel/Fraunhofer far-field sphere-to-plane propagator.
4. Fresnel near-field plane-to-plane propagator for collimated beams.

The Fresnel propagators are used for the free-space propagations in much the same way as in standard codes. They determine the amplitude on the downstream reference surface based on the amplitude on the upstream reference surface. These propagators are described in more detail in Section 6.

MACOS uses the geometric propagator to determine phase changes across optical elements. The geometric propagator uses rays as *phase probes* aligned with particular amplitude matrix grid points. The phases are determined directly from the exact optical path lengths calculated for the rays from the first reference surface, through the optical elements, to the next reference surface. If necessary due to aberrations through the optical elements, the rays can be regridded to assure alignment with the diffraction grid points. Compared to thin-phase, the MACOS approach

- Does not assume that the beam is paraxial through the optical element (though an assumption of paraxial beams is built into the Fresnel free-space propagators).
- Does not assume that the beam out of the lens is the same size as the beam into the lens.
- Does not assume all phase changes occur in a plane.
- More accurately accounts for geometric effects, such as field-dependant aberrations.

The MACOS geometric propagator does not compute magnitude changes unless polarization ray-trace is being used.

2.4 Polarized Light

MACOS polarized light summary: wht polarized light is; representation of polarized TE fields using rays; polarization rt; interfaces; coatings; vector diffraction.

SECTION 3: User Interface

We begin this section by discussing the mechanics of interacting with the MACOS application. Basic file-oriented functions are discussed, as are various graphics options. Memory requirements are discussed. The commands covered in this section are:

```
Help EXPort JOUrnal EXEcute
```

3.1 Starting and Stopping MACOS

The particular command used to start the MACOS application program depends—of course—on which machine you are using. If your machine is a…

- Macintosh: open the folder containing the MACOS application and double-click on the MACOS 1.0 icon
- Unix: type the command MACOS with appropriate path names to point to the MACOS executable file

On all machines, the response is the same. A window appears with the MACOS command dialog:

```
Modeling and Analysis for Controlled Optical Systems MACOS Version 2.8, May 31, 1999
This run is limited to 70 surfaces, 69 segments, 4226
rays ( 441 for sensitivity calcs). Up to 3 diffraction planes are stored. Up to 10 interpolated
surfaces of up to 255 data points are supported.
Maximum pixel array size is 128 by 128. Diffraction grid size is 128 by 128.
Glass table /home/comp/macosData/macos.glass read, with 202 glasses included. For a list of commands,
type H
```

MACOS>

MACOS>**q**

The MACOS> prompt indicates readiness of the MACOS application to take commands. To exit MACOS, type Quit or BYE to end a session. The response you get depends on whether a PGPLOT graphics window is open (Unix machines running XWindows only). If so, MACOS will ask you to hit return before it goes away. On Sun machines a warning that “non-standard” floating point results may have been produced will be printed. This can be ignored.

Type <RETURN> for next page:

3.2 Help

Type HELP (or H) at the MACOS> prompt to see the list of all commands. The output is paginated: hit return to advance to the next screen. Commands are grouped into 15 categories:

- Session & files: QUit, END, HELP, REset, STatus, SUMmarize, ELTS, EXECute, JOUrnal, MREset, !, PWD, CD, RX/LS/LL, VI/EMAcs
- Prescription I/O: NEW, OLD, FID, VALidate, SAVe, EXPort, SHOw, MODify
- Source & wavelength: CHIefray, WLENS, SWL, MULtispec, NFIlt/RFIlt/SFIlt, ATMosphere, SETC, SAOpt
- Ray tracing: RAY, SEGraytrace, PRAy/RRAy/TPR, MAP, COOrd, ACOor, EFL, BWK, FEXit, FSR, CENter, STOp, CENTRoid, FFP, PFP, FDP, SPCenter

- Behavior toggles: LNEg/NOLNeg, POLarized/NOPol, OBS, REGrid/NOREGrid, ORS, SRS, NONe
- Surface data: SINt, UDSinit
- Perturbation: PERturb, GPERturb, PREad
- Linear model: BUild, DMBuild, PARtials, LPErturb, LPRead, LREset, LSPot, LOPd, LPIxilate
- Diffraction: PROpagate, BEAm, VECtor, SCAlar, COMpose, ADD, DAdd, CADD, NOIse, SEEd, STRetch
- Outputs: OPD, SPOt, ZABerr, ZCOef, LOS, METcalc, INTensity, PIXel, AMPlitude, PHAse/PH, REAl, LOG, MTF/CMTF, IMG
- Plot style: GRAy, WIRe, SLIce, COLumn, CONtour, IMGmode, CIR, GIR, PGP
- File output: TEXt, BINary, FITs/WFIts, MAT, GETMatlabmatrix
- Window & post-processing: PLOcate, NOPLOC, PWIn, SZCo, GBS, BLUr/GBLur, GAIn, ODRaw, PGD, ROW
- System optimization: AVAR, MVAR, DVAR, VARS, AFOV, DFOV, FOVS, CALib, SFOV
- Misc / debug: SRAy, SRT, SRTrace, DOPD, DOPL, ZRM, JWST_v3d / Vis3d

Casing convention. Commands are shown as <UPPER prefix><lower tail>, e.g. RAYtrace, SUMmarize, PERturb. The uppercase part is the minimum-match abbreviation tested by the dispatcher - typing just RAY, SUM, or PER (case-insensitive) is sufficient.

Prerequisite tags. Lines in the HELP output may end in:

[Rx] - needs a loaded prescription (run OLD or NEW first)

[BLD] - needs a built linear model (run BUild or DMBuild)

[DIFF] - needs a propagated wavefront (run PROpagate first)

Commands without a tag work in any state. A complete reference of all commands appears in Appendix C.

3.3 MACOS Model Size Specification

The first thing the user will be asked to do after starting MACOS is to specify a model size. The model size determines MACOS computational accuray, such as the number of rays traced and image grid resolution, and obviously also has an impact on the overall computer simulation time. MACOS currently supports the following model sizes: 128, 256, 512, 1024 and 2048 mainly because the fast Fourier transorm routine is optimized for power-of-2 arrays.

The MACOS model size can be changed at any time during a MACOS session. Use the *mreset* command to set a new model size, as shown in the following example to set a model size to 256:

MACOS> **mreset 256**

```
or
```

MACOS> **mreset**

```
Enter new size (128, 256, 512, 1024, 2048): [128]: 256
Modeling and Analysis for Controlled Optical Systems MACOS Version 3.2, February, 2003
This run is limited to 70 surfaces, 69 segments, 16385
rays ( 500 for sensitivity calcs). Up to 3 diffraction planes are stored. Up to 10 interpolated
surfaces of up to 255 data points are supported.
Maximum pixel array size is 256 by 256. Diffraction grid size is 256 by 256.
```

In previous versions of MACOS, separate model parameter files are used for different model sizes. For example, the model parameter file *param256.cmn* is used for the model size 256. In MACOS 3.2b, a single model parameter file called *macos_param.txt* is used for all model sizes. *macos_param.txt* contains parameter sets for all supported model sizes. To run MACOS, either *macos_param.txt* or a link to it must exist in the MACOS execution directory.

3.4 Entering Commands

MACOS>**old**

MACOS commands can be abreviated using the first letters of the command. The neccessary letters are denoted in capital letters. However, they do not need to be entered in capitals as MACOS is case insensitive.

MACOS prompts the user for the next input. For example, as shown below, to enter an old file, the user types OLD. MACOS then prompts for the filename.

```
Enter file name: lens_array
Input file lens_array.in being loaded. Default used for SegXgrid
Optical Train Summary:
Tracing rays past 4 elements.
Aperture grid type = Circular with 512 grid points.

Elt 1: LensArray : Conic with Kc= 0.0000D+00, Kr=-8.0000D-01, nECoords = -6
Elt 2: Refractor : Conic with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6
Elt 3: FocalPlane : Flat with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6

Elt 4: FocalPlane : Flat with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6
Number of output coordinates is 5
Too many grid points. Resetting npts to 128 MACOS>
```

MACOS also allows commands to be stacked on a single input line. The commands must be separated by a space.

MACOS>**old lens_array show 2**

```
Enter file name: lens_array
Input file lens_array.in being loaded. Default used for SegXgrid
Optical Train Summary:
Tracing rays past 4 elements.
Aperture grid type = Circular with 512 grid points.

Elt 1: LensArray : Conic with Kc= 0.0000D+00, Kr=-8.0000D-01, nECoords = -6
Elt 2: Refractor : Conic with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6
Elt 3: FocalPlane : Flat with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6
Elt 4: FocalPlane : Flat with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords = -6
Number of output coordinates is 5
Too many grid points. Resetting npts to 128 MACOS> show
Enter number of element (0=aperture): [0]: 2

    iElt=  2
 EltName=  rs
 Element=  Refractor
 Surface=  Conic
    fElt=  1.000000000D+18
    eElt=  0.000000000D+00
   KrElt=  -1.000000000D+18
   KcElt=  0.000000000D+00
```

```
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 2.000000000D-01
  RptElt=  0.000000000D+00  0.000000000D+00 1.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  None
    zElt=  0.000000000D+00
PropType=  Geometric
 nECoord=  -6
```

MACOS>

MACOS>**show;;**

MACOS provides defaults for nearly all requested parameters or values. These are usually computed from contextual information. The value of the default for a particular parameter is shown in the prompt line enclosed in square brackets. To indicate that the default is to be accepted, the user can simply hit return when the prompt is displayed. If typing ahead, a semicolon can also be used to indicate that a default should be accepted. In the example below, the default element number 0 is used to select the aperture.

```
Enter number of element (0=aperture): [0]:
ChfRayDir= 0.000000000D+00 0.000000000D+00 1.000000000D+00 ChfRayPos= 0.000000000D+00 0.000000000D+00
-1.000000000D+00 zSource= 1.000000000D+22
IndRef= 1.000000000D+00 Extinc= 0.000000000D+00 Wavelen= 1.000000000D-04 Flux= 1.000000000D+00
GridType= Circular Aperture= 5.000000000D-01 Obscratn= 0.000000000D+00
nGridpts= 128
xGrid= 1.000000000D+00 0.000000000D+00 0.000000000D+00 yGrid= 0.000000000D+00 1.000000000D+00
0.000000000D+00
nElt= 4
```

MACOS>

3.4.1 Interrupting MACOS (Ctrl-C)

Pressing Ctrl-C at the MACOS> prompt - or during a long trace, build, or propagation - prints ‘MACOS: interrupted’ and exits with status 0. Internally MACOS installs a SIGINT handler at startup that uses _exit(0) so it is async-signal-safe and fires regardless of what the runtime is doing. This replaces the Intel-runtime ‘forrtl: severe (69)’ traceback that earlier MACOS versions produced on Ctrl-C.

3.4.2 Empty input

Pressing return at the MACOS> prompt with no command is a no-op - useful for clearing the input buffer or scrolling history without re-running the last command.

3.4.3 Unknown commands

Unknown commands print a single-line warning rather than the verbose multi-line traceback that v3.2 produced.

3.4.4 Session options and LOAD

Five options are properties of the session, not of the prescription: POLARIZATION (and its SCALAR/VECTOR choice), NOLNEG, CENTROID (the FEX axis, Section 5) and CIR (the color palette). Set once, they stay in force across every OLD or LOAD until changed again; a load that finds any of them away from its default prints one line, for example `LOAD: session modes in force: POLARIZATION, VECTOR`. STATUS lists them; RESET returns all but the palette to their defaults. The same holds for the OPD reference set with OPDREF (Section 5) and for IMGMODE (Section 7). Everything else a command sets about the loaded system, such as the stop or the variable elements for CALIB,

belongs to that prescription and is reset by the next load. The MATLAB and Python bindings keep the same options across their loads and warn in the same words.

3.5 Graphics Display

MACOS provides in-line graphical displays on Unix machines using the PGPLOT package from CalTech (see installation notes for information on how to obtain PGPLOT). PGPLOT is started up at the first invocation of a MACOS graphics function, such as INTensity. PGPLOT prompts the user to define the desired form for the graphical output:

MACOS>**stretch**

```
Enter image stretch type (LINEAR, LOG10, SQRT): [LINEAR]: log10
```

MACOS>**intensity**

```
Enter number of element where data is to be generated: [9]: 9
Tracing 3210 rays and propagating 16384 grid points... Scalar FF Prop between Elt 8 and Elt 9 to WF
1:
z1=-9.1481D+01 dx1= 1.0581D-01 min= 1.0581D-01 max= 1.0581D-01 dev= 2.6446D-06 lin=100.00%
z2= 1.0000D+22 dx2=-1.0996D-03
Compute time was 1.4453 sec
Graphics device/type (? to see list, default /NULL): /xwindow
Starting /opt/local/bin/pgxwin_server.
```

The normal selection for Unix machines is “/XWindow.” PGPLOT then sets up a graphics window in which plots are displayed. Once selected, the graphical output option cannot be changed without quitting and restarting MACOS.

3.6 Off-Line Graphics Display

MACOS graphics can also be displayed and analyzed using other applications. This is the only display option for Macintosh. It is also extremely useful for users on all platforms who wish to study their results in ways not provided by MACOS.

One such application is Matlab®, which provides an easy-to-use, powerful environment for manipulating and displaying data in many forms. Matlab is a product of The Math Works, Inc., 24 Prime Park, Natick, MA, USA (508)653-1415. We have included Matlab “.m-file” functions that provide some of the standard MACOS display types. These scripts import a specified datafile, generate a plot according to specified plot and stretch types, and return the data values. They functions also provide templates for modification or translation to other languages. By keeping the display application (e.g. Matlab) running in one window, and MACOS running in another, nearly seamless generation and display of results can easily be accomplished.

Generating the data for off-line display is as simple as selecting a file output “plot type,” using the BINARY, FITS, TEXT or MATLAB commands (Section 7). Subsequent use of graphics-generating commands such as INTensity then create a file and store the requested data in that file. If the file already exists, the user is asked if the file is to be overwritten; if not, a new name is requested. Other applications can then access and display the data. Repeating the previous example:

MACOS>**binary**

```
Plot type set to BINARY
```

MACOS>**intensity** Enter number of element where data is to be generated: [9]: 9 Tracing 3210 rays and propagating 16384 grid points… Scalar FF Prop between Elt 8 and Elt 9 to WF 1: z1=-9.1481D+01 dx1= 1.0581D-01 min= 1.0581D-01 max= 1.0581D-01 dev= 2.6446D-06 lin=100.00% z2= 1.0000D+22 dx2=-1.0996D-03 Compute time was 1.0508 sec Wavefront Propagation Data Summary: Wavelength=

1.6280000D-04; Source Flux= 1.0000000D+00 u-v plane diam= 1.3438007D+01 du= 1.0581108D-01 x-y plane diam=-1.3965216D-01 dx=-1.0996233D-03 Peak intensity= 1.1584734D-01; Peak occurs at i= 65, j= 65 Sum of intensity= 7.6908687D-01 Writing Intensity, Elt=9 DIRECT ACCESS File=tmo.int9 Data file is “tmo.int9” Binary array is of dimension 128 by 128. Magnitude of central pixel is 0.3161060810D-01

MACOS>

The output file name (in this case, “tmo.int9”) is nominally generated from the prescription file name, with the datatype (“int” for intensity) and element number appended after a period. Fits, text and Matlab format files have “.fit,” “.txt,” or “.mat” appended as well, respectively.

The Matlab command required to read the file, generate the plot (at log10 stretch), and return the data for further analysis is:

```
>>intensity=macosIMG('gray','tmo.int9','log10');
```

Here intensity is the returned intensity data, in this case a 128 by 128 matrix, ‘gray’ is the plot type (gray scale), ’tmo.int9’ is the file name, and ’log10’ is the stretch option. The result is a plot functionally identical to what MACOS would produce with the same settings.

Other applications and environments that you may find useful for analyzing and displaying MACOS data include include SAOImage (available free from the Smithsonian Astrophysical Observatory for Unix only), Spyglass Transform, MatrixX, IDL, plotting packages such as DeltaGraph, and spreadsheets such as Excel.

3.7 PostScript and GIF Graphics Output

MACOS provides PostScript and GIF graphical file output directly from PGPLOT (Unix only). When PGPLOT is started up, at the first invocation of a MACOS graphics function, it prompts the user to define the desired form for the graphical output. Typing

? prints a list of PGPLOT output options:

```
Graphics device/type (? to see list, default /NULL): ?
PGPLOT v5.0.0 Copyright 1994 California Institute of Technology Legal PGPLOT device types are:
/GIF (Graphics Interchange Format file, landscape orientation)
/VGIF (Graphics Interchange Format file, portrait orientation)
/NULL (Null device, no output)
/PS (PostScript file, landscape orientation)
/VPS (PostScript file, portrait orientation)
/CPS (Colour PostScript file, landscape orientation)
/VCPS (Colour PostScript file, portrait orientation)
/TEK4010 (Tektronix 4010 terminal)
/RETRO (Retrographics VT640 Tek emulator)
/XTERM (XTERM Tek terminal emulator)
/XWINDOW (X window window@node:display.screen/xw)
/XSERVE (A /XWINDOW window that persists for re-use)
/XDISP (pgdisp or figdisp server)
Graphics device/type (? to see list, default /NULL):
```

The GIF options generate separate GIF files for each graphic, named pgplot.gif, pgplot.gif_2, etc. in sequence.

The PS options generate a single file, pgplot.ps, containing all of the plots generated in a single session. It can be printed (on Unix machines) with the command:

```
% lpr pgplot.ps
```

The file can be renamed if you wish to save it.

Screen capture utilities and applications, such as SAOimage or XV, can also be very useful in recording MACOS graphical outputs.

3.8 MACOS Files

3.8.1 .In-File

An optical system prescription created in MACOS is saved in a “filename.in” file. This file stores the prescription in a text format which can be edited offline and reused later. The MACOS prescription files are generated by the NEW command (Section 4).

3.8.2 .Out-File

MACOS no longer generates a “filename.out” file for recording each result generated in a session.

3.8.3 Glass Tables

Glass data is provided in a text file, named macos.glass, which is located in a special MACOS data directory (see next section). The form and content of this file is discussed in Section 4.

Alternatively, glass data can be provided as part of the optical prescription “.in-file.”

3.8.4 MACOS Data Directory

Beginning with Version 2.8, certain MACOS data files, such as glass tables, are being placed in a special directory. On Unix systems, the path to this directory can be specified by the user by defining an environment variable, MACOS_DATA. This is done in response to the Unix shell prompt, as in:

```
> setenv MACOS_DATA /home/macos/data/
```

The above command establishes the directory /home/macos/data as the location of the data files for subsequent MACOS sessions. The user may specify any directory as MACOS_DATA.

If MACOS_DATA is not specified, 3 other directories will be searched for data files. In order these are:

1. Directory macosData in the user’s home directory
2. Directory /home/comp/macosData
3. The current directory

If the required data file is not found in any of these directories, a warning message is issued. In this event, the user is required to supply all relevant data, such as glass coefficients, through the prescription file.

3.8.5 Data Export

The EXPort command writes certain *variables* to a file, in several different formats (Section 7 discusses writing *graphics* data to files). The variables are useful when linking linear models to thermal or structural codes (see Section 8).

The following variables can be exported:

- CMAtrix: optical sensitivity matrix in global or local coordinates
- CmDM: deformable mirror sensitivity matrix
- RayPos: current position 3-vectors for each ray

- RayDir: current ray direction 3-vectors for each ray
- CumRayL: current optical pathlengths for each ray Files may be saved in the following formats:
- BINary: data is saved to a Fortran binary file, filename.bin
- TEXt: data is saved to an ASCII text file, filename.txt
- MFILE: data is saved to a Matlab file, filename.m

The following is a sample EXPort session.

MACOS>**build ;**

```
Enter terminal element number: [1]:
Tracing 12669 rays but not BUILDing linear model. BUILD limited to 500 rays. MOD npts and recompute.
MACOS>export
Current element is 1
Enter EXPORT mode (BINary, TEXt, MFIle, NAStran, H or Quit): [BINARY]: tex
Exporting data to lens_array.txt
Export data for CHFRay only, ALLRays, or NOChfray? [CHFRay]: chfr
EXPORT (CMAtrix, RayIndex, RAYPos, RAYDir, RAYL, H or Q): [QUIT]: raypos
% RayPos 3-vectors for 1 rays at Element 1
EXPORT (CMAtrix, RayIndex, RAYPos, RAYDir, RAYL, H or Q): [QUIT]: q MACOS>
```

3.9 Using Macros

MACOS has a scripting or macro capability. This is provided by keystroke recording and play-back commands, JOUrnal and EXEcute. Macros can call other macros, up to 10 deep. Comments (anything following a “%” on a line) and blank lines are ignored. Looping, conditionals and other “language” capabilities are not currently supported.

3.9.1 Journal Files

The JOUrnal command is used to start and stop a journal of all the commands the user inputs to the terminal. Typing the JOUrnal command initiates keystroke recording, which continues until toggled off by typing the JOUrnal command a second time.

JOUrnal allows the user to keep track of the commands entered in a MACOS session. The resulting .jou-file can later be used as a macro command (see Section 3.9.2). The

.jou-file can also be edited and modified offline.

The JOUrnal command has one argument, the filename. The filename entered by the user will automatically have the file extension .jou appended, to denote that the file is a JOUrnal file. If the file already exists, then the contents of the file are deleted before the JOUrnalizing begins. It is imperative that the user select a unique filename, if is it desired that old JOUrnal files be kept.

3.9.2 Executing Macros

The EXEcute command allows the user to designate a .jou-file from which further MACOS commands are read. Each command (including other EXEcute commands) is executed in sequence, until an END OF FILE symbol is read from the .jou-file.

MACOS then returns to interactive mode and the user can once again enter commands from the keyboard.

The EXEcute command has one argument, the filename of the batch file. The filename must have the file extension .jou to denote that the file is an journal file.

The user can edit a file produced with the JOUrnal comand to produce a macro. For example, if the .jou-file was created with an OPD at the 10th element, it is trivial to modify it to calculate the OPD at the 14th element. The user only needs to change the 10 in the .jou-file to a 14 using a text editor.

Examples are provided in the macosDemo directory distributed with the MACOS code.

3.10 Memory Requirements

The MACOS application currently uses a fixed memory allocation set at compile time. Different run modules are supplied for different sized problems, depending on the machine and available memory. Table 4 summarizes requirements for specific platforms.

TABLE 4. Memory Requirements

executable	Sun/Solaris	PowerMacintosh
macos128	9.75 MB	
macos256	18.3 MB	
macos512	38.4 MB	
macos1024	115 MB	

SECTION 4: Describing Optical Systems

Systems

A first MACOS session typically begins by invoking the NEW command. This begins a dialog whose function is to define the data used by MACOS in its computations. This data comprises the optical prescription, and is saved by MACOS as an ".in-file," which can be edited and reused later using the OLD command. The contents of the .in-file can be altered within a MACOS session using MODIFY, PERTURB or other commands, and saved as a new .in-file using the SAVE command. The .in-files can also be created and edited off-line using a text editor. The commands covered in this section are:

```
NEW DRAw
OLD (LOAd)
SUMmarize STAtus RESet SHOw
MODify BEAM SAVe
```

4.1 Specifying the Light Source

The NEW dialog begins by asking the user for information about the light source, such as where it is located, in what direction it is oriented, and what its optical properties are. After the source is defined, similar questions are asked about each optical element in turn. This section is intended to help the user provide the appropriate answers to these questions. It is also intended to define MACOS variable names, which are used in the

.in-files to specify aspects of the prescription.

MACOS represents light sources as *beams*, consisting of a ray bundle and an associated complex amplitude matrix (or matrices). Beams can be collimated or focused, as per Figure 11. In addition, the illumination profile of a beam can be specified as uniform, gaussian, dipole or cosine-to-a-power, using the BEAM command discussed in a later section. Collimated beams are composed of a bundle of parallel rays, originating at a plane in front of the entrance aperture. Collimated beams are used to represent the light received from a point-source at an infinite distance from the system. They are also used when collimated light sources, such as some lasers, are to be modeled. Focused beams are bundles of rays converging to or diverging from a point in space and represent the light from a nearby point source. Focused beams are defined on a spherical surface whose center of curvature is at the source point.

The properties that must be specified by the user include:

- Source position and direction, defined by the position and orientation of the "chief" ray (really the gut ray – the ray that lies in the center of the beam)
- Aperture size (collimated source) or angle (point source)
- Aperture shape (circular, square, hexagonal, pie, flower)
- Illumination profile (flat, gaussian, dipole or cosine-to-a-power)
- Wavelength
- Flux through the defined aperture
- Polarization state (if using polarization ray-trace analysis functions)
- Segmentation (unsegmented, hexagonal segments, or "pie" segments)
- Coordinates to define the orientation of the beam to the optical elements

4.1.1 Source Position and Orientation

In MACOS, a *light source* is a surface but not an element. The surface is defined by a single wavefront emitted from a single point source. The zSource parameter is the distance from the origin of the light to the light source surface, as defined by the location of the chief ray, ChfRayPos (see Figure 11). Collimated sources are specified by zSource=1d22. For point sources, the convention is that the sign of zSource is positive for a converging beam and negative for an expanding beam.

for a converging beam and negative for an expanding beam.

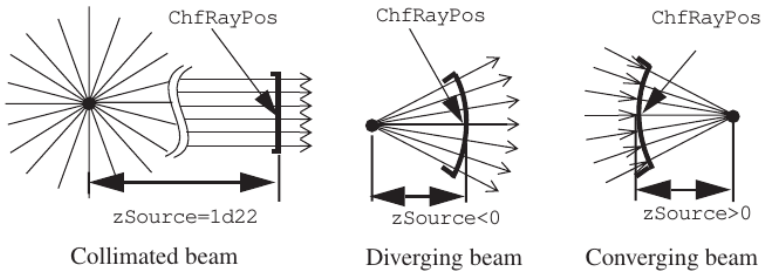


FIGURE 11 Light sources

The location and orientation of the light source is determined from the *chief ray*. The MACOS "chief ray" is the central or gut ray of the beam, as discussed in Section 2.2. It is the center ray of the incident beam bundle. It is normally a true chief ray as well, passing through the center of the system stop. This condition is enforced by the user, using the STOP or CENTER commands, as described later.

To define the source, the user must specify in the global coordinate system:

- Chief ray position vector
- Chief ray direction vector

These are set by the 3-vector inputs ChfRayPos and ChfRayDir. If the source is collimated, then every ray in the beam starts out with its direction equal to ChfRayDir. The position of each ray is calculated from ChfRayPos, the shape and type of aperture, and the specified ray density. If the source is a point source, appropriate position and direction is computed so that each ray starts from the surface of a sphere centered at the source point.

NOTE: Converging or diverging sources **should not be located at the source point**, or, for example, divide-by-zero errors will result when performing diffraction analysis.

4.1.2 Refractive Index and Extinction Coefficient at the Source

The refractive index is, in general, a complex number. The real part is the conventional index, giving the speed of light in the particular medium, and so defining the direction

changes a ray experiences at refractive interfaces. The imaginary part is the extinction coefficient, which defines the attenuation of the field in the medium. Only the index (real part) is significant in MACOS scalar ray-tracing calculations. The extinction coefficient is used only when polarization ray-trace is selected, by use of the POL command.

- The real part of the refractive index of the source medium is specified by IndRef. If ChfRayPos is in a vacuum, then IndRef=1. For many applications it is sufficient to approximate the index of refraction of air as 1, although the exact value depends on the pressure and temperature.

- The negative of the imaginary part of the refractive index, the extinction coefficient, of the source medium is specified by Extinc. For lossless media such as vacuum and air, set ExtInc=0. For perfect metals, set ExtInc=1d22. This parameter is used only when polarization ray-tracing is turned on.

The user must specify the index explicitly for the source only. For other elements, particular glasses can be specified, and the index will be varied as a function of the wavelength of the beam. The source IndRef and Extinc are not changed automatically with changes in wavelength. The user can implement such changes as needed using the MODIFY command.

4.1.3 Wavelength

The wavelength is specified by the wavelen variable. For most calculations, MACOS uses one wavelength at a time. Multiple-wavelength images can be generated using the COMPOSE command to define detector parameters, using use the MODify command to change the wavelen (and Flux) variables (see Section 4.11.3), then using the ADD command to separately propagate and accumulate different wavelength and flux level images at the detector. This procedure can be automated by using ".filt" filter files and the MULtispec command.

4.1.4 Flux

The total flux of the source illumination through the aperture area (defined by the Aperture, Obscuration and GridType parameters) is specified by Flux.This parameter is used in MACOS to facilitate the radiometric modeling of systems. It is useful when constructing composite images of multiple sources or at multiple wavelengths. This procedure can be automated by using ".filt" filter files and the MULtispec command.

4.1.5 Aperture

The aperture size is set by the Aperture parameter, whose interpretation is different for collimated or point sources. If the source is collimated (zSource=1d22), Aperture is the *diameter* of the input beam. If it is a point source(zSource<1d22), Aperture is the full *angle* (entered in radians like all angles in MACOS) subtended by the two marginal rays (see Figure 12). Thus, the numerical aperture of the system is

$$N.A.=n\sin(Aperture) \quad (4.1)$$

where n is the index of refraction.

Obscurations can be defined in the source beam if desired. Obscuration size is specified in the same way as the aperture, with the parameter Obscratn setting the diameter or angle of a *central* obscuration.

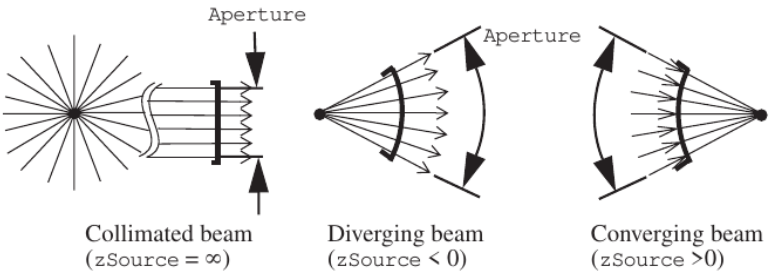


FIGURE 12 Aperture sizing

NOTE: This aperture obscuration option can be useful, but is not the best way to represent most beam-train obscurations. It is better to explicitly place obscurations on the appropriate elements as discussed in Section 4.7.

The entrance aperture can have several shapes, as shown in Figure 13, depending on whether the system is monolithic – in which all optics are of one piece – or segmented – in which at least one mirror is split into separate segments. For monolithic systems, the aperture shape can be round or square, as set by the GridType parameter. For segmented systems the options are hexagonal array, round with 6 pie-shaped outer segments, or round with an arbitrary number of outer segments. For computational efficiency, the segmentation is defined at the light source.

segmentation is defined at the light source.

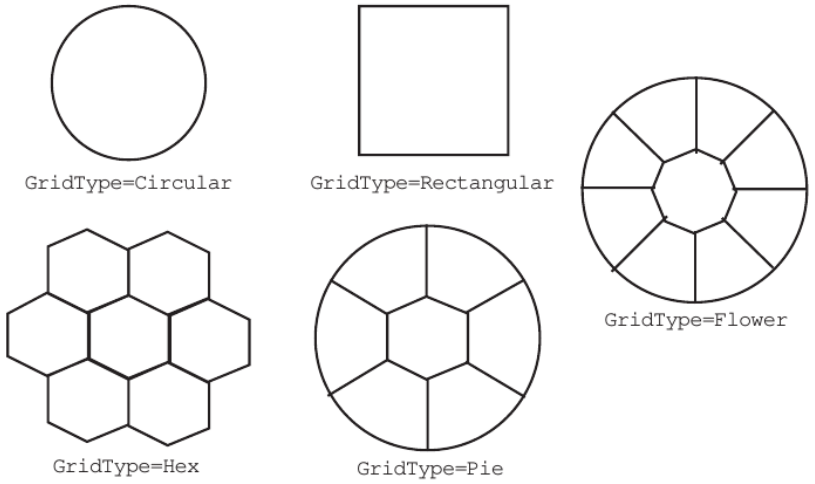


FIGURE 13 Aperture shapes

It is not necessary to segment the aperture to use lenslet arrays, as discussed later.

NOTE: Registration of a segmented aperture on a segmented mirror may shift when the beam incidence angle changes, or when intervening optics are perturbed. This can cause rays that nominally strike one mirror segment to move to another. In these conditions, the user should check to ensure proper registration.

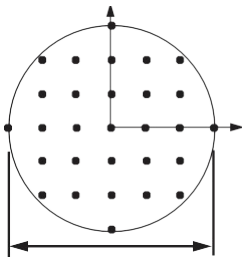
4.1.6 Ray Grids

The density of rays across the beam and the orientation of the ray grid are set by three

parameters:

- Number of ray grid points across the aperture is set by nGridPts
- Ray grid coordinate vectors xGrid and yGrid determine the global orientation of the ray grid and so the beam coordinates

The ray grid is laid out on the surface that defines the light source, according to a coordinate system for which zGrid equals the negative chief ray direction ChfRayDir and xGrid is as specified by the user (see Figure 14). The number of rays across the aperture is set by nGridPts.



```
yGrid
```

Coordinates

```
yGrid=zGrid x xGrid
```

is set by nGridPts.

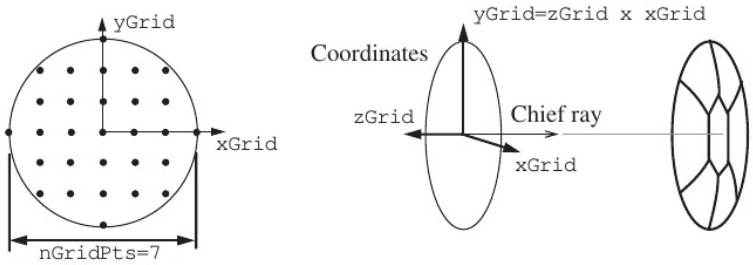


FIGURE 14 Ray grid layout

4.2 Example Light Source

MACOS obtains the information it needs to build an optical prescription (an ".in-file" dataset) through the NEW file definition dialog. The first questions in this dialog are about the light source and the latter questions pertain to the optical elements.

For the first example, we start the prescription for a Cassegrain telescope, Cassegrain.in, shown in Figure 15. The light source is a point source at infinity so the wave-front incident on the telescope is collimated. In the example dialog, the prompts are exactly as MACOS gives them. The correct user response is indicated in **bold** text, and the name of the variable being set is in ***italic*** text.

the name of the variable being set is in *italic* text.

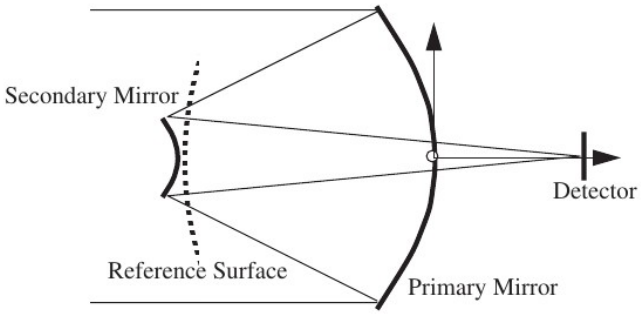


FIGURE 15 Cassegrain telescope layout

```
Prompt Response VariableName
```

MACOS>**new**

```
Enter file name: Cassegrain
File Cassegrain.in being created. filenam
Enter chief ray input direction vector (x,y,z) [0.,0.,1.0]:0,0,1 ChfRayDir Enter chief ray location
vector (x,y,z) [0.,0.,0.]:0,0,-5 ChfRayPos Enter distance from source [1.000000E+22]:1d22 zSource
Enter chief ray index of refraction [1.00000]:1 IndRef
Enter chief ray extinction coefficient [0.]:0 Extinc
Enter monochromatic wavelength [6.000000E-07]:1d-6 Wavelen
Enter intensity (flux over aperture area) [1.00000]:1 Flux
+ +
| Source ray grid types: |
| Circular Square Hex |
| Pie Flower |
+ +
Enter source grid type [Circular]: Circular GridType
Enter aperture diameter or angle [1.00000]:4 Aperture
Enter number of grid points [65]:15 nGridPts Enter input plane x-axis vector (x,y,z)
[1.00000,0.,0.]:-1,0,0 xGrid Enter input plane y-axis vector (x,y,z) [0.,1.00000,0.]:0,-1,0 yGrid
Enter number of elements [no default]: 5 nElt
```

This dialog produces the following lines in the prescription file Cassegrain.in: ChfRayDir= 0.000000000D+00 0.000000000D+00 1.000000000D+00

```
ChfRayPos= 0.000000000D+00 0.000000000D+00 -5.000000000D+00 zSource= 1.000000000D+22
IndRef= 1.000000000D+00 Extinc= 0.000000000D+00 Wavelen= 1.000000000D-06 Flux= 1.000000000D+00
GridType= Circular Aperture= 4.000000000D+00 Obscratn= 0.000000000D+00
nGridpts= 15
xGrid= -1.000000000D+00 0.000000000D+00 0.000000000D+00 yGrid= 0.000000000D+00 -1.000000000D+00
0.000000000D+00
nElt= 5
```

The Cassegrain.in example is finished in Section 4.9.

Now consider the case of a non-collimated input beam, zSource not equal to infinity. A Cassegrain telescope has been turned into a beam projector. The light source is now located where the detector was previously located as shown in Figure 16.

MACOS>**new**

```
Enter file name: zsourc
File zsourc.in being created.
Enter chief ray input direction vector (x,y,z) [0.,0.,1.0]:0,0,1 ChfRayDir Enter chief ray location
vector (x,y,z) [0.,0.,0.]:0,0,-1 ChfRayPos Enter distance from source [1.000000E+22]:-0.1 zSource
Enter chief ray index of refraction [1.00000]:1 IndRef
```

```
Enter chief ray extinction coefficient [0.]:0 Extinc
Enter monochromatic wavelength [6.000000E-07]:1d-3 Wavelen
Enter intensity (flux over aperture area) [1.00000]:1 Flux
+ +
| Source ray grid types: |
| Circular Square Hex |
| Pie Flower |
+ +
Enter source grid type [Circular]: Circular GridType
Enter aperture diameter or angle [1.00000]:0.1076 Aperture
Enter number of grid points [65]:25 nGridPts
Enter input plane x-axis vector (x,y,z) [1.00000,0.,0.]:-1,0,0 xGrid
```

-1.1 -1.0

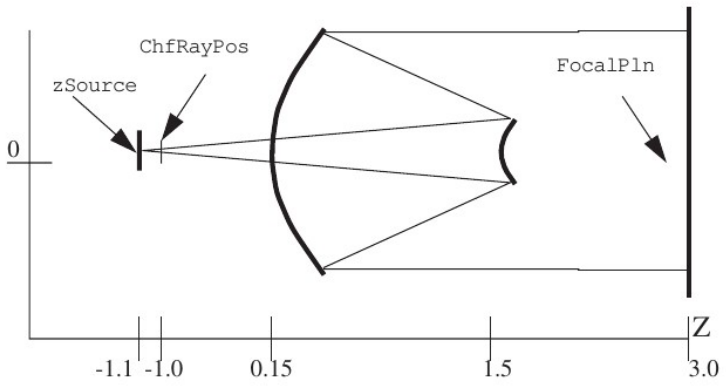


FIGURE 16 Non-collimated light source example

```
Enter input plane y-axis vector (x,y,z) [0.,1.00000,0.]:0,-1,0 yGrid
Enter number of elements [no default]: 6 nElt
```

This results in the following lines in the prescription file zsource.in:

```
ChfRayDir= 0.000000000D+00 0.000000000D+00 1.000000000D+00 ChfRayPos= 0.000000000D+00 0.000000000D+00
-1.000000000D+00 zSource= -1.000000000D-01
IndRef= 1.000000000D+00 Extinc= 0.000000000D+00 Wavelen= 1.000000000D-03 Flux= 1.000000000D+00
GridType= Circular Aperture= 1.076000000D-01 Obscratn= 0.000000000D+00
nGridpts= 25
xGrid= -1.000000000D+00 0.000000000D+00 0.000000000D+00 yGrid= 0.000000000D+00 -1.000000000D+00
0.000000000D+00
nElt= 6
```

Notice that the Aperture entry has changed significantly, since Aperture now represents an angle, instead of the diameter of the input beam. The ChfRayPos is now a small distance in front of the source, so zSource is now negative to indicate an diverging beam.

4.3 Specifying Optics

The next part of the new dialog is the specification of the optical elements. The first step in defining the beam train is actually the last step in the source dialog: specifying the number nElt of optical elements in the beam train. The number of elements is equal to the number of active optical surfaces, plus any reference (dummy) surfaces, plus 2 return surfaces for an exit pupil, if desired.

The new dialog continues by prompting the user for all data needed to define each optic. Optics are specified one at a time, in the sequence in which they are encountered by the light from the source (except nonsequential and segmented optics). Some optics, such as mirrors, are realized by a single *element*, or optically active surface. Other optics require

two or more elements. Lenses require two refracting elements, one for each surface (see the example in Section 4.4.2). Corner cubes require three non-sequential reflecting elements. Each element in multiple-element optics is entered separately.

The first step in specifying an element is to define its *type* (e.g., Reflector) and its *surface type* (e.g., Conic). Table 5 lists the elements currently supported by MACOS, and Table 6 lists the surface types. Further data required to define the surfaces may be requested, depending on specified surface type.

TABLE 5. Element types

Element Type (Element=..**)**	**Allowed Surface Types**	**Description**
Reflector	all	reflective surfaces (e.g., mirrors)
Refractor	all	refractive surfaces (e.g., lenses)
LensArray	conics or aspheres	lens arrays
Grating	all	reflective diffraction gratings
HOE	all	reflective holographic optical elements
NSReflector	conics	non-sequential reflective surfaces
NSRefractor	conics	non-sequential refractors
Segment	all	segmented mirrors
ObsSrf	conics	obscuring surfaces
FocalPln	flats	focal planes
RefSrf	conics	reference surfaces
ReturnSrf	conics	return surfaces

A non-sequential group (`NSReflector`/`NSRefractor` elements) may carry an optional `NSCount=` keyword on each member: the **hit budget**, i.e. the number of surface hits a ray may take inside that group before it is forced on to the next sequential element. `NSCount=` absent or `0` means **unlimited** (needed where a ray legitimately bounces among the members, as in a corner cube); `NSCount= 1` is the single-hit case of a segmented mirror. The budget is authoring intent — set it to `1` on a grid-surfaced non-sequential group, where an unbounded search can send a ray's surface-solve bracket to divergence.

TABLE 6. Surface types

Surface Type (Surface=..**)**	**Surface Description**
Flat	Planar
Conic	Conicoid of revolution. Precise shape is defined by conic constant KcElt and radius KrElt parameters (or fElt and eElt)
Toric	Toric or cylindrical shapes, defined by separate radius parameters in x and y directions.

Surface Type (Surface=..**)**	**Surface Description**
Anamorphic	“Saddle” shapes, defined by separate conic and radius parameters in x and y directions.
Aspheric	Base conicoid, with 10th-order aspheric deviation.
Monomial	Base conicoid, with 14th-order cartesian polynomial deviation.
Zernike	Base conicoid, with deviation defined by Zernike polynomials.
Interpolated	Base conicoid, with deviation defined by X-Y-Z data triplets
GridData	Base conicoid, with deviation defined by square matrix pixels
User-defined	Deformable mirrors defined using influence functions.

The second step is to define element location, orientation and rotation point, as discussed in Section 2.1:

- Element *location* is specified by the VptElt vector. This is a three-vector in global coordinates giving the location of the vertex of the element.
- Element *orientation* is specified by the psiElt vector. This is a unit magnitude three-vector in global coordinates giving the direction of the principal axis of the element.
- Element *rotation point*, RptElt, defines the pivot point for rotations of the element implemented using the Perturb command. It also defines the coupling between rotation and translation for the Build command. Together with any element coordinates, Telt, RptElt governs the element response to perturbations.

The third step is to define any aperture or obscurations that might occur at the element. Apertures define the margins of the element, and can be circular or rectangular. Obscurations are circular or rectangular regions on the surface of the element where light is completely absorbed. Negative obscurations “un-absorb” the light, countering the effect of an obscuration in regions where they overlap. Obscurations and apertures can be placed anywhere on the surface.

The fourth step is to define the zElt and PropType diffraction propagation parameters, which are needed if the element is at the start or end of a diffraction propagation.

The final step is to specify element coordinates (Telt), if they are desired.

In the next several subsections, each of these steps is discussed more carefully, and examples are shown.

4.4 Element Types

Element type and surface shape are each most critical in the determination of the properties of an optic. In this subsection we define the MACOS element types and illustrate their use through a sequence of specific examples.

4.4.1 Reflectors

Mirrors are entered as type Reflector. Reflectors can have any surface type or shape; the user will be prompted for different parameters depending on which surface type is chosen. No element-specific parameters are required for reflectors unless polarization ray-tracing is turned on. In that case, Extinc and IndRef should be carefully specified, as they determine the transmission of the beam past the reflective interface.

This example shows how to enter a flat mirror in MACOS:

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: flatMirror EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :reflect Element
+ +
```

Surface types:		
Flat	Conic	Aspheric
Anamorphic	Zernike	Monomial
Interpolated	UserDefined	

```
+ +
Enter surface type :flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,-1 psiElt Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]: no
Enter element index of refraction [1.00000]:1 IndRef
Enter element extinction coefficient [1.000000E+22]:1d22 Extinc
```

This results in the following lines in the prescription file:

```
    iElt=  1
 EltName=  flatMirror
 Element=  Reflector
 Surface=  Flat
    fElt=  1.000000000D+22
    eElt=  0.000000000D+00
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
    nObs=  0
  ApType=  None
    zElt=  1.000000000D+22
PropType=  Geometric
 nECoord=  -6
```

4.4.2 Refractors

A refracting element is the interface between two media of differing index of refraction, such as air and glass. Rays traversing the interface are, of course, bent at the interface. lenses, windows and other refracting optics are actually modeled using 2 optical elements, both of type Refractor. The first element

defines the front surface of the optic and sets the index for the ray traveling inside the optic. The second element defines the back surface, and resets the index to the value outside the optic.

Some refracting elements, such as prisms, use more than two elements, and may require use of nonsequential reflectors and refractors as well – see Section 4.4.15. Arrays of lenslets are covered in Section 4.4.4.

As with reflectors, refractors in MACOS may have any surface shape. The index of refraction IndRef can be set directly in the prescription, or indirectly, by specifying a particular glass for the element. To set the index directly, select FixedIndex glass type and then specify the index. No other element-specific parameters are required, unless polarization ray-tracing is turned on. In that case, Extinc must be specified as well.

This example lens has a spherical front surface and a planar rear surface, as illustrated in Figure 17:

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: lens_front EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating TrGrating |
| RfPolarizer TrPolarizer Obscuring |
+ +
Enter element type :refractor Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined Grid data |
| Toric InfluenceFcn |
+ +
Enter surface type :conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,-1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:1 fElt
Enter element eccentricity [0.]:0 eElt
Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]: no
Enter glass type [FixedIndex]: BK7 GlassElt
Enter Element 2 Data:
Enter element name (no blanks) [Elt]: lens_back EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating TrGrating |
| RfPolarizer TrPolarizer Obscuring |
+ +
Enter element type :refractor Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined Grid data |
| Toric InfluenceFcn |
```

```
+ +
Enter surface type :flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0.1 VptElt Enter element rotation point
(x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1 psiElt
Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]: no
Enter glass type [FixedIndex]: FixedIndex GlassElt
Enter element index of refraction [1.00000]:1 IndRef
Enter element extinction coefficient [0.00000]:0 Extinc
```

A Refractor may carry a gradient-index model (`GradInd=` with its coefficients). Only model 5, the radial profile $n(r) = n0 + n2\ r^2 + n4\ r^4$ integrated with a Runge–Kutta step, is implemented; a ray meeting any other model is lost by name (`GRIN model not implemented`, counted in the WARN summary) rather than traced with undefined values. Model 5 cannot presently be entered from a prescription file; this is an open item.

NOTE: It is not uncommon to “lose” rays, if they miss the properly defined area of a lens, that is, if the rays miss the region where the 2 surfaces of the lens are sequential. If rays extend beyond the intersection point of the two elements, the rays become undefined and are not traced further as shown in Figure 17.

This ray hits the elements out of sequence, becomes undefined

shown in Figure 17.

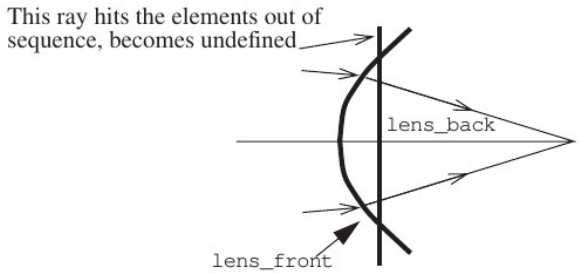


FIGURE 17 Ray bundle must pass within surface intersection points of the refracting elements for all rays to be traced properly

4.4.3 Glass Tables

Version 2.8 of MACOS comes with a single glass table, which incorporates the Schott glass types as of May 1999 (from http://schottglass.com). This table is a text file, macos.glass, located in the MACOS data directory. See Section 3.8.4 for details on how to specify this directory.

The user may specify the index of glasses or other optical materials in several ways:

- By selecting from the materials in the provided macos.glass file, as in the first part of the example in Section 4.4.2.

- By using a different macos.glass file, located in the MACOS data directory. The user can easily add other materials to a private copy of macos.glass, using the MACOS_DATA environment variable to define its location.
- By directly specifying the 6 Sellmeier glass coefficients in the prescription. These coefficients are B1, B2, B3, C1, C2, C3 from the manufacturers catalog. Note that the MACOS SAVE command saves the coefficients of each glass in the prescription file, even if the glass data was originally derived from the MACOS glass table.
- By declaring a material to be FixedIndex, and then specifying the index, in the prescription. This approach uses a single value of the index for any wavelength of light.

The form of the glass table is simple. Each glass is defined in a single row of the file, with the first entry being the name of the glass, followed by the 6 Sellmeier coefficients. The following shows the first few entries from macos.glass:

Air 0d0 0d0 0d0	0d0 0d0 0d0			
BaF13 1.57068590E+00 02 1.13934126E+02	1.51 295152E-01	1.07 638026E+00	9.39 377487E-03	4.49 479866E-
BaF3 1.32064267E+00	1.33 572683E-01	8.85 521821E-01	8.87 798715E-03	4.20 290346E-
02 1.11729167E+02				
BaF4 1.37492590E+00	1.46 354500E-01	9.12 643359E-01	9.29 554343E-03	4.40 426654E-
02 1.16607899E+02				

4.4.4 Lens Arrays

Lens arrays (a.k.a. lenticular arrays or lenslet arrays) are refracting optics. Usually, a lens array will have one flat surface and one surface figured into a mosaic of regularly distributed lenslets (Figure 18). Lens arrays are used most often in Hartmann wavefront sensors. An example adaptive optics wavefront sensor is provided in Appendix A.4.

MACOS uses a LensArray element type to model the bumpy surface of a lens array. LensArray elements are specified much as Refractor elements are, with the addition of a few new parameters. LensArray elements can have conic or aspheric lenslet shapes. Enter shape parameters that describe a single lenslet, and all the other lenslets will be automatically replicated. The location and orientation parameters VptElt and psiElt define the plane containing all of the lenslet vertices, and specify the orientation of each lenslet.

The first new parameter is LensArrayType, which can have two values: LensArray-Typ=1 for hexagonal arrays; and LensArrayTyp=2 for rectangular arrays. The second new parameter is LensArrayWidth, which is the width of a single lenslet (flat to flat for hexagonal lenslets) (Figure 18).

will be automatically replicated. The location and orientation parameters VptElt and psiElt define the plane containing all of the lenslet vertices, and specify the orientation of each lenslet.

The first new parameter is LensArrayType, which can have two values: LensArrayTyp=1 for hexagonal arrays; and LensArrayTyp=2 for rectangular arrays. The second new parameter is LensArrayWidth, which is the width of a single lenslet (flat to flat for hexagonal lenslets) (Figure 18).

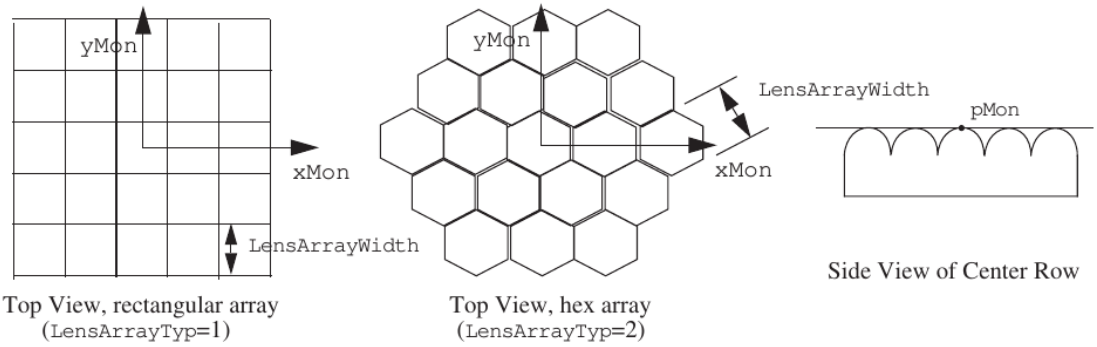


FIGURE 18 Hexagonal lens array

LensArray elements use a surface coordinate system to define the location and orientation of the lenslet grid on the underlying plane. The variable pMon is the center of the coordinate system, defining the vertex position of the “central” lenslet. pMon can differ

from VptElt, though only the projection of pMon onto the plane is significant. The variables xMon, yMon, and zMon define the surface coordinate frame as shown in Figure 27.

An additional example of the use of lenslet arrays is provided in Appendix A.4.

```
Enter element name (no blanks) [Elt]: LensArray EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :LensArray Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:2
Enter Conic Constant [0.]:0 KcElt
Enter Radius [-1.000000E+22]:-0.8 KrElt
Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [0.800000]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
```

```
Enter glass type [FixedIndex]: FixedIndex GlassElt
Enter element index of refraction [1.00000]:1.5 IndRef
Enter element extinction coefficient [0.]:0 Extinc
Enter surface reference point location (x,y,z) [0.,0.,0.]:0,0,0 pMon Enter surface x-axis unit vector
(x,y,z) [1.00000,0.,0.]:1,0,0 xMon Enter surface y-axis unit vector (x,y,z) [0.,1.00000,0.]:0,1,0
yMon Enter surface z-axis unit vector (x,y,z) [0.,0.,1.00000]:0,0,1 zMon Orthoganalized zMon= 0. 0.
1.0000000000000
Enter lenslet width [1.00000]:0.1 LensArrayWidth
Enter lens array type (1=hex, 2=square) [1]:1 LensArrayType
```

This results in the following lines in the prescription file:

```
iElt= 1
EltName= LensArray Element= LensArray Surface= Conic
fElt= 8.000000000D-01 eElt= 0.000000000D+00 KrElt= -8.000000000D-01 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00
IndRef= 1.500000000D+00

Extinc= 0.000000000D+00
pMon= 0.000000000D+00 0.000000000D+00 0.000000000D+00 xMon= 1.000000000D+00 0.000000000D+00
0.000000000D+00 yMon= 0.000000000D+00 1.000000000D+00 0.000000000D+00 zMon= 0.000000000D+00
0.000000000D+00 1.000000000D+00
LensArrayType= 1 LensArrayWidth= 1.000000000D-01
nObs= 0
ApType= None
zElt= 1.000000000D+22
PropType= Geometric nECoord= -6
```

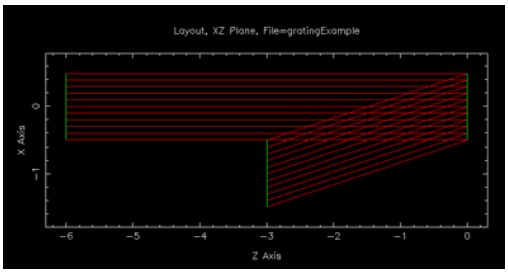
4.4.5 Reflective Gratings

Diffraction gratings are surfaces that have been ruled into a highly regular array of grooves. The rulings scatter light in all directions. The periodicity of the ruling causes there to be coherence in the light scattered in particular discrete directions. These directions correspond to various “diffraction orders,” which are functions of the ruling direction, the ruling width, and the wavelength of the incident beam (ref. Hecht). The amount of power scattered into each order is a more subtle matter, depending on the the detailed shape of the rulings. MACOS provides grating elements that model the beam diffracted into any specified order, but does not predict the intensity of the diffracted beams.

MACOS gratings can be defined on any surface shape. Enter surface parameters to define the underlying element, just as you would to define a regular mirror. Three additional element-specific parameters are required to define the grating geometry. These are:

- OrderHOE: The diffraction order you wish to trace off of the grating surface.
- h1HOE: A unit 3-vector perpendicular to the ruling direction and the psiElt vector. This vector is rotated and translated appropriately if the element is perturbed using the MACOS Perturb command.
- RulingWidth: The fixed distance between rules as projected to a flat plane underlying the surface. Distance between rules *along the curved surface* can vary if the surface shape is curved, which will be the case with a conic or aspheric surface type.

This is the convention of a straight-ruled grating: equidistant parallel groove planes cutting the surface, so on a curved grating the period measured along the surface grows where the surface tilts (the chord model, as in CODE V and Zemax); a flat grating is unaffected. Two consequences for analysis. The optical path difference across a grating is defined only modulo the wavelength (the engine’s smooth order-m phase equals the physical groove staircase mod λ), so compare OPDs across a grating as wrapped phase or as the complex field, never as unwrapped lengths. And a grating’s aperture and obscurations vignette rays as those of any other element do.



The following example shows the new dialog used to specify a grating on a flat surface. The complete prescription, with a collimated source illuminating the grating at normal incidence, is provided in Appendix A.2. Figure 19 shows the diffracted beam at a couple of orders.

Order=1 Order=-2

FIGURE 19 Diffraction grating showing diffraction orders 1 and -2.

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: Grating
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Grating
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Flat
Enter element vertex location (x,y,z) [no default]: 0,0,0
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0
Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1 Do you want an aperture on this element?
[NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [1.000000E+22]:1d22
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]: no
Enter element index of refraction [1.00000]:1
Enter element extinction coefficient [1.000000E+22]:1d22
Enter diffraction order [1.00000]:1 OrderHOE Enter spacing of grating ruling [1.00000]:18.97366596d-7
RuleWidth Enter vector perp to grating ruling (x,y,z) [no default]: 0,1,0 h1HOE
```

This dialog produced the following prescription data:

```
iElt= 1
EltName= Grating Element= Grating Surface= Flat
KrElt= -1.000000000D+22 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00 IndRef= 1.000000000D+00
Extinc= 1.000000000D+22
h1HOE= 0.000000000D+00 1.000000000D+00 0.000000000D+00 OrderHOE= 1.000000000D+00
RuleWidth= 1.897366596D-06
nObs= 0
ApType= None
zElt= 1.000000000D+22
PropType= Geometric nECoord= -6
```

4.4.6 Transmissive Gratings

The transmissive grating elements, like the reflective gratings of the previous section, use surfaces that have been ruled into a highly regular array of grooves to diffract light. The periodicity of the ruling causes there to be coherence in the light scattered in particular discrete directions. These directions correspond to various “diffraction orders,” which are functions of the ruling direction, the ruling width, and the wavelength of the incident beam (ref. Hecht).

MACOS grating elements model the beam diffracted into any specified order, but do not predict the intensity of the diffracted beams.

MACOS gratings can be defined on any surface shape. Enter surface parameters to define the underlying element, just as you would to define a regular mirror. Three additional element-specific parameters are required to define the grating geometry. These are:

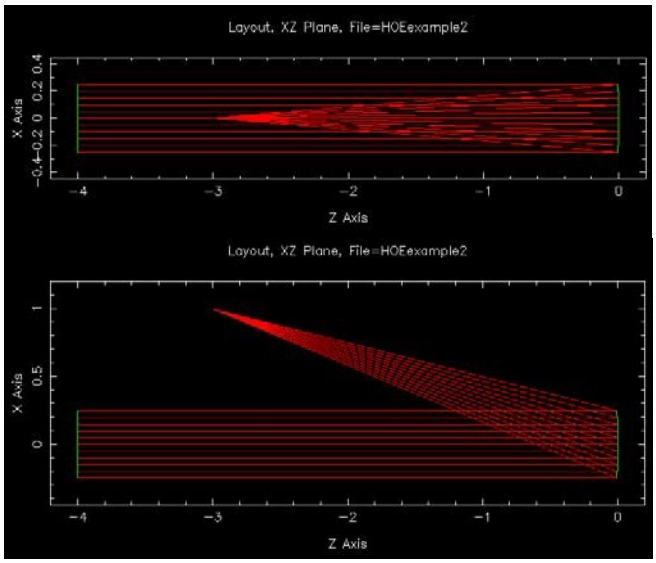
- OrderHOE: The diffraction order you wish to trace off of the grating surface.
- h1HOE: A unit 3-vector perpendicular to the ruling direction and the psiElt vector. This vector is rotated and translated appropriately if the element is perturbed using the MACOS Perturb command.
- RulingWidth: The fixed distance between rules as projected to a flat plane underlying the surface. Distance between rules *along the curved surface* can vary if the surface shape is curved, which will be the case with a conic or aspheric surface type.

4.4.7 Reflective Holographic Optical Elements (HOEs)

Holgraphic optical elements (HOEs) are diffractive elements that can be used to produce a more complex output beam than is possible with straight-ruled gratings. They are typically formed by interfering two coherent beams on the surface of the underlying optic. The surface is covered with photo-sensitive material, so the interference is recorded as areas of exposed and unexposed fringes. The piece is then etched to turn the interference photograph into permanent features on the surface of the optic. Subsequent illumination by one of the 2 reference beams produces a diffracted beam in the form and direction of the second beam. The result is a diffraction grating. MACOS currently supports reflective HOEs only.

MACOS HOEs can be defined on any surface shape. Enter surface parameters to define the underlying element, just as you would to define the shape of a regular mirror. The HOE grating is then defined using additional element-specific parameters. These parameters set up the original HOE interference geometry, so the grating on any point of the surface can be computed from the intended reference beams. The reference beams are rotated and translated appropriately if the element is perturbed using the MACOS Perturb command. The parameters are:

- h1HOE: The first is the input reference beam point source location in x,y,z. A collimated reference beam is located at infinity (1d22). Note that the reference beams have spherical (or flat) wavefronts only.
- h2HOE: The second is the ouput reference beam point source location in x,y,z.
- OrderHOE: The third is the diffraction order to be used to define the grating and to be ray-traced through the rest of the optical system. MACOS currently does not allow you to trace a different order than was used to define the grating.
- WaveHOE: The last is the construction or reference wavelength. The beam is traced using the source wavelength (Wavelen) which can be different.



The following example shows the new dialog used to specify a HOE on a parabolic surface. The complete prescription, with a collimated source illuminating the HOE at normal incidence, is provided in Appendix A.2. Figure 20 shows the output beam for both the diffracted beam and the reflected beam. Both will be present in an actual system, illustrating one use of the HOE: a means for splitting light off the same surface.

Diffracted beam Reflected beam

FIGURE 20 Holographic optical element (HOE), showing diffracted and reflected beams.

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: HOEonMirror
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :HOE
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic
Enter element vertex location (x,y,z) [no default]: 0,0,0
Enter element rotation point (x,y,z) [0.,0.,0.]:
Enter element principal axis (x,y,z) [0.,0.,-1.00000]: Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:3 Enter element eccentricity [0.]:1
Do you want an aperture on this element? [NO]: Enter number of obscurations [0]:
Enter Fresnel propagation distance [3.00000]:1d22
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]: no
Enter element index of refraction [1.00000]:1
Enter element extinction coefficient [1.000000E+22]:1d22

Enter input reference point (x,y,z) [no default]: 0,0,-1d22 h1HOE
Enter output reference point (x,y,z) [no default]: 1,0,-3 h2HOE
```

```
Enter diffraction order [1.00000]: 1 OrderHOE
Enter reference wavelength [1.000000E-07]:6e-7 WaveHOE
```

This dialog produced the following prescription data:

```
    iElt=  1
 EltName=  HOEonMirror
 Element=  HOE
 Surface=  Conic
   KrElt=  -6.000000000D+00
   KcElt=  -1.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
   h1HOE=  0.000000000D+00  0.000000000D+00 -1.000000000D+22
   h2HOE=  1.000000000D+00  0.000000000D+00 -3.000000000D+00
OrderHOE=  1.000000000D+00
 WaveHOE=  6.000000000D-07
    nObs=  0
  ApType=  None
    zElt=  1.000000000D+22
PropType=  Geometric
 nECoord=  -6
```

4.4.8 Reference Surfaces or Dummy Elements

Reference surfaces are “dummy” elements, with no direct effect on the light. They are used in MACOS to define the start and stop points of diffraction propagations. They are also used to provide a screen on which to project the beam for viewing using ray-trace or diffraction commands. They have many other uses as well.

A Reference may be given `Surface= Zernike` or `Surface= Aspheric` so that it carries a basis definition (modes, coefficients, aperture) for a program to read; the surface terms are stored but never applied to the rays. The one active exception is a Reference with grid data, which is a deliberate phase screen (Section 6). Obscurations on a Reference clip rays but do not touch the diffraction-grid field; a mask for a diffraction propagation must be an `Element= Obscuring` (Section 6.2).

As discussed in Section , reference surfaces can be aligned with the nominally spherical (or flat) wavefront of a particular beam. When done correctly, the reference surface then defines the nominal reference wavefront, or the wavefront the beam would have if it were perfect. The OPD of the actual beam wavefront on the surface (Section 5.2) then represents the deviation from perfection of the beam wavefront. These differences drive the MACOS diffraction propagators. This approach greatly reduces the sampling density required to avoid undersampling a wavefront for diffraction (ref. Siegman and Szicklas).

MACOS provides functions that help assure that reference surfaces used for diffraction calculations are correctly set up. These are:

- ORS: Optimizes a reference surface to provide best sampling of a beam (Section 6.3.1).
- SRS: Slaves one reference surface to another to properly set up a near-field diffraction propagation (Section 6.3.2).

Figure 21 illustrates a typical scenario, with a reference surface inserted in the focused beam of a lens. The reference surface has its center located at the focus of the lens, and so is aligned with the ideal (spherical) wavefront. The Cassegrain example (see Appendix A.1) provides another example; indeed all of the examples in the Appendix use reference surfaces.

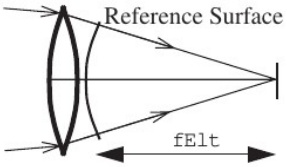


FIGURE 21 Reference surface correctly aligned with wavefront.

```
Enter element name (no blanks) [Elt]: ref_surf EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Reference Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:5.4 fElt
Enter element eccentricity [0.]:0 eElt
Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [5.40000]:5.4 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]: no
```

This dialog produced the following prescription data:

```
iElt= 3
EltName= ref_surf Element= Reference Surface= Conic
KrElt= -5.400000000D+00 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00 IndRef= 1.000000000D+00
Extinc= 0.000000000D+00
nObs= 0
ApType= None
zElt= 5.400000000D+00
PropType= Geometric
nECoord= -6
```

4.4.9 Focal Planes

The focal plane is a reference element, but one that always has a flat surface. The only difference between MACOS FocalPlane and Reference elements is in the way that linear models are computed for them. Use focal planes to end a prescription that is to be used for linear models! As a a reference element, a focal plane element has no direct optical effect. Focal planes are commonly used to terminate far-field diffraction propagations.

```
Enter element name (no blanks) [Elt]: focal_plane EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :FocalPlane Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,2 VptElt
Enter element rotation point (x,y,z) [0.,0.,2.00000]:0,0,2 RptElt Enter element principal axis
(x,y,z) [0.,0.,-1.00000]:0,0,1 psiElt Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
```

This dialog produced the following prescription data:

```
iElt= 5
EltName= focal_plane Element= FocalPlane Surface= Flat
KrElt= -1.000000000D+22 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
2.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 2.000000000D+00 IndRef= 1.000000000D+00
Extinc= 0.000000000D+00
nObs= 0
ApType= None
zElt= 1.000000000D+22
PropType= Geometric nECoord= -6
```

4.4.10 Return Surfaces

The return surface is another element which is used for calculations. It has no effect on wavefront propagation. Specifically, two return surfaces in sequence are used to find the position of the exit pupil of the system in the FEX command. This discussed more thoroughly in Section 5.3.

```
Enter element name (no blanks) [Elt]: return EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Return Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,1 VptElt
Enter element rotation point (x,y,z) [0.,0.,1.00000]:0,0,1 RptElt Enter element principal axis
(x,y,z) [0.,0.,-1.00000]:0,0,1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:5 fElt
Enter element eccentricity [0.]:0 eElt
Do you want an aperture on this element? [NO]: no
Enter number of obscurations [0]:0
```

```
Enter Fresnel propagation distance [5.00000]:5 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]: no
```

This dialog produced the following prescription data:

```
iElt= 1
EltName= return Element= Return Surface= Conic
KrElt= -5.000000000D+00 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
1.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 1.000000000D+00 IndRef= 1.000000000D+00
Extinc= 0.000000000D+00
nObs= 0
ApType= None
zElt= 5.000000000D+00
PropType= Geometric nECoord= -6
```

4.4.11 Polarizers and Waveplates

Two element types apply a polarization transformation to the beam without bending it: `TrPolarizer`, an ideal linear polarizer, and `WavePlate`, a linear retarder. Both are transmissive and both are active only when polarization is enabled (the `POLARIZATION` command, or `macos.polarization` / `pymacos` equivalents). With polarization off they behave exactly as `Reference` surfaces, so adding one to a prescription cannot change a scalar result.

Geometrically they are `Reference` surfaces: the surface may be `Flat` or `Conic`, the ray is intersected and its path length accumulated in the usual way, and the ray direction is unchanged. What they add is a 2×2 Jones matrix applied to the ray's electric field in the plane transverse to the ray.

Two new keywords describe them:

Keyword	Applies to	Meaning
`PolAxis=`	`TrPolarizer`, `WavePlate`	Reference axis as a 3-vector in global coordinates: the TRANSMISSION axis of a polarizer, the FAST axis of a waveplate. Required.
`Retardance=`	`WavePlate`	Retardance in waves at the prescription `Wavelen`. Required for a waveplate; 0.25 is a quarter-wave plate, 0.5 a half-wave plate.

`PolAxis=` is a direction, not a position, and it does not have to lie in the element's surface — MACOS derives the element's material axis from it (see below) and projects that into each ray's transverse plane, normalizing as it goes. The projection must not degenerate: an axis that leaves the element with no transverse direction extinguishes the ray rather than having MACOS choose an arbitrary basis. Both keywords are required on the element types that use them: a defaulted axis would orient a polarizer arbitrarily, and a defaulted retardance would make a waveplate a silent no-op.

`Retardance=` follows the same convention as `Coating=` layer thickness. The prescription value is in waves at the `Wavelen` current when the file is parsed, and MACOS converts it to a physical retardance ($n$slow − $n$fast)·d at load. The trace then divides by the wavelength in force at the time. A plate is therefore a fixed piece of glass and a wavelength sweep is chromatic, as a real plate is: a quarter-wave plate at 1 µm is an eighth-wave plate at 2 µm.

The retardance sign follows MACOS's propagation convention (`exp(+iωt)` time dependence, so a field accumulates `exp(−i·2π·L·N/λ)` through a medium). The slow axis therefore takes the more negative phase, and the element's Jones matrix in its own (fast, slow) eigenbasis is diag(1, exp(−iδ)) with δ = 2π·`Retardance`. Fast axis leads.

Both elements are thin idealizations. There is no ray splitting, no walk-off, no Fresnel loss at the plate faces and no substrate thickness, and the output field is purely transverse. In particular a polarizer discards the rejected component rather than emitting it, so a polarizing beamsplitter needs two traces — one per output port. For a beamsplitter at a substantial angle of incidence, a coated `Reflector` is usually the better model: there the s and p directions are set by the physical plane of incidence, and the thin-film recursion gives real diattenuation and retardance rather than an idealization.

A waveplate's retardance is likewise independent of the angle of incidence, where a real crystal plate's is not — the field-of-view effect that drives compound and Pancharatnam designs. Bounding that needs a birefringent-plate model with ordinary and extraordinary indices and a thickness, which MACOS does not provide.

Off normal incidence, what MACOS projects into the transverse plane is the element's *material* axis — the direction fixed in the substance of the element. This matters because projection does not preserve orthogonality, so projecting the declared pass axis is not the same operation as projecting the block axis and transmitting the complement; the two differ by acos(2 cos a/(1 + cos² a)) of transmitted-axis orientation, which is 3.56° at 20° of incidence and identically zero at normal incidence. For a waveplate the material axis is the declared (fast) axis. For a polarizer it is the *absorbing* direction — the wires of a wire grid, the aligned dipole chains of a dichroic sheet — which is the in-element complement of the declared pass axis; MACOS forms it, projects it, extinguishes it, and transmits its orthogonal partner. The keyword still declares the pass axis, so only its component perpendicular to the element normal has any effect, and a pass axis parallel to the normal extinguishes the ray. This is the dipole model, which Korger et al. (*Opt. Express* **21**, 27032, 2013) measured on a tilted dichroic polarizer.

The reflective polarizer type `RfPolarizer` is reserved but not implemented: a reflective wire grid carries grid reflection efficiency and the substrate's own s/p response beyond the axis rule.

```
iElt= 3
EltName= QuarterWavePlate
Element= WavePlate
Surface= Flat
KrElt= -1.000000000D+20 KcElt= 0.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00
VptElt= 0.000000000D+00 0.000000000D+00 2.000000000D-01
RptElt= 0.000000000D+00 0.000000000D+00 2.000000000D-01
PolAxis= 1.000000000D+00 1.000000000D+00 0.000000000D+00
Retardance= 2.500000000D-01
IndRef= 1.000000000D+00
Extinc= 0.000000000D+00
nCoat= 0
nObs= 0
ApType= None
PropType= Geometric
zElt= 2.000000000D-01
nECoord= -6
```

4.4.12 Hex and Pie Segmented Mirrors

Segmented systems (e.g. telescopes with segmented primary mirrors) are defined at the light source using GridType=hex or GridType=pie. This section describes entering the segmented surfaces. Figures 22 and 23 show the geometry for a segmented pie-shaped aperture and a large array of hexagonal segments, respectively. In both cases, the pannel coordinates are given by three variables (X,L,R). The segments must be entered in as elements in the same order they are listed in SegCoord. Segmented surfaces also require

- nSeg: number of segments
- gap: spacing between segments
- width: width of segments (flat to flat)

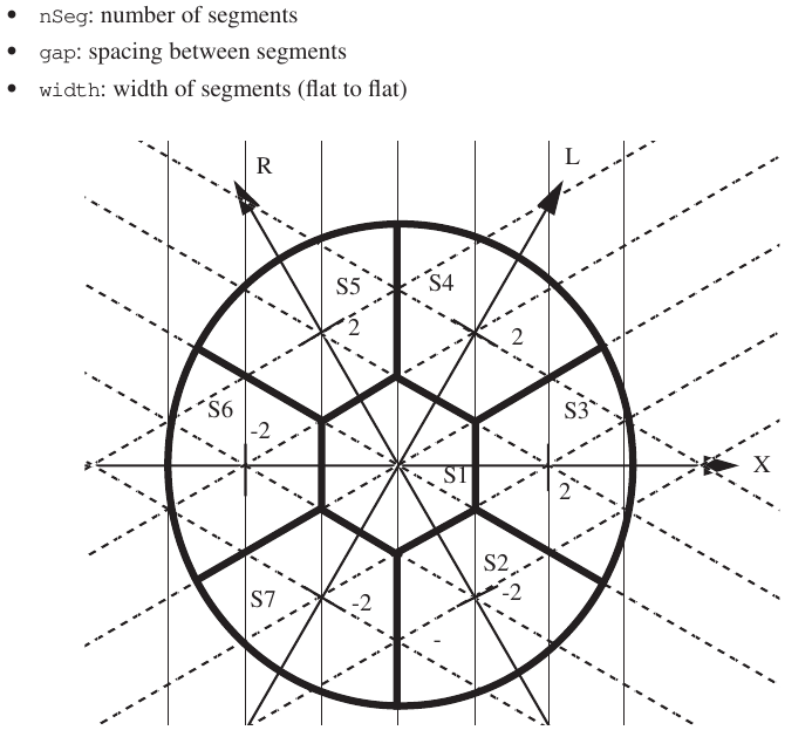


FIGURE 22 Hexagonal coordinates X,L,R for a pie-shaped segmented mirror.

More than 1 segmented mirror can be used in a single prescription. In the case of 2 segmented mirrors, the segments are required to be lined up, so that the segmentation defined at the source is valid for both beams.

An example of a segment mirror .in-file (seg.in) is in Appendix A.5. The panels are numbered in the same order as they are in the Figure 23.

Input data:

```
+ +
| Source ray grid types: |
| Circular Square Hex |
| Pie Flower |
+ +
Enter source grid type [Circular]: Pie GridType
```

```
Enter aperture diameter or angle [1.00000]:3.65 Aperture
Enter number of grid points [128]:31 nGridPts
```

| | | |

			67	66	65	64	63	62	
3	2	1	0	-1					
74	48	28	14	15	16	33	56	85	
-8, -7, 1	-6, -7, 0	-4, -5, -1	-2, -3, -2	0, -3, -3	2, -2, -4	4, -1, -5	6, 0, -6	8, 1, -7	
75	49	29	30	31	32	55	84		
-7, -8, -1	-5, -7, -2	-3, -6, -3	-1, -5, -4	1, -4, -5	3, -3, -6	5, -2, -7	7, -1, -8		
76	50	51	52	53	54	83			
-6, -9, -3	-4, -8, -4	-2, -7, -5	0, -6, -6	2, -5, -7	4, -4, -8	6, -3, -9			
77	78	79	80	81	82				
-5, -10 ,-5	-3, -9, -6	-1, -8, -7	1, -7, -8	3, -6, -9	5, -5, -10				

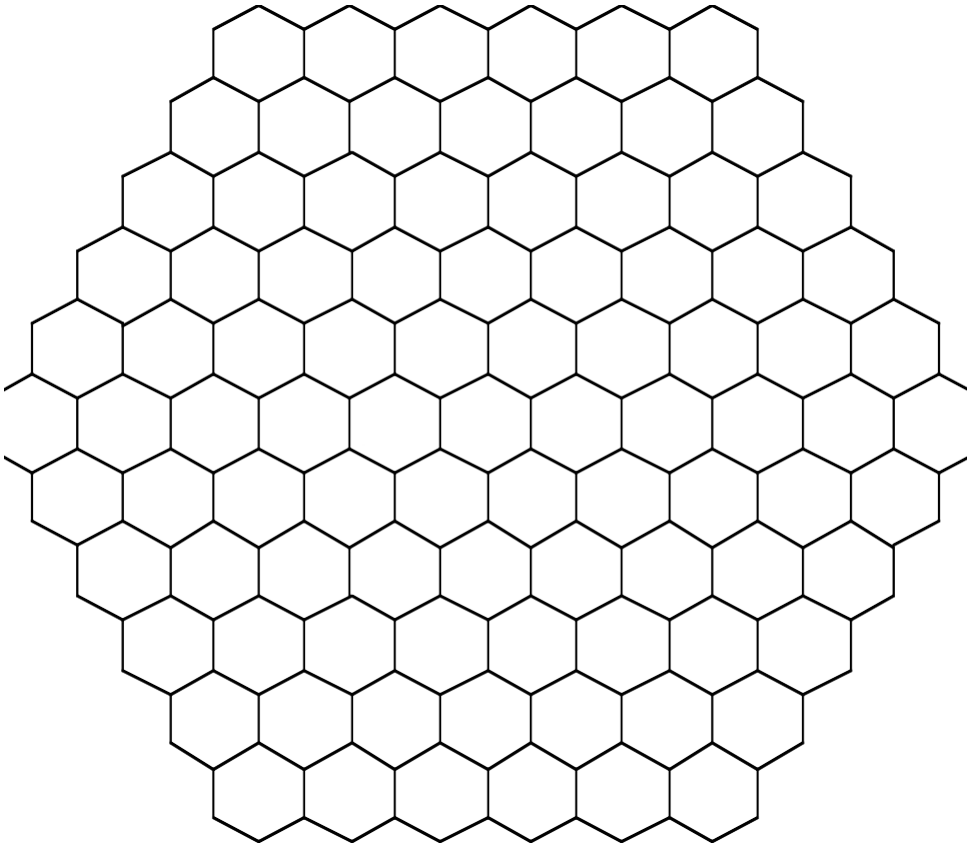


FIGURE 23

Hexagonal coordinates X,L,R for a large array

```
Enter input plane x-axis vector (x,y,z) [1.00000,0.,0.]:1,0,0 xGrid Enter input plane y-axis vector
(x,y,z) [0.,1.00000,0.]:0,1,0 yGrid Enter number of segments [no default]: 7 nSeg
Enter segment width: [3.65000]:1.2 width
Enter gap between segments: [0.]:0 gap
Enter 3 segment 1 hex coords [no default]: 0,0,0 SegCoord
Enter 3 segment 2 hex coords [no default]: 1,-1,-2
Enter 3 segment 3 hex coords [no default]: 2,1,-1
Enter 3 segment 4 hex coords [no default]: 1,2,1
Enter 3 segment 5 hex coords [no default]: -1,1,2
Enter 3 segment 6 hex coords [no default]: -2,-1,1
Enter 3 segment 7 hex coords [no default]: -1,-2,-1
Enter segment plane x-axis vector (x,y,z) [1.00000,0.,0.]:1,0,0 SegXgrid
```

Element data:

```
Enter Element 1 Data:
Enter element name (no blanks): (Elt): s1 EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Segment Element
+ +
| Surface types: |
```

	Flat	Conic	Aspheric
	Anamorphic	Zernike	Monomial

Flat	Conic	Aspheric
Interpolated	UserDefined	

```
+ +
Enter surface type :Conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,-1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:1.46 fElt
Enter element eccentricity [0.]:1 eElt
Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.46000]:1.46 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:

    iElt=  1
 EltName=  s1
 Element=  Segment
 Surface=  Conic
    fElt=  1.460000000D+00
    eElt=  1.000000000D+00
   KrElt=  -2.920000000D+00
   KcElt=  -1.000000000D+00
  psiElt=  0.000000000D+00 0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00 0.000000000D+00 0.000000000D+00
  RptElt=  0.000000000D+00 0.000000000D+00 0.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
    nObs=  0
  ApType=  None
    zElt=  1.460000000D+00
PropType=  Geometric
 nECoord=  -6

Is this correct? [YES]:
Enter Element 2 Data:
Enter element name (no blanks) [Elt]: s2 EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Segment Element
+ +
| Surface types: |
| Flat Conic Aspheric |
```

An amorphic	Zernike	Mon omial
Inte rpolated	User Defined	

```
+ +
Enter surface type :Conic Surface
Enter element rotation point (x,y,z) [0.,0.,0.]:.6124997330,-1.060881274,
.2569563356
Do you want an aperture on this element? [NO]:
```

```
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.46000]:1.46 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:
iElt= 2
EltName= s2 Element= Segment Surface= Conic
fElt= 1.460000000D+00 eElt= 1.000000000D+00 KrElt= -2.920000000D+00 KcElt= -1.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= 6.124997330D-01 -1.060881274D+00 2.569563356D-01 IndRef= 1.000000000D+00
Extinc= 1.000000000D+22
nObs= 0
ApType= None
zElt= 1.460000000D+00
PropType= Geometric nECoord= -6
Is this correct? [YES]:
...
Enter Element 7 Data:
Enter element name (no blanks) [Elt]: s7 EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Segment Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic Surface
Enter element rotation point (x,y,z) [0.,0.,0.]:-.612500534,-1.060880811,
.2569563356
Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.46000]:1.46 zElt

+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:
iElt= 7
EltName= s7 Element= Segment Surface= Conic
fElt= 1.460000000D+00 eElt= 1.000000000D+00 KrElt= -2.920000000D+00 KcElt= -1.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= -6.125005340D-01 -1.060880811D+00 2.569563356D-01 IndRef= 1.000000000D+00
Extinc= 1.000000000D+22
nObs= 0
ApType= None
zElt= 1.460000000D+00
PropType= Geometric nECoord= -6
Is this correct? [YES]:
```

4.4.13 Generating segmented prescriptions: SegMirMaker

Hand-editing a .in file to define every segment of a 7- or 19-segment mirror is tedious. MACOS v4.00 ships with a separate tool, SegMirMaker, in MACOS_resources/segmirmaker/, that generates a .presc (suitable for inclusion in a .in file) and an Hx.m edge-sensor measurement matrix from a parent prescription.

SegMirMaker accepts:

A MACOS .in parent prescription. The parent element can be any conic or FreeForm surface.

The number of rings (nRing = 1 for 7-segment, 2 for 19-segment, etc.).

The axis orientation psi, segment size or aperture diameter, inter-segment gap, and segment standoff.

3-DOF (piston/tip/tilt) or 6-DOF segments.

For FreeForm parents, SegMirMaker copies the parent's lFF, FFCoef, FF coordinate frame, grid data, and grid frame verbatim into every segment, so all segments share the parent's underlying FreeForm shape. Each segment's Mon slot is left empty (MonZernCoef = 0) but with lMon = L2 (half the segment width); the user can later add per-segment figure error via the Mon Zernike coefficients without re-running SegMirMaker.

SegMirMaker is the modernized successor to the SMPGe ('Segmented Mirror Prescription Generator') VAX Fortran tool from 1992; the original is preserved in MACOS_resources/segmirmaker/Archive/SMPGe.for. See MACOS_resources/segmirmaker/README.md for full usage.

4.4.14 Flower Segmented Mirrors

Flower segmented mirrors consist of a central segment surrounded by "petal" segments. As such, they are like the "pie" segmented mirrors of the previous section, except flower segmentation is not limited to a hexagonal geometry. Flower segmentation is defined at the light source using GridType=Flower. Two other parameters are used to define the segmentation:

- nPetals: number of petals surrounding the central segment
- radCtr: radius of the largest inscribed circle defining the size of the center segment

Only 1 segment coordinate is needed per segment, to specify its location in sequence around the central segment, with the 0th segment being the center segment, the 1st segment having one edge along the SegXgrid axis and the other along a line rotated 360°/

nPetals towards the SegYgrid axis, and the 2nd and subsequent segments located in the same sequence around the center segment. See Fig. 24 for an illustration.

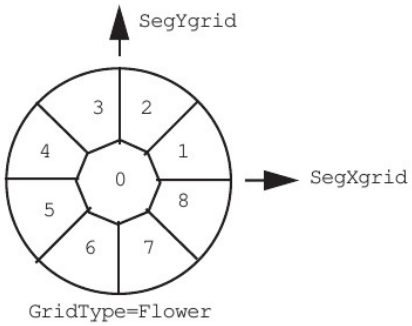


FIGURE 24 Flower segmentation, showing segment coordinates

4.4.15 Reflective Non-Sequential Surfaces

In most optical systems, all rays strike one element before passing on to the next element. There are exceptions, such as corner cube reflectors, where some rays will strike an element before others. These are modeled in MACOS using non-sequential surfaces. There is no limit on the number of non-sequential elements and the order that the non-sequential elements are entered into MACOS does not matter. However, there must be sequential surfaces before and after the group of non-sequential surfaces. For the purpose of the following discussion, we call them the first and second surfaces, respectively. After the ray passes the first sequential surface, MACOS solves for the next surface that the ray strikes which can be any of the group of non-sequential surfaces or the second sequential surface. If the ray hits a non-sequential surface, the procedure continues until the second sequential surface is encountered. After the second sequential surface is encountered the ray trace proceeds normally. Figure 25 shows the corner cube reflector example. The .in-file is in Appendix A.3

NOTE: Geometric propagation only is supported through non-squential elements.

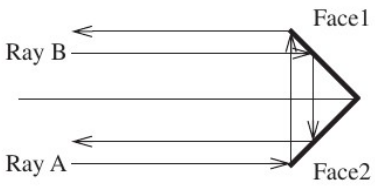


FIGURE 25 Non-sequential surface: Ray A has Face2 as first element, Ray B's first element is Face1.

Corner cube example

```
Enter element name (no blanks) [Elt]: Face1 EltName
+ +
```

Element types:		
Reflector	NSRe flector	Segment
Refractor	NSRe fractor	LensArray
FocalPlane	Re ference	Return

```
| HOE Grating Obscuring |
+ +
Enter element type :NSReflector Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:-.7071,0,-.7071 psiElt Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:1d0 Enter element extinction coefficient [1.000000E+22]:
Enter element name (no blanks) [Elt]: Face2 EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :NSReflector Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:.7071,0,-.7071 psiElt Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
```

Geometric	GeomUpdate	FarField
NFSpherical	NFS1surf	
NFPlane	NFP1surf	

	NF1	NF2	
	Spati alFilter	SF1surf	

```
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:1d0 Enter element extinction coefficient [1.000000E+22]:
```

4.4.16 Refractive Non-Sequential Surfaces

Refractive non-sequential surfaces are very similiar to reflective non-sequential surface (see Section 4.4.15). An example illustrating their application is a Luneberg Lens Antenna is provided in Appendix A.3. The Luneberg Lens is a stepped-gradient index lens consisting of concentric spherical shells, which grow denser as they decrease in radius. A point source on the surface produces rays that follow nearly elliptical trajectories, emerging on the other side of the lens in a collimated beam. Using multiple point sources distributed on the surface, the Luneberg Lens can provide instantaneous, solid-state beam coverage over nearly 4π steradians.

4.5 Element Surface Types

4.5.1 Flat Surfaces

Flat surfaces are completely defined by a point (the "vertex" VptElt) on the surface, and the unit vector normal to the surface at that point (psiElt). Technically, the *vertex* of a plane or sphere is undefined; the user is free to pick any point on the surface as the vertex. In MACOS, a flat surface is a spherical surface with an infinite radius of curvature, KrElt=1d22. See Section 4.4.1 for an example of entering flat surface data.

4.5.2 Conic Surfaces

Conic surfaces – really conicoids of revolution – include the most common optical shapes: spheres, ellipsoids, paraboloids, hyperboloids. Flat surfaces are spheroids with an infinite radius of curvature. MACOS specifies the shape of a conic surface using two surface-specific parameters: the conic constant KcElt and the "radius," KrElt. Table 7 lists the common names associated with special values of the conic constant.

A hyperboloid or an ellipsoid has two sheets. A ray is intersected with the sheet the vertex lies on (the two are told apart by the sign of the surface normal's axial component), and the far sheet is never taken even when a ray crosses it first; on a Return or Reference the root nearest the element's reference point RptElt is taken, which lets the exit-pupil sphere of FEX be traced back from the focus. An intersection choice that differs from the pre-2026 rule is counted and reported once per run in the WARN summary; such a count on an old prescription means it was tracing onto the wrong sheet.

TABLE 7. Conic coefficients

Conic coefficient	Conic section
KcElt<-1	hyperboloid
KcElt=-1	paraboloid
-1<KcElt<0	ellipsoid with major axis on the principal axis (also known as prolate spheroid), psiElt
KcElt=0	sphere
KcElt>0	oblate ellipsoid

MACOS also accepts input of conic parameters in terms of the geometric focal length (which is *not* the same as optical focal length of a spherical mirror) and eccentricity which are denoted fElt and eElt, respectively. Eccentricity and focal length are related to the conic constant and radius by:

$$eElt = \sqrt{-KcElt} \quad (4.1)$$

$$f\,Elt = \frac{-KrElt}{1+\sqrt{-KcElt}} \quad (4.2)$$

The “sag equation” for a conic surface, which is to say, the equation which defines all points on the surface, is:

$$Z(h) = \frac{h^2}{KrElt + \sqrt{(KrElt)^2 - (1+KcElt)h^2}} \quad (4.3)$$

Here h is the radial distance to the projection of the point of incidence onto a plane perpendicular to the principal axis and including the surface vertex; and Z is the height of the surface above that plane (Figure 26).

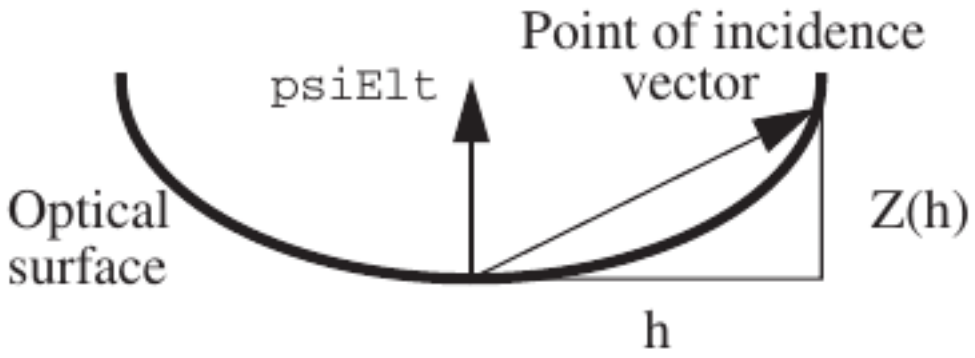


FIGURE 26 Conic mirror parameters

Note that the *vertex* of a conic refers to the geometric vertex except in the case of an oblate ellipsoid. For an oblate ellipsoid (KcElt>0), the “vertex” is taken as the point where the minor axis intersects the surface.

See Section 4.4.2 for an example of entering conic surface data.

4.5.3 Generalized (10th-Order) Aspheric Surfaces

Generalized aspheric surfaces use a conic surface as a base, and then impose a deviation on it, defined by additional, higher order, symmetric terms in radius h. In MACOS, generalized aspheric surfaces take the form

$$Z(h) = \frac{h^2}{KrElt + \sqrt{(KrElt)^2 - (1+KcElt)h^2}} + Ah^4 + Bh^6 + Ch^8 + Dh^{10} \quad (4.1)$$

where h is the radial pupil coordinate (the magnitude projection of the of the point of incidence vector onto the plane perpendicular to psiElt as shown in Figure 29), KrElt is the radius of curvature of the element, KcElt is the conic constant, and A, B, C, D are the aspheric coefficients, AsphCoef(i), with AsphCoef(1) = A, AsphCoef(2) = B, AsphCoef(3) = C, and AsphCoef(4) = D.

Note: The A,B,C,D parameters are used by optical design codes as well, except that the signs of the coefficients may differ in some cases. The sign of the coefficients in some codes is a function of the direction of the incident ray. MACOS references surfaces to a global coordinate system and does not

assume a particular direction of the beam in defining the optical train. In MACOS the coefficients are positive in the direction of the element psiElt vector.

This example shows how to enter a higher order aspheric reflector.

```
Enter element name (no blanks) [Elt]: gen_asphere EltType
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Reflector Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Aspheric Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt
Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,1 psiElt
Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:2
Enter Conic Constant [0.]:-1 KcElt
Enter Radius [-1.000000E+22]:5 KrElt
Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [2.50000]:5 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:
Enter aspheric coefficients A,B,C,D [0,0,0,0]:1e-9,2e-19,3e-17,4e-20 AsphCoef
```

This dialog produced the following prescription data:

```
    iElt=  1
 EltName=  gen_asphere
 Element=  Reflector
 Surface=  Aspheric
   KrElt=  -5.000000000D+00
   KcElt=  -1.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00  1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00  0.000000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00  0.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
AsphCoef=  1.000000000D-09  2.000000000D-19  3.000000000D-17  4.00000000D-20
    nObs=  0
  ApType=  None
    zElt=  5.000000000D+00
PropType=  Geometric
 nECoord=  -6
```

An `AsphCoef=` (or `AnaCoef=`) line that carries fewer values than the surface needs is accepted: the missing higher-order coefficients are zero, and one line naming the element and the count is printed at load.

4.5.4 Zernike Surfaces

Zernike surfaces add a deformation—described by a Zernike polynomial—to the surface of a conicoid. The deformation is entered as the “excess sag” of the surface (*not* the wavefront error). MACOS allows

up to the first 45 Zernike terms to be added to a conicoid base surface shape. The Zernike polynomial set that is used is that defined by Malacara in his book, “Optical Shop Testing.” The ordering of the polynomials used in MACOS, as well as two other popular orderings, are shown in Table 8.

The ordering and normalization are selected per element with the `ZernType=` keyword: `ANSI` (the MACOS/Malacara column of Table 8; the legacy name `Malacara` is accepted), `BornWolf`, `Fringe`, `Noll`, `ExtFringe`, and the RMS-normalized `NormANSI`, `NormBornWolf`, `NormFringe`, `NormNoll`, `NormHex`, `NormAnnularNoll` (which takes the annular ratio as a trailing value). **If no `ZernType=` is given, the ANSI ordering of Table 8 is assumed** — for every surface that carries Zernike terms (`Zernike`, `ZrnGrData`), whatever the element type. **A prescription converted from Code V must declare `ZernType= Fringe`** (or `Noll`) explicitly; the converter’s ordering is not inferred. A `nZernCoef=` line on a surface that carries no Zernike terms is refused at load, naming the element. Fewer than `nZernCoef` coefficients may be entered; the rest are zero. The same keyword family applies to the Mon and FF channels of FreeForm surfaces (`MonZernType`, `FFZernType`; see “FreeForm Surfaces” below).

TABLE 8. Zernike polynomials

MACOS (M alacara)	**Code V (Noll)**	**F RINGE (Fr inge)**	**Polynomial**
1	1	1	1
2	3	3	R sin(A)
3	2	2	R cos(A)
4	6	6	R^2 sin(2A)
5	5	4	2R2-1
6	4	5	R^2 cos(2A)
7	10	11	R^3 sin(3A)
8	9	8	($3R^3$-2R) sin(A)
9	8	7	($3R^3$-2R) cos(A)
10	7	10	R^3 cos(3A)
11	15	18	R^4sin(4A)
12	14	13	($4R^4$-$3R^2$) sin(2A)
13	13	9	6R4-6R2+1
14	12	12	($4R^4$-$3R^2$) cos(2A)
15	11	17	R^4 cos(4A)
16	21	27	R^5 sin(5A)
17	20	20	($5R^5$-$4R^3$) sin(3A)
18	19	15	(10R5-12R3+3R) sin(A)
19	18	14	(10R5-12R3+3R) cos(A)
20	17	19	($5R^5$-$4R^3$) cos(3A)
21	16	26	R^5 cos(5A)
22	28		R^6 sin(6A)
23	27	29	($6R^6$-$5R^4$) sin(4A)
24	26	22	(15R6-20R4+6R2) sin(2A)

MACOS (M alacara)	Code V (Noll)	F RINGE (Fr inge)	Polynomial
25	25	16	$20R^6-30R^4+12R^2+1$
26	24	21	(15R6-20R4+6R2) cos(2A)
27	23	28	$(6R^6-5R^4)$ cos(4A)
28	22		R^6 cos(6A)
29	36		R^7 sin(7A)
30	35		$(7R^7-6R^5)$ sin(5A)
31	34	31	(21R7-30R5+10R3) sin(3A)
32	33	24	(35R7-60R5+30R3-4R) sin(A)
33	32	23	(35R7-60R5+30R3-4R) cos(A)

TABLE 8. Zernike polynomials

MACOS (M alacara)	Code V (Noll)	F RINGE (Fr inge)	Polynomial
34	31	30	(21R7-30R5+10R3) cos(3A)
35	30		$(7R^7-6R^5)$ cos(5A)
36	29		R^7 cos(7A)
37	45		R^8 sin(8A)
38	44		$(8R^8-7R^6)$ sin(6A)
39	43		(28R8-42R6+15R4) sin(4A)
40	42	33	(56R8-105R6+60R4-10R2) sin(2A)
41	41	25	70R8-140R6+90R4-20R2+1
42	40	32	(56R8-105R6+60R4-10R2) cos(2A)
43	39		(28R8-42R6+15R4) cos(4A)
44	38		$(8R^8-7R^6)$ cos(6A)
45	37		R^8 cos(8A)

KcElt and KrElt (or eElt and fElt) are the parameters that describe the conicoid. The deviations occur only in the z-direction of a special coordinate frame. The variable pMon is the center of the coordinate frame. The variables xMon, yMon, and zMon define the special coordinate frame as shown in Figure 27.The variable lMon is the semi-diameter of the circle to which the Zernike deviations are scaled (remember that Zernikes are defined on a unit circle).

defined on a unit circle).

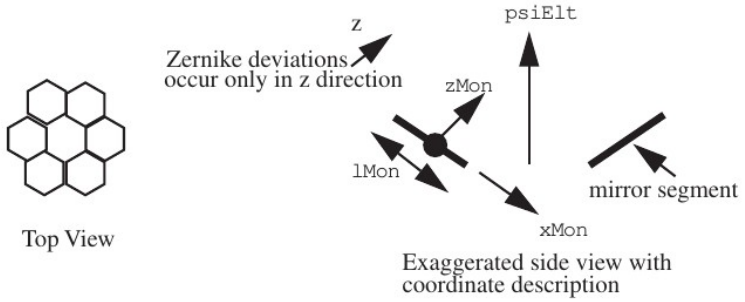


FIGURE 27 Specialized coordinate frame

Zernike polynomials, listed in Table 8, are orthogonal over a unit circle. The polar coordinates (R, A) are related to the coordinates of the projection of the point of incidence vector onto the plane perpendicular defined by xMon and yMon (X, Y) by X=R lMon cos(A) and Y=R lMon sin(A). The more common Seidel aberrations can be calculated from the Zernike coefficients (ref. Malacara).An example using a Zernike surface to simulate deformations on the primary mirror of a small telescope is provided in Appendix A.7. The following dialog illustrates the responses the NEW command expects in entering a Zernike reflector:

```
Enter element name (no blanks) [Elt]: zern_surface EltName
+ +
```

Element types:		
Reflector	NSRe flector	Segment
Refractor	NSRe fractor	LensArray
FocalPlane	Re ference	Return

```
| HOE Grating Obscuring |
+ +
Enter element type :Reflector Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Zernike Surface
Enter element vertex location (x,y,z) [no default]: 0,0,0 VptElt
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0 RptElt Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,-1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:2
Enter Conic Constant [0.]:0 KcElt
Enter Radius [-1.000000E+22]:1d22 KrElt
Do you want an aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
Enter element index of refraction [1.00000]:1
Enter element extinction coefficient [1.000000E+22]:
Enter Zernike coefficients 1-6 [0.,0.,0.,0.,0.,0.]:.0001,0,.00002,0,0, .00001
```

```
ZernCoef
Enter Zernike coefficients 7-12 [0.,0.,0.,0.,0.,0.]:0,0,.000001,.00002,0,0 Zern-Coef
Enter Zernike coefficients 13-18 [0.,0.,0.,0.,0.,0.]:0,.000003,.000001,0,0, .000002
ZernCoef
EnterZernikecoefficients19-24[0.,0.,0.,0.,0.,0.]:.0000005,.0000001,0,0,.000002,.00004
ZernCoef
Enter Zernike coefficients 25-30 [0.,0.,0.,0.,0.,0.]:.0000001,0,0,0, .000001,0
ZernCoef
Enter Zernike coefficients 31-36 [0.,0.,0.,0.,0.,0.]:0,0,0,0,0,0 ZernCoef
Enter Zernike coefficients 37-42 [0.,0.,0.,0.,0.,0.]:0,0,0,0,0,0 ZernCoef
Enter Zernike coefficients 43-45 [0.,0.,0.]:0,0,0 ZernCoef Enter surface reference point location
(x,y,z) [0.,0.,0.]:0,0,0 pMon Enter surface x-axis unit vector (x,y,z) [-1.00000,0.,0.]:1,0,0 xMon
Enter surface y-axis unit vector (x,y,z) [0.,-1.00000,0.]:0,1,0 yMon Enter surface z-axis unit vector
(x,y,z) [0.,0.,-1.00000]:0,0,1 zMon Orthoganalized zMon= 0. 0. 1.0000000000000
Enter scale length [1.00000]:1 lMon
```

This dialog produced the following prescription data: iElt= 1 EltName= zern_surface Element= Reflector Surface= Zernike KrElt= -1.000000000D+22 KcElt= 0.000000000D+00 psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00 IndRef= 1.000000000D+00 Extinc= 1.000000000D+22 ZernCoef= 1.000000000D-04 0.000000000D+00 2.000000000D-05 0.000000000D+00 0.000000000D+00 1.000000000D-05 0.000000000D+00 0.000000000D+00 1.000000000D-06 2.000000000D-05 0.000000000D+00 0.000000000D+00 0.000000000D+00 3.000000000D-06 1.000000000D-06 0.000000000D+00 0.000000000D+00 2.000000000D-06 5.000000000D-07 1.000000000D-07 0.000000000D+00 0.000000000D+00 2.000000000D-06 4.000000000D-05 1.000000000D-07 0.000000000D+00 0.000000000D+00 0.000000000D+00 1.000000000D-06 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 0.000000000D+00 pMon= 0.000000000D+00 0.000000000D+00 0.000000000D+00 xMon= 1.000000000D+00 0.000000000D+00 0.000000000D+00 yMon= 0.000000000D+00 1.000000000D+00 0.000000000D+00 zMon= 0.000000000D+00 0.000000000D+00 1.000000000D+00 lMon= 1.000000000D+00 nObs= 0 ApType= None zElt= 1.000000000D+22 PropType= Geometric nECoord= -6

4.5.5 Sparse Zernike input

In addition to the dense MonCoef form (120 coefficients in Cartesian monomial order), MACOS reads Zernike coefficients in a sparse ‘modes + coefs’ form:

```
MonZernType= BornWolf
nMonZernCoef= 4
MonZernModes= 4 9 16 25
MonZernCoef= -4.819E+00 4.001E+00 1.197E+00 2.269E-03
```

MonZernType selects the Zernike ordering and normalization. Supported types: ANSI, BornWolf, Fringe, NormANSI, NormBornWolf, NormFringe, NormNoll, Noll, NormAnnularNoll, NormHex, ExtFringe. The Norm* variants use RMS normalization; the unprefixed variants are unnormalized.

Conversion to the internal monomial form (MonCoef) happens automatically at trace time. The same scheme applies to FF Zernikes: FFZernType, nFFZernCoef, FFZernModes, FFZernCoef. Modes are 1-indexed; a modes line may be written on one line or wrapped over several.

4.5.6 Monomial Surfaces

MACOS has a second polynomial-based deformed-surface type, evaluated as a sum of Cartesian monomials. The generating function is:

$$Z(x,y)=\sum_{i=0}^{14}\square\sum_{j=0}^{i}c_k\acute{x}^j\acute{y}^{i-j},\acute{x}=\frac{x}{lMon},\acute{y}=\frac{y}{lMon},(4.1)$$

where (x, y) are the coordinates of the point of incidence in the special frame (pMon, xMon, yMon, zMon) of Figure 27, the deviation is along zMon, and the 120 coefficients MonCoef(k) are taken in that order: k = 1 is the constant term, then for each total degree i = 1 … 14 the terms j = 0 … i (so MonCoef(2) multiplies $\bar{y}$, MonCoef(3) multiplies $\bar{x}$, MonCoef(4) $\bar{y}^2$, MonCoef(5) $\bar{x}\bar{y}$, MonCoef(6) $\bar{x}^2$, …). (The printed 3.2 manual left this equation blank; the form is the engine's MonomialEval.)

The user must specify 120 coefficients in the prescription file.

4.5.7 Toric Surfaces

Toric surfaces are surfaces that have different shapes in some specified orthogonal x-and y-axes.Cylindrical and toroidal surfaces are two examples. Toric surfaces are distinguished from the anamorphic surfaces by there being a non-zero conic constant in ony 1 of the axes. MACOS toric surfaces are entered using the standard KrElt and KcElt format, except that two sets of these conic parameters are entered: one set for each of the x-and y-axis directions. The variable AnaCoef holds the values in the following order: Krx, Kry, Kcx. In addition, a specialized coordinate frame must be used to define the local x- and y-axes for these variables. This is the same specialized coordinate frame used in Section 4.5.4 and is defined by pMon, xMon, yMon, and zMon as shown in Figure 27. An example of an (cylindrical) anamorphic surface as defined in a .in-file is shown below

4.5.8 Anamorphic Surfaces

Anamorphic surfaces are surfaces that have different (conic) shapes in some specified orthogonal x- and y-axes. Cylindrical optics and astigmatic plates are two examples. MACOS anamorphic surfaces are entered using the standard KrElt and KcElt format, except that two sets of these conic parameters are entered: one set for each of the x- and y-axis directions. The variable AnaCoef holds the values in the following order: Krx, Kry, Kcx, Kcy. In addition, a specialized coordinate frame must be used to define the local x- and y-axes for these variables. This is the same specialized coordinate frame used in Section 4.5.4 and is defined by pMon, xMon, yMon, and zMon as shown in Figure 27. An example of an (cylindrical) anamorphic surface as defined in a .in-file is shown below.

```
iElt= 1
EltName= ana_surf Element= Refractor Surface= Anamorph
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00
0.000000000D+00 RptElt= 0.000000000D+00 0.000000000D+00 0.000000000D+00 IndRef= 1.000000000D+00
Extinc= 1.000000000D+22
AnaCoef= -1.700000000D+02 -1.000000000D+00 -1.000000000D+22 0.000000000D+00 pMon= 0.000000000D+00
0.000000000D+00 0.000000000D+00
xMon= -1.000000000D+00 0.000000000D+00 0.000000000D+00 yMon= 0.000000000D+00 1.000000000D+00
0.000000000D+00 zMon= 0.000000000D+00 0.000000000D+00 -1.000000000D+00
nObs= 0
ApType= None
zElt= 1.000000000D+22
PropType= Geometric nECoord= -6
```

4.5.9 User-Defined Surfaces: Deformable Mirrors

MACOS comes with 5 built-in user-defined surfaces. These surfaces use influence functions to model standard deformable mirror configurations for adaptive optics applications. They assume a grid of push-pull actuators spread out in regular patterns across the surface of the element. Coefficients that define the extension of each actuator are read from a file (or are directly modified by a calling program, if you are using SMACOS). The mirror coefficient file is specified in the prescription (or through the MODIFY command) as:

```
UDSrfFile='filename'
```

The surface types, in order of UDSrfType (as `usersub.F` dispatches them):

1. 1313-actuator deformable mirror.
2. 349-actuator deformable mirror.
3. 397-actuator triangular-array deformable mirror.
4. 349-actuator deformable mirror with squared-out influence function.
5. 349-actuator deformable mirror with ideal actuator influence functions (bilinear approximation).
6. 349-actuator deformable mirror with squared-out influence function.
7. 149-by-149 x-y grid (written for the Hubble primary mirror).
8. 115-by-115 x-y grid (written for the Hubble secondary mirror).
9. Radial-actuator deformable mirror.
10. SQAna-by-SQAna deformable mirror with damped-sine influence functions.
11. x-y grid surface.

Each type takes its parameters from

```
UDSrfParam= p1 p2 ... p6
```

up to six reals on ONE line, given BEFORE `UDSrfFile=` (the file is read with them; with no `UDSrfParam=` the engine sets the type's defaults at load and says so). Six is the width everywhere – the prescription parser, MODIFY, the element-entry prompt and SAVE; values past the sixth are ignored, and a short line is padded with 0 with a one-line note. The models read at most three:

- types 1-6 and 10 (influence-function mirrors): p1 = beta, the influence-function shape parameter; p2 = dAct, the actuator spacing; type 10 also p3 = SQAna, the actuators per side;
- type 9 (radial): p1 = drad, the radial actuator spacing; p2 = dq, the angular spacing (rad); p3 = minrad, the inner radius;
- types 7, 8 and 11 (x-y grids): p2 = the grid spacing; p1 is not read.

(The element arrays hold ten per element, `mUDSrfParam`; only six are ever read or written. Measured 2026-10-17, PLAN_CONSOLIDATION 1.2 (h).)

An example of the use of deformable mirrors, as well as lenslet arrays and the ATMOS

atmospheric phase disturbance command, is provided in Appendix A.4.

4.5.10 Interpolated Surfaces Using Regularly Gridded Data

MACOS provides interpolated surfaces of 2 types. The first type uses regular square gridded data, in the form of a matrix, whose rows and columns are filled with values giving the surface height deviation from a base conicoid of revolution. This form of interpolated surface, described in this section, is compatible with the high-density information typically generated by optical test interferometers. It uses linear interpolation to determine the intercept point of each ray at the optical surface, rather than the more

compute-intensive 3rd-order interpolation of the more general X-Y-Z interpolated surface discussed in the next section.

Grid-data surfaces are defined using 3 additional parameters. These are:

- mGridMat: the dimension of the gridded data matrix (nGridMat by nGridMat)
- GridSrfdx: spacing of the data grids in base units
- GridFile: name of the text file containing the nGridMat by nGridMat surface matrix

The maximum number of grid-data surfaces, and the maximum number of points for grid-data surfaces, are limited at compile time, by setting the mGridSrf and mGridMat parameters in the param.cmn file.

The surface data file GridFile should be in ASCII text, with nGridMat rows and

nGridMat columns. The data is read in using the following Fortran READ sequence:

```
((GridMat(i,j,jGridSrf),i=1,nGridMat),j=1,nGridMat)
```

The i and j indices correspond to the xMon and yMon coordinates, respectively:. The matrix is assumed centered at the pMon point. Increasing index values represent steps of size GridSrfdx along that axis. The maximum extent of the surface is ±(nGridMat/ 2)*GridSrfdx in either direction. Rays that fall outside the covered area are passed, but using only the base conic to determine direction and pathlength.

The grid spans (nGridMat − 1)·GridSrfdx base units in each direction, centered on the grid frame origin pData; a ray outside the span sees no figure (this is not an error), so a GridSrfdx too small for the element leaves figure only near its center. A grid declared larger than the compile-time mGridMat is not traced: the file is declined by name at load and the grid term is inert. When an element carries both a grid and Zernike terms, the grid frame xData, yData should equal the Zernike frame xMon, yMon, or a figure applied to one segment will not localize.

The data is read in when the prescription is read. No setup computations are required (no SINT command). Interpolation data can also be provided to SMACOS directly from a calling program that includes the appropriate common statements (see Section 9.2.3).

4.5.11 Interpolated Surfaces Using X-Y-Z Data

Interpolated surfaces using X-Y-Z data triplets allow arbitrary perturbations to be applied to a base conic-of-revolution surface, using irregularly sampled data. Compared to the gridded-data surface of the previous section, the X-Y-Z interpolated surfaces are slower to compute, using a 3rd-order, triangular, continuous surface interpolation algorithm. They also require large amounts of RAM.

The maximum number of interpolated data surfaces, and the maximum number of points for interpolated data surfaces, are limited at compile time, by setting the mIntSrf and mDP parameters in the param.cmn file. These settings are shown in the command window as MACOS is started up.

The interpolation data is entered from an input file, which is not read until the user executes a SINT (Surface INTerpolator) command. The SINT command reads the data files defining the surface(s) and precomputes interpolation coefficients used in the ray trace.

If rays are traced prior to executing the SINT command, the interpolated surfaces are converted to conic surfaces and the interpolation data is not used.

Interpolation data can also be provided to SMACOS directly from a calling program that includes the appropriate common statements (see Section 9.2.3).

The input data file can be in either binary or text data format. A binary surface data file is expected to have the name “filnam.srf*N*.bin,” where filnam is the loaded prescription’s file name *including* its `.in` and *N* is the element number (for `Cassegrain.in`, element 3: `Cassegrain.in.srf3.bin`). A text surface data file is expected to have the name “filnam.srf*N*.txt.” If no file with the expected name is found, the user is prompted for a different file name. If none is provided, the surface is not initialized and will be converted to a conic when rays are traced.

The first line of the file contains two integer numbers: nDP, the number of nodes for the optical element; and iComputeDZ, which indicates whether slope data is provided or not. If iComputeDZ=1, slopes are to be estimated from the height data; iComputeDZ=0 indicates that slopes are provided in the data file.

The nDP subsequent lines of the file contain the data for the interpolation. The data for one node is in one row in the file. Each row has 3 or 5 real numbers, giving the x and y coordinates of the data point (in the element’s local frame), the surface height at that point — the deviation from the element’s base conic, positive along psiElt, toward the incoming beam — and then, if iComputeDZ=0, the x and y slopes at that point. The grid defining the data does not need to be a regular grid. The grid points need not be in any particular order in the data file.

4.5.12 FreeForm Surfaces

FreeForm (SrfType = 14, Surface = FreeForm) is a single MACOS surface type combining up to four independent contributions:

```
z(rho) = z_conic(rho) + z_Mon(rho) + z_FF(rho) + z_grid(rho)
```

where:

z_conic is the conic of revolution defined by KrElt, KcElt, and the optical axis psiElt through the vertex VptElt.

z_Mon is a Cartesian-monomial sag in the local frame (pMon, xMon, yMon, zMon), normalized by aperture lMon, with coefficients MonCoef (or sparse MonZernCoef). Active only when lMon > 0.

z_FF is a free-form Zernike sag in an independent frame (pFF, xFF, yFF, zFF), normalized by lFF, with coefficients FFCoef (or sparse FFZernCoef). Active only when lFF > 0.

z_grid is an interpolated sag from a regularly-gridded data array, on an independent frame (pData, xData, yData, zData) with grid spacing GridSrfdx. Active only when nGridMat > 0.

Mon, FF, and grid contributions can be enabled in any combination; setting the corresponding sentinel to zero (or omitting it from the prescription) disables that term.

Why three independent frames. In segmented mirrors generated by SegMirMaker (§4.4.11), the FF coefficients describe the parent surface and the FF frame is parent-aligned (so all segments share the same FF Zernike set), while the Mon frame is per-segment (face-aligned) so each segment can carry its own figure error via MonZernCoef. The grid frame is parent-aligned so the data array represents a global perturbation map.

Independence is enforced in the surface evaluation. The FreeFormSrf routine treats Mon, FF, and grid sags as displacements along their own z-axes (zMon, zFF, zData). Earlier versions assumed all three shared a single z-axis, which produced incorrect intersections when, for example, the segment Mon frame was face-aligned with zMon = -psi while zFF = +psi (a 180-deg mismatch).

Example. Conic + Mon + FF + grid:

```
iElt= 1
EltName= primary
Element= Reflector
Surface= FreeForm
KrElt= -1.000D+02
KcElt= -1.0
psiElt= 0 0 1
VptElt= 0 0 0
RptElt= 0 0 0
lMon= 50.0
pMon= 0 0 0
xMon= 1 0 0
yMon= 0 1 0
zMon= 0 0 1
MonZernType= BornWolf
nMonZernCoef= 16
MonZernModes= 1 3 4 5 9 10 11 12 13 19 20 21 22 23 24 25
MonZernCoef= 1.197E+0 ...
lFF= 100.0
pFF= 0 0 0
xFF= 1 0 0
yFF= 0 1 0
zFF= 0 0 1
FFZernType= BornWolf
nFFZernCoef= 16
FFZernModes= ...
FFZernCoef= ...
nGridMat= 256
GridFile= flat.txt
GridSrfdx= 31.25
pData= 0 0 0
xData= 1 0 0
yData= 0 1 0
zData= 0 0 1
```

4.6 Coatings and Polarization Data

The polarization ray trace (Section 5, *Polarized Ray Tracing*) reads three kinds of prescription data beyond the scalar trace: the complex index of each element, an optional thin-film coating stack per element, and the source polarization state.

Element index and extinction. Every element carries `IndRef=` and `Extinc=`, forming the complex index N = n − iκ (κ > 0 is loss, under the engine's exp(+iωt) time convention). In the scalar trace only the real index of refractors matters; with polarization on, a bare (uncoated) surface applies the Fresnel s/p coefficients computed from its complex index. The standard mirror idiom

```
IndRef=   1.0E+00
Extinc=   1.0E+22
```

is a perfect conductor: it reflects with RP = RS = −1 and is polarization-neutral (no diattenuation, no retardance). To model a real metal mirror, either give the element the metal's complex index directly, or (preferred) add a coating stack whose outer layer is an optically thick metal film — thickness much greater than the skin depth reproduces the bare metal's Fresnel coefficients regardless of substrate.

Thin-film coating stacks (`Coating=`). An element may carry a multilayer coating, applied by a recursive thin-film calculation at each ray's actual angle of incidence when polarization is on. The keyword gives the layer count, followed by one line per layer, ordered **outermost to innermost**, each line holding the layer's real index, extinction coefficient, and thickness:

```
IndRef=   1.0E+00
Extinc=   1.0E+22
Coating=  2
  1.38  0.0   0.25
  2.30  0.10  0.125
```

Two rules deserve emphasis:

- **Layer thickness in the prescription is in units of waves** — waves of the `Wavelen=` value current at the point the parser reaches the `Coating=` line. It is converted to physical thickness (thickness·Wavelen/IndRef) at load time, so the loaded stack is physically fixed and wavelength sweeps after loading are handled correctly. (The programmatic interface, `coating`/`coat_set` in the bindings, takes **physical** thickness in BaseUnits directly and has no wavelength coupling.)
- **`Coating=` must appear after the element's `IndRef=`/`Extinc=` lines** — the parser snapshots the boundary media when it reads the coating block.

The incident medium for the coating calculation is the medium the ray is actually traveling in when it reaches the surface (the running index, e.g. air), and the substrate is the element's own complex index.

Source polarization state (`Ex0Ey0=`). The complex source field amplitudes can be carried in the prescription:

```
Ex0Ey0=   1.0  0.0  0.0  0.0
```

giving Ex real, Ex imaginary, Ey real, Ey imaginary, in the source grid frame. This sets the *state* only; turning polarization tracing on remains a command/API decision (POLarization, or `polarization('on')` in the bindings, which also prompt for or accept the state directly). The keyword round-trips through SAVE.

4.7 Obscurations and Apertures

Apertures are the areas through which light can pass. MACOS supports circular and rectangular apertures as shown in Figure 28. When a wavefront is propagated, the intensity that does not go through the hole is set to zero. Likewise, obscurations block light that falls within their interior. Figure 29 shows the types of obscurations implemented in MACOS: circular, rectangular, annular, and triangular. Geometrical rays are unaffected by the obscurations; the rays are traced through the optical system regardless. However, the OBScuration command will allow one to view only the unobscured rays in the spot diagram.

When entering data for obscurations and apertures, fElt, eElt, and, psiElt are required inputs. This may seem unnatural, but including these properties allows curvature to be given to apertures and obscurations. If an aperture or obscuration is planar, then fElt=1d22 and eElt=0. In this case, psiElt is normal to the plane. However, if the obscuration or aperture is not planar, such as an obscuration caused by mirror support pads, fElt, eElt, and, psiElt are the same as they would be for that mirror. All obscuring apertures are defined on the plane defined by psiElt and VptElt and projected onto the surface.

This section has three different examples. The first example is of an element representing a square aperture in front of the primary mirror.

```
Enter element name (no blanks) [Elt]: Sq_Ap EltName
+ +
| Element types: |
```



FIGURE 28 Examples of apertures types.

Circle Rectangle Ring Triangle

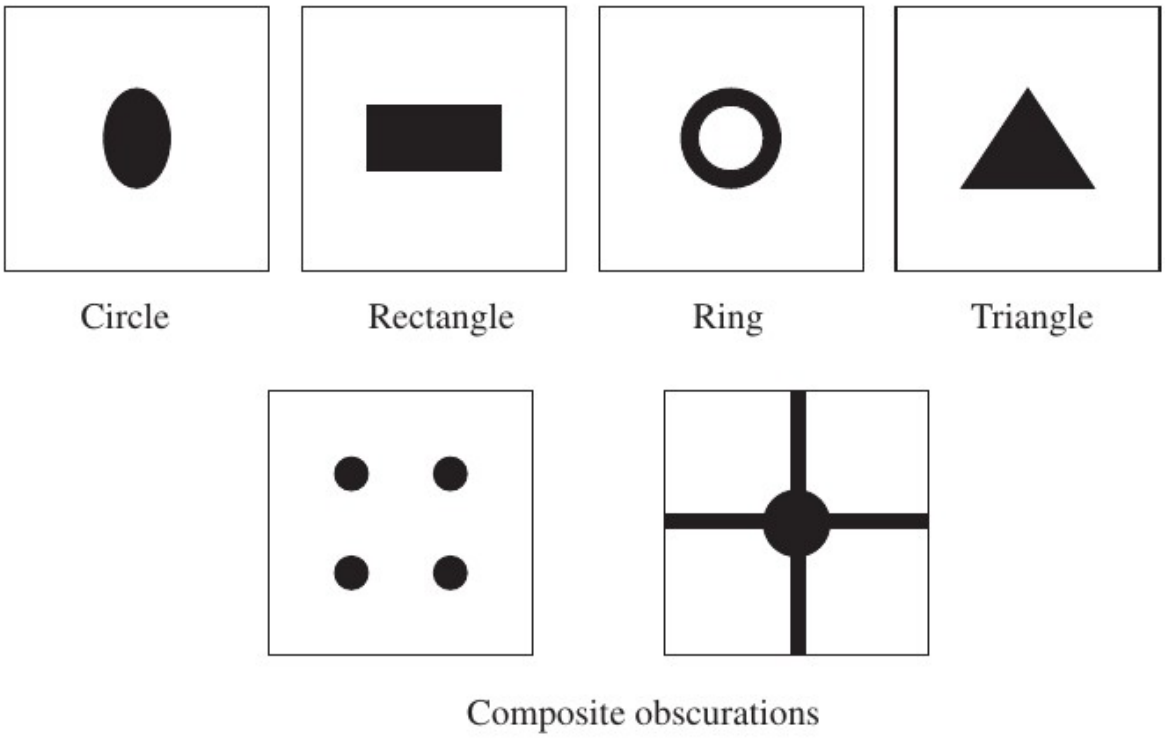


FIGURE 29 Examples of obscuration types and some composite obscurations.

	Reflector	NSReflector	Segment	
	Refractor	NSRefractor	LensArray	
	FocalPlane	Reference	Return	
	HOE	Grating	Obscuring	

```
 + +
 Enter element type :Obscuring Element
 + +
 | Surface types: |
 | Flat Conic Aspheric |
 | Anamorphic Zernike Monomial |
 | Interpolated UserDefined |
 + +
 Enter surface type :Flat Surface
 Enter element vertex location (x,y,z) [no default]: 0,0,-5.4 VptElt Enter element rotation point
 (x,y,z) [0.,0.,-5.40000]:0,0,-5.4 RptElt Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1
 psiElt Do you want an aperture on this element? [NO]: yes
 + +
 | Aperture types: |
 | Circular Rectangular |
 + +
 Enter aperture type [Circular]: Rectangular ApType Enter aperture xmin,xmax,ymin,ymax [-
 0.500000,0.500000,-0.500000,0.500000]: - 1.5,1.5,-1.5,1.5 ApVec
 Enter number of obscurations [0]:0 nObs
 Enter element x-axis vector (x,y,z) [-1.00000,0.,0.]:1,0,0 xObs
 Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
 + +
 | Propagation types: |
```

	Geometric	GeomUpdate	FarField	
	NFSpherical	NFS1surf		
	NFPlane	NFP1surf		
	NF1	NF2		
	SpatialFilter	SF1surf		

```
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
```

This dialog produced the following prescription data:

```
    iElt=  1
 EltName=  Sq_Ap
 Element=  Obscuring
 Surface=  Flat
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  Rectangular
   ApVec=  -1.500000000D+00  1.500000000D+00 -1.500000000D+00  1.50000000D+00
    zElt=  1.000000000D+22
PropType=  Geometric
 nECoord=  -6
```

The second example shows an obscuration consisting of four circles (see Figure 29). This is similar to what was done to model the Hubble Space Telescope's primary mirror which was drilled in three places for rubber retaining rings. This example is a Cassegrain telescope example with four support pads on the primary mirror. fElt and eElt are the same as the primary mirror. NGridpts was also increased so the obscurations could be seen in the focal plane. Note that the support pads are located just in front of the primary mirror.

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: 4CircObs EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Obscuring Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic Surface
Enter element vertex location (x,y,z) [no default]: 0,0,-.01 VptElt Enter element rotation point
(x,y,z) [0.,0.,-1.000000E-02]:0,0,-.01 RptElt Enter element principal axis (x,y,z) [0.,0.,-
1.00000]:0,0,-1 psiElt Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:5.4 fElt
Enter element eccentricity [0.]:1 eElt
Do you want an aperture on this element? [NO]: yes
+ +
```

```
| Aperture types: |
| Circular Rectangular |
+ +
Enter aperture type [Circular]: Circular ApType
Enter aperture radius, x, y [0.500000,0.,0.]:2,0,0 ApVec
Enter number of obscurations [0]:4 nObs
+ +
| Obscuration types: |
| Circle NegCircle |
| Rectangle NegRectangle |
| Annulus NegAnnulus |
| Triangle NegTriangle |
| Ellipse NegEllipse |
+ +
Enter obscuration type [Circle]: Circle ObsType
Enter obscuration radius, x, y [no default]: 0.2,1,1 ObsVec
Enter obscuration type [Circle]: Circle ObsType
Enter obscuration radius, x, y [no default]: 0.2,1,-1 ObsVec
Enter obscuration type [Circle]: Circle ObsType
Enter obscuration radius, x, y [no default]: 0.2,-1,1 ObsVec
Enter obscuration type [Circle]: Circle ObsType
Enter obscuration radius, x, y [no default]: 0.2,-1,-1 ObsVec
Enter element x-axis vector (x,y,z) [-1.00000,0.,0.]:1,0,0 xObs
Enter Fresnel propagation distance [5.40000]:5.4 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
```

This dialog produced the following prescription data:

```
iElt= 1
EltName= 4CircObs Element= Obscuring Surface= Conic
KrElt= -1.080000000D+01 KcElt= -1.000000000D+00
psiElt= 0.000000000D+00 0.000000000D+00 -1.000000000D+00 VptElt= 0.000000000D+00 0.000000000D+00 -
1.000000000D-02 RptElt= 0.000000000D+00 0.000000000D+00 -1.000000000D-02 IndRef= 1.000000000D+00
Extinc= 0.000000000D+00
nObs= 4
ObsType= Circle
ObsVec= 2.000000000D-01 1.000000000D+00 1.000000000D+00
ObsType= Circle
ObsVec= 2.000000000D-01 1.000000000D+00 -1.000000000D+00
ObsType= Circle
ObsVec= 2.000000000D-01 -1.000000000D+00 1.000000000D+00
ObsType= Circle
ObsVec= 2.000000000D-01 -1.000000000D+00 -1.000000000D+00 xObs= 1.000000000D+00 0.000000000D+00
0.000000000D+00
ApType= Circular
ApVec= 2.000000000D+00 0.000000000D+00 0.000000000D+00 zElt= 5.400000000D+00
PropType= Geometric nECoord= -6
```

The third example shows an obscuration produced by a secondary mirror and it’s support spider: two rectangles and a circle (see Figure 29). These planar obscurations obscure the primary mirror.

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: SpiderObs EltName
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Obscuring Element
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
```

```
| Interpolated UserDefined |
+ +
Enter surface type :Flat Surface
Enter element vertex location (x,y,z) [no default]: 0,0,-5.4 VptElt Enter element rotation point
(x,y,z) [0.,0.,-5.40000]:0,0,-5.4 RptElt Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1
psiElt Do you want an aperture on this element? [NO]: yes
+ +
| Aperture types: |
| Circular Rectangular |
+ +
Enter aperture type [Circular]: Circular ApType
Enter aperture radius, x, y [0.500000,0.,0.]:2,0,0 ApVec
Enter number of obscurations [0]:3 nObs
+ +
| Obscuration types: |
| Circle NegCircle |
| Rectangle NegRectangle |
| Annulus NegAnnulus |
| Triangle NegTriangle |
| Ellipse NegEllipse |
+ +
Enter obscuration type [Circle]: Circle ObsType
Enter obscuration radius, x, y [no default]: 0.5,0,0 ObsVec
Enter obscuration type [Circle]: Rectangle ObsType Enter obscuration xmin, xmax, ymin, ymax [no
default]: -.25,.25,-2,2 ObsVec Enter obscuration type [Circle]: Rectangle ObsType Enter obscuration
xmin, xmax, ymin, ymax [no default]: -2,2,-.25,.25 ObsVec Enter element x-axis vector (x,y,z) [-
1.00000,0.,0.]:1,0,0 xObs
Enter Fresnel propagation distance [1.000000E+22]:1d22 zElt
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric PropType
Do you wish to use element-based coordinates?: [NO]:
```

This dialog produced the following prescription data:

```
    iElt=  1
 EltName=  SpiderObs
 Element=  Obscuring
 Surface=  Flat
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  3
 ObsType=  Circle
  ObsVec=  5.000000000D-01  0.000000000D+00  0.000000000D+00
 ObsType=  Rectangle
  ObsVec=  -2.500000000D-01  2.500000000D-01 -2.000000000D+00  2.00000000D+00
 ObsType=  Rectangle
  ObsVec=  -2.000000000D+00  2.000000000D+00 -2.500000000D-01  2.50000000D-01
    xObs=  1.000000000D+00  0.000000000D+00 0.000000000D+00
  ApType=  Circular
   ApVec=  2.000000000D+00  0.000000000D+00  0.000000000D+00
    zElt=  1.000000000D+22
PropType=  Geometric
 nECoord=  -6
```

4.7.1 Obscuration types and the ObsVec= line

Every obscuration of an element is two prescription lines, `ObsType=` and `ObsVec=`, repeated `nObs=` times; the shapes lie in the element's local plane, x along `xObs=`, y = psiElt × xObs, origin at the vertex, in base units. The `ObsVec=` values by type (the interactive dialog asks for the same numbers, in the same order):

ObsType=	ObsVec= values	Meaning
Circle / NegCircle	r, xc, yc	radius and center
Rectangle / NegRectangle	xmin, xmax, ymin, ymax	axis-aligned box
Annulus / NegAnnulus	rOuter, xc, yc, rInner	ring
Triangle / NegTriangle	q1x, q1y, q2x, q2y, q3x, q3y	three corners
Ellipse / NegEllipse	xrad, yrad, xc, yc	axis-aligned ellipse
Matrix / NegMatrix	px, py, dx, dy	a 0/1 matrix on a pixel grid; the dialog then asks the matrix dimension and file name
Tapered_Ellipse	a, b, xc, yc, dr	ellipse whose transmission tapers over the outer fraction dr of a and b
Polygon / NegPolygon	xc, yc, n — then n lines of x y	convex polygon, vertices in order around it, relative to (xc, yc); see below
RotRectangle / RotNegRectangle	W, H, dx, dy, angle	box of width W and

ObsType=	ObsVec= values	Meaning
		height H centered at (dx, dy), rotated by angle (radians)
RotEllipse / RotNegEllipse	a, b, dx, dy, angle	ellipse of half-axes a, b centered at (dx, dy), rotated by angle (radians)

A **Neg** shape is a hole: a ray is obscured when it falls inside a positive shape and inside no negative one, so a NegCircle inside a Rectangle makes a plate with a round opening.

Polygon obscurations in three forms. A polygon must be convex, with at most 128 vertices (`mPolySide`), listed in order around its perimeter. Besides the 2-D form in the table, the vertices may be given in the GLOBAL frame as x y z triplets, one per line, after a `PolyObsVec= n` (or `Poly3DObsVec= n`) line in place of `ObsVec=`:

```
ObsType= Polygon
PolyObsVec= 4
0.5 0.5 -4.2
-0.5 0.5 -4.2
-0.5 -0.5 -4.2
0.5 -0.5 -4.2
```

MACOS takes the vertices' centroid as the polygon center and projects each vertex onto the element's local plane; the projection needs the element's `xObs=`, and the order of the lines no longer matters — a polygon written before its element's `xObs=` is held until `xObs=` is read (or until the load's default for it), where it used to be a hard error. The 2-D form of the same square is `ObsVec= 0 0 4` followed by `0.5 0.5`, `-0.5 0.5`, `-0.5 -0.5`, `0.5 -0.5`. On the Cassegrain example (4 m aperture, 51-point grid, 1877 rays) the secondary obscuration — a circle of radius 0.5 and its spider — loses 121 rays; this 1 m square in its place loses 145 in all three forms, the ray density of the grid times its area (`cli_tests/cmd/OBS_POLYGON.jou`). Apertures have the same polygon forms under `ApType= Polygonal` with `PolyApVec=`, but there `xObs=` must still come first.

4.8 Specifying Coatings

Multi-layer coatings are not supported in this release of MACOS.

4.9 Example Telescope

In Section 4.2 the source data for the Cassegrain telescope example was entered. This section finishes the example adding the necessary optical elements: the primary mirror, secondary mirror, reference surface, and focal plane. Figure 15 shows the telescope. Table 9 lists the optical design.

TABLE 9. Cassegraing telescope design

Element	Focal length	Ecc entricity	Diam eter	Distance from primary
Primary Mirror	5.4	1.0	4.0	
Secondary Mirror	1.338854098	1.634183595	1.0	4.061145901
Focal Plane	1d22	0.0		1.5

Continuation of MACOS input from Section 4.2.

```
Enter Element 1 Data:
Enter element name (no blanks) [Elt]: SecMirObs
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Obscuring
+ +
```

Surface types:			
Flat	Conic	A spheric	
Anamorphic	Z ernike	M onomial	

```
| Interpolated UserDefined |
+ +
Enter surface type :Flat
Enter element vertex location (x,y,z) [no default]: 0,0,-5.4 Enter element rotation point (x,y,z)
[0.,0.,-5.40000]:0,0,-5.4 Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1 Do you want an
aperture on this element? [NO]: yes
+ +
| Aperture types: |
| Circular Rectangular |
+ +
Enter aperture type [Circular]: Circular
Enter aperture radius, x, y [0.500000,0.,0.]:2,0,0 Enter number of obscurations [0]:1
+ +
| Obscuration types: |
| Circle NegCircle |
| Rectangle NegRectangle |
| Annulus NegAnnulus |
| Triangle NegTriangle |
| Ellipse NegEllipse |
+ +
Enter obscuration type [Circle]: Circle
Enter obscuration radius, x, y [no default]: 0.5,0,0
Enter element x-axis vector (x,y,z) [-1.00000,0.,0.]:1,0,0 Enter Fresnel propagation distance
[1.000000E+22]:1d22
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]:

    iElt=  1
 EltName=  SecMirObs
 Element=  Obscuring
 Surface=  Flat
    fElt=  1.000000000D+22
    eElt=  0.000000000D+00
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
```

```
  RptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  1
 ObsType=  Circle
  ObsVec=  5.000000000D-01  0.000000000D+00  0.000000000D+00
    xObs=  1.000000000D+00  0.000000000D+00  0.000000000D+00
  ApType=  Circular
   ApVec=  2.000000000D+00  0.000000000D+00  0.000000000D+00
    zElt=  1.000000000D+22
PropType=  Geometric
 nECoord=  -6

Is this correct? [YES]:
Enter Element 2 Data:
Enter element name (no blanks) [Elt]: Primary
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :Reflector
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic
Enter element vertex location (x,y,z) [no default]: 0,0,0
Enter element rotation point (x,y,z) [0.,0.,0.]:0,0,0
Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,-1 Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)?
[1]:1
Enter element focal length [1.000000E+22]:5.4 Enter element eccentricity [0.]:1
Do you want an aperture on this element? [NO]: Enter number of obscurations [0]:
Enter Fresnel propagation distance [5.40000]:5.4
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]: Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:

    iElt=  2
 EltName=  Primary
 Element=  Reflector
 Surface=  Conic
    fElt=  5.400000000D+00
    eElt=  1.000000000D+00
   KrElt=  -1.080000000D+01
   KcElt=  -1.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 0.000000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
    nObs=  0
  ApType=  None
    zElt=  5.400000000D+00
PropType=  Geometric
 nECoord=  -6

Is this correct? [YES]:
Enter Element 3 Data:
Enter element name (no blanks) [Elt]: Secondary
+ +
```

Element types:	
Reflector	NSReflector Segment |
Refractor	NSRefractor LensArray |
FocalPlane	Reference Return |
HOE	Grating Obscuring |

```
+ +
Enter element type :Reflector
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic
Enter element vertex location (x,y,z) [no default]: 0,0,-4.0611459018
Enter element rotation point (x,y,z) [0.,0.,-4.06115]:0,0,-5.4 Enter element principal axis (x,y,z)
[0.,0.,-1.00000]:0,0,-1 Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:1.3388540982 Enter element eccentricity [0.]:1.6341835953
Do you want an aperture on this element? [NO]: Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.33885]:1.3388540982
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]: Enter element index of refraction [1.00000]:
Enter element extinction coefficient [1.000000E+22]:

    iElt=  3
 EltName=  Secondary
 Element=  Reflector
 Surface=  Conic
    fElt=  1.338854098D+00
    eElt=  1.634183595D+00
   KrElt=  -3.526787502D+00
   KcElt=  -2.670556023D+00
  psiElt=  0.000000000D+00  0.000000000D+00 -1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -4.061145902D+00
  RptElt=  0.000000000D+00  0.000000000D+00 -5.400000000D+00
  IndRef=  1.000000000D+00
  Extinc=  1.000000000D+22
    nObs=  0
  ApType=  None
    zElt=  1.338854098D+00
PropType=  Geometric
 nECoord=  -6

Is this correct? [YES]:
Enter Element 4 Data:
Enter element name (no blanks) [Elt]: Ref_surf
+ +
```

Element types:	
Reflector	NSRe flector Segment |
Refractor	NSRe fractor LensArray |
FocalPlane	Re ference Return |
HOE	Grating Obscuring |

```
+ +
Enter element type :Reference
```

```
+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Conic
Enter element vertex location (x,y,z) [no default]: 0,0,-4.0601459018 Enter element rotation point
(x,y,z) [0.,0.,-4.06015]:0,0,-4.0601459018 Enter element principal axis (x,y,z) [0.,0.,-
1.00000]:0,0,1
Enter e&f or Kc&Kr (1=e&f,2=Kc&Kr)? [1]:1
Enter element focal length [1.000000E+22]:5.5601459018 Enter element eccentricity [0.]:0
Do you want an aperture on this element? [NO]: Enter number of obscurations [0]:
Enter Fresnel propagation distance [5.56015]:5.5601459018
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
Enter propagation type [Geometric]: FarField
Do you wish to use element-based coordinates?: [NO]:

    iElt=  4
 EltName=  Ref_surf
 Element=  Reference
 Surface=  Conic
    fElt=  5.560145902D+00
    eElt=  0.000000000D+00
   KrElt=  -5.560145902D+00
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -4.060145902D+00
  RptElt=  0.000000000D+00  0.000000000D+00 -4.060145902D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  None
    zElt=  5.560145902D+00
PropType=  FarField
 nECoord=  -6

Is this correct? [YES]:
Enter Element 5 Data:
Enter element name (no blanks) [Elt]: foc_pln
+ +
| Element types: |
| Reflector NSReflector Segment |
| Refractor NSRefractor LensArray |
| FocalPlane Reference Return |
| HOE Grating Obscuring |
+ +
Enter element type :FocalPlane

+ +
| Surface types: |
| Flat Conic Aspheric |
| Anamorphic Zernike Monomial |
| Interpolated UserDefined |
+ +
Enter surface type :Flat
Enter element vertex location (x,y,z) [no default]: 0,0,1.5 Enter element rotation point (x,y,z)
[0.,0.,1.50000]:0,0,1.5 Enter element principal axis (x,y,z) [0.,0.,-1.00000]:0,0,1 Do you want an
aperture on this element? [NO]:
Enter number of obscurations [0]:
Enter Fresnel propagation distance [1.000000E+22]:0
+ +
| Propagation types: |
| Geometric GeomUpdate FarField |
| NFSpherical NFS1surf |
| NFPlane NFP1surf |
| NF1 NF2 |
| SpatialFilter SF1surf |
+ +
```

```
Enter propagation type [Geometric]: Geometric
Do you wish to use element-based coordinates?: [NO]:

    iElt=  5
 EltName=  foc_pln
 Element=  FocalPlane
 Surface=  Flat
    fElt=  1.000000000D+22
    eElt=  0.000000000D+00
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00  1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00  1.500000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00  1.500000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  None
    zElt=  0.000000000D+00
PropType=  Geometric
 nECoord=  -6

nOutCord= 0
Tout=
Is this correct? [YES]:
Enter xOut (x,y,z) [-1.00000,0.,0.]:1,0,0
Enter yOut (x,y,z) [0.,1.00000,0.]:0,1,0
```

This completes the input example. The file is now saved as Cassegrain.in within the directory in which MACOS is running. The input file is included in Appendix A.1.

4.10 Editing .In-Files

Once a prescription has been entered and stored as a .in-file, the user can edit and reuse it for further analyses. Editing an .in-file is a matter of opening the file in the text editor

of your choice. On Unix machines this can be vi, Emacs, TextEdit, or another text editor. On the Macintosh, Qued-M, MPW Editor, BBEdit, Alpha, or other text or word processors can be used.

Files can be opened in directories other than the one MACOS is running in, by using appropriate path descriptors in specifying the file name. On Unix systems, the full name takes the form: “subdirectory/subdirectory/filename.” On Macintosh systems, the full name takes the form: “:subdirectory:subdirectory:filename.”

There are some simple rules for .in-files that must be followed to avoid ambiguities in the data. These are:

- The first data block must be for the source.
- Each element’s data must be in a block starting with ‘iElt=’ followed by the number of that element. All data within that data block refers to that element. The end of the data block is indicated by the next ‘iElt=’ line or the ‘nOutCord=’ line.
- Ordering within data blocks is generally not important, except that nObs must preceed the description of the obscurations.
- The last element data block is followed by the ‘nOutCord=’ specifying the number of output coordinates and the output coordinate matrix (`Tout=`). A prescription that ends without this block still loads: the loader proposes the beam frame at the last element (z along the beam from the previous element’s vertex to the last element’s vertex, or along the chief ray when there is one element; x from the element’s `xObs=`, or the global axis least aligned with z; y = z × x), prints it, and in the interactive program asks `Keep the proposed output coordinates? [YES]:` — NO reads

`nOutCord` and the `Tout` rows at the prompt. The MATLAB and Python bindings take the proposal and print the same lines. SAVE writes the block either way.

- A numeric `PropType=` (the legacy form, 0 through 15 in the order of the propagation-type table) is accepted and resolved to its name at load; `PropType= 0` is Geometric. SAVE writes the name. A value that is neither a name nor a number in the table is reported (`** PropType= <value> at element n not recognized: the default applies`) and the default is used.

- Blank lines can be used to separate data for clarity.

- Some optional keywords are never written by the NEW dialog and are added by hand (or by SAVE when the value differs from the default). In the source block: `UseChfRay4OPD= Y|N` selects the OPD reference (the chief ray, `Y`, is the default; `N` selects the aperture mean — Section 5.2) and `OPDRefRayLen=` fixes the reference pathlength directly. In an element block: `ZernType=` (ANSI when absent — see "Zernike Surfaces") and `NSCount=` (the non-sequential hit budget — see "Reflective Non-Sequential Surfaces").

- Comments. A whole line beginning with '%' is a comment, and SAVE writes it back (up to 500 such lines; an overflow is reported once at SAVE). A '%' after a value (`Key= value  % note`) ends the value: the note is read past but is NOT written back by SAVE. Block comments open with `/*` or `CommentBegin` and close with `*/` or `CommentEnd`, each marker first on its own line; SAVE writes the block back as it was. A one-line `/* note */` is not supported (VALIDATE refuses it as a comment block never closed).

- The design-optimization keywords (`OptTarget=`, `VarElt=`, `OptBeam*=`, …) are listed in Table 13 at the end of this section. They are reset at every load; SAVE writes the ones whose values differ from their defaults, and a load that reads any of them prints one line naming the setup in force.

- An unknown or misspelled keyword in the source block is reported once per load (`MBFile6: Unidentified header string = <key>`) and otherwise ignored; in an element block the message is `MBFile6: Unidentified string`.

- Data strings can be entered in free format. Up to 16 decimals of precision will be recognized.

Data for a single element must be grouped together, but the order of the data is not important as long as the 'iElt=' statement is first. When a new 'iElt=' statement is encountered, MACOS checks to make sure all the necessary data has been provided for the previous element. If data for an element has not been provided, MACOS will do one of two things. For some elements, MACOS will provide "default" data for that element. Other data is "necessary" for an .in-file and MACOS will print a warning that the .in-file was not loaded and the name of the omitted variable. A listing of the defaults provided for variables is included in Section 4.12.

4.11 Commands

MACOS provides the user several commands to help setup the optical prescription, to modify or perturb the optical prescription, and to list parameters of the optical system. Commands to help Some of these are described here; others are mentioned here but described elsewhere.

4.11.1 OLD and LOAD

Previously created .in-files are loaded using the OLD command. This command has one argument, the name of the input file (minus the ".in" suffix). For instance, to load the Cassegrain.in prescription:

MACOS> old Cassegrain

LOAd also work identically:

MACOS> loa Cassegrain

4.11.2 VALIDATE

The VALidate command syntax-checks a .in prescription file without loading it into MACOS. This is useful when sharing or hand-editing prescription files: rather than discovering a typo halfway through a load, the user can check the file first. Like OLD, the command takes the filename (with or without the .in suffix):

MACOS> validate Cassegrain

If the file is well-formed, MACOS prints “Cassegrain.in: OK”. If a problem is detected, the message identifies the offending line and key, for example:

Cassegrain.in: line 76: key “ZernType” has no value

The same check runs automatically as part of the OLD and LOAD commands; if it fails, the load is aborted and the user is re-prompted for a different filename, leaving the previously loaded prescription untouched.

The validator reads the file as text only. It refuses a key with no value (`EltName=`, `BaseUnits=` and `WaveUnits=` excepted, since older prescriptions leave them blank), a blank line inside a multi-row block such as a `Tout=` matrix, and a comment block that is never closed. It accepts `%` and `!` comment lines and the block-comment forms described under “Editing .In-Files”; note that the loader itself treats only `%` as a comment character. Files with CR or CRLF line endings load through a temporary LF copy and need no conversion.

4.11.3 MODIFY

The MODify command allows the user to change the current optical prescription without editing the .in-file. These changes are not saved in a .in-file unless the SAVe command is used. This allows the user to perturb elements, modify the prescription, and save the results if desired. Any of the .in-file variables listed in Section 4.12 can be changed. Write the assignment without spaces around the equals sign (`mod nGridpts=33`); `mod nGridpts = 33` is rejected with a message rather than read.

MACOS>**mod**

```
MOD: (q): h
MACOS MODify Function MODify Syntax Examples:
Flux=3
ChfRayPos(1)=2-MOD 1st element of vector ChfRayPos(:)=2,0,0-MOD all elements of vector RptElt(3,2)=3-
MOD 3rd element of vector for iElt=2 RptElt(:,2)=1.5,3,0-MOD all elements of vector for iElt=2
Quit: quit Modify function.
```

To use the MODify command, just type MOD. The user will then be prompted for the modification. If a MODification is successful, it will be echoed at the terminal. For instance to change the eccentricity of element 2 to 1.03.

MACOS>**mod**

```
MOD: (q): eElt(2)=1.03 eElt( 2) = 1.030000000D+00 KcElt( 2) = -1.060900000D+00 KrElt( 2) = -
2.842000000D+00
```

If a MOD is not successful, it will not be echoed.

```
MOD: (q): eeElt(2)=1.04
MOD: (q):
```

There are 2 ways to enter use MODify for vectors and arrays. The first is changing one element of the vector or array at a time:

```
MOD: (q): VptElt(3,2)=4.2
```

The second is to re-enter the entire vector:

```
MOD: (q): VptElt(:,2)=1.0,0.0,3.0
```

The SHOw command can be used to display the element's data.

4.11.4 SAVE

The SAVe command writes a new .in-file containing the current optical prescription data to disk. Parameters changed as the result of a PERturb or MODify command will be written as modified. This command takes one argument: the new .in-file name (without the ".in" suffix). If the specified file already exists, the user is asked if it should be overwritten.

What SAVE writes is the optical system as loaded and modified: every element block, the whole-line and block comments of the original file (in-line `% notes` are dropped), the stop (`ApStop=`), and the optional keywords whose values differ from their defaults (`UseChfRay4OPD= N` when the aperture-mean OPD reference is in force, `ZernType=`, `NSCount=`, `OPDRefRayLen=`). A ray trace never changes what SAVE writes. SAVE also writes the design-optimization keywords of Table 13 whose values differ from their load defaults (the target, the variable elements and their bounds, the field list and weights, the beam targets, the iteration cap and tolerance), so a saved prescription carries the CALIB setup made on it, and the next load announces that setup (`LOAD: CALIB setup in force from the prescription: …`). SAVE does not write the session options set by command (POLARIZATION, VECTOR, NOLNEG, CENTROID, CIR), which belong to the session rather than the system. Loading a saved file and saving it again reproduces the file: the test suite checks this round trip on every prescription in the repository.

4.11.5 SUMMARIZE

The SUMmarize command displays a brief summary of the loaded optical system, listing each element and showing its type, conic constant KcElt, and radius of curvature KrElt.

4.11.6 STATUS

The STAtus command displays information about some of the current states.

MACOS>**status**

```
Status for file cass_ap3:
Current element for OPD calculations= 4 Current element for WF/Spot calculations= 5 Obscuration
option for ray-trace= 0 Current WF plot type=SLICE
REGRID option= F Composed image= T
Pixel location set by chief ray option= F
```

4.11.7 RESET

The RESet command reinitializes some of the current states (see below) to their default values.

MACOS>**reset** MACOS>**status**

```
Status for file cass_ap3:
Current element for OPD calculations= 0 Current element for WF/Spot calculations= 0 Obscuration
option for ray-trace= 0 Current WF plot type=GRAY
REGRID option= F Composed image= F
Pixel location set by chief ray option= F
```

4.11.8 SHOW

The SHOw command displays all the prescription data for a particular element. It has one argument, the element number.

4.12 Summary of Prescription Variables

Tables 10 through 12 provide a summary of the MACOS prescription variables and their default values. When MACOS loads an .in-file, these defaults are used variables, if the user has not supplied values. When a default is used, the user is notified, e.g.:

```
Default used for zSource
```

Not all variables have defaults, and these variables must be specified by the user. If one of these variables is not given a value in the .in-file, then the .in-file is not loaded and the user is notified; for instance:

```
VptElt(1) must be specified by user Input file not properly loaded
```

The variables are shown with their index types in parentheses. These indices are:

- iObs: which obscuration in an element (can be up to 6 obscurations per element).
- iElt: number of the element in the .in-file.
- iAxis: ranges over the x,y,z coordinates in the global coordinate system.
- iPert: ranges over x,y,z rotations, x,y,z translations in the u-vector.
- iLin: ranges over the x,y,z components of the ray direction perturbation, the x,y,z components of the ray transverse aberration and the pathlength perturbation in the linear state vector x.
- iAsCoef: A, B, C, and D coefficients described in Section 4.5.3.
- iAp: either radius,x,y, or xmin,xmax,ymin,ymax depending on ApType.
- iZern: the 36 Zernike coefficients.
- iLayer: ranges over the coating layers.

All variables can be changed with the MODify command.

TABLE 10. Light source variables

Source Variables	Defaults	Description
Aperture	1.0D0	input aperture
BeamType	1 (uniform)	beam intensity
ChfRayDir(iAxis)	NO DEFAULT	chief ray direction
ChfRayPos(iAxis)	(0.0 D0,0.0D0,0.0D0)	chief ray location
CosPower		
ExtInc(0)	0.0D0	extinction coefficient
Ex0		electric field strength of source along x-axis
Ey0		electric field strength of source along x-

Source Variables	Defaults	Description
		axis
Flux	1.0D0	total intensity received across the unobscured aperture
gap	0.0D0	spacing between segments in seg-mented apertures
GridType	1 (circular)	type of input aperture
IndRef(0)	1.0D0	real part of index of refraction of input medium
nElt	NO DEFAULT	number of elements
nGridPts	IFIX(mpts/2)	number of ray grid points across the aperture
nSeg	0	number of segments
Obscratn	0.0D0	input obscuration
rxBeam		beam diameter in x-direction
ryBeam		beam diameter in y-direction
SegCo-ord(xyz,iElt)	NO DEFAULT	segment location in the entrance pupil
Wavelen	6.0D-7	wavelength of light
width	Aperture	width of segments
xGrid(iAxis)	NO DEFAULT	orientation of input x-axis vector
yGrid(iAxis)	cross product of xGrid and ChRay-Dir	orientation of input y-axis vector
zGrid(iAxis)		orientation of input z-axis vector
Zelt(0)		
zSource	1.0D22	distance of light source from point of origin

TABLE 11. Element variables

Element variables	Default values	Description
AnaCoef(iAna,iElt)		anamorphic coefficients
ApType	1 (circle)	aperture type
ApVec(iAp,iElt)	NO DEFAULT	aperture definition informa-tion
AsphCoef(iAsCoef,iElt)	(0.0 D0,0.0D0,0.0D0, 0.0D0)	asphere coefficients (4)
CoatIndxElt(iLayer, iElt)		index of coating layers
CoatThkElt(iLayer,iElt)		thickness of coating layers
eElt(iElt)	0.0D0	element eccentricity
EltName(iElt)	’ ’ (blanks)	element name
Element(iElt)	NO DEFAULT	element type
ExtInc(iElt)	0.0D0	extinction coefficient
fElt(iElt)	1.0D22	element focal length

Element variables	Default values	Description
h1HOE(iAxis,iElt)		
h2HOE(iAxis,iElt		
IndRef(iElt)	1.0D0	real part of refactive index for element
KcElt(iElt)	0.0D0	element conic constant
KrElt(iElt)	-1.0D22	element conic radius
LensArrayType(iElt)	1	type of lens in array (hex, square)
LensArrayWidth(iElt)	1	width of a single lens in the array
lMon(iElt)	Aperture	scale length for monomial (Zernike) element calcula-tions
MonCoef(iMon,iElt)		monomial coefficients
nCoatElt(iElt)		number of coatings
nECoord(iElt)	-6	number of element coordi-nates
nLayer		number of layers in coating, maximum 20
nObs(iElt)	0	number of obscurations
ObsType(iObs,iElt)	1 (circle)	obscuration type
ObsVec(iAp,iObs,iElt)	(0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0)	obscuration definition infor-mation
OrderHOE(iElt)		diffraction order of HOE to trace
PinHole()		pin hole
pMon(iAxis,iElt)	VptElt	reference for monomial (Zernike) surface calcula-tions

TABLE 11. Element variables

Element variables	Default values	Description
PropType(iElt)	1 (Rays Only)	type of propagation from element iElt to element iElt+1
psiElt(iAxis,iElt)	NO DEFAULT	element principal axis vec-tor
RptElt(iAxis,iElt)	VptElt	element rotation point
RuleWidth(iElt)		grating ruling width
Surface(iElt)		surface type
T Elt(iPert,iElt-Cord,iElt)	(1.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,1.0D0,0.0D0,	element local coordinates transformation matrix

Element variables	Default values	Description
	0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 1.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 1.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,1.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,1.0D0)	
UDSrfCoef(iCoef,iElt)		user defined surface coeffi-cients
UDSrfType(iElt)		user defined surface type
VptElt(iAxis,iElt)	NO DEFAULT	element vertex point
xMon(iAxis,iElt)	xGrid	x-axis for monomial (Zernike) surface calcula-tions
xObs(iAxis,iElt)	NO DEFAULT	orientation of x-axis for each element
yMon(iAxis,iElt)	cross product of psiElt and xMon	y-axis for monomial (Zernike) surface calcula-tions
WaveHOE(iElt)		recording wavelength for HOE
zElt(iElt)	fElt	Fresnel propagation distance

TABLE 11. Element variables

Element variables	Default values	Description
ZernCoef(iZern,iElt)	(0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0)	Zernike coefficients

Element variables	Default values	Description
	(0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0, 0.0 D0,0.0D0,0.0D0) (0.0 D0,0.0D0,0.0D0)	
zMon(iAxis,iElt)	psiElt	z-axis for monomial (Zernike) surface calcula-tions

TABLE 12. Output variables

Output Variables	Defaults	Description
Tout(iLin,iPert)	beam frame at the last element	the output coordinate transformation matrix (proposed when absent; the interactive program asks)
nOutCord	5	number of output coordinates

4.13 Design Optimization (CALIB) Keywords

The CALib command runs the native multi-field design optimizer: a Levenberg–Marquardt least-squares solve over the degrees of freedom of the variable elements, scored at up to 12 fields of view and 6 wavelengths (Appendix C lists the command; the Command Reference carries its dialog and the related commands SETC, AVAR, MVAR, DVAR, VARS, AFOV, DFOV, SFOV, FOVS, WLENS). Its setup is written into the prescription with the keywords of Table 13. Header keywords belong in the source block, before the first `iElt=`; element keywords belong in the block of the element they vary. Every keyword in the table is prescription state: it returns to its default at every OLD or LOAD. SAVE writes the keywords whose values differ from their defaults (header keys in the source block, element keys in their element's block), so the optimization set up on a prescription — interactively or by the bindings' `calib_set_*` calls — transfers through a saved file; a load that reads such a setup says so in one line (`LOAD: CALIB setup in force from the prescription: target SPOT at element 2, 1 field(s), 1 wavelength(s), 1 variable element(s), OptMaxItrs 10 -- VARS/FOVS/WLENS list them; SAVE writes them`). The same settings can be made interactively (SETC, AVAR, AFOV) and from the MATLAB and Python bindings (`calib_set_target`, `calib_set_var_elt`, `calib_set_iter`, `calib_set_tol`, `calib_set_beam`, `calib_set_beam_pos_fov`, `calib_set_beam_wt`; Command Reference Part II).

TABLE 13. Design-optimization (CALIB) keywords. All are reset at load; SAVE writes those whose values differ from their defaults, and a load that reads them announces the setup.

Keyword	Block	Value	Default	Meaning
`OptTarget=`	source	`WFE`, `SPOT`, `ZWF`, `Beam`	`WFE`	The merit. `OPL` is no longer a target; a prescription naming it is refused at load.
`OptTgtElt=` (`OptWFElt=`)	source	element number	nElt − 1	The element at which the merit is evaluated.
`OptTgtWF=`	source	file name; a name ending in `T` is read	none	A target wavefront to

Keyword	Block	Value	Default	Meaning
		as text, any other as binary		match instead of a null.
`OptSpotSize=`	source	size, base units	0	SPOT target: the spot size aimed for (a negative value reads as 0).
`OptZernModes=`	source	n m1 … mn	none	ZWF target: the Zernike modes scored (Noll ordering, 1-based; 11 = primary spherical).
`OptRayGrid=`	source	points across the grid	nGridpts/2	The optimizer's own ray grid, same convention as `nGridpts=`; capped at the model size.
`ChfRayDir=` / `ChfRayPos=` (`OptChfRayDir=` / `OptChfRayPos=`)	source, repeated	one 3-vector each per field	the source's own	The field list: the first pair is the nominal field; up to 12 pairs.
`OptFOVWt=`	source, after the field pairs	w1 … wn	1 each	Field weights (a row is divided by √w); FOVS lists them. Before 2026-10-17 a prescription with no `OptFOVWt=` loaded with every weight 0, read by the solver as 1e-10: merits printed then are 1e-10 × today's for the same solve.
`Wavelen=`, then `OptWaveLen=` (repeated) or `ArrWaveLen=`	source	wavelengths	the source's	The optimization wavelengths, up to 6; the first is the source wavelength.
`OptMaxItrs=` (`OptMxItrs=`)	source	n	10	Iteration cap.
`OptTol=`	source	> 0	1e-12	Convergence tolerance: the absolute change of the rms merit

Keyword	Block	Value	Default	Meaning
				between accepted steps, in the target's units. A value ≤ 0 is ignored with a note.
`OptFEX=`	source	`Y` / `N`	`N`	Evaluate the WFE at the exit pupil found by FEX at each field; needs a Return or Reference at element nElt − 1.
`OptAlg=`	source	`NonLin` / `Lin`	`NonLin`	Levenberg–Marquardt, or the linear (SVD) solve.
`OptSvdSvCut=`	source	cut	machine epsilon	Singular-value cut of the linear solve.
`OptNomSens=`	source	`Y`	off	Run the nominal sensitivity at load.
`OptSavePinv=`, `OptUseSavedPinv=`, `OptPinvFile=`	source	`Y`; `Y`; file name	off; off; `dady0_saved.txt`	Save, or reuse, the linear solve's pseudo-inverse.
`VarSrc=`	source	six flags	none	The source as a variable (TIP TILT CLOCK DX DY PIST); moving the source needs a STOP.
`OptSrcRpt=`	source	x y z	none	Rotation point of a variable source.
`VarElt=`	element	eight flags	none	The element's degrees of freedom: TIP TILT CLOCK DX DY PIST ROC CONIC, nonzero = vary. A linked (double-pass) element cannot be variable; its master moves it.
`VarCons=`	element, after `VarElt=`	lower upper, per varied flag in order	none	Bounds on the varied degrees of freedom; any `VarCons=` switches

Keyword	Block	Value	Default	Meaning
				the solve to the bound-constrained SLSQP driver.
`OptZern=` (`VarZern=`)	element	n t1 … tn	none	Surface Zernike terms as degrees of freedom; marks the element variable if `VarElt=` is absent.
`OptAsph=` (`OptAsphCoef=`)	element, after `VarElt=`	n t1 … tn	none	Aspheric terms as degrees of freedom. Above its `VarElt=` it is ignored with a message.
`OptBeamDir=`	element	dx dy dz (unitized)	none	Chief-ray direction target at this element.
`OptBeamPos=`	element	x y z	none	Chief-ray position target at this element; also makes this element `OptTgtElt`.
`OptBeamPosFov=`	element, repeated	x y z per field, in field order	none	Per-field position targets (up to 12).
`OptBeamSize=`	element	size	none	Beam size target at this element.
`OptBeamRefRayDir=`	element	dx dy dz	none	Reference-ray direction target at this element.
`OptBeamWt=`	element	w	1	Weight of the beam rows against the WFE or SPOT rows (≤ 0 reads as 1).
`OptBeamCentroid=`	element	`Y` / `N`	`N`	The position row reads the centroid of the rays that pass the train to this element, not the chief ray.

Conventions that go with the table:

- **Rows and units.** Every (wavelength, field) block of the least-squares problem holds the target's rows followed by any beam rows. WFE rows are in the target's units; SPOT and beam rows are

in base units. Beam rows ride on any target, so a WFE solve can hold a pupil position or a chief-ray direction at the same time; weight them with `OptBeamWt=`.

- **Stopping.** The solve stops *converged* when two consecutive accepted steps each change the rms merit by no more than `OptTol` and the sum of squares by no more than one part in 10^6; at the *round-off floor* after ten rejected trials in a row; at the *cap* of `OptMaxItrs`; *failed* when an evaluation fails (the chief ray lost) with no accepted step, or *stalled* when no step is accepted and none fails — the start is already a stationary point of the merit. A failed evaluation inside a trial counts as a rejected trial. The CLI prints `Optimization iterations =`, `Final LM merit (chisq) =` and `Stopped: …`; the bindings return the same three through `calib_stats`. The stopping rule does not depend on an earlier CALIB in the same session.

- **Aspheric degrees of freedom.** The differential step of a zero aspheric coefficient (an asphere started from a conic) is sized from the sag at the element's circular aperture (`ApType= 1`). Declare that aperture; without it the step is a fixed small number that is round-off on a prescription in meters, and one line says so.

- **Fields.** Repeat the chief-ray pairs in the source block, one pair per field, the nominal field first, and give `OptFOVWt=` after them (it reads as many weights as there are fields). The FOVS command lists the result.

- **Targets at the exit pupil.** `OptFEX= Y` evaluates the WFE at the exit pupil FEX finds for each field (the chief-ray axis convention of Section 5). The default is off because FEX is applied at element nElt − 1, which must be a Return or Reference surface for that to work.

SECTION 5: Ray-Trace Analysis

MACOS ray-trace commands define a bundle of rays at the light source, and then advance each ray element-by-element through the beam train. Individual rays can be followed through the system, or the entire bundle can be traced to display OPD maps or spot diagrams at particular elements. Placement of reference surfaces using ORS and SRS is introduced. The system exit pupil can be placed using the FEX command. Using the PERturb and MODify commands, individual elements can be altered and the analyses repeated. The commands defined in this section include:

```
DRAW MAP
RAY SPOt
OPD OBS
PERturb PREad
CENter STOP
FEX COORDinates
FFP PFP
POLarization NOPolarizaton SINT
```

5.1 Background

Having loaded the .in-file prescription that describes the optical system to MACOS, the user can trace rays to analyze the system. MACOS provides ray-trace commands that produce useful graphical (or file) outputs, such as SPOT and OPD. As discussed earlier, spot diagrams are graphs of the pierce points of the beam rays on a particular surface. They provide a measure of the geometric beam or image shape and are extremely useful in gauging performance. OPD maps show the wavefront profile at a particular surface. When used with an appropriate reference or return surface, OPD generates the wave-front error map of the system.

MACOS also provides commands to help setup appropriate reference surfaces. These include FEX, ORS and SRS (ORS and SRS are discussed later, in Section 6.3). Both SPOT and OPD provide means to quantify system performance, whether for design, tolerancing or simulation, for instrument-level or system-level analyses.

Detailed summaries of the paths taken by single rays can be generated using the RAY command. The optical train can be visualized using the DRAW command. These commands are useful in verifying design layout and correctness, as well as for single-ray performance measurements.

In addition to these commands, which generate specific data products, MACOS provides commands that can be used to change the system or the beam path in specific ways. The PERturb command implements a kinematically correct single-axis rotation and translation perturbation of an element or the beam source, and can be used to alter the system according to effects of misalignments or disturbances. The CENTER and STOP commands are used to set up the correct beam path through the optical train, by enforcing the system stop. The FEX command sets up return surfaces at exit pupils for OPD evaluation and for diffraction calculations. Commands such as FFP and PFP are

used to find the field angles that place the beam at particular points on an element, and are very useful in simulating imaging systems.

Before we describe the MACOS ray-trace commands, we will summarize how MACOS computes a ray-trace and introduce some of the internal MACOS variables that store ray-trace information. These variables are directly accessible to user programs that use the SMACOS include files.

A full beam ray-trace is initiated when the user issues an OPD or SPOT command (other commands, such as BUILD, INTensity or several of the other diffraction commands also initiate a full-beam ray-trace). Along with the command, the user specifies the element where the requested data is to be generated.

MACOS keeps track of the current position of the beam in the beam train. If the starting element for the trace is the source, then MACOS generates the starting ray states at the source point, using the source prescription data. Otherwise the trace starts where it left off previously. From the starting element, the beam is traced to the specified ending element. MACOS works in 2 loops to trace the beam from the starting element to the end element. The inner loop traces a single ray past each element until it reaches the specified ending element. The outer loop traces successive rays, following the order illustrated by the MAP command, until all rays have been traced.

In MACOS, “tracing rays” means updating specific ray-state variables so that they hold values appropriate for the specified end element. These variables include:

- RayPos(1:3,iRay) stores a 3-vector giving the position of the current segment of ray iRay in global coordinates. The current segment is the one that ends at the last element specified in an OPD, SPOt, BUild command. The last element after a diffraction propagation command (such as INTensity) depends on the type of propagation used just prior to the current element.

- RayDir(1:3,iRay) stores a unit magnitude 3-vector giving the direction of the *next* segment of ray iRay. If the current element is a mirror, RayDir gives the direction of the reflected ray. If it is a refracting surface, RayDir gives the direction of the refracted ray.

- RayL(iRay) stores a scalar giving the *geometric* length (not the optical path length which includes the effect of the index of refraction) of the *current segment* of ray iRay.

- CumRayL(iRay) stores a scalar giving the *cumulative optical* length (includes effects of non-unity index of refraction) of the ray iRay up to and including the current segment. It gives the optical pathlength from the source to the current element.

- RayIndex(iRay)stores the current index of refraction of the ray. These definitions are illustrated in Figure 30.

Repeating a trace of an unchanged system reproduces it bit for bit. The source coordinate frame and the stop aim are re-derived at every trace, but an update smaller than round-off (16 units in the last place) is not applied, so a re-trace never walks the frame by a bit and finite-difference sensitivities see no noise floor from it. An update between 2 and 16 units in the last place that was withheld is reported once per run (`** Note: a ... update of N ulp ... was NOT applied`); such a note means a program is stepping the source direction or the stop position at round-off level.

5.2 Ray-Trace Commands

1. **DRAW**

The DRAW command produces a drawing of the beam train, showing the optical element surfaces and the beam propagating through them. The beam is represented by eleven rays. The elements are drawn in cross-section, using the ray-intercept points from 100 rays across the beam to define the surfaces. The drawing is in a specified (xz, yz or xy) plane, and the surfaces and rays are drawn as the projection of the onto that plane. Tilted or folded systems can sometimes produce strange results! Different colors are used to distinguish between rays and several different element types.

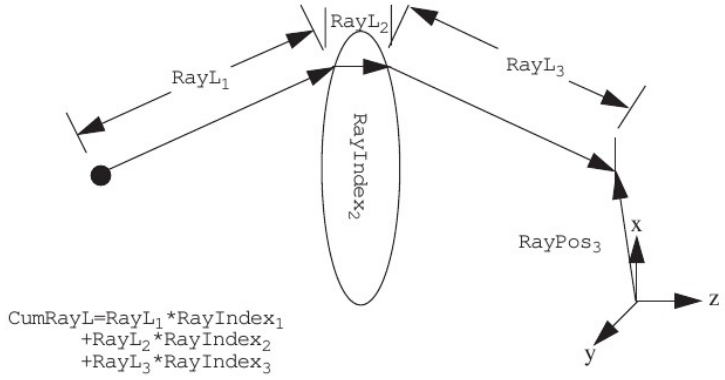


FIGURE 30 Ray state definitions

The user is asked to specify the starting element, the stopping element, and the viewing plane. The following example draws the ImageDemo.in prescription (see Appendix) beam train from the source (element 0) to element 11:

```
Enter first element to include: [0]: 0
Enter last element to include: [13]: 11
Enter drawing plane (XZ, YZ or XY in source coords): [XZ]: xz
```

The result is shown in Figure 31. The system is somewhat aberrated, so the beam does not come to a clean focus! A close-up of refractive elements 6-11 is specified by changing the starting element:

```
Enter first element to include: [0]: 6
Enter last element to include: [13]: 11
Enter drawing plane (XZ, YZ or XY in source coords): [XZ]: xz
```

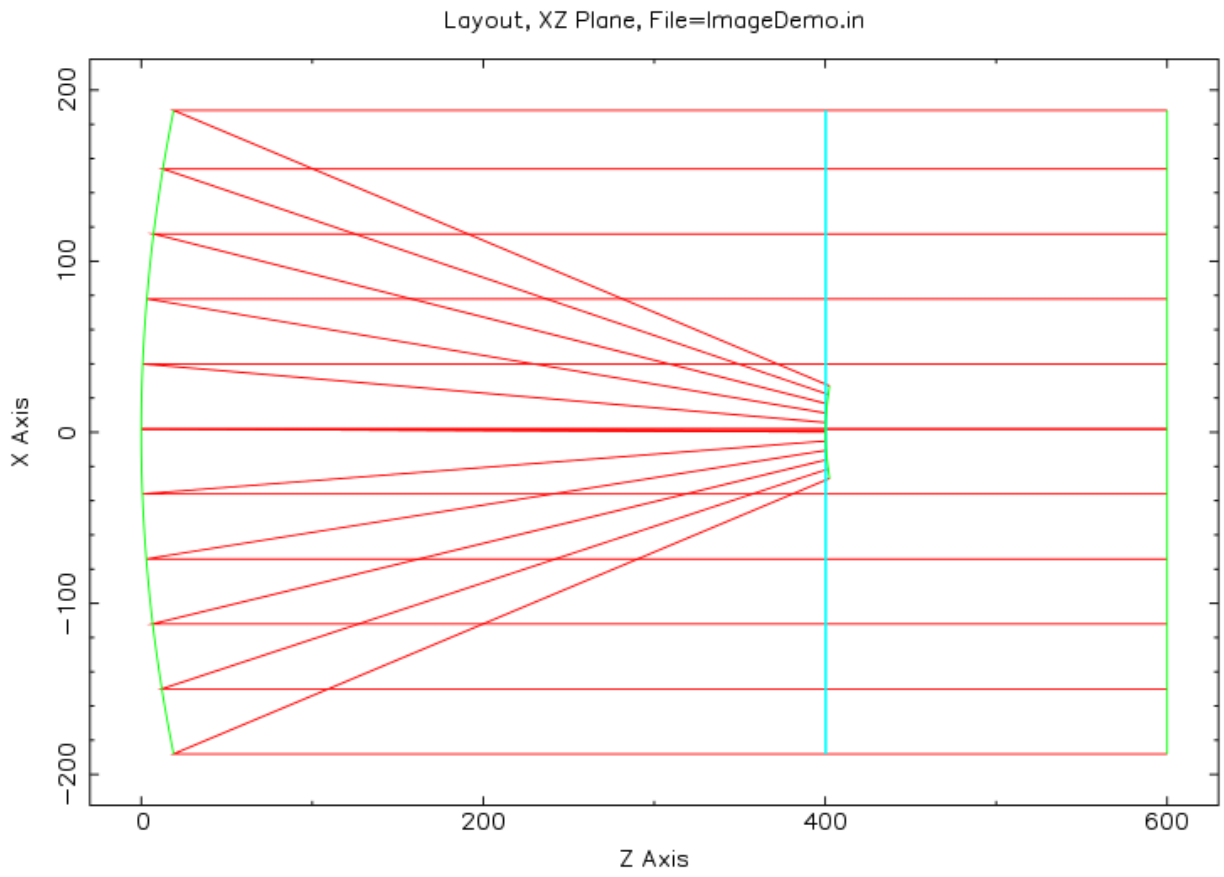


FIGURE 31 DRAW view of ImageDemo.in, the telescope (elements 0-5); the relay view waits on a DRAW repair – a refractor surface draws as two straight segments spanning the beam

2. MAP

Single rays are specified by their "ray ID" number, iRay. This number must be specified in tracing individual rays, printing individual ray partials, or in accessing ray state data directly using the SMACOS include files.

The relative location of each ray in the pupil can be displayed using the MAP command. When there are more than 15 rays across the aperture, only a subset of the ray numbers are shown. The chief ray is always ray number 1 (iRay=1) and is not displayed by MAP. For Cassegrain.in, the MAP command produces the following:

MACOS>**map**

Map of rays at input aperture:

0 84	85 86 87 88 89 90 91 92 93 94	9 5	9 6	0
69 70	71 72 73 74 75 76 77 78 79 80	8 1	8 2	83
0 56	57 58 59 60 61 62 63 64 65 66	6 7	6 8	0
0 43	44 45 46 47 48 49 50 51 52 53	5 4	5 5	0
0 30	31 32 33 34 35 36 37 38 39 40	4 1	4 2	0
0 0	19 20 21 22 23 24 25 26 27 28	2 9	0	0

0 84	85 86 87 88 89 90 91 92 93 94	9 5	9 6	0
0 0	0 10 11 12 13 14 15 16 17 18	0	0	0
0 0	0 0 3 4 5 6 7 8 9 0	0	0	0
0 0	0 0 0 0 0 2 0 0 0 0	0	0	0

MACOS>

MACOS>**map**

The lower left corner is the (x=0, y=0) point for the aperture coordinates defined by the ChfRayDir (=-zGrid) and xGrid vectors. The view is looking into the optical system from the source.

When using MAP with a segmented system, a map of the segments is also printed, showing which segment each ray hits. For SegmentDemo.in, the following map is produced.

Map of rays at input aperture:

```
Hit Map
```

MACOS>

3. **Undefined Rays**

If a MAP command is used after a ray-trace or diffraction command such as OPD, SPOt or INTensity, some of the rays may disappear. This indicates that those rays became *undefined* because the rays:

- missed an optical element
- hit an optical element out of order
- hit two non-sequential surfaces simultaneously

as shown in Figure 32. Undefined rays are called out during the trace command. The surface where a ray became undefined can be pinpointed using the RAY command.

Misses element Must go backwards to hit next element.

Hits elements simultaneously.

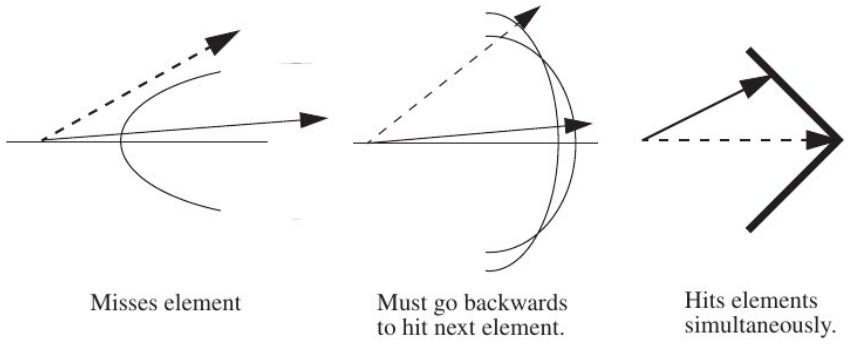


FIGURE 32 Example with defined rays (solid) and undefined rays (dashed)

1. **RAY**

The RAY command traces specific rays and prints the results. RAY has one argument, the number of the ray to be printed (iRay). The chief ray for Cassegrain.in is printed as:

MACOS>**ray**

```
Enter number of ray (1=chief ray, 0=quit):1 Ray 1 is the chief ray.
Ray 1 segment from Element 0 (InputRay) to Element 1 (SecMirOb): Starting point= 0.000000000D+00
0.000000000D+00 -6.000000000D+00
End point= 0.000000000D+00 0.000000000D+00 -5.400000000D+00 Direction= 0.000000000D+00
0.000000000D+00 1.000000000D+00
Segment Length= 6.000000000D-01 Total Length= 6.000000000D-01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 1 is obscured by element 1
Ray 1 segment from Element 1 (SecMirOb) to Element 2 (Primary ): Starting point= 0.000000000D+00
0.000000000D+00 -5.400000000D+00
End point= 0.000000000D+00 0.000000000D+00 -8.881784197D-16 Direction= 0.000000000D+00
0.000000000D+00 1.000000000D+00
Segment Length= 5.400000000D+00 Total Length= 6.000000000D+00
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 1 segment from Element 2 (Primary ) to Element 3 (Secondar): Starting point= 0.000000000D+00
0.000000000D+00 -8.881784197D-16
End point= 0.000000000D+00 0.000000000D+00 -4.061145902D+00 Direction= 0.000000000D+00
0.000000000D+00 -1.000000000D+00
Segment Length= 4.061145902D+00 Total Length= 1.006114590D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 1 segment from Element 3 (Secondar) to Element 4 (Ref_surf):
Starting point= 0.000000000D+00 0.000000000D+00 -4.061145902D+00 End point= 0.000000000D+00
0.000000000D+00 -4.060145902D+00 Direction= 0.000000000D+00 0.000000000D+00 1.000000000D+00
Segment Length= 1.000000000D-03 Total Length= 1.006214590D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 1 segment from Element 4 (Ref_surf) to Element 5 (foc_pln ): Starting point= 0.000000000D+00
0.000000000D+00 -4.060145902D+00
End point= 0.000000000D+00 0.000000000D+00 1.500000000D+00 Direction= 0.000000000D+00 0.000000000D+00
1.000000000D+00
Segment Length= 5.560145902D+00 Total Length= 1.562229180D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
```

This printout gives the start and end point of each segment of the ray in global coordinates. The direction is a unit vector in global coordinates. The segment length refers to the geometric distance between the ray start and end points; it is not scaled by index of refraction. The total length gives a running sum of the length of the ray that is scaled by the index of refraction. The index give the index of refraction for the previous and current element.

To print the lower marginal ray, we see from the MAP command that the iRay number is 2, and respond accordingly:

MACOS>**ray**

```
Enter number of ray (1=chief ray, 0=quit):2
Ray 2 has aperture coordinates 8 and 1.
WF coordinates are 129 and 122.
Adjacent rays are 0, 0, 0, 6.
Ray 2 segment from Element 0 (InputRay) to Element 1 (SecMirOb): Starting point= -4.440892099D-16 -
2.000000000D+00 -6.000000000D+00
End point= -4.440892099D-16 -2.000000000D+00 -5.400000000D+00 Direction= 0.000000000D+00
0.000000000D+00 1.000000000D+00
Segment Length= 6.000000000D-01 Total Length= 6.000000000D-01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 2 segment from Element 1 (SecMirOb) to Element 2 (Primary ): Starting point= -4.440892099D-16 -
2.000000000D+00 -5.400000000D+00
End point= -4.440892099D-16 -2.000000000D+00 -1.851851852D-01 Direction= 0.000000000D+00
0.000000000D+00 1.000000000D+00
Segment Length= 5.214814815D+00 Total Length= 5.814814815D+00
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 2 segment from Element 2 (Primary ) to Element 3 (Secondar): Starting point= -4.440892099D-16 -
2.000000000D+00 -1.851851852D-01
End point= -1.110223024D-16 -4.999999999D-01 -4.096296297D+00 Direction= 7.951199381D-17
3.580901857D-01 -9.336870027D-01
Segment Length= 4.188888889D+00 Total Length= 1.000370370D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
```

```
Ray 2 segment from Element 3 (Secondar) to Element 4 (Ref_surf): Starting point= -1.110223024D-16 -
4.999999999D-01 -4.096296297D+00
End point= -1.098674950D-16 -4.947992095D-01 -4.038085968D+00 Direction= 1.975982230D-17
8.899032834D-02 9.960324902D-01
Segment Length= 5.844219807D-02 Total Length= 1.006214590D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 2 segment from Element 4 (Ref_surf) to Element 5 (foc_pln ):

Starting point= -1.098674950D-16 -4.947992095D-01 -4.038085968D+00 End point= -1.877761358D-26 -
8.456685352D-11 1.500000000D+00 Direction= 1.975982230D-17 8.899032834D-02 9.960324902D-01
Segment Length= 5.560145902D+00 Total Length= 1.562229180D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
```

This printout is similar to the chief ray printout. However, the first three lines are new. They specify the ray grid coordinates, the coordinates of the diffraction amplitude matrix grid point with which the ray is associated, and the iRay numbers of the adjacent rays. The adjacent rays are listed in (-x, +x, -y, +y) order.

The RRAY, PRAY and TPR commands pick reference rays and trace them one at a time. RRAY finds the chief ray and the four marginal rays nearest the +x, −x, +y and −y edges of the source aperture; PRAY finds the chief ray and eight rays on a ring at 0.9 of the aperture radius. Each prints the rays it picked with their XY offsets from the chief ray in source-aperture units and their ray numbers (the same numbers RAY and MAP use). TPR then traces each picked ray from the source to the last element, or to the RayTgtElt element when one is set, and prints where it lands; a ray lost on the way is named. None of the three takes an argument, and TPR refuses until RRAY or PRAY has run. On CassWithExitPupil.in (the detector is element 6):

MACOS>**rray**

```
 Source aperture diameter =       4.0000 none
  Ref. ray XY offsets from chief ray:      1.6923,     -0.1538,   ID =      63
  Ref. ray XY offsets from chief ray:     -0.1538,      1.6923,   ID =     122
  Ref. ray XY offsets from chief ray:     -1.6923,     -0.1538,   ID =      52
  Ref. ray XY offsets from chief ray:     -0.1538,     -1.6923,   ID =       4
 Reference ray indices (chief + 4 marginal) =            1          63         122          52
4
```

MACOS>**tpr**

```
 TPR ray 1 at element 6:   0.000000000000000E+00   0.000000000000000E+00   1.500000000000000E+00
 TPR ray 63 at element 6:  -9.075851181705730E-11   8.250726490910409E-12   1.500000000000000E+00
 TPR ray 122 at element 6:   8.250726490910409E-12  -9.075851181705730E-11   1.500000000000000E+00
 TPR ray 52 at element 6:   9.076017715159423E-11   8.250934657727527E-12   1.500000000000000E+00
 TPR ray 4 at element 6:   8.250934657727527E-12   9.076017715159423E-11   1.500000000000000E+00
```

The marginal rays land within 1e-10 of the chief ray at the focus, the residual of the design. FSR (under the beam set-up commands below) finds the ray nearest a given fractional source position the same way and reports its number and offset. The MATLAB and Python bindings expose the finders as ref_rays; TPR has no binding yet.

2. OPD

The commands that perform full-beam ray-trace analyses are SPOt and OPD. The OPD (for Optical Path Difference) command traces the full bundle of rays to a specified element. At that element it computes an OPD for each ray and plots the beam OPD map. It also computes and prints the RMS OPD for the full beam.

The optical path difference of a ray is the difference between the optical pathlength of that ray and the chief ray. A positive OPD indicates that the ray travelled farther than the chief ray.

By default, then, the OPD is referenced to the chief ray (ray 1). The alternative — the mean pathlength over every valid ray in the aperture — is selected per prescription with `UseChfRay4OPD= N` (the default is `Y`,

and SAVE writes `UseChfRay4OPD= N` only when the mean is in force), per session with the `OPDREF MEAN` command (`OPDREF CHIEF` restores the default; the setting survives later loads), or from the bindings with `opd_ref`. Every OPD run prints the reference it used, e.g. `OPD reference: CHIEF RAY (default)` or `OPD reference: APERTURE MEAN (UseChfRay4OPD= N)`. The two maps differ by one constant per field, so the printed RMS (mean-removed) and P-V do not depend on the choice; `Average OPD` is that constant. The choice matters on a segmented pupil: under the mean reference, moving one segment shifts the aperture mean and attributes a spurious piston to every other segment; under the chief-ray reference the untouched segments read exactly zero. Use the chief ray. If the chief ray is geometrically lost (a surface miss, not an obscuration — an obscured chief ray still serves as the reference), the engine falls back to the mean and says so: `OPD reference: APERTURE MEAN (chief ray lost)`. Note that under the chief-ray reference a valid ray whose path equals the chief ray's has an OPD of exactly zero, the same value the OPD map holds where there is no ray; programs that read the map should take the valid-ray mask (`opd_mask` in mmacos and pymacos) rather than test for nonzero values.

The OPD map as plotted is always a square grid corresponding to the aperture ray grid printed out in the MAP command. To see the true locations of the rays the SPOt command must be used. Rays which are obscured or vignetted by obscurations or apertures on the current or previous elements are not plotted by default. The OBScuration command changes which rays are plotted and is discussed in Section 5.2.

OPD plots will be representative of the system wavefront error *if evaluated at an exit pupil or other appropriate reference surface*. Such a surface will generally be located in the near-field region of the beam, at a point where the ray grid is regular.

NOTE: OPD plots made in a caustic or other region where the beam sampling geometry is disrupted will not auccurately capture system wavefront error. Better results may usually be obtained by using a near- or far-field diffraction propagation to the point of interest, and then plotting the phase of the complex amplitude (see Section 6).

The OPD command has a single argument iElt which is the number of the element where the OPD map or spot diagram is to be computed. The element numbering is as specified when creating the .in-file.

OPD plots are not affected by STRETCH setting. OPD plots are always on a linear scale.

The default plot type is GRAy scale. The plot type can be changed with the following commands: WIRe, SLIce, GRAy, COLumn, ROW, CONtour, TEXt, MATlab,FITs, or BINary. These plotting commands are discussed in more detail in Section 7.4.

The following examples show output of the OPD command with the GRAY, ROW and BINary plot types for Cassegrain.in, at the secondary mirror (element 3), where the wavefront is far from flat.

MACOS>**opd 3**

```
Enter element where OPD is to be evaluated: [0]: 3
OPD: Rays being traced =         1877
OPD: nPassRays =    1756,    DAvgCumL =  90.30828646D-01
 OPD reference: CHIEF RAY (default)
RMS OPD error is   1.545357155D-02
P-V OPD error is   5.323448819D-02
Average OPD is    -3.0317256240508069D-02
A total of      121 rays were lost
  Obscured:            121
    at element    1:       121
Graphics device/type (? to see list, default /xw):
```

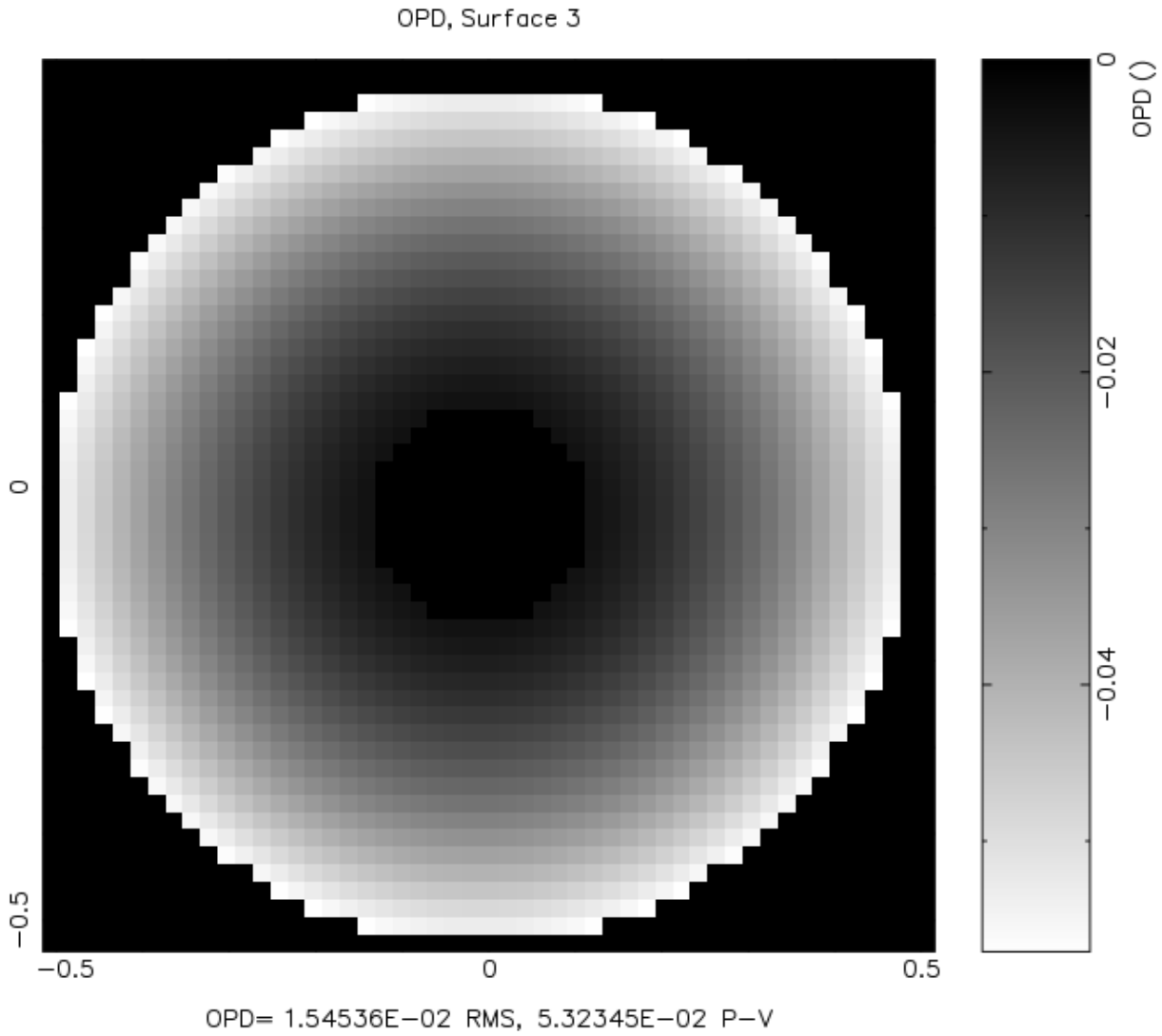


FIGURE 33 OPD GRAy scale plot

MACOS>**row**

```
 Plot type set to ROW
```

MACOS>**opd 3**

```
 Enter element where OPD is to be evaluated: [3]: 3
 ...
 RMS OPD error is   1.545357155D-02
 Enter number of row to be plotted: [26]: 26
```

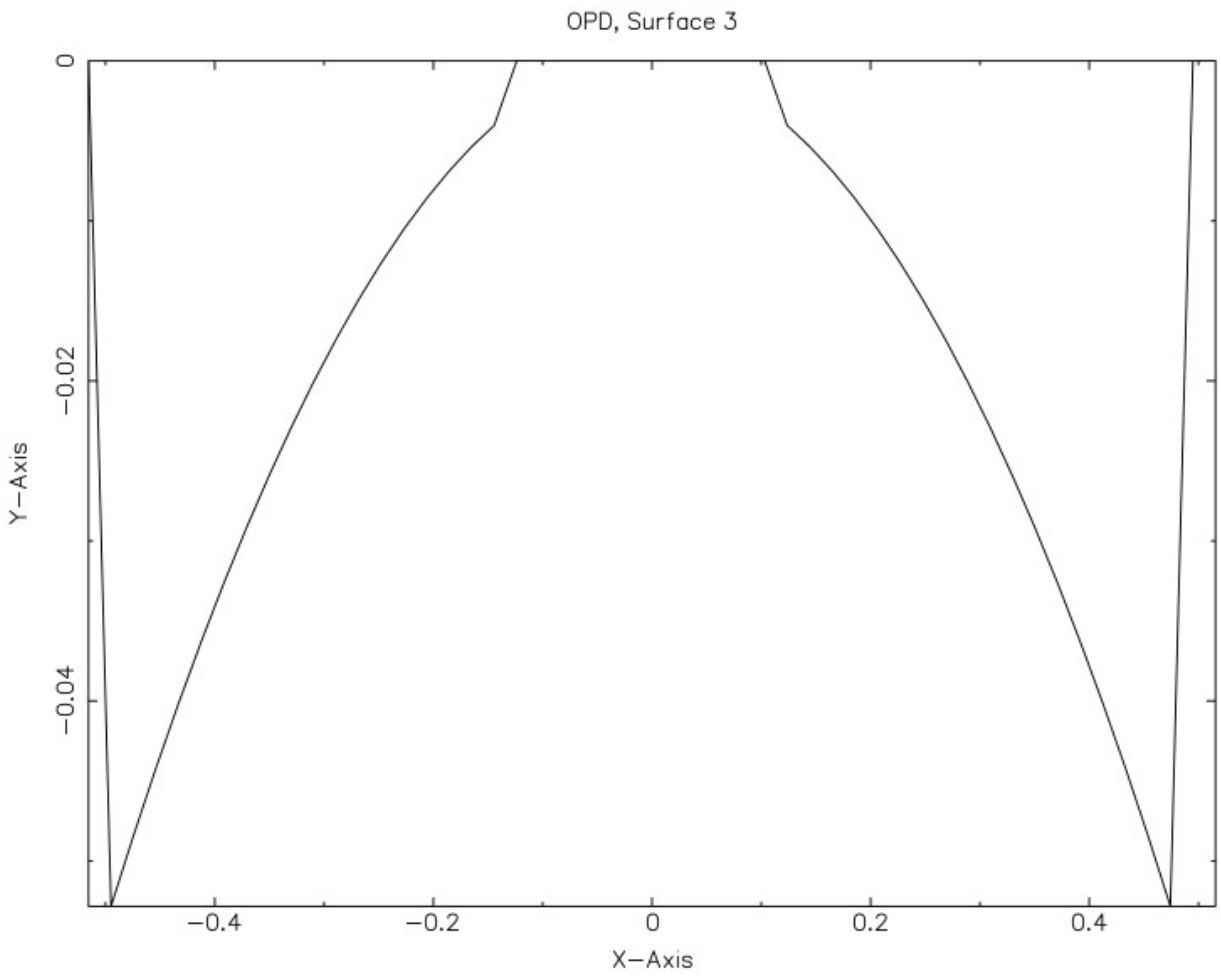


FIGURE 34 OPD ROW plot (the center row; the map is 0 outside the aperture and in the central obscuration)

MACOS>**bin**

Plot type set to BINARY

MACOS>**opd**

```
Enter element where OPD is to be evaluated [3]:3 RMS OPD error is 1.545357E-02
Writing OPD Map, Elt=3
DIRECT ACCESS File=Cassegrain.OPD3

Binary array is of dimension 128 128 0. Magnitude of central pixel is 0.0000000000D+00
```

3. SPOT

Like the OPD command, the SPOt command traces the full bundle of rays to a specified element. At that element it computes and plots a spot diagram.

A spot diagram is an x-y plot across the beam. It puts a dot at the point of incidence of each ray as projected onto a plane across the specified element (see Figure 35).

Rays which are obscured or vignetted by obscurations or apertures on the current or previous elements are not plotted by default. The OBScuration command changes which rays are plotted and is discussed in Section 5.2.

The SPOt command has the argument iElt: the number of the element where the spot diagram is to be computed. In addition, the user is asked to select which output coordinates to use to define the x and y axes. The choices are Tout, Enter or Beam (see Section 4). The Beam frame is built from a trace of the chief ray and two differential rays; a chief ray that is obscured but geometrically defined, such as the on-axis chief ray of a centrally obscured telescope, still defines it. The spot is plotted about the element's reference point by default; the SPCenter command (`chfray`) plots it about the chief ray instead, which makes the spot a measure of shape only: a tilt of a fold mirror then moves the printed chief-ray location but not the spot. SPOt plots an x-y scatter format unless plot types TEXt, BINary, or MATlab are selected, in which case a ".spot" file is written to disk. Figures 4 and 36 show spot diagrams for Cassegrain.in.

diagrams for `Cassegrain.in`.

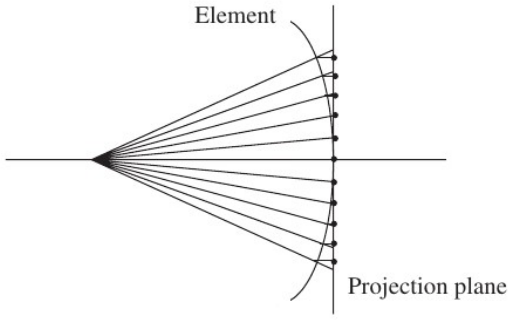


FIGURE 35 Spot on the projection plane

MACOS>**spot**

```
Enter element where spot diagram is to be computed [0]:4 Enter output coordinate option (Tout, Enter
or Beam): [Tout]: Tracing 150 rays...
Compute time was 0.3086 sec
Graphics device/type (? to see list): /xw
Chief ray location: x= 0.0000000D+00 y= 0.0000000D+00 Centroid of spot diagram: x=-5.2080625E-18 y=
0.0000000E+00 MACOS>
```

4. OBSCURATION

By default, the SPOt and OPD commands do not plot rays which are obscured or vignetted by obscurations or apertures on the current or previous elements. The OBScuration command is used to change this default.

```
OBS has three settings. OBS=ALL plots every ray -- even rays that are obscured. OBS=POSITIVE, the
default, plots only rays that are not obscured on the current or on any previous element.
OBS=NEGATIVE plots only those rays that are obscured some-
```

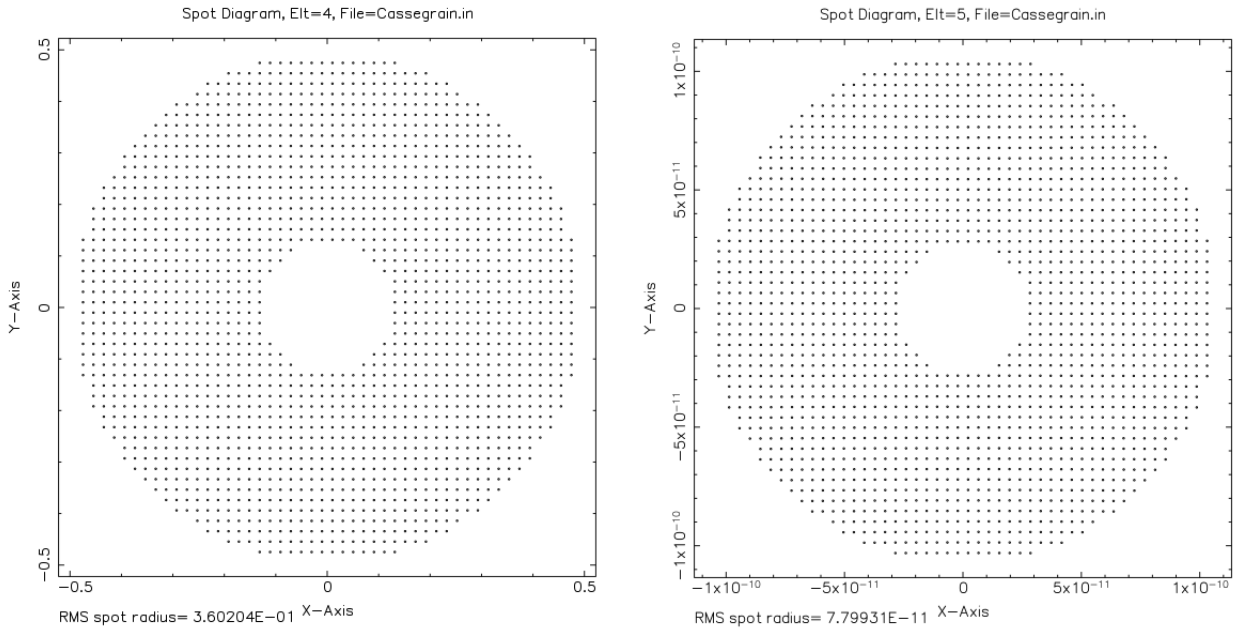


FIGURE 36 Exit pupil and detector SPOt diagram for Cassegrain.in.

where upstream of the specified element. Of course, rays that become undefined upstream are not plotted under any of these options.

Figure 37 shows spot diagrams of the Cassegrain telescope, for comparison with Figure

36. The spot diagram on the left is plotted with OBS=ALL, showing all rays. The spot diagram on the right is plotted with OBS=NEGATIVE, showing only obscured rays.

MACOS>**spot**

```
Enter element where spot diagram is to be computed [4]:5 Enter output coordinate option (Tout, Enter
or Beam): [Tout]: Tracing 150 rays...
Compute time was 0.1211 sec
Chief ray location: x= 0.0000000D+00 y= 0.0000000D+00 Centroid of spot diagram: x=-1.1055660E-16 y=-
1.1055660E-16 Type <RETURN> for next page:
```

MACOS>**obs**

```
Enter ray-trace obscuration option (ALL, POSITIVE, NEGATIVE) [POSITIVE]: ALL
```

All rays plotted on spot diagrams

MACOS>**spot**

```
Enter element where spot diagram is to be computed [5]:5 Enter output coordinate option (Tout, Enter
or Beam): [Tout]:
Chief ray location: x= 0.0000000D+00 y= 0.0000000D+00 Centroid of spot diagram: x= 7.1761210E-17 y=
7.1761210E-17 Type <RETURN> for next page:
```

MACOS>

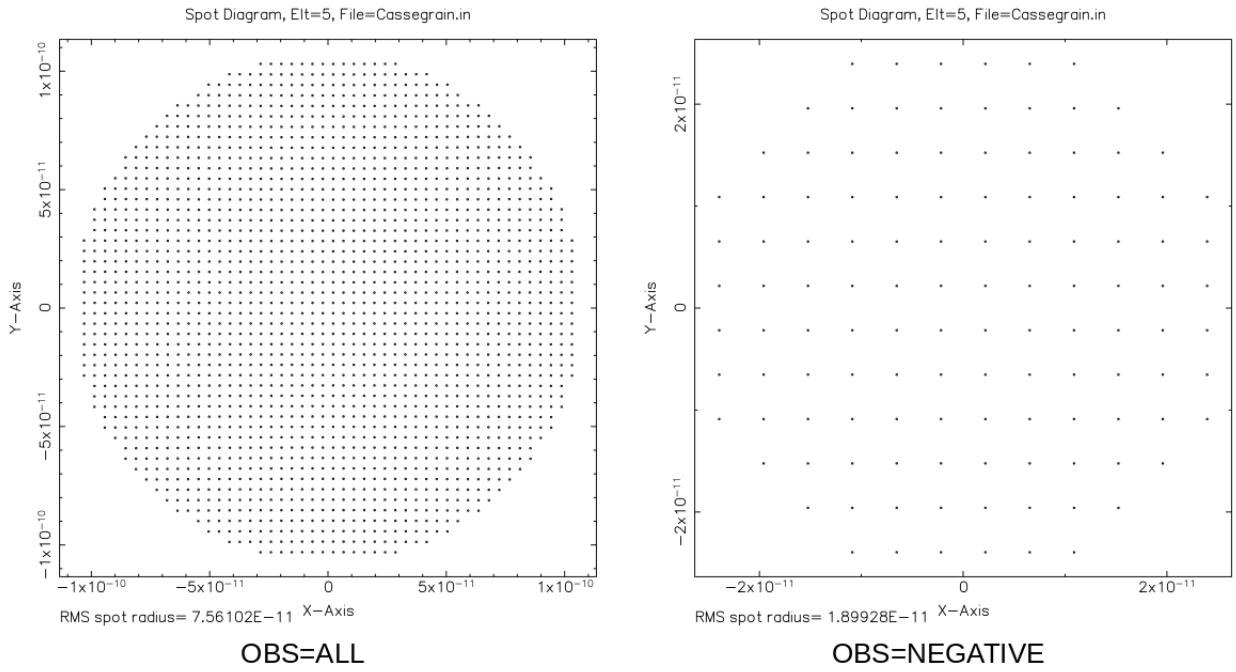


FIGURE 37 Exit pupil Spot diagrams with alternate OBS settings

5.2.1 SEGRAYTRACE

Trace a single ray that passes through the geometric center of a chosen segment element, printing the ray state at every downstream surface. Useful for verifying that segmented-mirror prescriptions place segments where you expect, by comparing the ray interception point to the segment's rotation point (its RptElt). The ray starting point is derived from the source segment coordinates.

```
MACOS> SEGRAYTRACE
Enter segment number (0=quit): [0]: 5
Ray 1 segment from Element 0 (InputRay) to Element 5 (Seg5):
Starting point= ...
End point = ... (should be RptElt(:,5))
Direction = ...
...
Enter segment number (0=quit): [0]: 0
MACOS>
```

The source ray's position is automatically offset from ChfRayPos so the resulting ray (parallel to ChfRayDir for a collimated source, or aimed from ChfRayPos for a point source) lands exactly at the segment's center. The segment-dispatch path is forced to fire only on the chosen segment via RayToSegMap; other segments are skipped.

If SegMirMaker utility was used to generate the segment prescription, the trace endpoint at the segment's element should match RptElt to within SFFZPSolve's tolerance (~1e-11 mm). Larger mismatches diagnose generator bugs:

- ~180-deg rotational mismatch (e.g. Seg2's endpoint matches Seg5's RptElt): the segment-numbering psi-flip in the generator didn't fire when it should have.
- ~mm radial mismatch in the surface-normal direction: the generator computed RptElt on the conic only, missing the FF Zernike sag.

The command requires an Rx loaded with at least one Segment element; otherwise it prints ‘Element N is not a Segment’ and re-prompts.

5.2.2 Per-ray status reporting

Every ray in a trace records its terminal status:

OK (0) – successfully terminated at last requested element

Obscured (1) – blocked by an obscuration or aperture

Miss (2) – missed a surface (no intersection found)

Bracket (3) – root-finding bracket failure on a composite (FreeForm / aspheric / grid) surface

MaxIter (4) – root-finder hit its iteration limit

Undef (5) – initial state, never updated

The end-of-trace WARN summary breaks down failures by category and records, for each ray, the element of its first failure; for an obscured ray that is the element that clipped it, and WARN lists the obscured count per element. A trace restarted part way through the train (an OPD after an earlier trace) keeps the statuses of the earlier part. Bracket / iter messages are throttled after the first 20 to keep the terminal usable when many rays fail. The bindings return the per-ray status and element (`ray_status_get`; mmacos `get_ray_status`).

5.3 Beam Set-Up Commands

The next several commands are designed primarily to help the user correctly set up particular computations, such as computation of beam wavefront error, or diffraction computations. These commands help the user implement perturbations of the optical train to evaluate performance impacts of misalignments, different field angles, or “zoom” settings. The STOP command specifies the system stop, either at an element or at a point in global coordinates, to help in defining the beam. It is required by other commands, such as PERTURB, FFP, PFP and FEX, that might move the source. The CENTER command finds source positions that cause the beam to be centered at a specified element. The FFP and PFP commands find source field angles that place the beam at a particular point on a particular surface, such as on a detector. The FEX command finds the system exit pupil and places a return surface at that point. These commands, together with the ORS and SRS commands discussed in the next section, provide a full capability for correctly setting up most ray-trace and diffraction analyses.

1. **STOP**

The lateral position of a beam propagating through a system is constrained by the presence of a system stop. The system stop is an aperture somewhere in the system through which the beam passes. It defines the beam position and (usually its shape), as per Figure 38. Certain commands (PERturb, FFP, PFP and FEX) require knowledge of the stop position to enforce the correct beam geometry.

The system stop is specified using the STOp command. STOp does not center the beam in the stop (CENTEr can be used to do this). It merely defines the system stop position. This information is used to constrain the beam when setting exit pupils or perturbing the source.

System stop aperture defines beam position for changing field angles

Changing field angle moves image position

source.

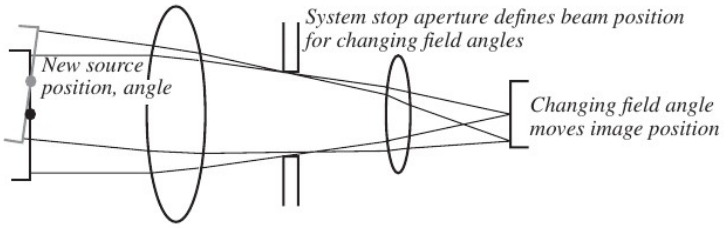


FIGURE 38 System stop

STOp has 2 or 3 arguments, depending on whether the stop is to be specified at an element or at a point in object space. The first argument is a character string specifying whether the stop is to be tied to a particular element (ELT) or to a particular point in object space (OBJ).

If the former, the next parameter requested is the element number. This is followed by a request for an offset vector: a 2-vector in local beam coordinates giving the center of the stop with respect to the vertex of the element (VptElt).

If the stop is to be specified in object space, the next request is for the 3-vector coordinates of the stop position. The three values may be typed on one line separated by commas or spaces; a short line is completed from the next tokens, with a note saying how many values are still expected.

The stop element may be a Segment of a segmented mirror (the chief ray is mapped to that segment); it may not be a non-sequential element, which is refused by name. In segmented prescriptions every segment's VptElt is the parent vertex, so an offset of 0,0 on any segment gives the same stop; a segment's own center needs an offset measured from its RptElt. An element stop rebuilds the source coordinate frame about the new chief ray but keeps the frame's handedness (a left-handed xGrid, yGrid, ChfRayDir triad stays left-handed), so the ray grid of a segmented source stays on its segments.

MACOS>**stop**

The following examples illustrate the setting of stops for the Cassegrain.in example prescription. The first example sets the stop at the nominal location in 3-space of the primary mirror vertex:

```
Stop at ELT or OBJect point? [OBJ]: object
Enter stop position in object space (x,y,z): [0.,0.,0.]:0,0,0 MACOS>
```

MACOS>**stop**

The second example puts the stop at the secondary mirror. The STOp command computes the equivalent object-space stop position:

```
Stop at ELT or OBJect point? [OBJ]: elt Enter system stop element number: [1]:3 Enter offset vector
(dx,dy): [0.,0.]:0,0
Computed StopPos= 0.000000000D+00 0.000000000D+00 1.637981902D+01 MACOS>
```

2. **CENTER**

The CENTEr command solves for a new source position that places the beam at a specified point on a specified element. This command is useful for enforcing the system stop; it has other applications as well. This command has two arguments: the element number of the element where the beam is to be placed; and the offset of the center of the beam from the vertex of the element. The CENTEr comand changes the source chief ray position ChfRayPos but not the field angle.

The following example uses the Cassegrain.in example to center a perturbed beam, with the result shown by tracing the chief ray to the specified element:

MACOS>**center**

```
Enter system stop element number: [1]:2 Enter offset vector (dx,dy): [0.,0.]:0,0
Beam centered in 2 iterations, error= 3.231486333D-15 MACOS>ray
Enter number of ray (1=chief ray, 0=quit): [0]: 1
Ray 1 is the chief ray.
Ray 1 segment from Element 0 (InputRay) to Element 1 (SecMirOb): Starting point= 6.055112155D-17
5.000007357D-03 -5.000005690D+00
End point= 6.055112155D-17 4.200001400D-03 -4.200000000D+00 Direction= 0.000000000D+00 -9.999998333D-
04 9.999995000D-01
Segment Length= 8.000060899D-01 Total Length= 8.000060899D-01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
Ray 1 is obscured by element 1
Ray 1 segment from Element 1 (SecMirOb) to Element 2 (Primary ): Starting point= 6.055112155D-17
4.200001400D-03 -4.200000000D+00
End point= 6.055112155D-17 -3.230922474D-15 0.000000000D+00 Zeroed!
Direction= 0.000000000D+00 -9.999998333D-04 9.999995000D-01
Segment Length= 4.200002100D+00 Total Length= 5.000008190D+00
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
(etc...)
```

3. COORD

The COOrdinates command computes beam coordinates at any specified element. Beam coordinates are the projection onto that element of the xGrid and yGrid axes defined at the source element (see previous section). The COORDinates command takes

a single argument: the element number iElt. It prints out two orthogonal unit vectors in

MACOS>**coord**

response. These vectors are the projection onto the surface at the chief ray of the xGrid

and yGrid axes.

These coordinates are computed using four differential chief rays. The first two differ slightly in x- and y-*angle* from the nominal chief ray. In the near-field of the source, such as at a pupil element, the incidence points of these rays on the element differ enough to indicate the projected axes directions. The second two differential chief rays differ in x- and y-*position* from the nominal chief ray. They are used to indicate axes direction in the far-field of the beam, such as at a focal plane.

Note: The beam coordinates may undergo a sign change in the vicinity of a focus. The unit vectors will be correct and consistent, however, their signs may abruptly flip. Use common sense in interpreting the results of the COORD command!

The following example uses the Cassegrain.in example, determining the coordinates projection onto the secondary obscurations (iElt=1), the exit pupil (iElt=5), and the detector (iElt=6):

```
Enter element number: [1]:1
Computed xLocal= -1.000000000D+00 0.000000000D+00 0.000000000D+00 Computed yLocal= 0.000000000D+00 -
1.000000000D+00 0.000000000D+00 Computed zLocal= 0.000000000D+00 0.000000000D+00 1.000000000D+00
```

MACOS>coord 5

```
Enter element number: [1]: 5
Computed xLocal= -9.999999998D-01 0.000000000D+00 2.229180881D-05 Computed yLocal= 0.000000000D+00 -
9.999999998D-01 2.229180881D-05 Computed zLocal= 2.229180880D-05 2.229180880D-05 9.999999995D-01
```

MACOS>coord 6

```
Enter element number: [1]: 6
Computed xLocal= -1.000000000D+00 0.000000000D+00 0.000000000D+00 Computed yLocal= 0.000000000D+00 -
1.000000000D+00 0.000000000D+00 Computed zLocal= 0.000000000D+00 0.000000000D+00 1.000000000D+00
```

MACOS>pert 6

4. **FFP**

The find field point (FFP) command finds the field angles for which the chief ray intersects a specified element at a specified point. It resets both the source position and angle (ChfRayAng and ChfRayPos) to both satisfy the system stop condition and to place the beam at the given point.

The FFP command has two arguments: the element number of the surface at which the chief ray intercept is specified; and the offset of the chief ray from the vertex of the target element in global coordinates. The STOP command must be used prior to invoking FFP.

The following example uses the Cassegrain.in example, finding the beam that offsets an image by (1e-3, -2e-3):

MACOS>**stop**

```
Stop at ELT or OBJect point? [Elt]: obj 0,0,0
Enter stop position in object space (x,y,z): [0.,0.,0.]: 0,0,0 MACOS>ffp 6
Enter element number: [6]:
Enter offset vector in global units (dx,dy): [0.,0.]:1e-3,-2e-3 Field angle finder results:
Did 1 iterations, error= 6.936774704D-11
Old dx= 0.000000000D+00 New dx= 1.000000031D-03 0.000000000D+00 -2.000000062D-03
Old crd= 0.000000000D+00 New crd= 4.458354140D-05 0.000000000D+00 -8.916708280D-05
1.000000000D+00 9.999999950D-01 Old crp= 0.000000000D+00 New crp= -2.229177070D-04 0.000000000D+00
4.458354140D-04
-5.000000000D+00 -4.999999975D+00
Accept the new chief ray? [YES]: yes
```

MACOS>**ray 1** ***Trace ray to show new incidence point…***

```
Enter number of ray (1=chief ray, 0=quit): [0]: 1 Ray 1 is the chief ray.
Ray 1 segment from Element 0 (InputRay) to Element 1 (SecMirOb): Starting point= -2.229177070D-04
4.458354140D-04 -4.999999975D+00
End point= -1.872508748D-04 3.745017496D-04 -4.200000000D+00 Direction= 4.458354140D-05 -
8.916708280D-05 9.999999950D-01
Segment Length= 7.999999791D-01 Total Length= 7.999999791D-01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
(etc...)
Ray 1 segment from Element 5 (ExitPupi) to Element 6 (Detector): Starting point= 1.812075347D-04 -
3.624150694D-04 -4.060145887D+00
End point= 1.000000031D-03 -2.000000062D-03 1.500010000D+00 Direction= 1.472607007D-04 -2.945214014D-
04 9.999999458D-01
Segment Length= 5.560156189D+00 Total Length= 1.462230217D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
```

MACOS>

5. **PFP**

The pixel field point (PFP) command is the same as the FFP command, in that it finds the field angles for which the chief ray intersects a specified element at a specified point. It differs in that the incidence point is specified in pixel coordinates. PFP resets both the source position and angle (ChfRayAng and ChfRayPos) to both satisfy the system stop condition and to place the beam at the target point.

The PFP command has two arguments: the element number of the surface at which the chief ray intercept is specified; and the offset of the chief ray from the vertex of the target element in global coordinates. The STOP command must be used prior to invoking PFP.

The following example uses the Cassegrain.in example, finding the beam that puts an image at pixel location (20, -100):

MACOS>**PFP 6**

```
Enter element number: [6]:6
Enter pixel size for placing image on pixel array: [0.]:15e-4 Enter image position in pixel units
(dx,dy): [0.,0.]:20,-100 Field angle finder results:
Did 2 iterations, error= 1.581159622D-12
Old dx= 1.000000031D-03 New dx= 3.000000000D-02
-2.000000062D-03 -1.500000000D-01 Old crd= 4.458354140D-05 New crd= 1.337301749D-03
-8.916708280D-05 -6.686508743D-03
9.999999950D-01 9.999767508D-01 Old crp= -2.229177070D-04 New crp= -6.686508743D-03 4.458354140D-04
3.343254372D-02
-4.999999975D+00 -4.999883754D+00
Accept the new chief ray? [YES]:
```

MACOS>**ray 1** ***Trace ray to show new incidence point…***

```
Enter number of ray (1=chief ray, 0=quit): [0]: 1 Ray 1 is the chief ray.
Ray 1 segment from Element 0 (InputRay) to Element 1 (SecMirOb): Starting point= -6.686508743D-03
3.343254372D-02 -4.999883754D+00
End point= -5.616797930D-03 2.808398965D-02 -4.200000000D+00 Direction= 1.337301749D-03 -
6.686508743D-03 9.999767508D-01
Segment Length= 7.999023513D-01 Total Length= 7.999023513D-01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
(etc...)
Ray 1 segment from Element 5 (ExitPupi) to Element 6 (Detector): Starting point= 5.436452666D-03 -
2.718226333D-02 -4.060076800D+00
End point= 3.000000000D-02 -1.500000000D-01 1.500010000D+00 Direction= 4.416714741D-03 -2.208357371D-
02 9.997463720D-01
Segment Length= 5.561497351D+00 Total Length= 1.462402454D+01
Index(i-1)= 1.000000000D+00 0.000000000D+00 Index(i)= 1.000000000D+00 0.000000000D+00
```

MACOS>

6. **FEX**

The Find EXit pupil (FEX) command computes the location of the system exit pupil and places an optimized spherical reference surface at that point. By “optimized,” we mean that the spherical reference surface is correctly set up to perform a far-field diffraction propagation to the next element, which is usually a detector.

To use FEX, the prescription must be set up with two return surfaces followed by a detector focal plane or reference surface. The first return surface is placed at the location of the focal plane detector, and the second is placed in the vicinity of the exit pupil. FEX locates the second precisely at the exit pupil location and computes its orientation and curvature based on the position of the detector element.

FEX works for systems where the exit pupil is located in front of the detector. It is not useful for telecentric systems (where the exit pupil is at infinity) or for cases where the exit pupil is located behind the detector.

FEX traces five rays: the nominal chief ray and four differential chief rays whose field angles differ from it by about 1 arcsecond (5e-6 rad), in the plus and minus directions about each of two perpendicular axes. All are traced to the detector return surface and then backwards to find their intersections with the nominal chief ray; the mean of the four crossing points defines the center of the exit pupil. Averaging the plus and minus probes cancels the term linear in the probe angle, and averaging the two axes takes the medial of the tangential and sagittal pupils, so the result does not depend on the orientation or handedness of the source coordinate frame (before 2026-09-08 a single probe about xGrid was used). The parameters of the exit pupil (the second return surface) are set as follows:

- vertex, VptElt, to the center of the exit pupil

- radius of curvature, KrElt, to the chief-ray distance from the exit pupil to the next element (iElt+1), whatever that element is: a focal plane, a mask or a reference surface. When iElt+1 is a curved focal surface the chief ray is intersected with the surface itself, not its tangent plane; the plane would read as defocus in the off-axis OPD (zero on axis, which is why it goes unnoticed).
- principal axis, PsiElt, to the chief ray direction, ChfRayDir

The result is a reference sphere aligned with the spherical wavefront converging to the point where the chief ray hits the detector. This configuration correctly models diffraction images at that point field point. If the optical system is perturbed, the FEX command must be rerun.

The axis of the reference sphere is the chief ray by default. The CENTROID command aims it at the centroid of the beam at iElt+1 instead, and CHIEFRAY restores the default; the prescription keyword `FEXCentroid= N` vetoes the centroid for a deck. On an obscured, segmented or aberrated beam the centroid walks off the chief ray, so the centroid axis tilts the sphere and reads as tip and tilt (and, against a chief-ray OPD reference, piston) in the OPD. Leave the default. Each FEX run prints the axis it used and the crossing it found, `EP crossing = mean of N probes; spread S`; the spread is the difference between the tangential and sagittal pupils, a measure of pupil astigmatism.

FEX guards its own setup and says so. If the four differential chief rays are parallel to the nominal one (a telecentric system) the crossing is undefined; FEX keeps the element where it is and makes it flat. If the reference sphere would be smaller than the beam footprint at the exit pupil (marginal rays would miss it) FEX switches to the legacy radius, the distance to the preceding focal return, and reports the two values. If some probe rays are lost it averages the survivors with a warning. The SXP command (Set eXit Pupil) is FEX without these guards and gives the same sphere on every prescription checked; XPS writes the per-ray crossing cloud of the exit pupil for a pupil-quality evaluation.

FEX requires two arguments: the number iElt of the exit pupil return surface and the location in global coordinates of the vertex of the aperture stop.

For example, we added two return surfaces to the Cassegrain.in prescription just before the detector using a text editor (see Section 4.9). The first is a return surface at the detector location. The second is close to the exit pupil. The entire .in-file is in Appendix A.1.4.

MACOS>**show 5**

```
Enter number of element (0=aperture): [0]: 5

    iElt=  5
 EltName=  rtn1
 Element=  Return
 Surface=  Conic
    fElt=  1.000000000D+22
    eElt=  0.000000000D+00
   KrElt=  -1.000000000D+22
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00 0.000000000D+00 1.000000000D+00
  VptElt=  0.000000000D+00 0.000000000D+00 1.500000000D+00
  RptElt=  0.000000000D+00 0.000000000D+00 1.500000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  None
    zElt=  0.000000000D+00
PropType=  Geometric
 nECoord=  -6

nECoord= -6
```

MACOS>**show 6**

```
Enter number of element (0=aperture): [0]: 6

    iElt=  6
 EltName=  rtn2
 Element=  Return
 Surface=  Conic
    fElt=  6.000000000D+00
    eElt=  0.000000000D+00
   KrElt=  -6.000000000D+00
   KcElt=  0.000000000D+00
  psiElt=  0.000000000D+00  0.000000000D+00 1.000000000D+00
  VptElt=  0.000000000D+00  0.000000000D+00 -4.500000000D+00
  RptElt=  0.000000000D+00  0.000000000D+00 -4.500000000D+00
  IndRef=  1.000000000D+00
  Extinc=  0.000000000D+00
    nObs=  0
  ApType=  None
    zElt=  6.000000000D+00
PropType=  Geometric
 nECoord=  -6
```

Then the exit pupil is optimized:

MACOS>**fex**

```
Enter number of exit pupil return surface: [5]:6 Tracing 173 rays...
Chief ray location: x= 0.0000000D+00 y= 0.0000000D+00 z=-
4.5000000D+00
Centroid location: x= 7.9797280D-17 y= 8.6736174D-17 z=-
4.4875916D+00
Centroid offset from chief ray: x=-7.9797280D-17 y=-8.6736174D-17 z=-1.2408352D-02
Exit pupil finder results:
Old f= 6.000000000D+00 New f= 6.790667844D+00

Old z= 6.000000000D+00 New z= 6.790667844D+00 Old psi= 0.000000000D+00 New psi= 9.936451891D-17
0.000000000D+00 1.080049119D-16
1.000000000D+00 1.000000000D+00 Old Vpt= 0.000000000D+00 New Vpt= 0.000000000D+00 0.000000000D+00
0.000000000D+00
-4.500000000D+00 -5.290667844D+00
Accept the new element? [YES]:
```

MACOS>

7. **PERTURB**

The PERturb command allows the user to change the current optical prescription by rotating or translating the light source or any of the optical elements. PERturb accepts as arguments rotation and translation perturbation vectors, which it applies directly to the prescription data. Rotations of an element are taken about the prescription rotation point RptElt. Subsequent ray traces or propagations will use the new prescription.

PERturb differs from MODify in that it changes only the position and orientation of elements (MODIFY can be used to change any parameter in the prescription). PERturb is also different from MODify in that PERturb uses the rotation point and proper kinematics to apply rotational perturbations (PERturb rotations are relative to the old orientation) and MODify specifies a absolute location. PERturb is not limited to small motions.

To reset the prescription to its unperturbed condition, it is necessary to explicitly reverse the PERturb commands executed previously or to reload the prescription using the OLD command. Due to the noncommutativity of rotations, the latter method is to be preferred if more than one rotation has been applied.

The PERturb command takes arguments specifying the element to be perturbed and the perturbation vectors. The element is specified by number iElt which is in the range: 0 (the source) to nElt (the number

of elements in the active prescription). Units for rotation perturbations are radians. The units of the translation perturbations are in the chosen base units (meters, inches, whatever).

Rotations are applied using Euler axis/angle parameters (Ref. Hughes). The axis of the rotation is given by the direction of the rotation perturbation vector. The angle is given by its magnitude. This approach is used in preference to Euler angles or quaternions as it provides a smooth transition to the usual small-angle linear approximations.

If iElt=0, indicating the source is to be perturbed, the user is prompted for the location of the aperture stop in global coordinates. For instance to look 0.5 deg (8.72664d-3 rad) off the x-axis, a rotation about the y-axis is commanded:

MACOS>**perturb**

```
Enter element to be perturbed: [0]:0
Enter rotational perturbation vector (x,y,z): [0.,0.,0.]:0,8.72664d-3,0 Enter translational
perturbation vector (x,y,z): [0.,0.,0.]:0,0,0 Computing new perturbed system parameters
```

MACOS>

If iElt>0, indicating a regular element, the user is asked whether the perturbation vectors are in local element coordinates or in global coordinates. If in local coordinates, the user is prompted for a single 6-vector perturbation in those coordinates. If in global

coordinates, the user is prompted for a rotational perturbation 3-vector, then a translational 3-vector. For instance, to translate the secondary mirror in Cassegrain.in:

MACOS>**pert**

```
Enter element to be perturbed:3
Enter coordinate system for perturbation (ELEMENT or GLOBAL): [GLOBAL]: Enter rotational perturbation
vector (x,y,z): [0.,0.,0.]:0,0,0
Enter translational perturbation vector (x,y,z): [0.,0.,0.]:0,0,1d-6 Computing new perturbed system
parameters
```

MACOS>

8. **PREAD**

The PREad command reads element perturbations from a specified datafile. MACOS assumes that the name of the perturbation file is infilename.pert. If this file does not exist, MACOS prompts for ther perturbation file name.

The perturbation file must be in the following all numeric format. The perturbations must be in global coordinates. The order of the perturbation vectors in the .pert-file must be identical to the order of the elements (the first perturbation vector corresponds to the first element, the sixth perturbation vector corresponds with the sixth element). Each element must have a perturbation vector. If an element is unperturbed, then a string of zeroes should be entered as its perturbation vector. The perturbation vectors are rotational perturbations and translational perturbations. The 6 components of the perturbation vector must be separated by spaces.

MACOS>**pre**

```
File Cassegrain.pert read. MACOS>
```

9. **ATMOS**

MACOS provides an atmospheric phase disturbance function with the ATMos command. ATMos imposes a ray pathlength and direction perturbation on the beam at a specified element. The ray

perturbations are derived from a Kolmogorov phase disturbance model, parameterized by the atmospheric turbulence coherence length r0, and driven by a random number generator. The seed for the random number generator can be set using the SEED command.

ATMos provides two options for specifying the atmospheric grid. The RAY option treats each ray as an independant but correlated random variable, determining the phase disturbance (and so the direction and OPD) for each ray directly from the Kolmogorov generating function. The ATM option implicitly defines a coarser grid of “atmospheric states” than the ray grid provides. Essentially, the phase disturbance is determined for cells whose size is specified by the user, but which will be in general larger than the ray grid cell size. Ray direction and OPD values are computed by interpolating from the surface defined by the atmospheric phase states to the ray grid cells.

ATMOS takes four or five arguments, depending on how the grid is to be specified (RAY or ATM). An example of the use of ATMOS in an adaptive optics system model is provided in Appendix A.4. A sample dialog showing use of ATMOS follows:

MACOS>**atm**

```
Enter number of atmosphere phase screen element: [0]:1 Enter atmosphere r0: [12.4492]: 24
Enter atmosphere wavelength: [2.854331E-05]: 2.854331E-05 Enter atmosphere tilt participation (0-1):
[1.00000]:0 Use RAY grid or separate ATMospheric grid? [RAY]: atm
Computed xLocal= 0.000000000D+00 -1.000000000D+00 0.000000000D+00 Computed yLocal= -1.000000000D+00
0.000000000D+00 0.000000000D+00 Computed zLocal= 0.000000000D+00 0.000000000D+00 -1.000000000D+00

Tracing 12113 rays... Compute time was 0.2266 sec
Enter grid spacing: [1.56840]:20 MACOS>
```

Here the “tilt participation” regulates the percentage of the Kolmogorov that is to be included in the phase screen. Tilt is explicitly removed when this factor is set to zero, and increases proportionally as the factor goes to 1, which signifies full Kolmogorov tilt. An example, showing a typical atmospheric wavefront from AOexample.jou (Appendix A.4), is shown in Figure 39.

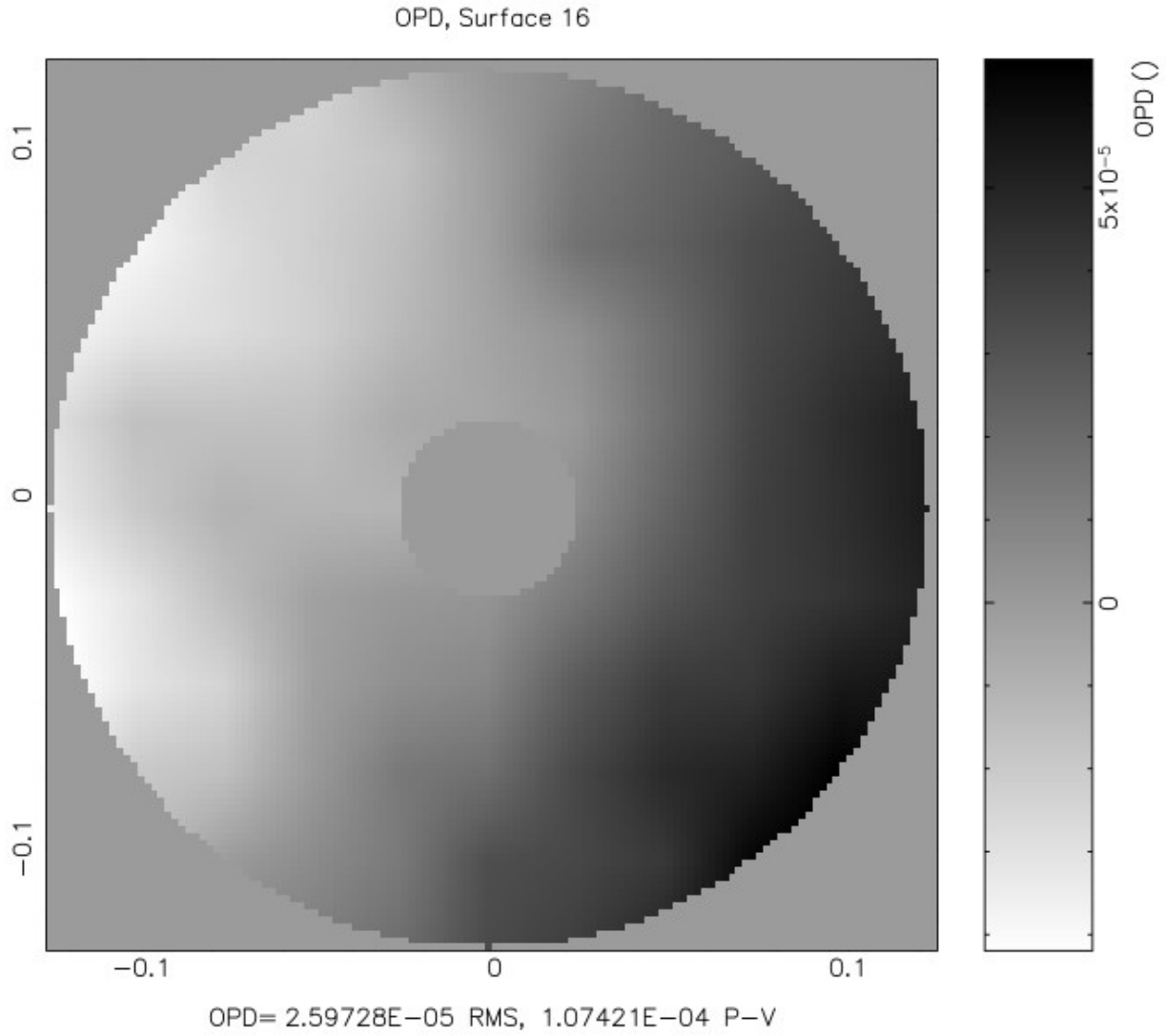


FIGURE 39 Atmospheric disturbances (AOexample).

10. **POLARIZATION**

The POLarization command turns on polarization ray tracing. The default is no polarization ray tracing. The command prompts for the complex source field components Ex0 and Ey0, and (when the model supports it) also enables vector diffraction. See the *Polarized Ray Tracing* subsection below for the physics, conventions and analysis tools.

11. **NOPOLARIZATION**

The NOPOlarization command turns off polarization ray tracing. This is the default setting.

12. **SINT**

The Surface INTerpolate (SINT) command activates the surface interpolation of elements. Prior to using the SINT command, the surface is assumed to be a conic-of-revolution. MACOS assumes that the interpolation input file has the name in-file.srf#.txt (or .bin), where in-file is the prescription's file name

including its .in and # is the number of the element the data describes. If that file does not exist, MACOS prompts the user for the name of the file.

MACOS>**SINT**

```
Computing interpolated surface data... Compute time was 5.7871 sec
```

MACOS>

5.4 Polarized Ray Tracing

Polarization ray tracing — the per-ray electric field, surface Fresnel and coating effects, the polarizing elements, vector diffraction and the polarization-aberration analysis — has its own chapter, Section 11. Turn it on with the POLarization command (off with NOPOLARIZATION); both are session modes that survive a LOAD.

SECTION 6: Diffraction Analysis

This section presents commands for physical optics modeling. Most optical systems are adequately modeled using a single-plane, exit-pupil diffraction model. Other systems are better modeled using multiple planes of diffraction. This section describes how to set up .in-files and other necessary data for both cases. The commands discussed in this section are:

```
ORS SRS
BEAm REGrid
SCAlar VECtor
```

The diffraction modeling routines of MACOS can accurately compute both near-field

(Fresnel diffraction) and far-field (Fraunhoffer diffraction) effects. Roughly speaking, near-field propagation takes the beam from pupil to pupil, regions where the beam is large compared to a wavelength. Far-field propagations take the beam from a pupil to a focal point, or from a focal point to a pupil. In the vicinity of a focus, the beam diameter approaches a wavelength.

Examples of systems where near-field diffraction is important include laser beam transfer systems, or spatial filters, where the beam propagates from pupil to pupil via a far-field plane where an aperture or obscuration is placed. Examples of far-field diffraction include imaging systems and high-gain directional antennas. In some cases both types of diffraction are important, and multiple planes of diffraction should be used.

6.1 Single- versus Multi-plane Diffraction

The first step in setting up a problem is to decide whether the important diffraction effects can be captured using a single-plane model, or whether multi-plane diffraction is necessary. Most imaging systems are adequately modeled using the single-plane approach, with one far-field diffraction being used to create the image. Section 6.2.2 discusses the set up of the .in-file for a single far-field propagation.

Systems for which diffraction effects of beam truncation due to apertures or obscurations in the beam train may be significant may be best modeled using multiple planes of diffraction. Examples are systems with spatial filters, such as coronograph instruments. Examples of systems requiring near-field propagation, whether multi-plane or no, include telecentric systems, defocused systems, or any system for which intermediate-plane intensities are desired.

If a multi-plane model is necessary, the first step is to lay out the optical system and determine the sequence of propagations. Consider the Cassegrain telescope with relay optics (including a fold near the focus of the main telescope) shown in Figure 40. Figure 41 shows reference surfaces added for multiplane diffraction calculations. The beam is partially obscured at the main and relay telescope secondary mirror supports (“spiders”). There are 4 diffraction propagations which are clearly shown in the diffraction schematic, Figure 42. Note that points A and D are used in the model twice. The first time each is used to model the spider obscurations. The second time, the points are used as a reference surfaces for diffraction calculations.

Spiders

Primary

Relay secondary Relay primary Focal plane

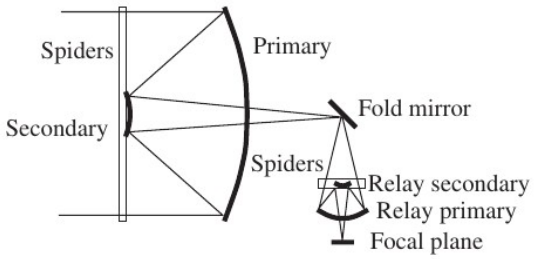


FIGURE 40 Optical schematic of Cassegrain telescope and relay optics example

The first near-field propagation, PropType=5, occurs between the main telescope

3 zElt6

A 5 6 7 B

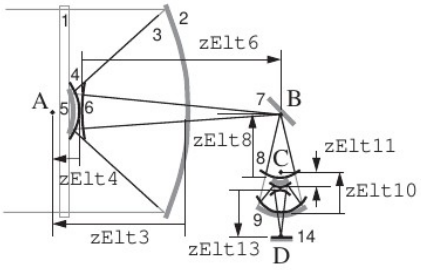


FIGURE 41 Reference surfaces for multi-plane diffraction in Cassegrain telescope and relay optics example

```
2 3 4 5 6 7 8 9 10 11 12 13 14
```

A A B C D D

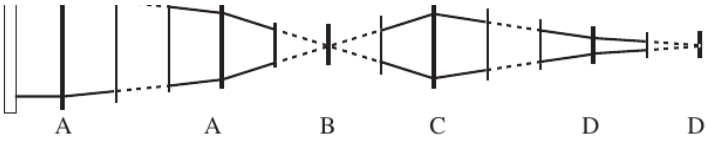


FIGURE 42 Diffraction schematic for Cassegrain telescope and relay optics example

primary and secondary mirror (between reference surfaces 3 and 4 which are located close to these optics). The reference surfaces share a common center of curvature at point A. Since this is a converging beam, the zElt parameters are positive for both reference surfaces (see Section 6.1).

The second near-field propagation occurs between the secondary mirror and the relay camera's secondary mirror support "spiders." It is a "through-focus" propagation, with zElt=6 positive (converging beam) and zElt=8 negative (diverging beam). The reference point for this propagation is point B. The fold mirror is included in the propagation.

The third propagation is at the converging beam between the relay primary and secondary. This near-field propagation is centered at point C.

The last propagation is a far-field propagation, PropType=3, to the focal plane. It is centered at the focal plane, point D.

6.2 Setting up Diffraction Propagations

To use MACOS diffraction features, the .in-file prescription must be set up correctly. A surface which does not change the wavefront (reference, obscuring, return, or focal plane elements) must ideally be placed at the start and end of each diffraction propagation. Each pair of reference surfaces must be defined together—*slaved* to each other—so as to avoid spurious phase errors. Slaving reference surfaces to each other means ensuring that they share a common axis and a common center of curvature. The reference surfaces should be *optimized* to minimize sampling errors. The beam must be adequately *padded* to avoid aliasing. In certain conditions, the rays that are used to determine geometric phase should be *regridded*.

MACOS provides several commands to help ensure that the prescription data is set up correctly. The FEX command (discussed earlier in Section 5.3) optimizes placement of a return surface at an exit pupil for far-field diffraction propagation. The ORS command optimizes a reference surface for minimum sampling error by matching it to the best-fit sphere and aligning it to the chief ray. The SRS command slaves one reference surface to another, solving for location, orientation, and radius of the starting (or ending) surface of a near- or far-field diffraction propagation as a function of the parameters of the ending (or starting) surface. Used together, FEX, ORS and SRS enable the correct setup of any MACOS diffraction propagation.

The FEX, ORS and SRS commands are run after a .in-file prescription is loaded. They *refine* the prescription for elements defined in the .in-file. The results can be saved in a new .in-file using the SAVe command. If a PERturb, MODify, FFP, PFP, CENTER, or other command changes the source position or angle, or the position, angle or figure of an element in the propagation path, FEX, ORS and SRS should be rerun.

The results of propagation commands (such as INtensity or PIXilate) are stored in particular internal MACOS arrays. These arrays are accessible through SMACOS common statements and the SMACOS call line. These include the ray-trace data variables defined in Section 5.1. Additional diffraction-specific arrays include:

- WFElt(mdttl,mdttl,mWF) holds complex amplitude matrices that store the results of propagation commands (such as INtensity). Here mdttl is the maximum dimension of the “wavefront” amplitude arrays, and is defined in the paramXXX.cmn include files.
- PixArray(mPix,mPix) stores the results of a pixilated image generation command (such as PIXilate or ADD). Here mPix is defined in the paramXXX.cmn include files.

MACOS implicitly enforces a padding requirement of at least 50% at compile time through internal data array declaration statements. This is accomplished by restricting the ray grid, which defines the maximum extent of the ray beam, to occupy the central half of the WFElt(mdttl,mdttl,mWF) matrices (see Section 2.3).

Once the surfaces for the diffraction calculations are properly set up, the beam can be propagated through the system by invoking one of the propagation commands described in this section (such as INtensity). These commands use geometric and Fresnel propagators to propagate the complex amplitude matrix to the specified element. They then plot the appropriate data product.

Two rules about what the propagation sees. First, masks: an obscuration or aperture is applied to the diffraction-grid field only when its element is `Element= Obscuring`. On a Reference (or any other element type) the same obscuration clips rays in the ray trace but leaves the grid field untouched, so a focal-plane mask declared on a Reference looks like a coronagraph in the prescription and does nothing to the propagated field; declare masks for diffraction as Obscuring elements (CoroExample.in, element 6). Second, media: a propagation leg inside a material of index n runs at the wavelength λ/n, so its Fresnel number is the medium's; a leg is assumed homogeneous (a leg that crosses a refracting face is not modeled), and a line announces any leg with $n \neq 1$.

We list here the available diffraction propagators (Table 13).

TABLE 13 MACOS diffraction propagators

- *PropType=…

**	**Description**
Geometric	Geometric propagator, updates phase only as $e^{i\phi}$
FarField	Fresnel/Fraunhaufer far-field propagator: sphere- or plane- to-plane
NFSpherical	Fresnel near-field propagator: sphere- to-sphere
NFPlane	Fresnel near-field propagator: plane- to-plane
NF1	Part 1 of the near-field propagator: sphere- to-plane. Use when an obscuration is to be imposed at an intermediate focus
NF2	Part 2 of the near-field propagator: plane-to-sphere
GeomUpdate	Geometric propagator, forces update at each surface (don't use)
NFS1surf	Near-field propagator: sphere- to-sphere, using end ref surf only
NFP1surf	Near-field propagator: plane- to-plane, using end ref surf only
SpatialFilter	Near-field propagator: sphere- to-sphere, with spatial filter
SF1surf	Near-field propagator with spatial filter, using end ref surf only

6.2.1 Choosing Propagator Type

For imaging systems, it is usually clear whether a particular diffraction propagation is best modeled using the near-field or far-field propagator. If the light is to be evaluated at a focus, the far-field propagator should be used. Otherwise, the near-field propagator should be used. For far-field propagations to focal planes, it is important to place the focal plane element at the physical detector location, which may not coincide with the center of curvature of the exit pupil, which is to say, the system might be defocused (see Figure 43).

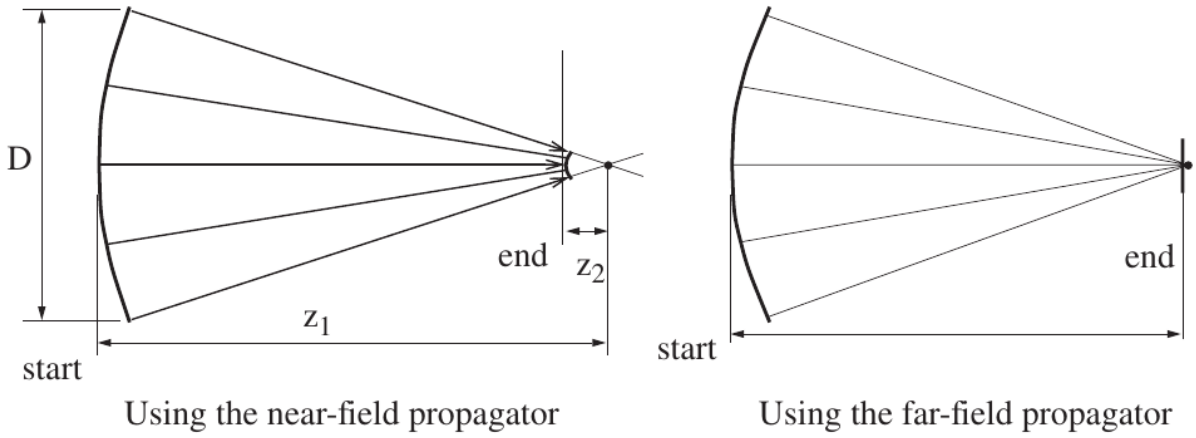


FIGURE 43 Imaging a defocused beam

Some systems require near-field propagation to model image formation. For example, spacecraft fine guidance sensors are defocused to spread a star image over multiple pixels to improve centroiding accuracies (see Figure 43). These images can be accurately generated using the near-field propagators followed by a geometric propagation to the focal plane. The intensity in each pixel can then computed using the PIXilate command described in Section 7.2.2.

Other conditions requiring near-field propagations include the presence of a spatial filter, occulting mask, or other intensity-altering object at an intermediate far-field point in the beam.

In some cases the choice of propagator will not be clear. Evaluating the Fresnel number *Fn* may help determine the choice. Consider the defocused beam of Figure 43. Here the first reference surface has diameter D and axial distance, i.e distance from the center of curvature, *z1*. The second reference surface is located at an axial distance *z2*. The Fresnel number *Fn* is

$$F_n = \frac{D^2}{\lambda\left(z_2 - z_1\right)} \left(EQ1\right)$$

Both far- and near-field propagators should be tested when *Fn* is in the range (0.1 – 1.0).

Obvious aliasing effects may also be mitigated by changing the sampling of the pupil, by increasing the diffraction grid size, the ray grid size, or both. This may require using larger MACOS executable sizes (e.g., macos512 if aliasing is seen with macos256), as well as larger values for nGridPts.

6.2.2 Far-Field

Consider the two cases of far-field diffraction shown in Figure 44. Both are typical of systems where a far-field diffraction to a focal plane is required to compute an image or point-spread function. This may be the only diffraction computation in the system (a single-plane model) or it may be the last of several propagations in a multi-plane model.

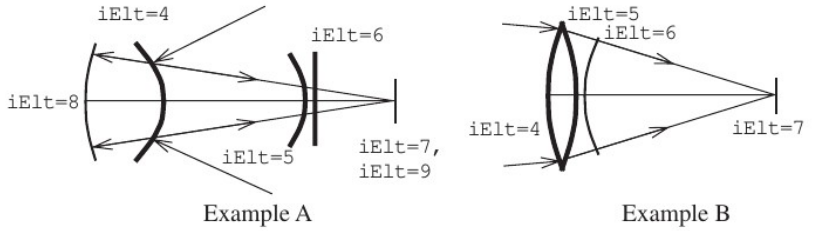


FIGURE 44 Far-field propagation examples

Example A in Figure 44 is set up for *exit pupil diffraction*. The light is incident on element 4 (the secondary mirror in some reimaging optics) and is reflected to two refractive surfaces (elements 5 and 6) which together form a lens. Finally, the light forms an image on element 7. For exit pupil diffraction, MACOS needs two additional surfaces in the model: elements 8 and 9. Elements 7 and 8 must be *return surfaces* with element 7 located at the focal plane and element 8 located at the exit pupil. From element 7, MACOS traces light backwards in image space (this is the way MACOS performs the calcualtion, light does not actually propagate in this fashion) until it reaches the exit pupil (element 8). The path length of this segment is negative for the purposes of geometric propagation. It then reverses direction until reaching the focal plane (element 9). Element 9 must be a *focal plane surface*.

The exit pupil surface must be a sphere with radius of curvature equal to the distance from the exit pupil to the image on the focal plane. The FEX command can be used to setup the exit pupil location, principal axis, and radius of curvature based on the chief ray. The Fresnel propagation distance for the exit pupil, zElt, must be set to the radius of curvature of the exit pupil element.

NOTE: The principal axis of the exit pupil should be set along the line connecting the center of the exit pupil and the center of the image. The FEX command sets it along the chief ray which is not the same for systems with a large amount of odd aberrations, such as coma. For some applications, there can be a significant difference.

We assign the geometric propagator (PropType=1) to all elements, except for the exit pupil (element 8) which is assigned the far-field propagator (PropType=3). This directs MACOS to use the far-field propagator to compute the amplitude matrix at the focal plane, based on the phases calculated at the exit pupil by the geometric propagator.

The main advantage of using the exit pupil for far-field diffraction is that the ray grid is usually well-formed there. If a system is significantly aberrated, sampling at non-pupil planes may give distorted ray grids which causes a poor match between the locations of the ray grid and the locations of the complex amplitude matrix grid used in the diffraction calculations. However if the system is well corrected and the ray grid is regular, a different plane may be used. A spot diagram at the plane provides a good qualitative check. Another check is provided by the propagation diffraction summary, which gives information on the “dx” or ray spacing across the reference surface.

Example B illustrates a system that uses a reference surface located away from the exit pupil to mark the start of a far-field diffraction calculation. Two fewer surfaces are required in this case, as the reference surface is placed in the beam between the final optical element (element 6) and the focal plane. All elements are assigned PropType=1, except the reference surface which is assigned PropType=3 to mark the start of the far-field propagation. Again, the reference surface is be a sphere with radius of curvature equal to the distance from the reference surface to the point of incidence of the chief ray with the focal plane. The SRS command can be used to find the orientation and radius for the reference surface.

A far-field leg maps spatial frequency f to the output coordinate x = λ z f. A pupil sampled finer than λ/2 (a 20 µm pinhole at 1 µm on a 64-point grid, or any heavily zero-padded fine pupil) therefore produces output pixels beyond |x| > z that carry no propagating energy. They are kept by default, so that the normalization of record is unchanged; the bindings' `ffcut` option zeroes them and reports how many it removed.

6.2.3 Near-Field

There are basically four cases of near-field diffraction (see Figure 45). The algorithm uses planar reference surfaces. The first three cases deal with converging/diverging beams and use spherical reference surfaces. They employ the angular spectrum propagation algorithm for the paraxial equation and the Sziklas/Seigman algorithm for spherical wavefronts.The fourth case, nearly collimated light, uses the propagation algorithm based on the standard form angular spectrum propagation for the paraxial wave equation(see, e.g. Goodman [2])

The first three types are distinguished by the signs of the Fresnel propagation distance z to the *reference point*, the center of curvature of *both* of the reference spheres. z is positive when the light is moving towards the reference point, as in converging beam. In a diverging beam, the light is moving away from the reference point and z is negative.

Both z's are infinite for a collimated beam.

WARNING: In aberrated systems, the ray grid can lose its regularity. This is likely to be more pronounced near to focus, but can occur anywhere. The grid regularity can be restored using the REGrid command described in Section 6.3.3. In some cases this should be accompanied by an OPD interpolation step, which is not included in this release of MACOS.

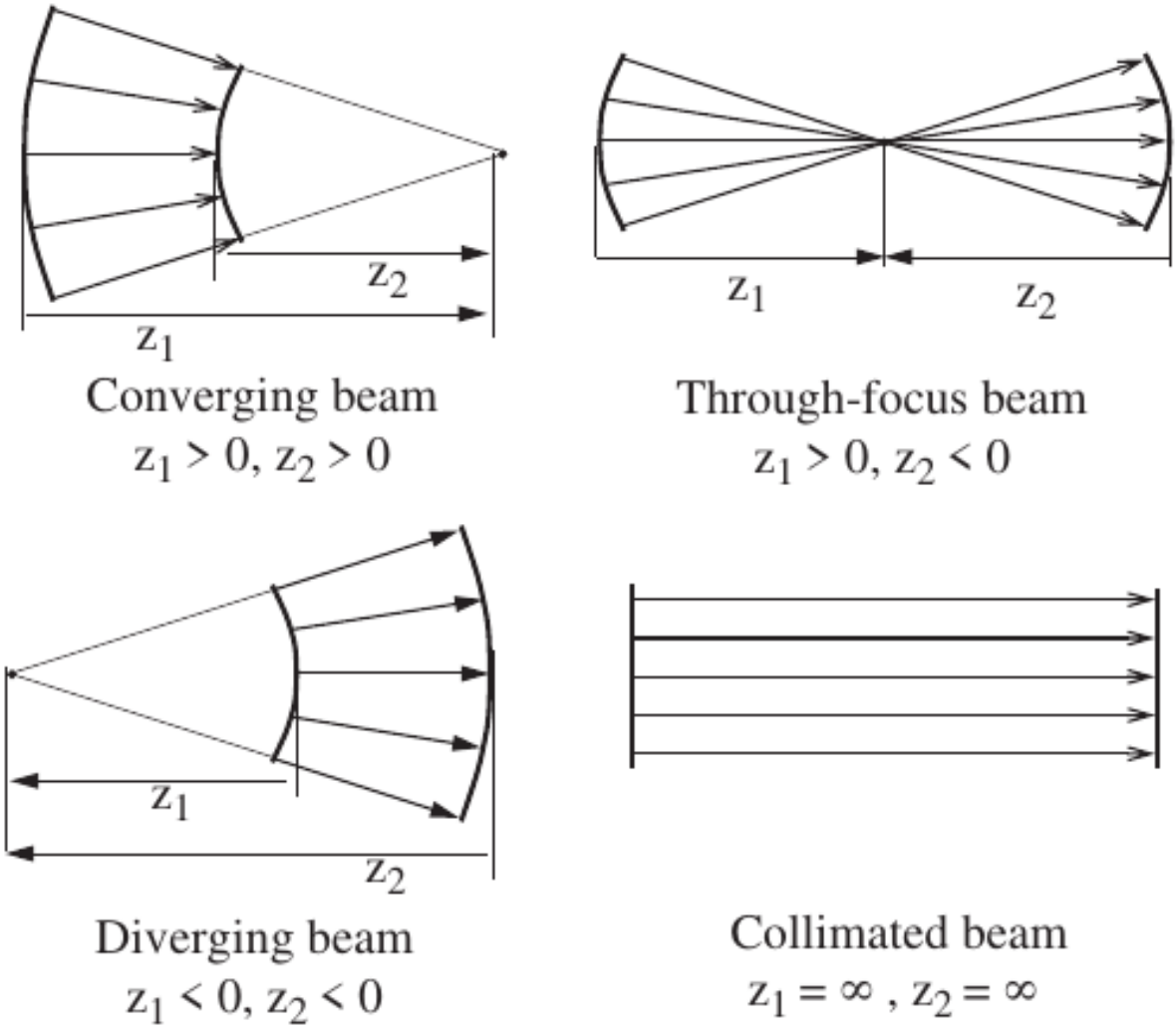


FIGURE 45 Near-field diffraction examples

Table 14 list the values for reference parameters for near-field propagations. The Fresnel propagation distances are entered both as the zElt parameters for each reference surface and as the focal length, fElt (or the radius of curvature, KrElt) of the element. The sphere-to-sphere propagations are all specified by setting PropType=5 for the first reference surface. The end reference surface has PropType=1, indicating geometric propagation following the surface. The plane-to-plane propagation has PropType=4. (Unpowered elements such as fold mirrors can be included within the scope of a propagation by specifying PropType=5 or 4.) The last element must have PropType=1.

NOTE: PropType=2 assumes no phase change from the previous element and should be used with extreme caution.

TABLE 14. Reference surface parameters for near-field propagation

Parameter	Referece Surface 1	Reference Surface 2
zElt	z1	z2
fElt	|z1|	|z2|

Parameter	Referece Surface 1	Reference Surface 2
eElt	0	0
KrElt	-|z1|	-|z2|
KcElt	0	0
PropType	4,5	1

The locations and orientations of the reference surfaces must be specified.

- Vertex location, VptElt, should be close to the optical element at the appropriate extreme of the propagation, but not so close that the surface intersects the optical element which would cause some rays to become undefined.
- The principal axis, psiElt, should be set so that the chief ray is normal to the reference surface.

Initial guesses can be refined later with the ORS and SRS commands.

6.3 Commands

6.3.1 ORS

The Optimize Reference Surface (ORS) command is used to refine the prescription for a reference surface used in near-field diffraction. To use ORS, the prescription must be set up with a reference surface at the desired location (VptElt) and with an approximate principal axis (psiElt). ORS requires one argument: the number iElt of the reference surface to be optimized.

ORS first relocates the reference surface so that the chief ray intersects the vertex point (VptElt). Then ORS tilts the specified reference surface so that psiElt is in alignment with the chief ray. Finally, ORS iteratively solves for the value of fElt that minimizes the OPD over a 15 by 15 grid of rays. The solution is the radius of the reference sphere that best fits the wavefront at that location. This process usually converges quite quickly. If it doesn't, a message may appear indicating a problem. The usual source of difficulty is that the surface has been put in backwards! Try changing the signs of psiElt using the MODify command and repeat.

NOTE: ORS may not find the optimal reference surface for systems with a large amont of odd aberrations, such as coma. In these systems, the minimum OPD is obtained when the reference sphere is tilted from the chief ray.

The ORS command prints the old and new values of the element parameters and asks the user if the new ones are acceptable. Enter yes to accept the new values for subsequent computations. Enter no to retain the previous values. The new element data will not be kept unless a SAVe command is used to store the data. For example, using the CoroExample.in near-field example of Appendix A: elements 5 and 7 are the reference surfaces that bracket the near-field propagation to the coronagraph mask (PropType NF1 on element 5). To show ORS at work, element 5's focal length is first set 4% long with MODify, and ORS recovers the design value (fElt = 6.3293) to the solve tolerance, a hundredth of a wave:

MACOS>**mod**

```
MOD: [q]: fElt(5)=6.6
fElt(  5) =   6.600000000D+00
KrElt(  5) =  -6.600000000D+00
MOD: [q]: q
```

MACOS>**ors**

```
Enter number of element to be optimized: [1]: 5
Reference surface optimization results:
TTL OPD=  9.264281133D-12  nCalls=  27
  Old f=  6.600000000D+00   New f=  6.329334638D+00
  Old z=  6.329334641D+00   New z=  6.329334638D+00
Old psi=  0.000000000D+00 New psi=  0.000000000D+00
          0.000000000D+00           0.000000000D+00
          1.000000000D+00           1.000000000D+00
Old Vpt=  0.000000000D+00 New Vpt=  0.000000000D+00
          0.000000000D+00           0.000000000D+00
         -4.829334641D+00          -4.829334641D+00
Accept the new element data? [YES]: yes
```

An exit-pupil Return surface for a far-field propagation is set up with FEX (Section 5), not ORS.

The reference sphere is fit to the rays that reach the surface before it. When fewer than three do, ORS names the starved trace and aborts rather than fit nothing. A copy of ImageDemo.in (Appendix A) with a 5 mm Zernike coefficient on its primary, which loses all but 15 of its 741 rays, shows the message:

MACOS>**ors**

```
Enter number of element to be optimized: [1]: 12
ORS: only     0 of   317 test rays reach element  11 (the surface before it); at least 3 are needed.
ORS: command aborted.
```

6.3.2 SRS

The Slave Reference Surface (SRS) command is used to slave one of the two reference surfaces used in a diffraction propagation to the other reference surface. It applies these changes directly to the element prescription. To use SRS, the prescription must be set up with a reference surface at the start and end of the propagation. If the propagation is a far-field propagation, the second surface can be a focal plane. To use SRS, the slaved reference surface must be in the MACOS prescription at the desired location (VptElt) and with an approximate principal axis (psiElt).

SRS is to be used in conjunction with ORS. ORS is run first for the first of the two reference surfaces, then SRS is run for the second. ORS requires two arguments: the number iElt of the reference surface to be slaved and the number iElt to which the reference surface is to be slaved.

SRS first relocates the reference surface so that the chief ray intersects the vertex point (VptElt). Then SRS tilts the specified reference surface so that psiElt is normal to the surface. Finally, SRS solves for the propagation distance zElt based on the zElt of the surface it is being slaved to and the optical path length between the surfaces along the chief ray. It computes the other element parameters accordingly.

The SRS command prints the old and new values of the element parameters and asks the user if the new ones are acceptable. Enter yes to accept the new values for subsequent computations. Enter no to retain the previous values. The new element data will not be kept unless a SAVe command is used to store the data. Continuing the CoroExample.in example, the second near-field reference surface (element 7, the end of the propagation through the coronagraph mask) is slaved to the first (element 5). To show SRS at work, element 7 is first disturbed with MODify (focal length 7.0, zElt -7.0); SRS puts it back at the design radius 6.3293, sharing element 5's center of curvature, and keeps zElt negative, the sign of a diverging beam at the end of a through-focus propagation (Table 14):

MACOS>**mod**

```
MOD: [q]: fElt(7)=7.0
fElt(  7) =   7.000000000D+00
KrElt(  7) =  -7.000000000D+00
MOD: [q]: q
```

MACOS>**mod**

```
MOD: [q]: zElt(7)=-7.0
zElt(  7) =  -7.000000000D+00
MOD: [q]: q
```

MACOS>**srs**

```
Enter number of element to be slaved: [1]: 7
Enter number of element to slave to: [1]: 5
Reference surface recalculation results:
  Old f=  7.000000000D+00   New f=  6.329334641D+00
  Old z= -7.000000000D+00   New z= -6.329334641D+00
Old psi=  0.000000000D+00 New psi=  0.000000000D+00
          0.000000000D+00           0.000000000D+00
         -1.000000000D+00          -1.000000000D+00
Old Vpt=  0.000000000D+00 New Vpt=  0.000000000D+00
          0.000000000D+00           0.000000000D+00
          7.829334641D+00           7.829334641D+00
Accept the new element? [YES]: yes
```

The near-field propagation to the mask (INT 7) then reproduces the design deck's result exactly. The pair can be slaved in either order: slaving element 5 to element 7 returns zElt +6.3293, the converging sign at the start of the propagation. For a far-field pair the second surface can be the focal plane itself (in CassWithExitPupil.in, the detector slaved to the exit-pupil surface).

6.3.3 REGRID

This feature is undergoing revision and not currently supported.

6.3.4 SCALAR

The SCAlar command sets a toggle to use a scalar formula when calculating single-plane diffraction (see Section 6.1). The SCAlar command has no arguments.

Scalar diffraction is the default and the only mode when polarization ray tracing is off (see Section 11). Vector diffraction is the default with polarization on.

6.3.5 VECTOR

The VECtor command sets a toggle to use vector formula when calculating single-plane diffraction (see Section 6.1). The VECtor command has no arguments.

Scalar diffraction is the default and the only mode when polarization ray tracing is off (see Section 11). Vector diffraction is the default with polarization on.

In vector mode the three field components Ex, Ey, Ez are propagated as independent scalar fields. This is rigorous rather than approximate: in a homogeneous isotropic medium each Cartesian component independently satisfies the scalar Helmholtz equation, and the angular-spectrum propagator is its solution operator, so the vector legs inherit exactly the scalar legs' paraxial validity envelope — no better and no worse. Component coupling enters at surfaces, where it is handled per ray by the s/p machinery of the polarized trace.

Scope: vector diffraction covers the whole propagation chain — the far-field (Fraunhofer) leg, every near-field, plane-to-plane, spherical, Fresnel and DFT leg, the diffraction-grid obscuration apply, and the ray-side aperture and apodization masking. A multi-leg chain therefore preserves the vector field between legs. (Before MACOS 4.1 only the far-field leg was vectorized; the other legs carried the two off-plane components without propagating them.)

In vector mode the three wavefront storage planes are repurposed as Ex/Ey/Ez, so only a single wavefront can be active — do not combine vector diffraction with multi-wavefront work such as COMPOSE.

The individual components are readable through the bindings: `complex_field` takes a `plane` selector, where 1, 2 and 3 return Ex, Ey and Ez and the default 0 returns the element's own wavefront. The three add in intensity, not amplitude, and their sum reproduces the ordinary intensity map. Requesting a component plane while vector diffraction is off is refused rather than answered, because in scalar mode that storage plane holds an unrelated wavefront and not a field component.

One limitation remains. Between two physical propagation legs the grid field is advanced by a scalar phase, which is exact when the intervening elements are non-polarizing — Obscuring, Reference and FocalPlane elements, which covers the standard coronagraph chain of pupil, focal-plane mask, Lyot stop and focal plane. A chain that places a COATED or reflecting surface between two physical legs needs the surface Jones matrix applied to the grid field as well as to the rays, which is not yet implemented; in that configuration the polarization transfer at the intervening surface is not carried into the diffracted field.

6.3.6 Propagation commands

Several commands, such as INT, LOG, ADD, etc. cause the beam to be propagated to the specified element using MACOS combined ray-trace and diffraction computations.

These commands are described in the next section.

6.3.7 BEAM

The BEAm command allows the user to select a source beam profile. The default is a uniform beam. Alternatives are Gaussian, cosine-to-a-power, and dipole beam profiles.

If a Gaussian beam is choosen, the user is then prompted for the diameter of the beam in x and y, where the diameter is the standard deviation of the Gaussian profile.

Figure 46 shows the intensity at the exit pupil (element 5) of CassWithExitPupil.in with the default uniform beam and with a Gaussian beam of waist radius 1.0 (half the 4 m aperture):

MACOS>**int 5**

```
Enter number of element where data is to be generated: [6]: 5
Tracing       125 rays and propagating     65536 grid points...
 Geometric Prop between Elt  0 and Elt  5 to WF  1
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06;  Source Flux= 1.0000000D+00
u-v plane diam= 0.0000000D+00  du= 0.0000000D+00
x-y plane diam= 1.9435037D+01  dx= 7.6215832D-02
Peak intensity= 8.0645161D-03; Peak occurs at i=  131, j=  123
Sum of intensity= 9.0322581D-01
```

MACOS>**beam**

```
Enter beam type (UNIFORM, GAUSSIAN, COS**POWER, DIPOLE): [UNIFORM]: gau
Enter x beam waist radius: [1.00000]: 1.0
Enter y beam waist radius: [1.00000]: 1.0
```

MACOS>**int 5**

```
Enter number of element where data is to be generated: [5]: 5
...
Peak intensity= 6.2289782D-01; Peak occurs at i=  127, j=  130
Sum of intensity= 8.9981120D+00
```

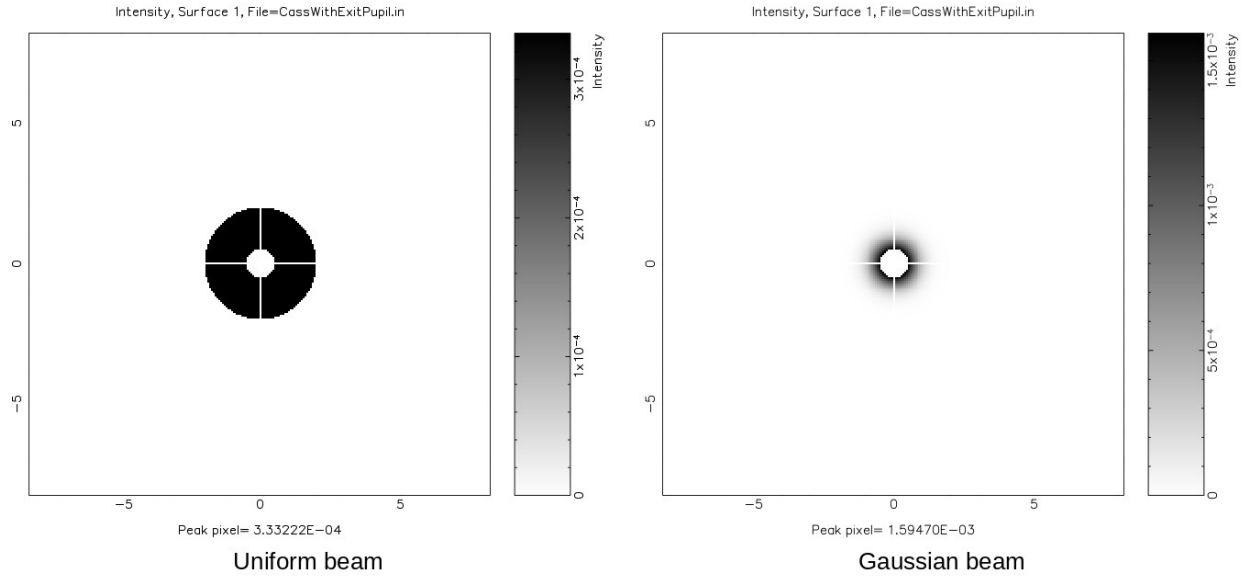


FIGURE 46 Propagation examples with Gaussian and uniform beams

SECTION 7: Beam Propagation and Image Simulation

Image Simulation

This section presents the MACOS image generation and plotting commands. The com-plex amplitude matrices can be plotted to show pupil functions, point images, or images of extended objects. Composed images can combine multiple objects or multiple colors. Pixel array detectors can be simulated. Noise effects and image blurring can be simu-lated. Plot types include wireframe, contour, grayscale, row, and columns. Text and binary outputs are also available. The commands discussed in this section are:

```
INtensity GAIn
PIxilate AMplitude
WINdow COMpose
ADD DADD
MULTISPEC RFILT
NOISE SEED
BLUR LOG
REAl IMAGinary
GRAy WIRe
SLIce CONtour
COLumn ROW
TEXt BINary
FITs MATlab
GBLUR
```

7.1 Background

MACOS propagation commands, such as INtensity, propagate the beam through the optics to produce a complex amplitude matrix or matrices representing the light beam as projected onto a particular surface. The commands described in this section generate the complex amplitude matrices, perform basic analyses on them, and plot the results. Plots of intensity, magnitude and phase, and the real and imaginary parts of the amplitude are available. Images, or intensity plots, can be digitized or *pixilated*, simulating the images received by a pixel array detector such as a CCD. Multiple-source images can be composed on a single pixel array, allowing the accumulation of multiple-color or multiple object images. Simulated noise can be added.

Propagation works as follows. If no propagation command has been executed on the current system, it starts at the source. The rays are initialized according to the aperture and source geometry. The complex amplitude matrices are initialized to zero for those grid points that are not associated with rays (e.g. padding in the wavefront array). The grid points which are associated with rays are initialized to achieve uniform amplitude with total magnitude equal to the value set by the Flux .in-file parameter. If a propagation has been performed previously, to an element upstream from the current element, then the previous values of the complex amplitude matrix are the starting point for the current computations.

The light is then advanced surface-by-surface through the system. Rays are traced to determine the phases that drive the geometric and Fresnel propagators. The propagators

are invoked at the end of each specified diffraction to update the complex amplitude matrices. Depending on the compile-time parameter mWF, one or more of these are stored in the WFElt array. These are accessible to the image generation commands for plotting. The propagation terminates when it has computed the complex amplitude at the specified last element.

Propagation commands print a summary of the scaling calculations for Fresnel propagation. These provide information on the regularity of the ray grid (the “lin” or linearity is displayed as a percentage) at each reference surface. Warnings are issued if rays become undefined (usually indicative of a problem with the .in-file prescription).

7.2 Beam Propagation Commands

To introduce and motivate the various beam propagation and image simulation commands, we will use a couple of examples. The first example is a simple non-optimized coronagraph. As shown on Figure 47, this system consists of three telescopes in sequence. At the focus of the first telescope is an occulting mask (iElt=6), which is designed to suppress the light from the core of an on-axis star. The beam is then re-expanded. A second mask is placed at the system stop (iElt=11), which is located in the collimated beam between the second and third telescopes. This forms a Lyot stop, which suppresses high-frequency light scattered from the occulted axial star. The detected on-axis image (iElt=16) is a somewhat complex and chaotic PSF, with greatly reduced core intensity.

Off-axis images are passed with very little attenuation, yielding near-ideal PSFs. The net effect is to greatly increase the visibility of dim off-axis objects compared to bright on-axis objects. Coronographs can greatly decrease dynamic range between 2 very close objects, such as a star and a planet might provide, enabling easier observation of the dimmer object.

The prescription and macro for generating the results plotted in this section are coroExample.in and coroExample.jou, respectively. These files are listed in Appendix A.6.

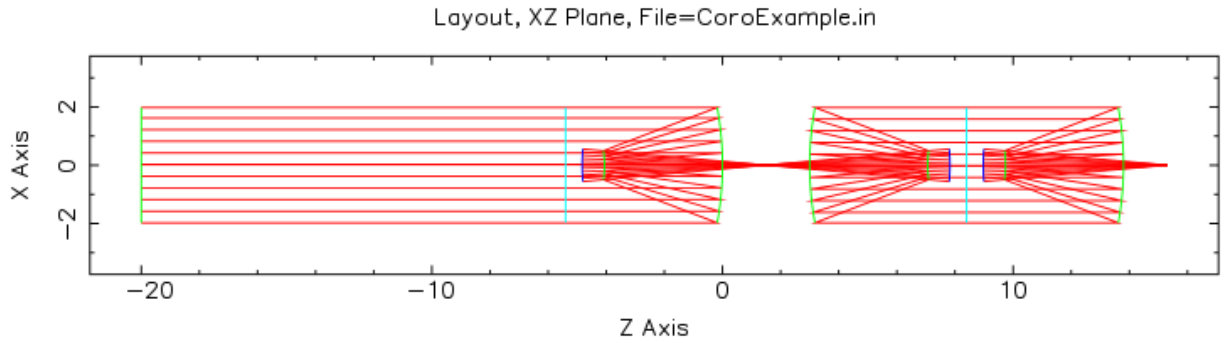


FIGURE 47 DRAW sketch of coroExample.in system.

7.2.1 INTENSITY

The INtensity command takes the complex amplitude matrix at the specified element, computes the modulus squared to determine the intensity, and plots or exports it according to the active plot option (see Section 7.4).

The INtensity command only accepts one argument–the number of the surface at which the intensity is viewed. This surface must be a stored wavefront. In the default gray scale plot, the intensity is displayed with black and white corresponding to the maixmum and minimum intensities, respectively.

We begin the example by propagating the on-axis beam through the first part of a near-field diffraction propagation (PropType=NF1), observing the image at the focus of the first telescope, both with and without the occulting mask:

MACOS> **mod**

```
MOD: [q]: nObs(6)=0 Turns off the occulting mask
nObs( 6) = 0 MOD: [q]: q MACOS> int 6
Enter number of element where data is to be generated: [1]: 6 Tracing 12661 rays and propagating
262144 grid points...
Prop to reference point Elt 5 and Elt 6 to WF 1:
z1= 6.3293D+00 dx1= 8.8773D-03 min= 8.8703D-03 max= 8.8832D-03 dev= 3.7728D-06 lin= 99.96%
z2= 1.0000D+30 dx2= 1.3925D-06
Compute time was 8.7598 sec Wavefront Propagation Data Summary:
```

```
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 4.5362873D+00 du= 8.8772746D-03
x-y plane diam= 7.1158918D-04 dx= 1.3925425D-06
Peak intensity= 3.7152264D-02; Peak occurs at i= 257, j= 257
Sum of intensity= 8.7709310D-01 MACOS>
```

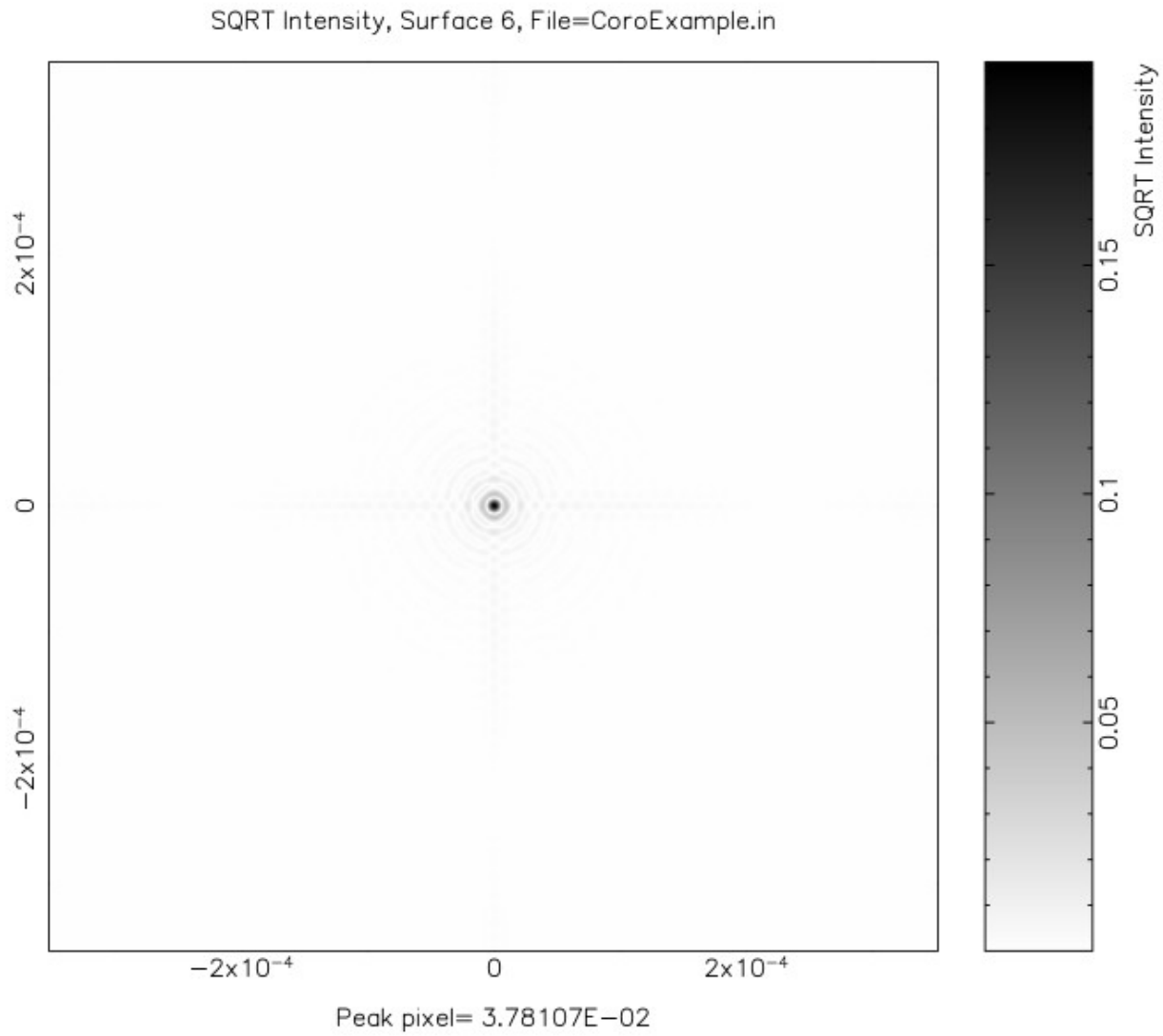


FIGURE 48 Unocculted image at first focal plane (coronagraph example)

Continuing with the example, the beam intensity can be observed at the other points of interest in the coronagraph, simply by specifying the appropriate element numbers. At the pupil, the Lyot stop, and then at the detector, the intensity plots are generated by:

MACOS> **int 10**

```
Enter number of element where data is to be generated: [6]: 10 Tracing 12661 rays and propagating
262144 grid points...
Prop from reference point Elt 6 and Elt 7 to WF 1:
z1= 1.0000D+30 dx1= 1.3925D-06
z2=-6.3293D+00 dx2= 8.8773D-03 min= 8.8703D-03 max= 8.8832D-03 dev= 3.7728D-06 lin= 99.96%
Geometric Prop between Elt 6 and Elt 10 to WF 1 Compute time was 6.8867 sec
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 0.0000000D+00 du= 0.0000000D+00
x-y plane diam= 1.6256657D+01 dx= 3.1813417D-02
Peak intensity= 7.6769075D-05; Peak occurs at i= 258, j= 297 Sum of intensity= 1.3185163D-01
```

MACOS> **int 11**

```
Enter number of element where data is to be generated: [10]: 11 Tracing 12661 rays and propagating
262144 grid points...
Geometric Prop between Elt 10 and Elt 11 to WF 1 Compute time was 0.4170 sec
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 1.6256657D+01 du= 3.1813417D-02
x-y plane diam= 1.6094488D+01 dx= 3.1496063D-02
Peak intensity= 2.9045304D-05; Peak occurs at i= 267, j= 320 Sum of intensity= 4.4725472D-02
```

MACOS>

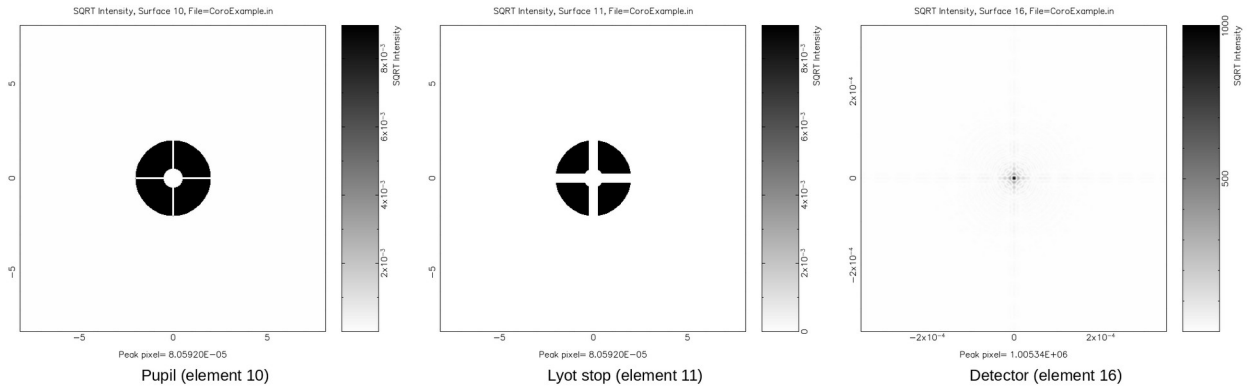


FIGURE 49 On-axis beam profile at a pupil; at the Lyot stop; and at the detector (coronagraph example)

7.2.2 PIXILATE

The PIxilate command is used to simulate discrete element detectors. MACOS calculates wavefront propagation on an array size which is independent of the detector size. The physical dimension of the diffracted image pixels is in general different from the detector pixel size being used to sample the signal. The PIxilate command imposes a square detector area over the intensity pattern. It averages the INtensity pattern within each detector pixel and sets that average equal to the value within the pixel. This is shown schematically in Figure 50. The PIxilated grid is centered on the chief ray (unless the PLOcate command has been executed) and the image location is determined by the Fourier transform of the exit pupil OPD map. Residual tilt in the OPD will shift the image from the chief ray intercept on the focal plane.

The PIxilate command has three arguments: the number of pixels per side (of the detector), the pixel width, and the element where the signal is to be PIxilated.

Continuing our example, we use the pixilate command to get a closer look at the occulted and unocculted images at the first focus:

Point-spread function Pixel boundaries Pixilated image

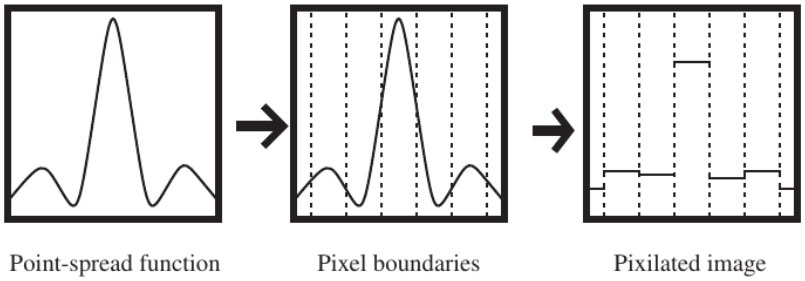


FIGURE 50 Pixilization of images

MACOS> **pix 6**

```
Enter number of element where data is to be generated: [6]: 6 Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 2.2394683D+00 du= 1.7633609D-02
x-y plane diam= 3.5613167D-04 dx= 2.8041864D-06 Enter number of pixels per side: [128]: 64
Enter size of pixel: [2.804186E-06]:
Peak intensity= 1.6217206D-01; Peak occurs at i= 65, j= 65 Sum of intensity= 9.0994070D-01
```

MACOS> **mod**

```
MOD: [q]: nObs(6)=1 Reinstates the occulting mask
nObs( 6) = 1
MOD: [q]: q
```

MACOS> **pix 6 64 2.804186E-06**

```
Enter number of element where data is to be generated: [16]: 6 Tracing 3210 rays and propagating
16384 grid points...
Prop to reference point Elt 5 and Elt 6 to WF 1:
z1= 6.3293D+00 dx1= 1.7634D-02 min= 1.7628D-02 max= 1.7637D-02 dev= 2.6054D-06 lin= 99.99%
z2= 1.0000D+30 dx2= 2.8042D-06
Compute time was 0.4272 sec Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 2.2394683D+00 du= 1.7633609D-02
x-y plane diam= 3.5613167D-04 dx= 2.8041864D-06 Enter number of pixels per side: [128]: 64
Enter size of pixel: [2.804186E-06]:
Peak intensity= 8.2110545D-04; Peak occurs at i= 65, j= 72
Sum of intensity= 1.1480418D-01 Note the reduced intensity
```

MACOS>

7.2.3 STRETCH

It is often important to be able to visualize very low-level details of an image or beam. This may require changing the stretch of the image: the scaling of the intensity. Plotting on a log10 or sqrt intensity scale greatly increases the dynamic range of an image. The MACOS STRetch command changes the intensity scaling, to log10, sqrt, or linear scale.

An example of setting the stretch is:

MACOS> **stretch**

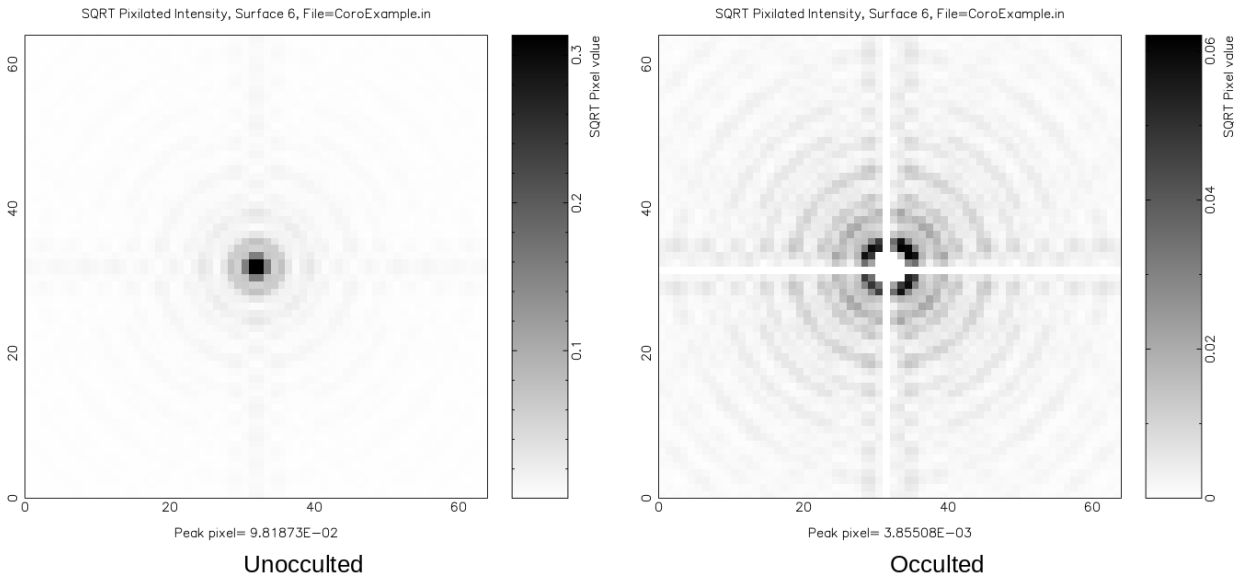


FIGURE 51 Pixilated images at first focal plane (coronagraph example)

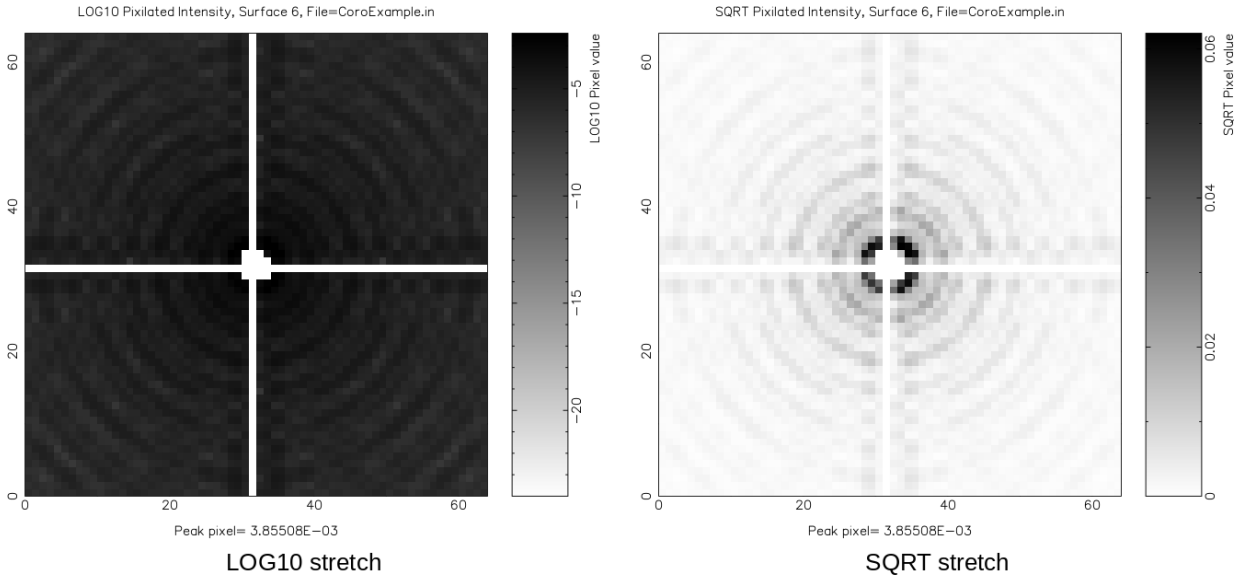


FIGURE 52 Pixilated images at log10 and sqrt stretch (coronagraph example)

```
Enter image stretch type (LINEAR, LOG10, SQRT): [LINEAR]: log10
```

7.2.4 WINDOW

Using PIXilate to simulate detector images has a drawback: if the FEX command is used correctly, the image will always be centered in the PixArray window, even at differing field angles. The WINdow command provides a means of fixing the detector in space, so that changes in field angle result in appropriate motion of the image on the detector. WINdow sets PixArray equal to a specific region on the detector.

WINdow has four arguments. The first is a character string indicating the user's choice of output coordinate options. The second is the pixel size. The third is the location of the element vertex in pixel coords (this is used to define the origin of a pixel coordinate system, whose axis directions are defined by

the previously selected coordinates. The fourth is the location in pixel coordinates of the center of the window:

MACOS> **win**

```
Enter output coordinate option (Tout, Enter or Beam): [Tout]: Tout
Enter pixel size for placing window: [0.]: 1.392542d-6 Enter pixel coords of element vertex (x,y):
[0.,0.]: 0,0 Enter window location in pixel coords (x,y): [0.,0.]: 0,0
```

7.2.5 COMPOSE

The COMpose command defines the pixel size and dimensions for a detector window. It also sets up an accumulate option, whereby images from a series of propagations, under arbitrarily different conditions, can be incoherently added. It is useful for creating composite images from multiple-source or multiple-wavelength objects.

COMpose images are created by successive applications of the ADD command. ADD propagates the current beam, pixilates the intensity to the COMpose grid, and adds it to what is already present. COMpose is also used with the NOIse and BLUR commands, to add simulated noise to an image. An example using the COMpose and ADD commands is presented in Section 7.2.6.

COMpose has three arguments: the number of pixels per side (of the detector), the pixel width, and the element where the signal is to be COMposed.

Continuing the coronagraph example, a pixel array is defined at the detector element:

MACOS> **compose**

```
Enter element where image is to be composed: [16]: 16
Enter number of pixels per side: [512]: 64
Enter size of pixel: [1.000000E-03]: 1.392542E-06
```

7.2.6 ADD

The ADD command propagates the current beam, pixilates the intensity to the COMpose

grid, and adds it to the composed pixel array image, which is stored in PixArray.

Continuing the coronagraph example, the on-axis image is added to the detector pixel array:

MACOS> **add**

```
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 4.5362873D+00 du= 8.8772746D-03
x-y plane diam= 7.1158918D-04 dx= 1.3925425D-06
Peak intensity= 3.8883158D-04; Peak occurs at i= 257, j= 257 Sum of intensity= 4.4725467D-02
Image added. Plot composed image? [YES]: yes
```

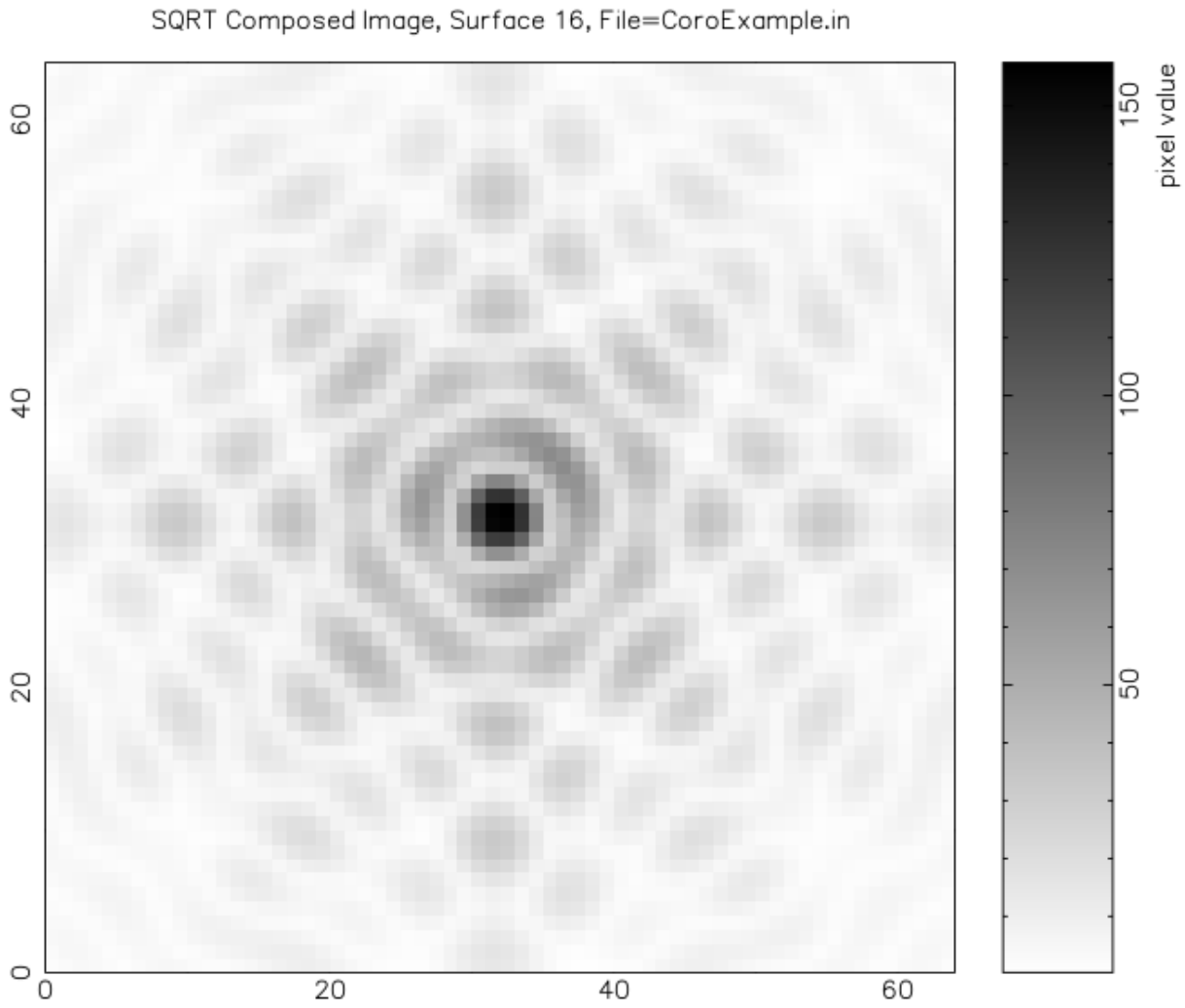


FIGURE 53 Axial image ADDed to begin composed image.

7.2.7 Propagating a Perturbed Beam

The next step in the coronograph example is to define an off-axis star and propagate it to the detector, adding it into the composed image. The result is an image with 2 PSFs on

it, allowing us to evaluate the effectiveness of the coronagraph in reducing dynamic range between neighboring objects.

MACOS> **stop**

```
Stop at ELT or OBJect point? [Elt]: elt Enter system stop element number: [1]: 11 Enter offset vector
(dx,dy): [0.,0.]: 0,0
Computed StopPos= 0.000000000D+00 0.000000000D+00 -1.427723273D+02
```

MACOS> **ffpˆ *Find field angles that put the off-axis*** Enter element number: [16]: **6 *spot right next to the occulting disk*** Enter offset vector in global units (dx,dy): [0.,0.]: **2d-5,2d-5**

```
Field angle finder results:
Did 1 iterations, error= 1.424113342D-15
Old dx= 0.000000000D+00 New dx= 2.000000000D-05 0.000000000D+00 2.000000000D-05
Old crd= 0.000000000D+00 New crd= 8.916721422D-07 0.000000000D+00 -8.916721422D-07
1.000000000D+00 1.000000000D+00 Old crp= 0.000000000D+00 New crp= 1.094726641D-04 0.000000000D+00 -
```

```
1.094726641D-04
-2.000000000D+01 -2.000000000D+01
Accept the new chief ray? [YES]: yes
```

MACOS> **ors** ***Find new reference surfaces for diffraction propagation***

```
Enter number of element to be optimized: [1]: 5
Reference surface optimization results:
TTL OPD= 1.656171318D-09 nCalls= 18
Old f= 6.329334641D+00 New f= 6.329334641D+00 Old z= 6.329334641D+00 New z= 6.329334641D+00 Old psi=
0.000000000D+00 New psi= -2.730547038D-06 0.000000000D+00 2.730547038D-06
1.000000000D+00 1.000000000D+00 Old Vpt= 0.000000000D+00 New Vpt= 3.728254596D-05 0.000000000D+00 -
3.728254596D-05
-4.829334641D+00 -4.829334641D+00
Accept the new element data? [YES]: yes
```

MACOS> **ors**

```
Enter number of element to be optimized: [1]: 7
Reference surface optimization results:
TTL OPD= 1.656171779D-09 nCalls= 18
Old f= 6.329334641D+00 New f= 6.329334641D+00 Old z= -6.329334641D+00 New z= -6.329334641D+00 Old
psi= 0.000000000D+00 New psi= 2.730547038D-06 0.000000000D+00 -2.730547038D-06
-1.000000000D+00 -1.000000000D+00 Old Vpt= 0.000000000D+00 New Vpt= 2.717454041D-06 0.000000000D+00 -
2.717454041D-06
7.829334641D+00 7.829334641D+00
Accept the new element data? [YES]: yes
```

MACOS> **fex** ***Find new exit pupil as well***

```
Enter number of exit pupil return surface: [15]: 15
Tracing 333 rays...
Chief ray location: x=-2.0000000D-05 y= 2.0000000D-05 z=
1.5300000D+01
Centroid location: x=-2.0042495D-05 y= 2.0042495D-05 z=
1.5300000D+01
Centroid offset from chief ray: x= 4.2495119D-08 y=-4.2495119D-08 z=-6.7501560D-14
Exit pupil finder results:
Old f= 6.329334641D+00 New f= 7.324539719D+00
```

	6.329334641D+00	
Old z=	New z=	7.324539719D+00
Old psi=	0.000000000D+00	-2.736348769D-06
	New psi=	
	0.000000000D+00	2.736348769D-06
	1.000000000D+00	1.000000000D+00
Old Vpt=	0.000000000D+00	1.263889675D-13
	New Vpt=	
	0.000000000D+00	-1.263889675D-13

```
8.970665359D+00 7.975460281D+00
Accept the new element? [YES]: yes
```

MACOS> **mod** ***This object is 100 times dimmer than the on-axis object***

```
MOD: [q]: flux=0.01 Flux = 1.000000000D-02 MOD: [q]: q
```

MACOS> **stretch**

```
Enter image stretch type (LINEAR, LOG10, SQRT): [LINEAR]: log10
```

MACOS> **int** ***This image is shown in Figure 54***

```
Enter number of element where data is to be generated: [16]: 6
Tracing 12661 rays and propagating 262144 grid points... Prop to reference point Elt 5 and Elt 6 to
WF 1:
```

```
z1= 6.3293D+00 dx1= 8.8773D-03 min= 8.8703D-03 max= 8.8832D-03 dev= 3.7730D-06 lin= 99.96%
z2= 1.0000D+30 dx2= 1.3925D-06
Compute time was 8.9688 sec Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D-02 u-v plane diam= 4.5362873D+00 du= 8.8772746D-03
x-y plane diam= 7.1158918D-04 dx= 1.3925425D-06
Peak intensity= 3.7145605D-04; Peak occurs at i= 257, j= 257 Sum of intensity= 8.6867251D-03
```

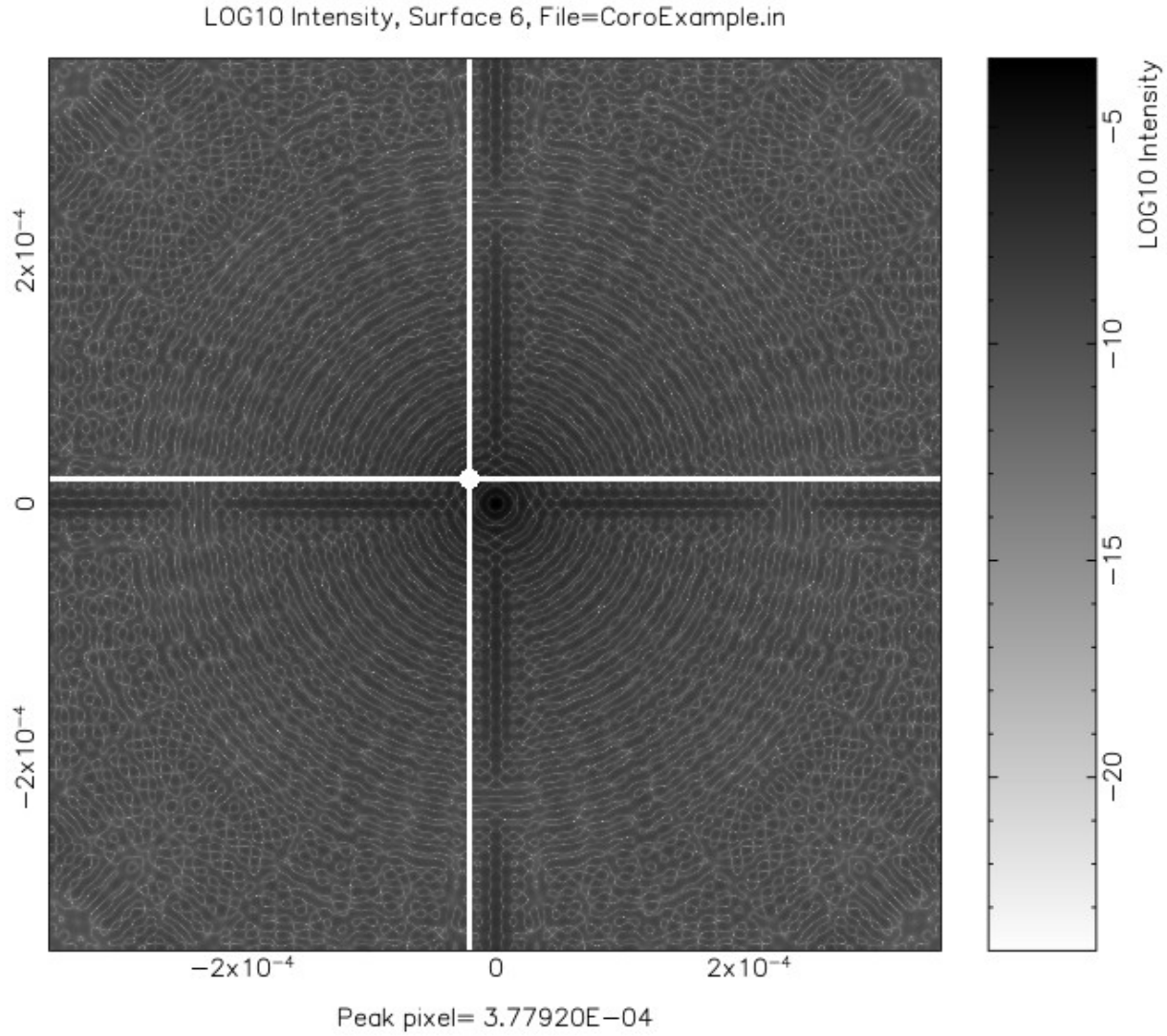


FIGURE 54 Off-axis image at the first focus, showing closeness to occulting mask.

MACOS> **add** ***Now the off-axis star is added to the composed image (Figure 55)***

```
Tracing 12661 rays and propagating 262144 grid points... Prop from reference point Elt 6 and Elt 7 to
WF 1:
z1= 1.0000D+30 dx1= 1.3925D-06
z2=-6.3293D+00 dx2= 8.8773D-03 min= 8.8703D-03 max= 8.8832D-03 dev= 3.7728D-06 lin= 99.96%
Scalar FF Prop between Elt 15 and Elt 16 to WF 1:
z1= 7.3245D+00 dx1= 1.0273D-02 min= 1.0265D-02 max= 1.0280D-02 dev= 4.3667D-06 lin= 99.96%
z2= 0.0000D+00 dx2= 1.3925D-06
Compute time was 14.0371 sec Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D-02
u-v plane diam= 5.2495594D+00 du= 1.0273110D-02 x-y plane diam= 7.1158918D-04 dx= 1.3925425D-06
Peak intensity= 2.2644562D-04; Peak occurs at i= 257, j= 257 Sum of intensity= 6.8026488D-03
Image added. Plot composed image? [YES]: yes
```

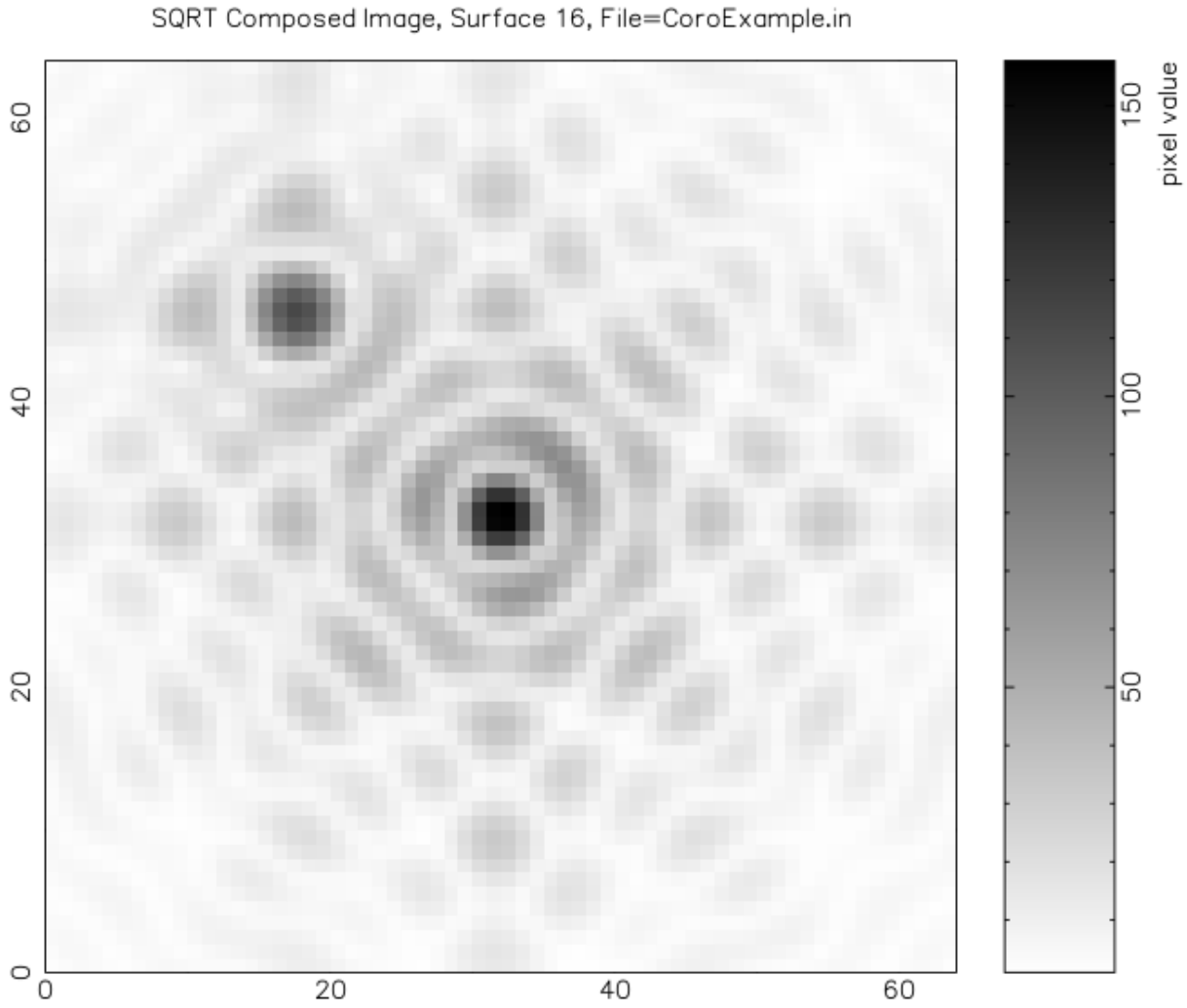


FIGURE 55 Off-axis image added to composed detector image. Note that the image dynamic range is about 1, while the objects differ by 100 in flux.

7.2.8 GBLUR

The GBLUR command is used to simulate “long-exposure” jitter effects or other phenomena that cause smooth spreading of the image. It propagates the beam to the specified surface, and then convolves the resulting intensity with a gaussian. It adds the result to the pixel array buffer (PixArray) and optionally displays it. The blurring is done at the WF sampling density, rather than the pixel density, which will generally be more accurate if the blurring occurs prior to the digitization of the image at the detector. Before using GBLUR, the user must invoke the COMPose command to set the detector pixel geometry. The width of the blur kernel is defined in the function arguments.

The following example sets up a typical case. The detector is first defined, at element 21, by specifying the number and size of the pixels. Note that the pixel size is specified to be the same as the WF pixel size (dx) for this example, which shows the blurred image at its highest resolution.

The GBLUR function is then invoked, first with a kernel width much smaller than the pixel size, and then with a kernel size much larger. The results

MACOS> **compose**

```
Enter element where image is to be composed: [21]: 21
Enter number of pixels per side: [128]: 128
Enter size of pixel: [1.000000E-03]: 1.0007177D-05
```

MACOS> **gblur**

```
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 1.6365868D-01 du= 1.2886510D-03
x-y plane diam= 1.2709115D-03 dx= 1.0007177D-05
Peak intensity= 9.5305603D-02; Peak occurs at i= 65, j= 65 Sum of intensity= 8.6054104D-01
Enter Gaussian blur kernel width in base units (0,0): [0.,0.]: 1d-6,1d-6
Image added. Plot composed image? [YES]: yes
```

Type <RETURN> for next page:

MACOS> **gblur**

```
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06; Source Flux= 1.0000000D+00 u-v plane diam= 1.6365868D-01 du= 1.2886510D-03
x-y plane diam= 1.2709115D-03 dx= 1.0007177D-05
Peak intensity= 9.5305603D-02; Peak occurs at i= 65, j= 65 Sum of intensity= 8.6054104D-01
Enter Gaussian blur kernel width in base units (0,0): [0.,0.]: 300d-6,300d-6
Image added. Plot composed image? [YES]: yes
```

Type <RETURN> for next page: MACOS>

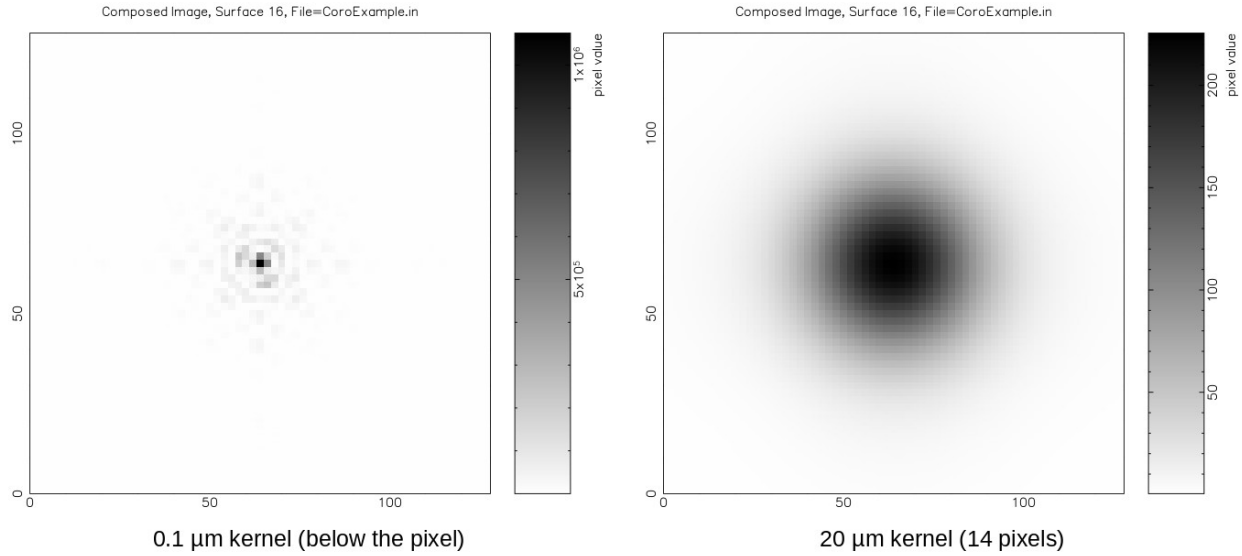


FIGURE 56 GBLUR example: the coronagraph's composed detector image (CoroExample.in, element 16, 128 pixels of 1.39 µm) after a 0.1 µm kernel, far below the pixel (left), and after a 20 µm kernel (right); each from a fresh COMPOSE.

7.2.9 DAD

The DAD command displays the current COMposed image. This is useful if you have forgotten what the composed image looks like, or if you want to view it using a different stretch or plot type. It has no arguments.

7.2.10 GAIN

The GAIn command computes the ratio of the light diffracted by the optical system to a uniformly radiating point source of the same intensity. It is used mainly to evaluate the far-field performance of

radio-frequency antennas. This command has one argument, the surface on which to compute the gain. An example is provided by the Luneberg Lens Antenna (Appendix A.3.3), and shown in Figure 58.

7.2.11 AMPLITUDE and PHASE

Each element in the MACOS wavefront propagation matrix is a complex number which can be expressed in terms of its real and imaginary components (i.e., a + b*i where a and b are real numbers). This complex number is a vector in the complex plane with two components, real and imaginary as shown in Figure 59.

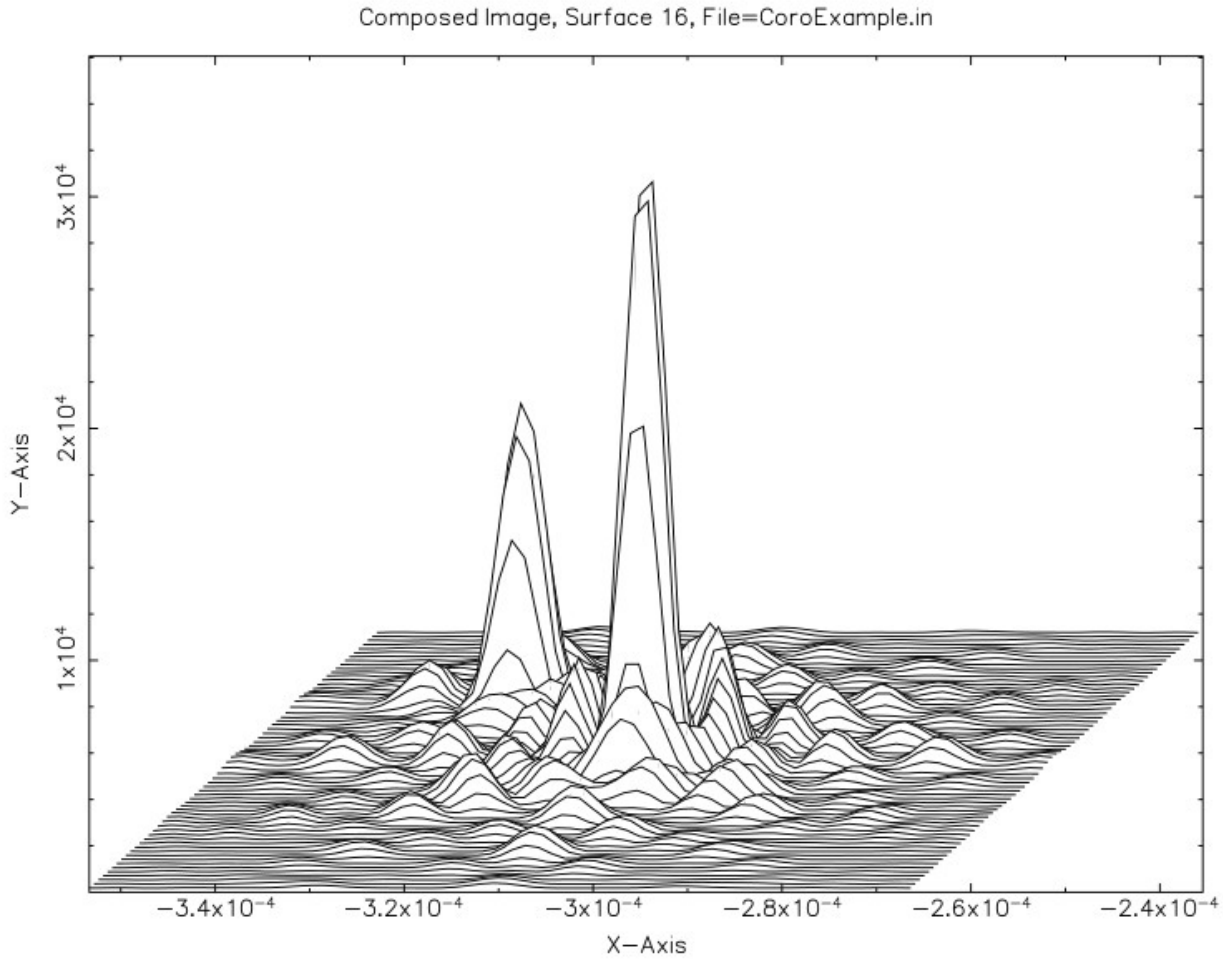


FIGURE 57 Composed detector image displayed at linear stretch, slice plot type.

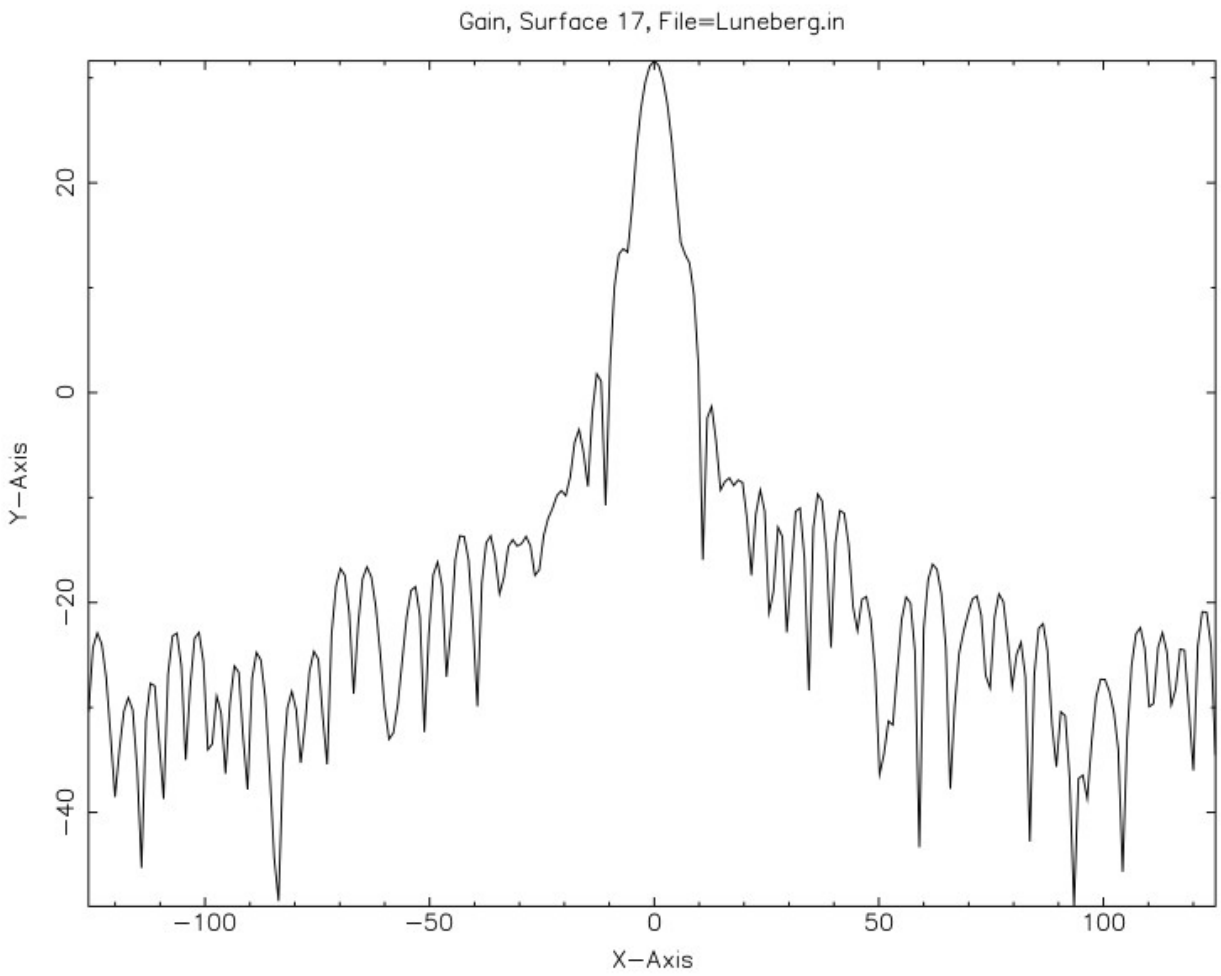


FIGURE 58 Far-field gain of Luneberg Lens Antenna.

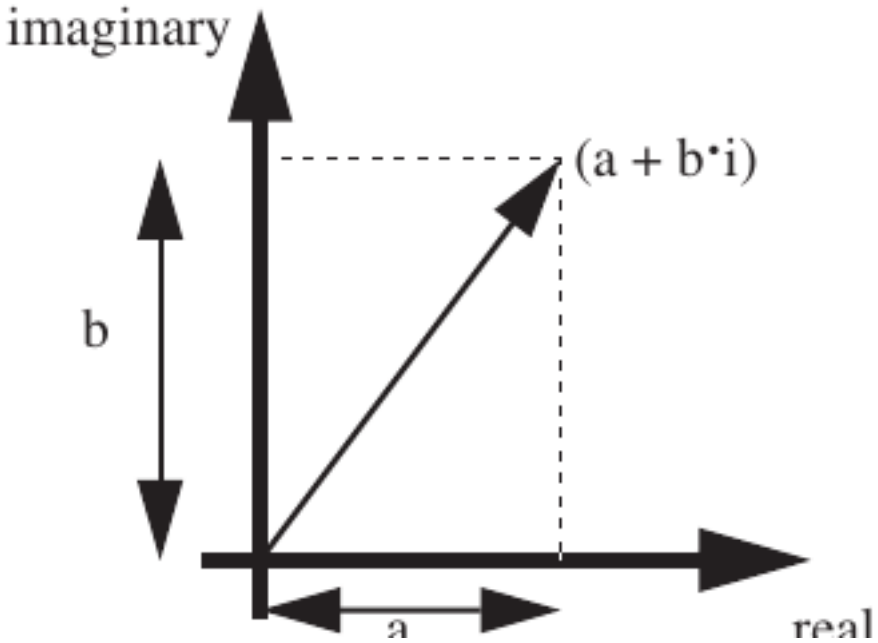


FIGURE 59 Graphing numbers in the complex plane

The AMplitude and PHASE commands directly plot the corresponding components of the complex amplitude matrix. Figure 60 provides an example. The first plot shows the amplitude or length of the vector. The second plot shows the phase of the vector, or the angle which the vector makes with the real axis modulo 360 degrees. The AMplitude command requires the wavefront element number nElt as input.

This example uses CassWithExitPupil.in in Appendix A.1.4, at the detector (element 6): AMplitude propagates to the element and plots the amplitude; PHAse plots the phase of the same field.

MACOS>**amp 6**

```
Enter number of element where data is to be generated: [6]: 6
Tracing       125 rays and propagating     65536 grid points...
===> 2: Far-field sphere-to-plane diffraction ...
 Scalar FF Prop between Elt  5 and Elt  6 to WF  1:
 z1= 5.5601D+00 dx1= 7.6216D-02 min= 7.6183D-02 max= 8.4127D-01 dev= 3.0183D-05 lin= 99.96%
 z2= 0.0000D+00 dx2= 2.8497D-07
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06;  Source Flux= 1.0000000D+00
u-v plane diam= 1.9435037D+01  du= 7.6215832D-02
x-y plane diam= 7.2667666D-05  dx= 2.8497124D-07
Peak amplitude= 1.0507790D+04; Peak occurs at i=  129, j=  129
```

MACOS>**phase 6**

```
Enter number of element where data is to be generated: [6]: 6
Wavefront Propagation Data Summary:
...
```

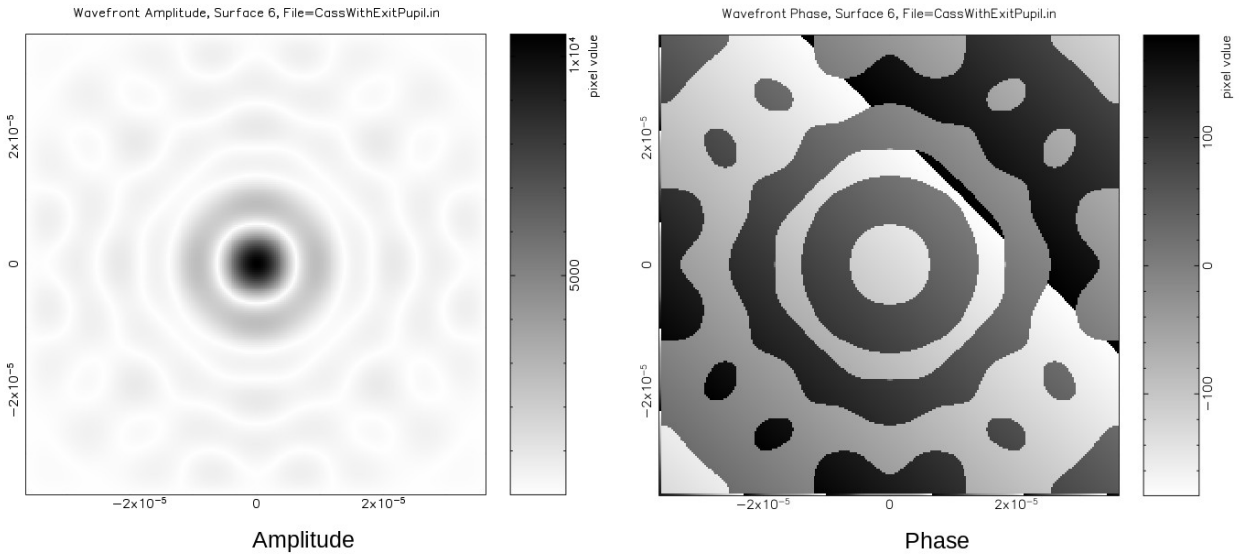


FIGURE 60 AMplitude and PHAse example

7.2.12 REAL and IMAGINARY

As discussed in Section 7.2.11, the wavefront propagation matrix elements are complex numbers. The REAl and IMAGINARY commands display these two components of the complex amplitude matrix. Figure 61 shows the real and imaginary components. The first graph displays the real component of each complex pixel number in the array. The second graph shows the imaginary component. The single command argument is the element number of interest.

This example uses CassWithExitPupil.in in Appendix A.1.4.

MACOS>**real 6**

```
Enter number of element where data is to be generated: [6]: 6
Wavefront Propagation Data Summary:
Wavelength= 1.0000000D-06;  Source Flux= 1.0000000D+00
u-v plane diam= 1.9435037D+01  du= 7.6215832D-02
x-y plane diam= 7.2667666D-05  dx= 2.8497124D-07
```

REAl plots the real part and then the imaginary part (two pages):

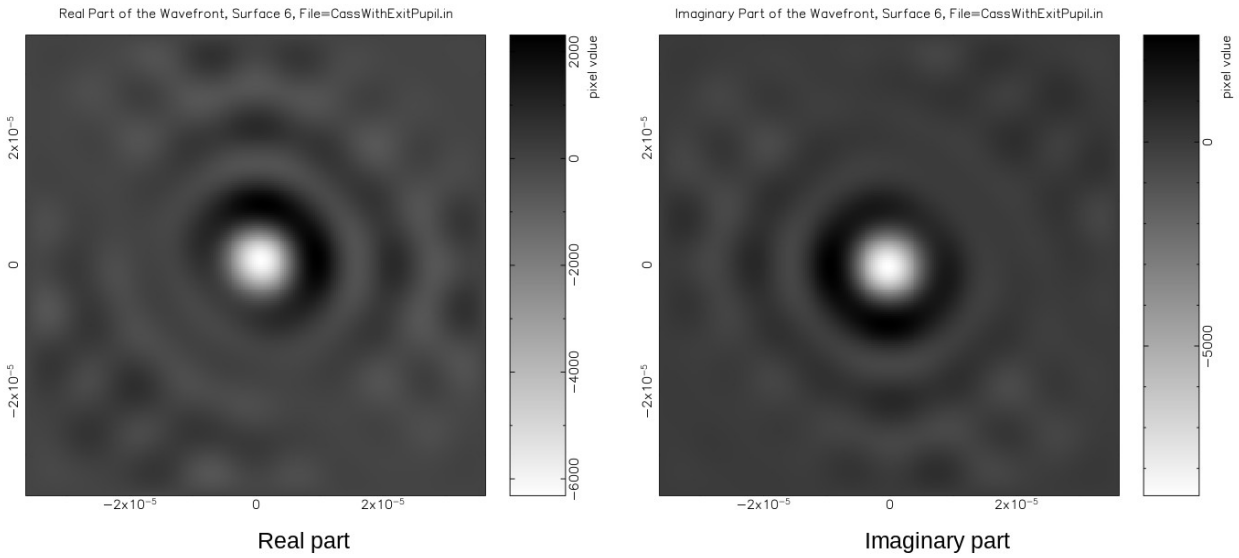


FIGURE 61 REAl example

7.3 Image polarity and stretch-aware labels

Image polarity (IMGMODE). Image-type plots (INTensity, PIXillated, OPD-as-image) can be rendered in either of two polarities:

- Astronomical (default, ‘ASTRO’ or ‘NEG’): large pixel values render dark, small values render bright. Matches the PGPlot convention many MACOS users are accustomed to.
- Conventional (‘CONV’ or ‘POS’): large values bright, small values dark. Matches general scientific imaging.

Toggle with the IMGMODE command:

```
MACOS> IMGMODE
Enter image polarity (NEG/POS, ASTRO/CONV): [NEG]: CONV
```

The setting persists for the session. It does not affect the non-image plot types (CONTOUR, SLICE, WIRE, SPOTDIAG, PLOTCOL), which always render with black ink on white.

Stretch-aware colorbar labels. When STRETCH is active (LOG10 or SQRT), the colorbar wedge label reflects the active stretch:

- INT, STRETCH=LIN: label ‘Intensity’
- INT, STRETCH=LOG10: label ‘LOG10 Intensity’
- INT, STRETCH=SQRT: label ‘SQRT Intensity’
- PIX, STRETCH=LOG10: label ‘LOG10 Pixel value’
- OPD with BaseUnits: label ‘OPD (mm)’ etc.

The label is set by the command handler before the draw routine emits it, so the polarity / stretch and the label always match.

Bottom-of-plot annotation. Most image plot types now print one or two annotation lines just below the colorbar:

- OPD: ‘OPD=<rms> <unit> RMS, <pv> <unit> P-V’
- SPOT: ‘RMS spot radius=<r> <unit>’
- INT: ‘Peak pixel=<MaxInt>’
- PIX: ‘Peak pixel=<maxval>’

7.4 Plot Type Commands

7.4.1 GRAY

The GRAy command resets the type of plot to GRAy scale. This is the default plot type. It has no arguments.

7.4.2 WIRE

The WIRe command takes all the values in an array and first plots them as three dimensional points. Then each point is then connected to its nearest neighbor on the grid by a line. Perspective is added and the obscured lines are removed. The result is a surface-like representation of the data as shown in Figure 62. There is no scale on the graph. The WIRe command is always used in conjunction with other commands and has no arguments.

MACOS>**wire**

```
Plot type set to WIRE
```

MACOS>**dad**

Figure 62 is the coronagraph’s composed detector image of Figure 55 (64 x 64 pixels) redrawn under the WIRE plot type as the current build draws it. The WIRE renderer is the hidden-line wireframe inherited from PGPLOT; under the built-in graphics it draws the sheet’s outline with the mesh all but invisible, and a 256-point intensity grid draws as a single spike. Its rendering is under review (the plan’s display items); use SLIce (Figure 57) for a surface view in the meantime.

7.4.3 SLICE

The SLIce command is very similar to the WIRe command in that it displays the array of data as a surface. In this instance, the graph also contains a scale so the data values can be read directly. Like the WIRe command, SLIce is used with other commands and has no arguments.

Figure 57 provides an illustration of the SLICE plot type.

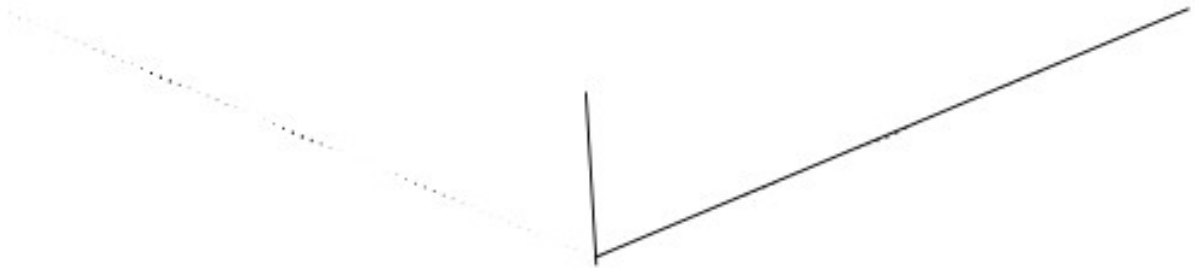


FIGURE 62 WIRe plot

7.4.4 CONTOUR

The CONtour command sets the plot type to display data using a contour map (an elevation map). Data points with similar values are connected by the same line. The CONtour map shows graphically those points with similar values. The CONtour command has no arguments.

This example uses CassWithExitPupil.in in Appendix A.1.4. Figure 63 shows the results.

MACOS>**con** MACOS>**opd**

```
Enter element where OPD is to be evaluated:5 Average OPD is -1.894794E-12
Average total path is 1.5622291803998E+01 Average delta path is -3.7770819600190E-01 RMS OPD error is
1.091947E-12
```

Type <RETURN> for next page:

MACOS>

7.4.5 COLUMN and ROW

The COLumn and ROW commands set the plot type to display a slice across the data surface. The slice is along a column or row of the data matrix, respectively. The user is prompted for the column or row number when the data is to be plotted out (for instance, when an INTensity command is issued). The COLumn and ROW commands have no arguments.

This example uses CassWithExitPupil.in in Appendix A.1.4. Figure 64 shows the results.

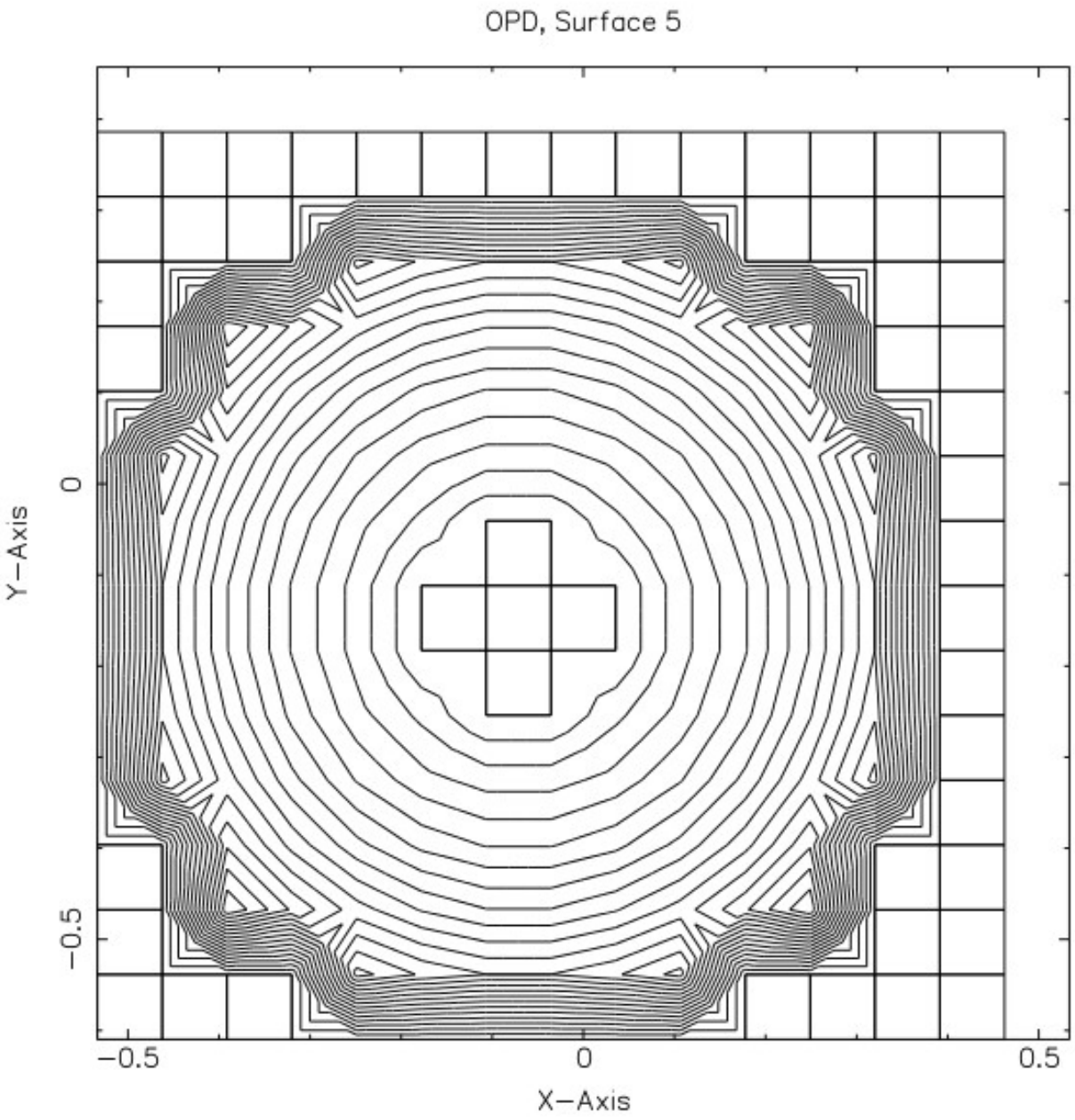


FIGURE 63 CONtour example

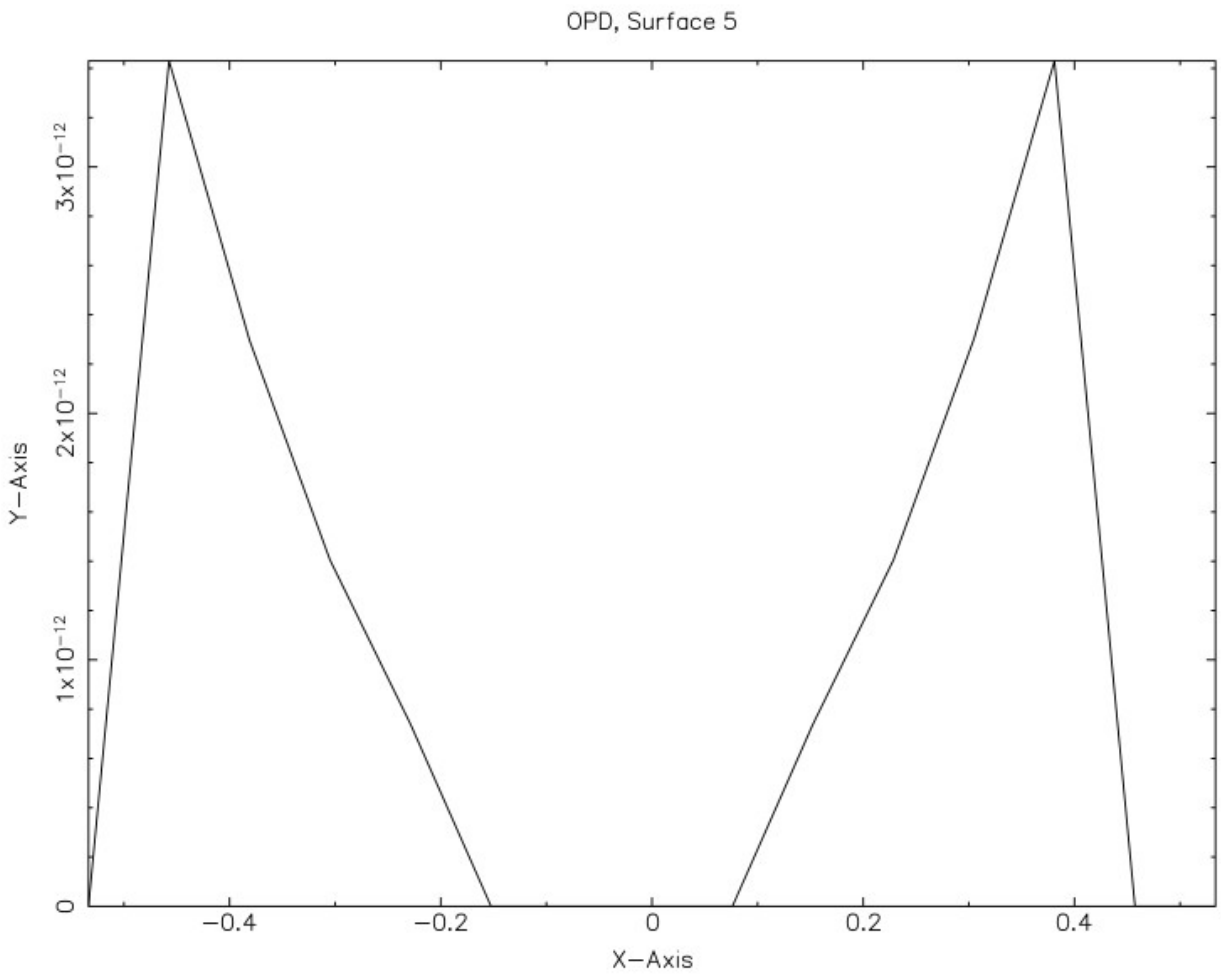


FIGURE 64 COLumn example

7.4.6 TEXT

The TEXt command writes the data points in the array to a file for further analysis. The file is in ASCII format and can be printed or read into a text editor. There are up to mdttl rows of numbers, each with up to mdttl numbers across: this can be quite a large file! The data points are the same as numbers used to create the graphical display. They are single precision. The TEXt command creates files as output and has no arguments. The filenames specify the contents and are of the format: [filename].[image type][surface number].txt where [image type] can be in (intensity), am (amplitude), ph (phase), re (real), im (imaginary), spot, and OPD.

This example uses CassWithExitPupil.in in Appendix A.1.4. The TEXt file is stored in **CassWithExitPupil.int5.txt**

MACOS>**text** MACOS>**intensity**

```
Enter element where WF is to be plotted:5 Wavefront plane 1 is evaluated at this element. Wavefront
Propagation Data Summary:
Wavelength= 1.0000000D-06; Transmission Distance= 5.5601459D+00 u-v plane diam= 8.4230375E+00 du=
3.3031520E-02
x-y plane diam= 1.6767097E-04 dx= 6.5753318E-07 Writing Wavefront Intensity, Elt=5
FORMATTED File=CassWithExitPupil.int5.txt MACOS>
```

7.4.7 BINARY

The BINary command writes numerical files which can be used by other signal/image processing software. It takes the array data and writes it out to a file in BINary format. The BINary command produces files for output and has no arguments. The filenames specify the contents and are of the format: [filename].[image type][surface number] where [image type] can be in (intensity), am (amplitude), ph (phase), re (real), im (imaginary), spot, and OPD.

7.4.8 MATLAB

The MATlab command writes the data arrays to MATlab format. The MATlab command produces files for output and has no arguments. The filenames specify the contents and are of the format: [filename].[image type][surface number].dat where [image type] can be in (intensity), am (amplitude), ph (phase), re (real), im (imaginary), spot, and OPD.

7.4.9 FITS

The FITs command writes the data arrays to fits format files. The FITs command produces files for output and has no arguments. The filenames specify the contents and are of the format: [filename].[image type][surface number].fit, where [image type] can be in (intensity), am (amplitude), ph (phase), re (real), im (imaginary), and OPD.

SECTION 8: Differential Ray-Tracing and Linear Optical Models

The MACOS differential ray-trace functions are used to compute optical sensitivity matrices. These provide linear models of the effects of optical element perturbations on optical performance. The linear models can be exercised internally to MACOS using functions such as LPErturb, LOPd and LINtensity which parallel commands introduced earlier. Linear optical models can also be EXPported for use external to MACOS.

```
BUild DMBuild PArtials LPErturb LPRead LREset LSPot LOPd
LPIxillate LINtensity EXPort
```

8.1 Background

linear models of optical systems provide a linear matrix transformation between element perturbations and ray perturbations. These optical models can be used in place of the full-blown ray-trace calculations for systems that undergo only small changes. They can also be used in ways that the nonlinear models cannot, such as in linear statistical studies.

Linear models are generally valid where

- The small-angle approximation is valid for changes in ray direction
- The ray transverse aberration is “small” relative to the curvature of the optical elements

The limits are not absolute, but vary from system to system. The user can check the range of validity of a linear model for a particular system. This is done by using the MACOS LPErtrurb, LOPd, LSPot and LINtensity commands to exercise the linear model. Results are compared with exact results produced for the same cases using the MACOS PERturb, OPD, SPOt and INtensity commands. If, over the expected range of perturbation values, the linear models produce results that are within the accuracy requirements of the particular analysis, then they can be safely used for that analysis. Otherwise a nonlinear SMACOS model should be used.

8.1.1 Linear Ray States

Rays are defined at any particular point in a beam train by their location, direction and length. In MACOS, the current *exact ray state* is defined as the position, direction and path length of a ray immediately after the current element (see Figure 65). The *position* of the iRayth ray is a 3-vector giving the point of incidence with the current element. MACOS stores it in the array RayPos as RayPos(1:3,iRay). The *direction* of the iRayth ray is a unit-magnitude 3-vector stored in RayDir(1:3,iRay). Note that if the current element is a mirror or lens, the direction is of the reflected or refracted ray. The *optical path length* of the iRayth ray from the source to the current element is a scalar and is stored in CumRayL(iRay).

and is stored in `CumRayL(iRay)`.

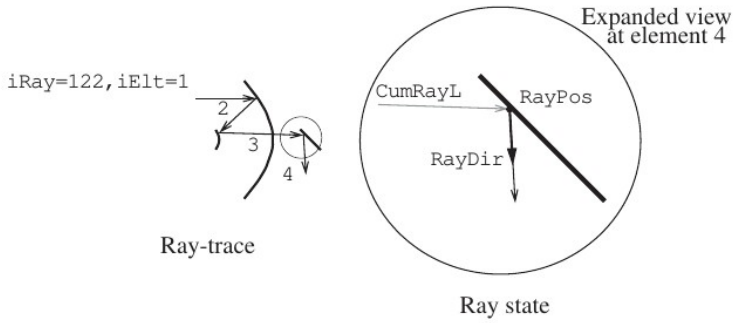


FIGURE 65 Ray state

The effect of a *perturbation* in the optical system, such as a rotation or translation of an optical element, is to *change the ray state* at every downstream element. Such perturbations can be applied using the PERturb command, and the ray retraced to find the new ray state.

If a perturbation is sufficiently small, it will effect the rays *linearly*. That is, the new ray state at every downstream element is very nearly a linear function of the perturbation. This is true for some range of perturbation magnitudes that varies with each system, but is in general many times the wavelength of light.

MACOS linear modeling features use *analytic differential* ray-trace equations to compute the partial derivatives of the rays with respect to element perturbations. The differential ray-trace equations are derived in References 1, 8, and 9. They provide a set of analytical formulas giving the ray partial derivative matrices in a form that is easily computed.

The *linear ray state* differs slightly from the exact ray state. The linear ray state is a single 7-vector, composed of a 3-vector of *ray direction perturbation*, a 3-vector of *ray transverse aberration*, and a scalar for *ray optical path difference*. It is written x

$$\hat{x} = \begin{bmatrix} d\hat{r} \\ d\hat{\gamma} \\ dL \end{bmatrix} \quad (8.1)$$

where $d\hat{r}$ is the ray direction perturbation. The ray direction perturbation is a 3-vector whose magnitude is small compared to 1 and which is perpendicular to the ray direction, RayDir. The total ray direction is equal to the vector sum of RayDir and $d\hat{r}$ (to the accuracy of the small angle approximation). Figure 66 shows the linear ray state.

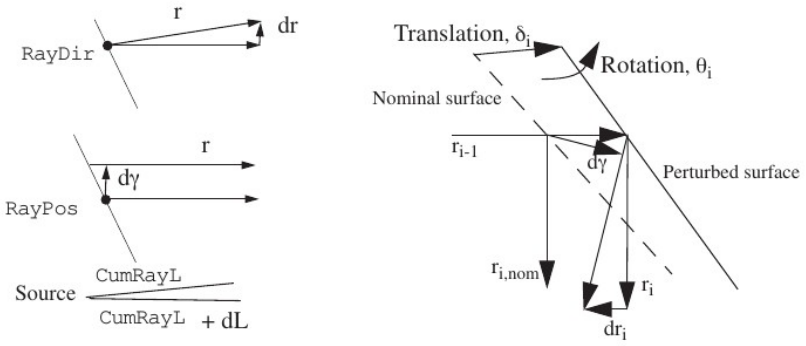


FIGURE 66 Linear ray state

The transverse aberration γ is the lateral displacement of the ray. It is a vector perpendicular to the ray direction, from the nominal ray position to a point on the perturbed ray. It is not necessarily located on the surface of the element and is not, in general, the derivative of the ray position, RayPos. Exceptions are the focal-plane element type and any surface that is perpendicular to the nominal ray direction.

The path length perturbation, dL, is the difference between the optical path length of the perturbed and nominal rays. It is not the OPD, as that is the difference of the ray and the chief ray. However, the OPD is easily computed from dL as discussed below.

8.1.2 Linear Models

The linear ray state at a particular element is a function of perturbations of the upstream elements and of perturbations in the input ray. These perturbations are with respect to a "nominal state" $(xn)nom$, which may or may not be zero. For each ray at an element n, the linear ray state has the functional form:

$$\hat{x}_n = \left(\hat{x}_n\right)_{nom} + \frac{\partial \hat{x}_n}{\partial \hat{u}_1}\hat{u}_1 + \cdots + \frac{\partial \hat{x}_n}{\partial \hat{u}_n}\hat{u}_n + \frac{\partial \hat{x}_n}{\partial \hat{x}_0}\hat{x}_0 \quad (8.1)$$

Here the *u* vectors are 6-vectors of element perturbations, composed of 3-vectors θ for element rotational perturbation and δ for element translation perturbation:

$$\hat{u} = \begin{bmatrix} \hat{\theta} \\ \hat{\delta} \end{bmatrix} \quad (8.2)$$

The individual optical sensitivity matrices can be printed using the PARtials command. The sensitivity of the nth ray to perturbations of the input (source) ray is a 7 by 7 matrix of the form:

$$\frac{\partial \hat{x}_n}{\partial \hat{x}_0} = \begin{bmatrix} \frac{\partial \hat{r}_n}{\partial \hat{r}_0} & \frac{\partial \hat{r}_n}{\partial \hat{\gamma}_0} & \frac{\partial \hat{r}_n}{\partial L_0} \\ \frac{\partial \hat{\gamma}_n}{\partial \hat{r}_0} & \frac{\partial \hat{\gamma}_n}{\partial \hat{\gamma}_0} & \frac{\partial \hat{\gamma}_n}{\partial L_0} \\ \frac{\partial L_n}{\partial \hat{r}_0} & \frac{\partial L_n}{\partial \hat{\gamma}_0} & \frac{\partial L_n}{\partial L_0} \end{bmatrix} \quad (8.3)$$

The sensitivity of the nth ray to perturbations of the ith element is a 7 by 6 matrix of the form:

$$\frac{\partial \hat{x}_n}{\partial \hat{u}_i} = \begin{bmatrix} \frac{\partial \hat{r}_n}{\partial \hat{\theta}_i} & \frac{\partial \hat{r}_n}{\partial \hat{\delta}_i} \\ \frac{\partial \hat{\gamma}_n}{\partial \hat{\theta}_i} & \frac{\partial \hat{\gamma}_n}{\partial \hat{\delta}_i} \\ \frac{\partial L_n}{\partial \hat{\theta}_i} & \frac{\partial L_n}{\partial \hat{\delta}_i} \end{bmatrix} \quad (8.4)$$

The complete expression for the linear ray state for a single ray in terms of element and input-ray perturbations is written in matrix form as:

$$\hat{x} = (\hat{x}_n)_{nom} + \begin{bmatrix} \frac{\partial \hat{x}_n}{\partial \hat{x}_0} & \frac{\partial \hat{x}_n}{\partial \hat{u}_1} & \cdots & \frac{\partial \hat{x}_n}{\partial \hat{u}_n} \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{u}_1 \\ \cdots \\ \hat{u}_n \end{bmatrix} \quad (8.5)$$

The same form is extended to give the linear ray state for multiple rays at the same element

$$\begin{bmatrix} \hat{x}_{ray1} \\ \cdots \\ \hat{x}_{raym} \end{bmatrix} = \begin{bmatrix} \hat{x}_{ray1} \\ \cdots \\ \hat{x}_{raym} \end{bmatrix}_{nom} + \begin{bmatrix} \begin{bmatrix} \frac{\partial \hat{x}_n}{\partial \hat{x}_0} & \frac{\partial \hat{x}_n}{\partial \hat{u}_1} & \cdots & \frac{\partial \hat{x}_n}{\partial \hat{u}_n} \end{bmatrix}_{ray1} \\ \cdots \\ \begin{bmatrix} \frac{\partial \hat{x}_n}{\partial \hat{x}_0} & \frac{\partial \hat{x}_n}{\partial \hat{u}_1} & \cdots & \frac{\partial \hat{x}_n}{\partial \hat{u}_n} \end{bmatrix}_{raym} \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{u}_1 \\ \cdots \\ \hat{u}_n \end{bmatrix} \quad (8.6)$$

Both of these equations have the form *x = xnom + Cu* , where the *C-matrix* is the *optical sensitivity matrix*, giving the change in ray state due to perturbations of the optical elements. The C-matrix provides a linear model of the optical system. The MACOS BUild command assembles these sensitivities from the individual ray partial derivatives.

It can be extremely useful to incorporate local coordinates directly into the optical sensitivity C-matrices. If the perturbations for one element are to be applied in coordinates other than the global coordinate system–perhaps in an actuator coordinate system–then that local coordinate transformation matrix can be included directly within the C-matrix. The transform from element iElt coordinates to global coordinates is written ${}^{G}T^{iElt}$.

Similarly, local coordinates can be used at the output end of the linear model. This allows for the creation of models for ray direction only, or transverse aberration or path length only. Focal-plane axes are easily incorporated. We write the transform for ray state from global coordinates to output coordinates as ${}^{out}T^{G}$. As an example, the 1x7 output transform matrix that captures only the path length ray state in global coordinates is

$$ {}^{pathlength}T^{G} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (8.7) $$

As another example, the 2x7 output transform which takes the transverse aberration in ray state and casts it into detector coordinates *x* and *y* is

$$ {}^{detector}T^{G} = \begin{bmatrix} 0 & 0 & 0 & \hat{x} & 0 \\ 0 & 0 & 0 & \hat{y} & 0 \end{bmatrix} \quad (8.8) $$

Both actuator and detector coordinates are specified by the user as part of the .in-file prescription.

With the input and output coordinate matrices included, the C-matrix takes the form:

$$ \begin{bmatrix} \hat{x}_{ray1} \\ \cdots \\ \hat{x}_{raym} \end{bmatrix} = \begin{bmatrix} {}^{out}T^{G} \left[\frac{\partial \hat{x}_n}{\partial \hat{x}_0} {}^{G}T^{0} \frac{\partial \hat{x}_n}{\partial \hat{u}_1} {}^{G}T^{1} \cdots \frac{\partial \hat{x}_n}{\partial \hat{u}_n} {}^{G}T^{n} \right]_{ray1} \\ \cdots \\ {}^{out}T^{G} \left[\frac{\partial \hat{x}_n}{\partial \hat{x}_0} {}^{G}T^{0} \frac{\partial \hat{x}_n}{\partial \hat{u}_1} {}^{G}T^{1} \cdots \frac{\partial \hat{x}_n}{\partial \hat{u}_n} {}^{G}T^{n} \right]_{raynRay} \end{bmatrix} \begin{bmatrix} \hat{x}_0 \\ \hat{u}_1 \\ \cdots \\ \hat{u}_n \end{bmatrix} \quad (8.9) $$

8.2 Commands for Building Models

8.2.1 BUILD

The BUild command traces all of the rays in a beam from the source to the specified final element. It then computes the full C-matrix linear model, giving the sensitivity of each ray at the last element to perturbations of each element in the beam train. It has one argument, the number iElt of the last element in the propagation.

BUild performs the MACOS differential ray-trace calculations, and must be performed before exercising commands that manipulate linear models, such as LPE, LOP, LSP, LIN, or before EXPorting C-matrix information.

Note that the BUild command ignores the presence of reference surfaces and obscuring surfaces unless they are the last surface. The BUild command currently does not work if there are return surfaces in the beam train.

MACOS>**build**

As an example, consider the Cassegrain.in (see Appendix A.1.2). This prescription has output coordinates set up to compute a wavefront model as discussed previously. Once the prescription is loaded using the OLD command, the C-matrix is easily built:

```
Enter terminal element number:4
Tracing 150 rays and BUILDing linear model... Compute time was 0.4141 sec
```

MACOS>

8.2.2 DMBUILD

The DMBuild command is not used in this version.

8.3 Commands for Changing Models

8.3.1 LPERTURB

The LPErturb command allows the user to enter an element perturbation for use with the linear model analysis commands. LPErturb takes different arguments depending on whether element input coordinates are to be used. The first argument iElt is the number of the element to be perturbed. Note that element 0 (the source) cannot be perturbed using LPErturb. The second argument is a yes-or-no response indicating whether input coordinates are to be used. If NO is selected, the next two prompts are for rotational and translational 3-vector perturbations. If YES is selected, a vector of the same length as the number of input coordinates is input in the input coordinate system.

After each LPErturb command, the current perturbation vector (in global coordinates) is printed out. The LPErturb command *adds* the perturbation data to the existing perturbation vector. Multiple LPErturb commands sum each perturbation with the previous perturbation vector. The perturbation vector can be reset to zero using the LREset command. Using Cassegrain.in:

MACOS>**lperturb**

```
Enter element to be perturbed:3 Use element coordinates? (NO):
Enter rotational perturbation vector (x,y,z):0,1e-7,0 Enter translational perturbation vector
(x,y,z):0,0,0
Element 1 Linear Perturbations:
Rotational= 0.000000D+00 0.000000D+00 0.000000D+00 Translational= 0.000000D+00 0.000000D+00
0.000000D+00
Element 2 Linear Perturbations:
Rotational= 0.000000D+00 0.000000D+00 0.000000D+00 Translational= 0.000000D+00 0.000000D+00
0.000000D+00
Element 3 Linear Perturbations:
Rotational= 0.000000D+00 1.000000D-07 0.000000D+00 Translational= 0.000000D+00 0.000000D+00
0.000000D+00
Element 4 Linear Perturbations:
Rotational= 0.000000D+00 0.000000D+00 0.000000D+00 Translational= 0.000000D+00 0.000000D+00
0.000000D+00
Element 5 Linear Perturbations:
Rotational= 0.000000D+00 0.000000D+00 0.000000D+00 Translational= 0.000000D+00 0.000000D+00
0.000000D+00 MACOS>
```

8.3.2 LPREAD

The LPRead command reads element perturbations from a specified datafile. MACOS assumes that the name of the perturbation file is infile.pert. If this file does not exist, MACOS prompts for the perturbation file name.

The perturbation file must be in the following all numeric format. The perturbations must be in global coordinates. The order of the perturbation vectors in the .pert-file must be identical to the order of the elements (the first perturbation vector corresponds to the first element, the sixth perturbation vector corresponds with the sixth element). Each element must have a perturbation vector. If an element is unperturbed, then a string of zeroes should be entered as its perturbation vector. The perturbation vectors have 6 components: a 3-vector rotational perturbations and a 3-vector translational perturbations. The 6 components must be separated by spaces.

MACOS>**lpr** FileCassegrain.pert read. MACOS>

8.3.3 LRESET

The LREset command zeros any previously entered linear perturbations. It should be used, when necessary, to reset the linear perturbation vector after LPErturb commands.

8.4 Commands for Exercisizing Models

8.4.1 PARTIALS

The PARtials command prints individual optical sensitivity matrices for specified rays. The user is asked whether to print in input/output or global coordinates. For Cassegrain.in, the partials of the chief ray in global coordinates are:

MACOS>**partials**

```
Enter ray number for partials (0=done, 1=chief ray):1 Print partials in actuator/sensor coordinates?
(YES): no
Partial of ray 1 at Element 4 (Ref_surf) to ray at Element 0 (Inpu-tRay)
0.3036E+01 0.0000E+00 0.0000E+00 -0.4458E-01 0.0000E+00 0.0000E+00
0.0000E+00
0.0000E+00 0.3036E+01 0.0000E+00 0.0000E+00 -0.4458E-01 0.0000E+00
0.0000E+00
0.0000E+00 0.0000E+00 0.1000E+01 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00
0.5549E+01 0.0000E+00 0.0000E+00 0.2479E+00 0.0000E+00 0.0000E+00
0.0000E+00
0.0000E+00 0.5549E+01 0.0000E+00 0.0000E+00 0.2479E+00 0.0000E+00
0.0000E+00
0.0000E+00 0.0000E+00 0.1006E+02 0.0000E+00 0.0000E+00 0.1000E+01
0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.1000E+01
Partial of ray 1 at Element 4 (Ref_surf) to Element 2 (Primary ) per-turbations
0.0000E+00 -0.6606E+01 0.0000E+00 0.6117E+00 0.0000E+00 0.0000E+00
0.6606E+01 0.0000E+00 0.0000E+00 0.0000E+00 0.6117E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
```

0.0000E+00	-0.8122E+01	0.0000E+00	0.0000E+00	0.7500E+00	0.0000E+00
0.8122E+01	0.0000E+00	0.0000E+00	0.0000E+00	0.7500E+00	0.0000E+00
0.0000E+00	0.0000E+00	0.0000E+00	0.0000E+00	0.0000E+00	0.0000E+00
0.0000E+00	0.0000E+00	0.0000E+00	0.0000E+00	0.0000E+00	0.2000E+01

```
Partial of ray 1 at Element 4 (Ref_surf) to Element 3 (Secondar) per-turbations
0.0000E+00 0.1241E+01 0.0000E+00 -0.5671E+00 0.0000E+00 0.0000E+00
-0.1241E+01 0.0000E+00 0.0000E+00 0.0000E+00 -0.5671E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 -0.2000E+01
Partial of ray 1 at Element 4 (Ref_surf) to Element 4 (Ref_surf) per-turbations
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
Enter ray number for partials (0=done, 1=chief ray):
```

The specific partials take the form in Eqns. 8.3 and 8.4. The first matrix shows that source chief ray direction perturbations are magnified 3.036 times at the reference surface. The effect on ray direction at the reference surface of primary mirror tilts and decenters are magnified by 6.606 and 0.5671 times, respectively.

The path length partials show a factor-of-two sensitivity for this ray. Looking at the OPD partials only for a marginal ray, these numbers change:

```
Enter ray number for partials (0=done, 1=chief ray):2 Print partials in actuator/sensor coordinates?
(YES): y
Partial of ray 2 at Element 4 (Ref_surf) to ray at Element 0 (Inpu-tRay)
0.2996E+01 -0.2639E-17 0.1976E-16 -0.4450E-01 0.3920E-19 0.0000E+00
0.0000E+00
-0.2639E-17 0.2984E+01 0.8899E-01 0.3920E-19 -0.4432E-01 0.0000E+00 0.0000E+00
0.5643E+01 -0.4971E-17 0.1977E-15 0.2500E+00 -0.2202E-18 0.1976E-
16 0.0000E+00
-0.4971E-17 0.5620E+01 0.8902E+00 -0.2202E-18 0.2490E+00 0.8899E-01 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.1000E+01
Partial of ray 2 at Element 4 (Ref_surf) to Element 2 (Primary ) per-turbations
```

			0.	0000 E+0			
-0. 4462E-16		-0 .6401E+01		0	0 .5827E+00	-0 .8407E-17	0. 4262 E-16
0.	6200 E+0 1	0 .4462E-16	0.	0000 E+0 0	-0 .8407E-17	0 .5448E+00	0. 1920 E+0 0
-0. 1069E-15		-0 .8239E+01	0.	0000 E+0 0	0 .7500E+00	-0 .1533E-16	0. 7920 E-16
0.	7757 E+0 1	0 .1069E-15	0.	0000 E+0 0	-0 .1533E-16	0 .6810E+00	0. 3567 E+0 0
-0. 3934E+01		0 .8735E-15	0.	0000 E+0 0	-0 .7951E-16	-0 .3581E+00	0. 1934 E+0 1

```
Partial of ray 2 at Element 4 (Ref_surf) to Element 3 (Secondar) per-turbations
-0.3440E-17 0.1228E+01 0.5551E-16 -0.5382E+00 0.8367E-17 -0.4262E-16
-0.1244E+01 0.3440E-17 0.0000E+00 0.8367E-17 -0.5005E+00 -0.1920E+00 0.3146E-16 -0.6986E-32
0.0000E+00 0.3062E-32 0.1379E-16 -0.9888E-16
0.1417E+00 -0.3146E-16 0.0000E+00 0.1379E-16 0.6210E-01 -0.4453E+00
0.6140E+00 -0.1363E-15 0.0000E+00 0.5975E-16 0.2691E+00 -0.1930E+01
Partial of ray 2 at Element 4 (Ref_surf) to Element 4 (Ref_surf) per-turbations
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00

0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00 0.0000E+00
Enter ray number for partials (0=done, 1=chief ray):
```

8.4.2 LOPD

The LOPd command generates an OPD map of the beam at the final surface of the linear model. It parallels the OPD command discussed in Section 5.2. Using Cassegrain.in:

MACOS>**lopd**

```
Tracing perturbed system Average OPD is 1.822578E-12
RMS OPD including piston is 3.023756E-08 RMS WFE excluding piston is 3.023755E-08 Type <RETURN> for
next page:
```

8.4.3 LSPOT

The LSPot command generates a spot diagram of the beam at the final surface of the linear model. It parallels the SPOt command discussed in Section 5.2. Using Cassegrain.in:

MACOS>**lspot**

```
Tracing perturbed system Average OPD is -9.502741E-07
RMS OPD including piston is 1.081200E-01 RMS WFE excluding piston is 1.081200E-01 Type <RETURN> for
next page:
```

8.4.4 LINTENSITY

The LINtensity command computes beam wavefront phase at the final surface of the linear model and then performs a single far-field propagation to the next element in the prescription. If the last element in the linear model is the last element in the prescription, the user is prompted for transmission distance and other information. LINtensity performs calculations that parallel the propagation commands discussed in Sections 6.3 and 7.2.1, respectively. Using Cassegrain.in:

MACOS>**lintensity** Tracing perturbed system

```
Average OPD is 1.822578E-12
RMS OPD including piston is 3.023756E-08 RMS WFE excluding piston is 3.023755E-08 FFT/Point Spread
Function Data Summary:
Wavelength= 9.9999999748E-07; Transmission Distance= 5.5601458549E+00 u-v plane diam=
1.8044868469E+01 du= 7.0764191449E-02
x-y plane diam= 7.8265948105E-05 dx= 3.0692527275E-07
```

Type <RETURN> for next page:

8.4.5 LPIXILATE

The LPIxilate command computes beam wavefront phase at the final surface of the linear model and then performs a single far-field propagation to the next element in the prescription. If the last element in the linear model is the last element in the prescription, the user is prompted for transmission distance and other information. Finaly, the image

MACOS>**lpix**

is pixilated. The LPIxilate command parallels the PIxilate command discussed Section 7.2.2. Using Cassegrain.in:

```
Tracing perturbed system Average OPD is 1.822375E-12
RMS OPD including piston is 2.104591E-12 RMS WFE excluding piston is 1.052736E-12 FFT/Point Spread
Function Data Summary:
Wavelength= 9.9999999748E-07; Transmission Distance= 5.5601458549E+00 u-v plane diam=
1.8044868469E+01 du= 7.0764191449E-02
x-y plane diam= 7.8265948105E-05 dx= 3.0692527275E-07
Enter number of pixels per side:10 Enter size of pixel:1e-7
```

Type <RETURN> for next page:

8.5 Commands for Exporting Models

The EXPort command provides a way to export the C-Matrix of the linear model and is discussed in Section 3.8.5.

SECTION 9: Subroutine MACOS and User Extensions

Subroutine MACOS (SMACOS) provides the full functionality of MACOS in a Fortran subroutine. Regular MACOS commands are available to any other program via a Fortran CALL statement. This provides a nonlinear optics modeling capability for dynamic simulations or optimization programs. This section describes SMACOS in 4 subsections:

- Background
- Passing data between SMACOS and user applications
- Example
- Command reference

9.1 Background

Certain tasks require a full nonlinear optical modeling capability in a form that can be directly called by another program. Examples include

- Dynamic simulation of integrated optics/structures/controls systems where structural motions determine optical outputs such as focal-plane images and drive controller actions
- Monte Carlo studies of optical systems where uncertain parameters are varied randomly, collected, and analyzed statistically
- Optical parameter estimation where optical model parameters are determined by driving modelled optical outputs, such as images, to match actual measured outputs
- Design optimization where sets of parameters are varied to extremalize an optical performance metric

SMACOS consists of source code files, object code files, and include files that are compiled and linked with the user's main program.

To use SMACOS, several steps must be followed:

1. Develop .in-file optical prescriptions for each system to be modeled
2. In the user's main program, use Fortran include statements to define SMACOS variables (see Section 9.2)
3. Set up the sequence of commands in the main program using SMACOS subroutine calls
4. Compile, link, and execute the code

The process of using SMACOS begins with the creation of standard MACOS .infile optical prescriptions. This can be done interactively using the standard MACOS appli-cation as described in Section 4 or an existing .infile can be modified using a text editor.

The prescription should be checked to verify that the light PROpagates through the systems and the expected images are produced.

SMACOS normally includes the standard MACOS graphics. In some cases this may not be desirable: these routines take up appreciable memory in the computer. An alternative library of routines is available for applications that do not require graphical output.

9.2 Call Line Variables

The user's main program must define the SMACOS call line dummy variables. SMACOS is called:

```
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OPDMat,RaySpot,RMSWFE,PixArray)
```

The variables are explained in below.

9.2.1 Dummy Command Variables

SMACOS calls using the following dummy variables to pass the arguments needed to carry out SMACOS commands:

1. The SMACOS dummy character variable command is set to the MACOS command name: command='OLD'

2. Any character arguments are stored in the SMACOS character argument array:

```
CARG(1)='Cassegrain'
```

3. Any logical arguments are stored in the SMACOS logical argument array:

```
LARG(1)=.TRUE.
```

4. Any double-precision arguments are stored in the SMACOS double precision argument array: DARG(1)=3.1415926d0

5. Any single-precision arguments are stored in the SMACOS double precision argument array: RARG(1)=3.1416e0

6. Any integer arguments are stored in the SMACOS integer argument array:

```
IARG(1)=12
```

These dummy variables are dimensioned:

```
CHARACTER*132 command CHARACTER*32 CARG(9) REAL*8 DARG(9)) INTEGER IARG(9) LOGICAL LARG
REAL*4 RARG(9)
```

9.2.2 Dummy Output Variables

SMACOS uses the following dummy variables to pass output:

7. OutRaySpot: spot diagram data generated by SPOt and LSPot

8. RMSWFE: RMS wavefront error generated by OPD and LOPd

9. OPDMat: wavefront generated by OPD and LOPd

10. PixArray: image data generated by ADD, BLUR, PIX, LPIX and other commands These dummy variables are dimensioned:

```
REAL*8 RaySpot(mRay, 2), OPDMat(mpts, mpts), RMSWFE REAL*4 PixArray(mPix, mPix)
```

9.2.3 Sharing Data via Fortran Modules

SMACOS shares data with the calling program through a small set of Fortran 90 modules. This replaces the COMMON-block / .cmn include-file mechanism used in MACOS 2.x; calling programs only need a list of `use` statements – no manual array sizing, no shared-memory layout to keep in sync, no preprocessor model-size switches. The canonical reference implementation is MACOS_resources/GMI/GMI.F (the MATLAB MEX interface), which exercises every module described here.

9.2.4 The SMACOS modules

A user program that drives SMACOS typically opens with this block:

```
use param_mod ! compile-time array maxima (mElt, mRay, ...)
use src_mod ! source / chief ray / wavelength
use elt_mod ! every per-element array
use macos_mod ! global flags and model state
use math_mod ! shared math helpers (DMPROD, DOT, ...)
use sourcsub_mod ! source-grid setup (SetSourceRayGrid, etc.)
use smacos_mod ! the SMACOS dispatcher itself
! optional, for advanced uses:
use traceutil_mod, only : CRIncidDir, opdRayMask
use design_optim_mod ! optimization (AVAR/MVAR/CALib)
implicit none
```

Most callers only need the first seven; the others are pulled in as needed. The `only :` form is preferred when you reference just a handful of names from a large module, both for compile speed and to make the dependency explicit.

9.2.5 What's in each module

param_mod – compile-time array bounds: mElt (max elements), mRay (max rays), mPix, mDP, mGridMat, etc. Every other module uses these to declare allocatable arrays.

src_mod – source description: ChfRayPos(3), ChfRayDir(3), Wavelen, Flux, Aperture, GridType, SegCoord(3,mElt), and the wavefront ray pos / dir grids RayPos(3,mRay) / RayDir(3,mRay).

elt_mod – per-element data: KrElt(mElt), KcElt(mElt), psiElt(3,mElt), VptElt, RptElt, MonCoef(mMonCoef,mElt), FFCoef(mFFCoef,mElt), pFF / xFF / yFF / zFF, GridMat, plus EltID(mElt) and SrfType(mElt) flags.

macos_mod – session state: ifPlot, ifGrid, ifBuild, macos_realloc, plus the macos_init_all(modelsize) entry point that allocates / re-allocates everything.

smacos_mod – the SMACOS subroutine plus its dispatch helpers. `call SMACOS('OLD', CARG, DARG, IARG, LARG, RARG, OPDMat, RaySpot, RMSWFE, PixArray)` is the call signature.

math_mod – helpers shared with macos_f90 internals: DMPROD (matrix product), DOT, DXPROD, DUNITIZE, DORTHOGANALIZE, etc.

9.2.6 Initializing the runtime

Module-scope arrays are ALLOCATABLE; they have zero size until the runtime is initialized for a chosen model size. Call macos_init_all once before the first SMACOS call:

```
integer, parameter :: modelsize = 256
call macos_init_all(modelsize) ! sizes mElt, mRay, mPix, ...
call dopt_init ! sizes optimization arrays
macos_init = .true.
```

Valid model sizes are 128, 256, 512, 1024, 2048, 4096, and 8192. Re-call macos_init_all with a different size to grow or shrink the runtime; existing arrays are deallocated and replaced. In interactive macos this is the MREset command; in SMACOS-driven code it's a single subroutine call.

9.2.7 A minimal SMACOS driver

The following is a complete program that loads a prescription, traces, and prints the OPD RMS. GMI.F is essentially a longer version of this same pattern wrapped behind a MATLAB MEX gateway.

```
program smacos_driver
use param_mod
use src_mod, only : ChfRayPos, ChfRayDir, Wavelen
use elt_mod, only : RptElt, KrElt, EltName, nElt
use macos_mod, only : macos_init
use sourcsub_mod, only : SetSourceRayGrid
use smacos_mod, only : SMACOS
implicit none

! SMACOS argument-list buffers
character(len=132) :: command
character(len=32) :: CARG(9)
real*8 :: DARG(9), RMSWFE
integer :: IARG(9)
logical :: LARG
real*4 :: RARG(9)
real*8, allocatable :: OPDMat(:,:), RaySpot(:,:)
real*4, allocatable :: PixArray(:,:)

integer :: modelsize

! 1. initialize the runtime
modelsize = 256
call macos_init_all(modelsize)
macos_init = .true.
allocate(OPDMat(mpts,mpts), RaySpot(mRay,2),
& PixArray(mPix,mPix))

! 2. load a prescription
command = 'OLD'
CARG(1) = 'mySystem.in'
call SMACOS(command, CARG, DARG, IARG, LARG, RARG,
& OPDMat, RaySpot, RMSWFE, PixArray)

! 3. set up source ray grid (uses src_mod state)
call SetSourceRayGrid(npts, Dicr, Djcr, dxSource, .false.,
& .false.)

! 4. build linear model + trace
command = 'BUILD'
IARG(1) = nElt ! terminal element
call SMACOS(command, CARG, DARG, IARG, LARG, RARG,
& OPDMat, RaySpot, RMSWFE, PixArray)

command = 'OPD'
IARG(1) = nElt
call SMACOS(command, CARG, DARG, IARG, LARG, RARG,
& OPDMat, RaySpot, RMSWFE, PixArray)

! 5. read results back from module-shared state
write(*,*) 'OPD RMS =', RMSWFE
write(*,*) 'KrElt(1) =', KrElt(1),
& ' RptElt(:,1) =', RptElt(:,1)
end program smacos_driver
```

9.2.8 Reading and writing module data

Once `use elt_mod` is in scope, every element-array is directly addressable. Reading is a simple subscript: `KrElt(3)`, `RptElt(:,iElt)`, `MonCoef(:,iElt)`. Writing follows the same pattern – but if a write changes anything that affects the ray trace (frame geometry, surface coefficients, etc.), the linear model becomes stale and you must re-run BUILD before exercising it. GMI.F's ApplyPerturbationToOpticalSystem is a

worked example: it mutates ZernCoef / MonZernCoef / TElt directly via the module, then issues `command = 'PERTURB'` for the dispatcher to recompute frames.

9.2.9 Nominal-state save/restore

Programs that apply repeated perturbations (sensitivity loops, tolerancing) need to restore the optical state between iterations. GMI.F shows the pattern: snapshot every per-element array that PERTURB modifies into a local *Nom array on first entry, then copy back at the start of every later call. For FreeForm surfaces this includes pFF/xFF/yFF/zFF and pData/xData/yData/zData; SMACOS does not snapshot these automatically. Do not store snapshots in a Fortran 77 COMMON block; allocate them in the calling module instead.

9.2.10 Legacy COMMON-block support

The old smacosvars.cmn / elt.cmn / src.cmn / param.cmn include files were retired in v3.2. Calling programs that still depend on them must be rewritten to `use` the modules above. In almost every case this is a one-for-one mapping: `KrElt(i)` from elt.cmn becomes `use elt_mod, only : KrElt` and the same subscript expression. See git history of macos_f90/elt_mod.F for the full alias map.

9.3 Specification of Model Size in SMACOS

A new subroutine is added to MACOS 3.2b, as part of the SMACOS library, that allows the user to change model size at runtime. The subroutine call syntax is

CALL SMACOS_Set_Model_Size(model_size)

where the argument *model_size* is a either a constant or a variale for the specified model size. The call to SMACOS_Set_Model_Size will release the dynamic storage used for the existing model size, and allocate storage for the new model size. It also does some necessary setup to make sure a proper SMACOS initialization is performed when the model size is changed. The model parameter file *macos_param.txt* must be accessible from the user's execution directory as discussed in Section 3.8.4.

9.4 Example

MACOS>**old**

The user code accesses MACOS functions using the SMACOS call line in a fashion analogous to the MACOS command line. For instance, the MACOS command sequence to load Cassegrain.in and compute the intensity at the final element is:

```
Enter file name: Cassegrain
Input file Cassegrain.in being loaded. Optical Train Summary:
Tracing rays past 5 elements.
Aperture type = 1 (Annular ) with 15 grid points.
Elt 1: Obscuration with Kc= 0.0000D+00, Kr=-1.0000D+20, nECoords
= -6
Elt 2: Conic Mirror with Kc=-1.0000D+00, Kr=-1.0800D+01, nECoords
= -6
Elt 3: Conic Mirror with Kc=-2.6706D+00, Kr=-3.5268D+00, nECoords
= -6
Elt 4: Reference Surface with Kc= 0.0000D+00, Kr=-5.5601D+00, nECoords
= -6
Elt 5: Focal Plane with Kc= 0.0000D+00, Kr=-1.0000D+18, nECoords
= -6
Number of output coordinates is 5 MACOS>intensity
Enter last element for propagation:5
Tracing 4396 rays and propagating 65536 grid points... FF Prop between Elt 4 and Elt 5 to WF 1:
z1= 5.5601D+00 dx1= 7.0764D-02 min= 7.0704D-02 max= 7.0825D-02 dev= 3.5308D-05 lin= 99.95%
z2= 0.0000D+00 dx2= 3.0693D-07
Compute time was 0.3236 sec MACOS>
```

The same functions can be performed using SMACOS calls as follows:

```
C Load Cassegrain.in prescription file command='OLD' CARG(1)='Cassegrain'
CALL SMACOS(command,CARG,DARG,IARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)

C Propagate
command='INTENSITY' IARG(1)=5
CALL SMACOS(command,CARG,DARG,IARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
```

The SMACOS call line passes results of some calculations in dummy output arrays (see Section 9.2.2).

9.5 Commands

Every MACOS command is available as an SMACOS command, with the exception of NEW. The MOD command has a different form in SMACOS than in MACOS, as it is expected that the SMACOS user will prefer to directly access and change any data values, rather than incur the overhead of a full SMACOS call. The SMACOS MOD command simply resets the current ray-trace and WF element counters to 0, so that subsequent computations will start from the source. It has no arguments.

One command is available for SMACOS that is not used in MACOS: the NONe command, which turns off the MACOS plot functions.

This section summarizes most of the SMACOS commands and their arguments.

9.5.1 File and Data Manipulation

The SMACOS commands for loading .in-files, displaying element data and exporting data are:

11. LOAD or OLD: load existing optical system dataset. This command has one argument, the name of the prescription file (minus the .in suffix). The command is set up in the main program as follows:

```
command='OLD'
CARG(1)=filename
```

and is executed using the call statement:

```
CALL SMACOS(command,CARG,DARG,IARG,RARG,…)
```

The call is the same for all commands and so is not repeated for each.

12. Help: display a summary of the SMACOS commands. This command has no arguments. The command is set up in the main program as follows:

```
command='HELP'
```

13. SUMmarize: display a summary of the optical system data on the user's terminal. This command has no arguments. It is set up:

```
command='SUMMARIZE'
```

14. STAtus: display a summary of the status of the current calculations. This command has no arguments. It is set up:

```
command='STATUS'
```

15. RESet: reset various internal flags and counters to their initial values. This has the effect of forcing the next calculations to be performed as if the current file was just loaded. This command has no arguments. It is set up:

```
command='RESET'
```

16. SHOw: display the data defining a specified element. This command has 1 integer argument: the number of the element to be displayed. The source is considered to be element 0. It is set up:

```
command='SHOW' IARG(1)=iElt
```

17. SAVe: write out the current optical data as a new .in-file. This command has 1 character argument: the name of the new file. It is set up:

```
command='SAVE' CARG(1)=new_filename
CARG(2)='YES' (to overwrite if file exists)
```

18. EXPort: this function writes specified data to disk using one of three formats: MATLAB ".m-file", fits, or Fortran binary formats. It is set up differently depending on the particular variable to be exported, its form, etc. The user is advised to prototype the particular command sequence using MACOS, and then set up the appropriate sequence of arguments.

```
command='EXPORT' IARG(1)=… CARG(1)=…
```

19. EXEcute: runs a journal file. This command has 1 character argument, namely the name of the journal file to be executed. The command is set up in the main program as follows:

```
command='EXECUTE' CARG(1)='journal_file_name'
```

20. JOUrnal: writes a journal file. This command has 1 character argument, namely the name of the journal file to be written. The command is set up as follows:

```
command='JOURNAL' CARG(1)='journal_file_name' CARG(2)='YES' (to overwrite if file exists)
```

21. MODify: resets the surface counters for ray-trace and diffraction computations, so that changes to elements will be picked up. This command has no arguments. It is set up as follows:

```
command='MODIFY'
```

Note that the SMACOS MODify command (unlike the MACOS MODify command) does not change any data elements. It is up to the user to do this directly, by assignment statements in the calling program.

9.5.2 Ray-Trace Analysis

22. MAP: this command displays and prints a cross-sectional map of the source light beam, as described in Section 5.2. It has no arguments. The command is set up as follows:

```
command='MAP'
```

23. RAY: trace a specified ray through the entire system. This command has one argument, the ray number. The command is set up as follows:

```
command='RAY' IARG(1)=iRay
```

24. FFP: find field point corresponding to a specified chief ray offset at a particular optical element. This command has 4 arguments: the element number of the surface at which the chief ray intercept is specified, the offset of the chief ray from the vertex of the element, and a prompt response to accept the new source parameters. The command is set up as follows:

```
command='FFP' IARG(1)=iElt
DARG(1)=chief ray offset x in base units DARG(2)=chief ray offset y in base units CARG(1)='YES' (to
accept changes)
```

25. PFP: find field point corresponding to a specified chief ray offset in pixel coordinates

at a detector. This command has 4 arguments: the element number of the surface at which the chief ray intercept is specified, the size of the detector pixels, the offset of the chief ray from the vertex, and a prompt response to accept the new source parameters. The command is set up as follows:

```
command='PFP' IARG(1)=iElt
DARG(1)=pixel size in base units DARG(2)=chief ray offset x in pixel units DARG(3)=chief ray offset y
in pixel units CARG(1)='YES' (to accept changes)
```

26. CENter: center the beam in the aperture stop. This command has three arguments: the element number of the aperture stop and its offset from the vertex. The command is set up as follows:

```
command='CENTER' IARG(1)=iElt
DARG(1)=stop offset x DARG(2)=stop offset y CARG(1)='YES' (to accept changes)
```

27. POLarization: turn on polarization ray-tracing. This command has 4 arguments specifying the polarization state of the source rays. The command is set up as follows:

```
command='POL' DARG(1)=TEM X (real part)
DARG(2)=TEM X (imaginary part) DARG(3)=TEM Y (real part) DARG(4)=TEM Y (imaginary part)
```

28. NOPolarization: turn off polarization ray-tracing. This command has no arguments. The command is set up as follows:

```
command='NOPOL'
```

29. PERturb: rotate and/or translate a specified optical element. This command has several arguments. An integer argument is used to specify the number of the element to be perturbed (element 0 is the source). Up to 9 double precision arguments are used to specify the perturbations. A character argument is used to specify global or element coordinates. The command is set up as follows:

```
command='PERTURB' IARG(1)=iElt
```

If iElt=0 (the source is to be perturbed) then the perturbation is entered in global coordinates

```
DARG(1)=rotational perturbation x DARG(2)=rotational perturbation y DARG(3)=rotational perturbation z
DARG(4)=translational perturbation x DARG(5)=translational y DARG(6)=translational z
```

and the system aperture stop location is entered:

```
DARG(7)=stop x DARG(8)=stop y DARG(9)=stop z
```

If iElt is not zero (an optical element is to be perturbed) then the coordinate system for the perturbation is specified using a character argument:

```
CARG(1)=eltcoord or globalcoord
```

and if element coordinates were selected, the perturbations are :

```
DARG(1)=perturbation in element coordinate 1 DARG(2)=perturbation in element coordinate 2
DARG(3)=perturbation in element coordinate 3 DARG(4)=perturbation in element coordinate 4
DARG(5)=perturbation in element coordinate 5
DARG(6)=perturbation in element coordinate 6
```

or if global coordinates were selected:

```
DARG(1)=rotational perturbation x DARG(2)=rotational perturbation y DARG(3)=rotational perturbation z
DARG(4)=translational perturbation x DARG(5)=translational perturbation y DARG(6)=translational
perturbation z
```

30. PREad: reads perturbations from an input file. This command has 1 argument, the name of the files. The command is set up as follows:

```
command='PREAD' CARG(1)=filename
```

31. OPD: trace the full set of rays from the source (or from the current element if an OPD, SPOt, or PROpagate command has been performed previously) up to a specified element, and then calculate the OPD distribution across that element. The result is stored in the array NomOPDMat and passed in the SMACOS call line. Depending on the selected plot option, the OPD map may be plotted. This command has one argument, the element number. The command is set up as follows:

```
command='OPD' IARG(1)=iElt
```

CARG(1)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

32. SPOt: trace the full set of rays from the source (or from the current element if an OPD, SPOt, or PROpagate command has been performed previously) up to a specified element, and then calculate the spot diagram at that element. The result is stored in the array OutRaySpot and passed in the SMACOS call line. Depending on the selected plot option, the spot diagram may be plotted. The way obscured rays are handled depends on the setting of the OBS option. This command has one argument, the element number. The command is set up as follows:

```
command='SPOT' IARG(1)=iElt
```

CARG(1)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

The SPOt command expects 5 output coordinates.

33. OBScuration: specify the way obscured rays are handled for SPOt diagrams. This command has one argument, the obscuration option number. If this is set to 0, all rays are plotted; if set to 1, only unobscured rays are plotted; if set to 2, only obscured rays are plotted. The command is set up as follows:

```
command='OBS' CARG(1)=Obscuration option
```

34. SINT: activates the surface interpolation algorithms. This command has no arguments. The command is set up as follows:

```
command='SINT'
```

9.5.3 Diffraction Analysis

35. INTensity: trace the full set of rays and propagate the diffraction wavefront from the source (or from the current element if a diffraction command has been performed previously) up to a specified element. The resulting complex amplitude is stored in the array WFElt, which is one of the variables defined in the include files. The modulus squared is plotted; it is stored in the MWFFT array. This command has one argument, the element number. The command is set up as follows:

```
command='INTENSITY' IARG(1)=iElt
```

CARG(1)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

36. GAIn: compute and plot the ratio of the light diffracted by the optical system to a point source. This command has one argument, the element number at which to comput the gain. The command is set up as follows:

```
command='GAIN' IARG(1)=iElt
```

CARG(1)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

37. SCAlar: calculate using scalar diffraction algorithm, the default. This command has no arguments. The command is set up as follows:

```
command='SCALAR'
```

38. VECtor: calculate using scalar diffraction algorithm, the default. This command has no arguments. The command is set up as follows:

```
command='VECTOR'
```

39. ORS: optimize a reference surface for sphere-to-sphere diffraction. This command finds the best-fit sphere to the wavefront at a specified reference surface element, which is then reset to match the best-fit sphere. This command has one argument, the reference surface element number. The command is set up as follows:

```
command='ORS' IARG(1)=iElt
```

40. SRS: slave a reference surface to a previously optimized reference surface for sphere-to-sphere near-field diffraction or for sphere-to-plane far-field diffraction. This command has two arguments: the element number of the surface to be slaved; and the element number of the surface to which it is to be slaved. It uses the parameters defining the second element to reset the first so that the diffraction is correctly defined. The command is set up as follows:

```
command='SRS' IARG(1)=iSlv1 IARG(2)=iSlv2
```

41. FEX: find the exit pupil of the system. This command traces two chief rays through the aperture stop of the system to the element preceeding the specified element and finds their crossing point in image space. The specified surface is then reset to be a sphere located at the crossing point and centered at the intersection of the source chief ray with the preceeding element. Both the specified

element and the one preceeding it must be return surfaces. The command has two arguments: the number of the element to be set to the exit pupil and the location of the system aperture stop in global coordinates. The command is set up as follows:

```
command='FEXIT' IARG(1)=iElt
CARG(1)='YES' (to accept changes)
```

42. REGrid: toggle the "regrid" option for sphere-to-sphere propagations. With regridding set on, the ray grid is recalculated after each near-field propagation to assure correct alignment of the rays with the diffraction wavefront grid points. This command is most useful in highly aberrated systems. In some circumstances it must be used with the OPD interpolation command (not currently implemented). The command has no arguments. It is set up as follows:

```
command='REGRID'
```

43. BEAm: allows the user to select between a uniform beam and a Gaussian beam. This command has three arguments: the beam type (uniform or Gaussian), the x beam diameter (for a gaussian beam) and y beam diameter (for a Gaussian beam). The command is set up as follows:

```
command='BEAM' IARG(1)=BeamType
DARG(1)=x beam diamter DARG(2)=y beam diamter
```

9.5.4 Wavefront and Image Plots

44. INtensity: compute and plot the intensity of the wavefront at the specified element. Plotting is done only if plot type is not NONe and the graphics options are included in the compile. The type of plot is as set by previous calls to SMACOS. This command has one argument, the element number. The command is set up as follows:

```
command='IN' IARG(1)=iElt
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

45. AMplitude: compute and plot the amplitude and phase of the wavefront at the specified element. This command has one argument, the element number. The command is set up as follows:

```
command='AM' IARG(1)=iElt
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

46. REAl: compute and plot the real and imaginary parts of the wavefront at the specified element. This command has one argument, the element number. The command is set up as follows:

```
command='REAL' IARG(1)=iElt
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

47. LOG: compute and plot the logarithm (base 10) of the intensity of the wavefront at the specified element. This command has one argument, the element number. The command is set up as follows:

```
command='LOG' IARG(1)=iElt
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

48. PIxilate: compute and plot the pixilated wavefront intensity at the specified element. This command has three arguments: the element number, the number of pixels across one side of the array (assumed square), and the size of a pixel. The minimum size of the pixel is half the wavefront grid spacing. This is not automatically checked, so be careful! The PIxilated image is stored in the PixArray variable and passed through the SMACOS call line. The PLOcate command sets the option to locate the center of the pixel array to the vertex point of the element. The command is set up as follows:

```
command='PIXILATE' IARG(1)=iElt
IARG(2)=number of pixels per side RARG(1)=size of pixel
```

49. COMpose: initiate a composed image to which subsequent intensity plots can be ADDed. This command has three arguments: the element number, the number of pixels across one side of the array (assumed square), and the size of a pixel. The minimum size of the pixel is half the wavefront grid spacing. This is not automatically checked, so be careful! The command is set up as follows:

```
command='COMPOSE' IARG(1)=iElt
IARG(2)=number of pixels per side RARG(1)=size of pixel in base units
```

50. WINdow: define a window on the detector that stays fixed wrt the detector. Subsequent images that fall on the window, as determined by the ray trace, are placed appropriately, provided the FEX command is used for each image. Used with the PIX,

COMPOSE, ADD, etc. commands. This command has 5 arguments. It is set up as follows:

```
command='WINDOW'
CARG(1)='TOUT' (detector coordinates option) DARG(1)=size of pixel in base units
DARG(2)=x element vertex location in pixel coords DARG(3)=y element vertex location in pixel coords
DARG(4)=x window center location in pixel coords DARG(5)=y window center location in pixel coords
```

51. ADD: add an intensity plot to a composed image. This command has two arguments: the element number and a character argument to specify whether to plot the composed image. The composed image is stored in the PixArray variable and passed through the SMACOS call line. The command is set up as follows:

```
command='ADD' IARG(1)=iElt
CARG(1)='PLOT' or 'NOPLOT'
```

CARG(2)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

52. DADD: display the currently composed image. This command has no arguments. The command is set up as follows:

```
command='DADD'
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

53. NOISE: add detection noise to the currently composed image. Noise here means random fluctuations in the values of the pixels of an image. This command has several arguments, which vary with context. The command is set up for Gaussian noise as follows:

```
command='NOISE' CARG(1)='GAUSSIAN'
DARG(1)=standard deviation of noise in DN CARG(2)='PLOT' or 'NOPLOT'
```

CARG(3)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

For photon noise, a Poisson process where the standard deviation is the expected value:

```
command='NOISE' CARG(1)='PHOTON' CARG(2)='PLOT' or 'NOPLOT'
```

CARG(3)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

For Poisson noise on a uniform background level:

```
command='NOISE' CARG(1)='POISSON'
RARG(1)=background level in DN
DARG(1)=detector quantum efficiency (in the range 0 to 1) CARG(2)='PLOT' or 'NOPLOT'
```

CARG(3)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

54. BLUR: blur the currently composed image using a 9-pixel kernel. This command is intended to simulate detector charge leakage. It has one argument, the blur kernel peak, which can take values between 1 and 0. The command is set up as follows:

```
command='BLUR' RARG(1)=blur kernel peak CARG(1)='PLOT' or 'NOPLOT'
```

CARG(2)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

55. GBLUR: propagate and blur an image, then add to the currently composed image. This command is more general than the BLUR command, using a Gaussian kernel. It is often used to simulate long-exposure jitter blurring. The command is set up as follows:

```
command='GBLUR'
DARG(1)=Gaussian blur kernel width in base units CARG(1)='PLOT' or 'NOPLOT'
```

CARG(2)=’YES’ (to overwrite if a file-writing plot type is selected and a file of the same name exists)

9.5.5 Plot Types

56. NONe: turns off plotting entirely. The command is set up as follows:

```
command='NONE'
```

57. NULl: turns off plotting entirely. The command is set up as follows:

```
command='NULL'
```

58. WIRe: sets surface plot type to “wireframe”. The command is set up as follows:

```
command='WIRE'
```

59. SLIce: sets surface plot type to “slice”. The command is set up as follows:

```
command='SLICE'
```

60. GRAy: sets surface plot type to “gray scale”. The command is set up as follows:

```
command='GRAY'
```

61. CONtour: sets surface plot type to “contour plot”. The command is set up as follows:

```
command='CONTOUR'
```

62. ROW: sets surface plot type to “row plot”. The command is set up as follows:

```
command='ROW'
```

63. COLumn: sets surface plot type to “column plot”. The command is set up as follows:

```
command='ROW'
```

64. TEXt: sets surface plot type to “text”. This provides for the plot data to be written to a text file instead of being plotted. The command is set up as follows:

```
command='TEXT'
```

65. BINary: sets surface plot type to “binary”. This provides for the plot data to be written to a binary file instead of being plotted. The command is set up as follows:

```
command='BINARY'
```

66. MATlab: sets surface plot type to “Matlab”. This provides for the plot data to be written to a Matlab .m-file instead of being plotted. The command is set up as follows:

```
command='MATLAB'
```

67. FITs: sets surface plot type to “fits”. This provides for the plot data to be written to a fits file instead of being plotted. The command is set up as follows:

```
command='FITS'
```

9.5.6 Linear Models

68. BUild: trace the full set of rays and the ray partial derivative matrices from the source up to a specified element, and then compute the composite C-matrix. The result is stored in the array Cmatrix and passed in the SMACOS call line. This command has one argument, the element number. The command is set up as follows:

```
command='BUILD' IARG(1)=iElt
```

69. DMBuild: performs as the DMBuild, but also includes interpolated surface partials. The result is stored in the array C-matrix and passed in the SMACOS call line. This command has one argument, the element number. The command is set up as follows:

```
command='DMBUILD' IARG(1)=iElt
```

70. PArtials: displays and prints the full set of ray partial derivatives for a specified ray. The command is set up as follows:

    command='PARTIALS' IARG(1)=iRay

71. LPErturb: rotate and/or translate a specified optical element. This command has several arguments. An integer argument is used to specify the number of the element to be perturbed (LPErturb does not work with element 0). Up to 9 double precision arguments are used to specify the perturbations. A character argument is used to specify global or element coordinates. The command is set up as follows:

```
command='LPERTURB'
IARG(1)=iElt
```

and the coordinate system for the perturbation is specified using a character argument:

```
CARG(1)=eltcoord or globalcoord
```

If element coordinates are choosen:

```
DARG(1)=perturbation in element coordinate 1 DARG(2)=perturbation in element coordinate 2
DARG(3)=perturbation in element coordinate 3 DARG(4)=perturbation in element coordinate 4
DARG(5)=perturbation in element coordinate 5 DARG(6)=perturbation in element coordinate 6
```

else if global coords are choosen:

```
DARG(1)=rotational perturbation x DARG(2)=rotational perturbation y DARG(3)=rotational perturbation z
DARG(4)=translational perturbation x DARG(5)=translational perturbation y DARG(6)=translational
perturbation z
```

72. LPRead: rotate and/or translate a specified optical element using data stored in a separate file. This command has no arguments. It assumes that properly formatted data (see Section 8.3.2) is stored in a file named filnam.pert where filnam is the name of the .in-file. The command is set up as follows:

```
command='LPREAD'
```

73. LPReset: reset the linear perturbation vector to zero. This command has no arguments. The command is set up as follows:

```
command='LRESET'
```

74. LSPot: use the linear model generated in the BUild and LPErturb commands to compute the spot diagram at the last element specified in the BUild command. This element should be a focal plane (EltType=3). The output of this command is stored in OutRaySpot and (depending on the active plot option) may be plotted on the screen. This command has no arguments. The command is set up as follows:

```
command='LSPOT'
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

75. LOPd: use the linear model generated in the BUILD and LPERTURB commands to compute the OPD map at the last element specified in the BUILD command. This element should be a reference surface (EltType=4). The output of this command is stored in PertOPDMat and (depending on the active plot option) may be plotted on the screen. This command has no arguments. The command is set up as follows:

```
command='LOPD'
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

76. LINtensity: use the linear model generated in the BUild and LPErturb commands to compute and plot the intensity of the wavefront at the last element specified in the BUild command. This element should be a reference surface (EltType=4) followed

by a focal plane set up for a far-field diffraction. This command has no arguments. The command is set up as follows:

```
command='LINTENSITY'
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

77. LPIxilate: use the linear model generated in the BUild and LPErturb commands to compute and plot the pixilated wavefront intensity at the last element specified in the BUild command. This element should be a reference surface (EltType=3) followed by a focal plane set up for a far-field diffraction. This command has two arguments: the number of pixels across one side of the array (assumed square) and the size of a pixel. The minimum size of the pixel is half the wavefront grid spacing. This is not automatically checked, so be careful! The PLOcate command does not apply to the LPIxilate command. The command is set up as follows:

```
command='LPIXILATE' IARG(1)=number of pixels per side RARG(1)=size of pixel
```

CARG(1)='YES' (to overwrite if a file-writing plot type is selected and a file of the same name exists)

9.6 GMI: MATLAB MEX interface (FreeForm updates)

GMI (MACOS_resources/GMI/) is a thin MATLAB MEX wrapper around SMACOS, packaging perturbations and pulling back OPD / PIX / SPOT / complex-EF arrays into MATLAB. V4.00 changes:

- NRHS = 14 (was 13). Old call_GMI.m callers passing 13 arguments will fail with 'GMI requires 14 input arguments'. The new 14th argument is pmonzern, a MonZernCoef perturbation vector parallel to pzern for FreeForm surfaces.
- param.monzernSrf – column vector of FreeForm element indices to perturb (analogous to param.zernSrf). Encoded in pflg with a 9999 sentinel when absent.
- param.pmonzern – per-mode coefficient vector, length length(monzernSrf) * nmonzern, where nmonzern defaults to param.mzern. Read inside call_GMI.m and forwarded as the 14th MEX argument; user code does not call GMI() directly.
- GMI.inc compile-time sizes: numseg = 7, mzern = 15 (up from 6 / 12) to support 7-segment FreeForm test cases with 15 modes per segment. When a runtime param.mzern exceeds the compile-time mzern, the MEX validation rejects with 'pzern must be scalar, empty or a mpzern x 1 matrix'. Bump mzern and force a rebuild (rm GMI.o GMIG.o; source makegmi.sh – make does not detect .inc changes).
- Nominal save/restore expanded to FreeForm frames. ObtainNominalSettings / SetToNominalSettings now snapshot and restore pFF/xFF/yFF/zFF and (when nGridMat > 0) pData/xData/yData/zData alongside the existing pMon / psiElt / vptElt / TElt entries. This fixes a v3.x bug where per-call prb perturbations on FreeForm elements leaked the FreeForm coordinate frames forward, so the 'nominal-equivalent' OPD drifted across iterations of a sensitivity loop.

See MACOS_resources/GMI/README.md and MACOS_resources/GMI/CLAUDE.md for full architecture details.

SECTION 10: Worked Examples

Four complete workflows, each a runner that exists in the companion repository `MACOS_resources` and prints its own table. They are run from MATLAB with the mmacos bindings (Section 9 describes the engine they call); every one reads a single parameter file, writes `<tag>_<stage>.txt` beside itself, and keeps that text as its record, so the test of this section is to run the command and compare. The numbers quoted here were produced on 2026-10-17 from the committed sources.

```
run('<path-to>/mmacos/mmacos_setup.m');     % once per session: the +macos package and the mex
```

10.1 10.1 A sensitivity workflow and an error budget (`e5hex1_budget`)

`templates/50_sensitivities/e5hex1_budget/run_budget_e5hex1.m` harvests the rigid-body sensitivity Jacobian of the public seven-segment telescope `e5hex1.in` — `macos.dw_dx_multi`, five field points, six degrees of freedom per optic, model size 128 — and turns it into an optical error budget after Redding 2014: a properly formed RSS budget is a covariance analysis, and a Monte-Carlo draw gives the same answer. The error groups, their distributions and the control set are a table (`budget_spec_e5hex1.m`): the seven primary segments and the secondary are controlled in six degrees of freedom each, 48 actuators, by the pseudo-inverse controller; a wavefront sensing row and one actuator row per control group are added.

```
cd <path-to>/mmacos/templates/50_sensitivities/e5hex1_budget
run_budget_e5hex1
```

The run takes a few minutes and leaves the roll-up table (`e5hex1_budget.csv`), the text report, a per-group bar chart and two workbooks, one of them (`e5hex1_wfe_budget.xlsx`) a live-formula calculator in the HabEx form whose white cells are the inputs.

From the report (`e5hex1_budget_report.txt`), with every error rigid-body on this deck:

line	WFE, nm rms
uncontrolled (pre-WFC) total	317.4
post-WFC total, covariance	3.156
post-WFC total, Monte-Carlo, 1000 draws with the controller, sensing and actuator draws in the loop	3.179
wavefront-sensing row (3 nm white noise, of which only the controllable part leaks: 48 of 51 225 rays)	0.092

The actuator rows dominate the corrected budget (primary piston 2.60 nm, primary tilt 1.28 nm, secondary tilt 1.25 nm). The covariance and the Monte-Carlo totals agree to 0.7 %, the point of the method. The 2026-10-17 run reproduced the record of the template's README and of the regression test `tRunBudget/test_wfc_stage_cov_equals_mc` to every printed digit.

10.2 10.2 A telescope design: the long-slit TMA front end (`tma_longslit`)

`templates/10_telescopes/tma_longslit/tma_longslit_run.m` designs a three-mirror telescope that feeds a straight slit telecentrically with a bounded cone, the SBG VSWIR zig-zag form: a weak concave M1, a small convex M2 that is the stop, M3 carrying the power, the slit 300 mm behind M3. The default instrument is the dyson5 3k front end: focal length 330 mm, aperture 183 mm, F/1.8, a ±4.7° strip onto a 54 mm slit. Two stages run in seconds:

```
addpath('<path-to>/mmacos/templates/10_telescopes/tma_longslit');
OUT = tma_longslit_run({'first_order','section'});
```

`first_order` is the paraxial zig-zag; `section` emits the three mirrors as off-axis conic sections (the first-order result forces this: a tilted sphere cannot supply the same power in and across the fold plane), traces the strip through the engine on a 41-point grid and measures it. Its table, from `tls_section.txt`:

quantity	measured	spec
plate scale, local / to the edge of the strip	329.99 / 329.87 mm	330.0 mm
image of the ±4.7° strip	54.24 mm	54.0 mm slit
F/# at the slit, along / across the strip, every field	1.92–1.93 / 1.95–1.96	1.8 cone bound
chief-ray angle to the slit normal at the strip edge (telecentricity)	0.093°	—
every ray passes	1.000	—
worst clearance, footprint plus a 5 mm mount (M3 → slit leg against M2)	+10.2 mm	> 0

The rms spot at this stage is millimetres (2230 µm at the strip edge): the conic sections only set the geometry. The `figure` stage, an engine-traced Levenberg–Marquardt ladder of 15–80 minutes a rung, brings the figure in; the `e2e` stage joins the best rung to the spectrometer of Section 10.3. The 2026-10-17 run reproduced the 2026-10-07 record to the printed digit; the only moving column was the chief-ray miss of the M2 pole, at 1e-7 µm.

10.3 10.3 A spectrometer design: the Dyson challenge (`dyson5`)

`challenges/dyson5/dyson5_run.m` designs an EMIT-class push-broom imaging spectrometer from a public specification: F/1.8 in air-equivalent image space, a 3000 × 500 pixel array at 18 µm (a 54 mm slit, 9 mm of spectrum), 380–2500 nm, smile and keystone under 0.1 pixel. Stage `s0` is engine-free and runs in under a minute; it sets the classical concentric Dyson at the spec and finds the scaling law the rest of the design obeys.

```
addpath('<path-to>/mmacos/challenges/dyson5');
OUT = dyson5_run();                    % stage s0 at the spec (then s1, the deck)
```

From `dyson5_s0_scaling.txt` (slit centre offset 6 mm, the slit corner at 27.66 mm from the axis):

- At the Dyson condition the blur grows as the fourth power of the field height (fitted exponent 4.17, a fifth-order residual) and falls as the cube of the block radius (fitted −3.18): blur ~ h^4 / r^3.
- At the slit corner the 0.25-pixel blur budget (4.5 µm rms) is met only at a block radius of 208 mm or more: grating radius 671 mm, a 257 mm grating. The verdict is to report the law and not add elements yet; the later stages (the meniscus, the fold prism, the telescope join) are the beats recorded in the challenge's README.

The stored record of this stage was taken on 2026-09-30 with the slit centre offset then in force, 8 mm (corner at 28.16 mm, 213 mm required); the law and its exponents are the same.

10.4 10.4 A gauge bench: COPHI on the TG96 interferometer (`cophi_dm96`)

`templates/40_benches/cophi_dm96/cophi_run.m` builds the heterodyne camera interferometer as a deformable-mirror surface gauge on the bench of record of the TG96 campaign: the reference flat's seat is taken by a traced, fiber-fed local-oscillator arm, so there is no analyzer and no quarter-wave plate and the polarization gain is absent by construction. The bench stage solves the clearances, builds and emits both

arms as prescriptions (`<tag>_test.in`, `<tag>_ref.in`) and registers them at the camera: pixel pitch, the chief-ray pixel, the magnifications from the DM and from the seat, the local oscillator's footprint over the test arm's pupil image and its own wavefront there, and the null phase on the flat DM.

```
cd <path-to>/mmacos/templates/40_benches/cophi_dm96
OUT = cophi_run('bench.optics','oap','tag','bench_oap','stages',{'bench'});   % the mirror rig of
record
```

The run takes about a minute at model size 1024 (a 385-point ray grid) and writes `runs/bench_oap/bench_oap_cophi_report.txt`. Its registration block:

quantity	measured	pass criterion
chief-ray pixel, LO minus test arm	0.000 px	< 1 px
LO footprint centroid offset from the test pupil image	1.625 px, inside the LO's 33.9 px overfill	< overfill
LO coverage of the test pupil (77 877 px), flatness	1.0000, 0.21 % rms	> 0.99
null on the flat DM, interference phase as surface after piston	0.782 nm rms (tilt 0.647, defocus 0.169, rest 0.405)	< 1 nm
fringe contrast over the test pupil	0.9817 median	—
magnification, DM to camera / seat to camera	10.5576 / 10.5600	—

The 1.6 px is the test pupil (the image of the DM stop) sitting off the chief ray, a property of the test arm, and the 0.65 nm of tilt in the null is the LO collimator's beam direction against the parabola's; both are fixed maps the frozen null subtracts, as on the record's interferometer (whose mirror-rig null is 12.9 nm). The 2026-10-17 run reproduced the README's record row of the mirror rig to every printed digit.

SECTION 11: Polarization

MACOS carries a complete vector-polarization physics: every ray can carry a complex three-vector electric field, every reflector and refractor applies its Fresnel or multilayer-coating coefficients to it at the ray's own angle of incidence, two element types polarize and retard the field, and the diffraction propagations can carry the three field components through the whole chain. Above the engine, the bindings assemble the Jones pupil and the standard polarization-aberration metrics and the polarization-limited contrast floor of a coronagraph. This section is the user's description of that machinery; every claim in it is pinned by a named test, and the measurements behind the claims are in the MACOS Polarization Validation Report (Section 11.8).

11.1 Turning polarization on

Polarization ray tracing is off by default and is a *session mode*: once turned on it survives every LOAD until it is turned off, and every LOAD prints a line naming the non-default modes in force.

```
MACOS>pol
 Polarization turned on
 Vector diffraction enabled
Enter Ex0 (real,imag): [0.,0.]: 1,0
Enter Ey0 (real,imag): [0.,0.]: 0,0
```

The POLarization command prompts for the complex source field amplitudes Ex0 and Ey0, in the source grid frame; `POL 1,0 0,0` on one line is the same. A prescription can carry the state in the `Ex0Ey0=` keyword (Section 4) — the state only; turning the trace on remains a command or API decision. NOPOLARIZATION turns it off. On every stock model size the command also enables vector diffraction; SCALAR and VECTOR (Section 6) toggle that part separately. A 0,0 launch is accepted silently and traces no field, so set the state before reading any polarization result.

From the bindings the same switch is `polarization('on', Ex0, Ey0)` in mmacos and pymacos (api `pol_set` / `pol_get`), and `vecdif_set` for the vector-diffraction part. Changing the polarization state marks the cached trace dirty, so the next query re-traces with the new state.

With polarization on, the geometry is unchanged: ray positions, directions and path lengths — and therefore every OPD and spot result — are bit-identical to the scalar trace. Polarization adds the field; it does not move the rays.

11.2 Conventions

Cross-code polarization disagreements are nearly always convention mismatches. MACOS's conventions were read off the engine source, are asserted by the tests, and are the authority for every number in this section and in the validation report.

Convention	Value
Time dependence	exp(+iωt); a field accumulates exp(−i·2π·L·N/λ) through a medium of complex index N over a path L, so the phase decreases as the path grows
Absorbing index	N = n − iκ with κ > 0 meaning loss, as entered in `IndRef=` / `Extinc=` and in `Coating=` layers
Jones storage basis	linear (x, y); circular states by a unitary change of basis
Jones pupil reference frame	double-pole (Chipman) by default; local s/p and global available as diagnostics

Convention	Value
Retardance sign	fast axis leads; a retarder's Jones matrix in its own eigenbasis is diag(1, exp(−iδ))
Diattenuation	intensity-based, D = (T_max − T_min)/(T_max + T_min) in [0, 1]
Coating thickness and retardance in a prescription	waves at the `Wavelen=` in force when the parser reaches the line; converted to a physical length at load, so a film or a plate is fixed glass and a wavelength sweep is chromatic
Field amplitude normalization	the uncoated and the coated transmission both carry the radiometric factor $\sqrt{(n_2 \cos\theta_2 / n_1 \cos\theta_1)}$, so $\lvert t\rvert^2$ is the power transmittance

Two consequences of the index convention are worth stating. A mirror described by the perfect-conductor idiom (`IndRef= 1.0`, `Extinc= 1.0E+22`) reflects with r_s = r_p = −1 in a fixed transverse basis and is polarization-neutral: it changes the ray geometry and introduces neither diattenuation nor retardance, so any stock perfect-conductor prescription is a unitarity check for free. A real metal mirror is modeled with a metal coating stack; an optically thick metal layer reproduces the bare metal's Fresnel coefficients whatever the substrate.

11.3 What the trace does with the field

Launch. A collimated source launches every ray with E = Ex0·xGrid + Ey0·yGrid, uniform across the beam. A point source launches each ray in its own frame, y = unit(RayDir × xGrid), x = y × RayDir, which reduces to the grid frame on the chief ray; the difference from a uniform frame is of order $NA^2/2$ and is measured, not assumed. The flux normalization is a common real scalar that every Jones quantity carries and no ratio sees.

At a bare surface. The field is decomposed into s and p components in the local plane of incidence, the surface's complex Fresnel coefficients for the ray's angle of incidence are applied, and the field is reassembled on the s and the *reflected* p directions, so the field stays transverse to the outgoing ray. The incident medium is the one the ray is actually traveling in; the other medium is the element's own complex index.

At a coated surface. An element carrying a `Coating=` stack (Section 4) is evaluated by the recursive thin-film (Airy) calculation through every layer at the ray's own angle of incidence, with the stack's complex indices, the incident medium the ray is in and the element's index as the substrate. Reflectors return the reflected field; refractors the transmitted one. The transmitted amplitude carries the radiometric factor once, boundary to boundary, so a coated lens and an uncoated lens agree on what an amplitude means. Transmission into a metal substrate is evanescent and carries no power; coated *lenses* are the use case. MACOS has a second, older coating description (`nCoatElt` and its companions, real indices only) that serves the non-sequential refractive trace and does not enter the polarization path; the `Coating=` stack is the one that does.

Two lessons from the engine's history are kept here because they are the shape of the errors a reader is most likely to make in another code. A reflection's p-coefficient sign depends on whether the p direction is held fixed or follows the outgoing ray; the wrong pairing reflects the transverse field about the p axis instead of negating it, which is unitary and an involution — it cancels exactly across any mirror pair, so every two-mirror and unitarity test passes while one near-normal mirror turns x-polarized light into equal x and y. The tell was a cross-polarization *flat in pupil radius* where an isotropic surface must give a ρ^2 law. And a transmission amplitude's radiometric factor must be applied once for the whole stack, never

per interface: the interior factors cancel identically, and a per-interface form is right only while every interface is present, in order, with the right media.

11.4 Polarizing elements

`TrPolarizer`, an ideal linear polarizer, and `WavePlate`, a linear retarder, are described in Section 4 (the `PolAxis=` and `Retardance=` keywords, with examples). Geometrically they are Reference surfaces: with polarization off they *are* Reference surfaces, and a prescription with them in it cannot change a scalar result. With polarization on each applies a 2×2 Jones matrix in the plane transverse to the ray — diag(1, 0) for the polarizer, diag(1, exp(−iδ)) for the retarder, in an eigenbasis built per ray.

The basis is the physical content. Away from normal incidence the declared axis has to be brought into the plane transverse to the ray, and orthographic projection does not preserve orthogonality, so it matters which axis is projected. MACOS projects the direction fixed in the *material* of the element — the absorbing direction of a polarizer (the wires of a wire grid), the crystal fast axis of a waveplate — and completes the partner axis by orthonormalization, which is what keeps a lossless retarder unitary. The same question was settled experimentally for a tilted dichroic polarizer in the dipole model's favor (Korger et al., Opt. Express 21, 27032, 2013). The effect is 3.6° of transmitted axis at 20° incidence and 45° azimuth, bounded by sin² of the incidence angle, and exactly zero at normal incidence and whenever the axis lies in or perpendicular to the plane of incidence — so the obvious test, axis along x with the tilt in the x–z plane, reports a reassuring zero that proves nothing. `PolAxis=` still declares a polarizer's transmission axis; the absorbing direction is derived from it.

The idealizations: no ray splitting (a polarizing beamsplitter is two traces, or better a coated Reflector at 45° where the s/p split is physical), no walk-off, no face Fresnel loss, no substrate, a purely transverse output, and a retardance that does not vary with incidence angle — a real crystal plate's does, which is the field-of-view effect behind compound and Pancharatnam designs. A reflective polarizer (`RfPolarizer`) is a named type that is not implemented.

11.5 Vector diffraction

With polarization on and VECTOR in force, every propagation leg carries the three Cartesian field components Ex, Ey, Ez as three independent scalar fields through the same kernel: the far-field sphere-to-plane leg, every near-field, plane-to-plane, surface-to-focus and DFT leg. Scalar mode runs the same code on one plane and is bit-identical to the engine before vector mode existed. The grid is seeded from the per-ray field at the first physical-optics leg and advanced through the following legs; rays the trace has clipped are extinguished at the seed, so polarization on with vector diffraction off is bit-identical to polarization off. A 0/1 mask is a diagonal Jones matrix and is applied to all three planes.

Three things to know when reading a vector-mode field:

- The three wavefront storage planes are repurposed as Ex, Ey, Ez of one wavefront, so vector mode and multi-wavefront composition (COMPOSE and the broadband commands) cannot run together.
- The components are reachable from the bindings: `complex_field(elt, 'plane', k)` returns Ex, Ey or Ez for k = 1, 2, 3 and refuses the request in scalar mode, where a plane is an unrelated wavefront and would read as a component. The vector intensity is the sum of the three; on an off-normal train it differs from the scalar intensity by the out-of-plane power and by the separate diffraction of Ey and Ez (0.26 % on the Cassegrain example, decomposed and pinned).
- The field's common phase is train-dependent relative to the OPD map. MACOS bookkeeps the OPD by subtracting return legs and the per-ray field phase by adding every leg, so on a Return-terminated train the assembled field matches the scalar convention, while a plain trace to a

detector measures the exact conjugate of the OPD map. There is no universal bridge; the Jones-pupil layer divides the common scalar out and never needs one, and anyone comparing a vector-mode field to a scalar one phase for phase must expect the conjugate on such a train.

The one limitation that remains: between two physical-optics legs the grid field is advanced by a scalar phase, which is exact only when the intervening elements are non-polarizing (Obscuring, Reference, FocalPlane). A coated or reflecting surface *between* two legs — folds between an interferometer's recombiner and its detector, an off-axis parabola between two masks — is not yet carried by the grid; the polarizing elements have the same restriction, being applied to rays before the seed. Results on such a chain are lower bounds, and the contrast-floor function measures the shortfall and says so.

11.6 Polarization analysis

Polarization analysis. The per-ray field at any element can be harvested through the bindings (`ray_field`), and the standard polarization-aberration analysis is built on top of it: `jones_pupil` assembles the 2×2 Jones matrix at every pupil point from two traces with orthogonal source states, and `pol_maps` decomposes it into diattenuation and retardance maps (with the pupil mean — a state change — separated from the spatially varying part that drives error budgets). `pol_zernike` then expands those maps onto a Zernike basis, giving the standard polarization-aberration terms — piston, tilt, defocus, astigmatism and up — so that a MACOS result can be compared with the published literature term by term rather than by map shape. For an on-axis rotationally symmetric two-mirror system the expansion reduces, as the theory requires, to pure *polarization astigmatism*: diattenuation and retardance grow as the square of the pupil radius with an axis locked to the pupil azimuth, and every other term — including the entire circular component — sits at round-off. These functions and their conventions, including the choice of pupil reference basis (double-pole by default), are documented in the MACOS Command Reference, Part II. The interactive CLI exposes the underlying trace and coating machinery; the Jones-pupil analysis layer is binding-only.

Polarization contrast floor. For a coronagraph the question is not the shape of the aberration but how much light it puts where the dark zone should be. `pol_contrast_floor` answers it directly: it propagates with vector diffraction on and splits the detector field into a co-polarized, a cross-polarized and a longitudinal channel. The cross-polarized channel is the part no scalar deformable-mirror control can remove, so its peak-normalized level is the polarization-limited contrast floor, and the function also reports how that floor moves with the coating choice.

Two points of method matter. First, the split is taken at the detector rather than in the pupil. Because the chain is linear in the input polarization state and all three components propagate with the same kernel, a spatially uniform analyzer commutes with propagation, so projecting after the propagation gives the same answer as projecting before it — and it avoids the Jones pupil, which cannot serve as a pupil multiplier because it carries the accumulated optical path length. Second, "co-polarized" is defined against the *mean output* state, not the input state. A train may rotate the polarization geometrically while introducing neither diattenuation nor retardance; charging that rotation to the cross-polarized channel would report an aberration where there is none, since an observer would simply align the analyzer to the output. The analyzer used is therefore the dominant eigenvector of the pupil coherency matrix, which is insensitive to the common wavefront and by construction minimizes cross-polarized power.

An unpolarized source is modeled as two traces with orthogonal input states summed in intensity, never as one trace with the second state inferred from it. The floor reported on a chain that places coated or reflecting surfaces between two propagation legs is a lower bound, for the reason given in Section 6; the function measures the shortfall and warns rather than leaving it to the reader.

The sharpest cheap check of a polarization set-up is the Zernike expansion on an on-axis rotationally symmetric two-mirror train: the theory requires pure polarization astigmatism — astigmatism-0 in the first Pauli component, astigmatism-45 in the second, equal in magnitude, no circular part, no defocus, a ρ^2

radial law whose on-axis extrapolation vanishes. MACOS puts every forbidden term at 1e-15 of the astigmatic one. A broken pupil frame or a wrong s/p sign breaks the mode *pattern*, not merely the map, which is why this test sees what magnitude tests miss.

11.7 Design rules the measurements taught

- **Specify a protective overcoat at λ/4 of the working wavelength.** An overcoat is neither a polarization cost nor a benefit in itself; the sign of its effect is its optical thickness at the working wavelength, and it reverses across the quarter-wave condition. Measured on the Cassegrain with a 110 nm MgF_2 overcoat on aluminium, cross-polarized power against the bare metal: 5.27× at 1 µm, 0.20× at 632.8 nm, a true quarter wave 0.053× at either.
- **Report two ratios.** The total cross-polarized power is what a designer sees and carries the irreducible geometric term; the coating's excess over the uncoated baseline is the coating-only quantity and the only one comparable with a pure-Fresnel analytic. They diverge most at the quarter wave, where the coating term is nearly extinguished and the total sits at 1.5× the uncoated floor.
- **An unpolarized source is two traces** with orthogonal input states summed in intensity, never one trace with the second state inferred.
- **Across wavelengths, measure total power**, never a fixed-pixel annulus mean, which subtends a different λ/D range at each wavelength and mixes the coating effect with the diffraction scale.
- **Write a gate's analytic from the textbook**, never from the engine's own expression: a reference transcribed from the code is circular in exactly the sign it should check.

11.8 Validation

The MACOS Polarization Validation Report (`docs/macos-manual/polval/`, built as `macosPolVal`) measures every claim above with a named test behind each number and the conventions table reproduced byte for byte from its own conventions section. Its ladder, from the bottom:

1. polarization on does not move geometry (OPD bitwise unchanged);
2. the two-trace Jones pupil is unitary on a lossless train (diattenuation residual 2.6e-15) and matches closed-form Fresnel coefficients on a coated fold to 1.2e-14;
3. the two-mirror train reduces to pure polarization astigmatism (forbidden terms at 7e-15 of the astigmatic term);
4. after one near-normal perfect-conductor mirror the whole single- reflection Jones matrix matches the Born and Wolf closed form at 2e-15 median, and the cross-polarization grows as ρ^2 (slope 1.87);
5. coated transmission matches the Abeles characteristic matrix (Macleod, ch. 2) at normal incidence and at 45°, the angle that exercises the cosine half of the radiometric factor;
6. the polarizing elements obey Malus's law, the retarder's signed circular Stokes parameter, the 2θ composition law and unitarity, on both element dispatch chains, at 20° incidence;
7. vector diffraction reproduces the scalar intensity to 4.5e-16 on a normal-incidence chain for x, 45° and circular input (a test with x-polarized input alone passes vacuously);
8. the coated reflector is anchored to a published protected-aluminium mirror model (van Harten, Snik and Keller, PASP 121, 377, 2009): diattenuation and retardance agree with their curves to 3e-14 and 5e-14 against their stated ±0.01, driven with *their* indices — index tables disagree, and that is not a machinery error.

Every fix the ladder found is pinned by a test that the pre-fix engine fails; the report names them.

11.9 Idealizations and gaps

No ray splitting at any element; no Mueller calculus and no partially coherent light (unpolarized light is two traces); no vector vortex coronagraph element; `RfPolarizer` not implemented; a waveplate's retardance independent of incidence angle; between-leg polarizing surfaces not carried by the grid (the Tranche-2 running Jones matrix per ray is designed, not built); spatially variable coatings not yet available; vector mode and multi-wavefront composition exclusive. The oblique-incidence radiometric factor and the ray-density bookkeeping of a diffraction seed overlap conceptually; whether that double-counts in oblique refractive seeding is an open audit question — the mirror trains the PROPER comparisons used never exercised it.

REFERENCES

1. D. Redding and W. Breckenridge, “Modeling Optics for Dynamics and Control Analysis,” Journal of Guidance, Control and Dynamics, September, 1991.

2. J. W. Goodman, *Introduction to Fourier Optics*, McGraw-Hill, New York, 1968.

3. Sziklas and Siegman, “Mode Calculations in Unstable Resonators with Flowing Sat-urable Gain2: Fast Fourier Transform Method,” Applied Optics, v14, p. 1874-1889, 1975.

4. Oppenheim and Schaefer, *Digital Signal Processing.*

5. Brigham,

6. G. Lawrence, OASIS Manual, Version 5.0, August, 1985 with August 1986 updates. Published by the Rocketdyne Division of Rockwell International Corporation, Albu-querque Optical Support Office.

7. G. Lawrence, Optical Modeling preprint

8. D. Redding and W. Breckenridge, “Coordinate-Free Raytrace Equations for Aspheric and Deformed Optical Surfaces,” Memo DCR-91-38, July 13, 1991.

9. D. Redding, “Mathematical Description of Generalized Optical Element Surfaces,” Memo DCR-91-26, May 27, 1991.

10. P. C. Hughes, *Spacecraft Attitude Dynamics*,1986.

11. H. Akima, “A method of bivariate interpolation and smooth surface fitting for irrr-regularly distributed data points,” ACM Trans. on Mathematical Software, V4, No 2, June 1978.

12. H. Akima, “ALGORITH 526:Bivariate interpolation and smooth surface fitting for irrrregularly distributed data points,” ACM Trans. on Mathematical Software, V4, No 2, June 1978.

13. D. C. Redding, D, P. Dumont, and J. Yu, “Hubble Space Telescope Prescription Retrieval,” Optical Society of America, Space Optics Meeting, Paper MF4. Will-iamsburg VA. 11/18-19/91.

14. D. C. Redding and W. Breckenridge, “Optical Modeling for Dynamics and Control Analysis,” Journal of Guidance, Control, and Dynamics, Vol. 14, No. 5. Sept.-Oct. 1991.

APPENDIX A: Examples and Tutorial

Demonstrations

This appendix contains prescriptions and macros that illustrate various aspects of MACOS usage. Electronic versions of these files have been provided with the MACOS distribution tapes. The interested user can execute these examples to gain experience using MACOS.

A.1 Cassegrain Telescope Examples

A.1.1 Cassegrain.jou

```
% This macro generates example figures
% for Section 2 of the MACOS User Manual
old Cassegrain modify ngridpts=51 q
spot 1 Tout % Fig 3a
draw;;;; % Fig 3b
spot 5 Tout % Fig 4a
perturb 2 global 0,1e-4,0 0,0,0
spot 5 Tout % Fig 4b
```

A.1.2 Cassegrain.in

```
% Cassegrain telescope example
ChfRayDir= 0d0 0d0 1d0 ChfRayPos= 0d0 0d0 -5d0 zSource= 1.d+21
IndRef= 1d0
Extinc= 0d0 Wavelen= 1.d-06
Flux= 1d0 GridType= Circular Aperture= 4d0 Obscratn= 0d0 nGridpts= 15
```

xGrid= -1d0 0d0 0d0 yGrid= 0d0 -1d0 0d0 nElt= 5

iElt= 1

EltName= SecMirObs Element= Obscuring Surface= Conic

KrElt= -1.d+20


```
KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 -4.2d+00 RptElt= 0d0 0d0 -4.2d+00 IndRef= 1d0
Extinc= 1d22
nObs= 3
ObsType= Circle
ObsVec= 5.d-01 0d0 0d0 ObsType= Rectangle
ObsVec= -0.003125 0.003125 -3 3
ObsType= Rectangle
ObsVec= -3 3 -0.003125 0.003125
xObs= 1d0 0d0 0d0 ApType= Circular ApVec= 2d0 0d0 0d0
zElt= 1.d+20 PropType= Geometric nECoord= -6
iElt= 2
EltName= Primary Element= Reflector Surface= Conic
KrElt= -1.08d+01
KcElt= -1d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= Circular
ApVec= 2.1d0 0d0 0d0 zElt= 5.4d+00
PropType= Geometric nECoord= -6
iElt= 3
EltName= Secondary Element= Reflector Surface= Conic
KrElt= -3.526787501D+00 KcElt= -2.670556022D+00
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -4.061145902D+00
RptElt= 0d0 0d0 -5.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
```

```
zElt= 1.338854098D+00
PropType= Geometric nECoord= -6
iElt= 4
EltName= Ref_surf Element= Reference Surface= Conic
KrElt= -5.560145902D+00
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -4.060145902D+00 RptElt= 0d0 0d0 -4.060145902D+00
```

	1d0 1d22	
	0	
	None 5.560145902D+00	
IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	FarField	
	-6	
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	5	
	Detector FocalPlane Flat	
	-1d22	
	0d0	
	0d0 0d0 1d0 0d0 0d0 1.5d+00	
	0d0 0d0 1.5d+00	
	1d0 1d22	
	0	
	None 0d0	
	Geometric	
	-6	
nOutCord=	5	0d0 0d0 0d0
Tout=	1d0 0d0 0d0 0d0	
	0d0 1d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0	0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0
	0d0 0d0 0d0 0d0	0d0 0d0 1d0

A.1.3 CassWithExitPupil.jou

```
% This macro generates more example figures
% for Section 2 of the MACOS User Manual
old CassWithExitPupil modify ngridpts=128 q
perturb 3 global 0,0,0 0,0,1e-4
stop obj 0,0,0
fex 5 yes
opd 5 Tout % Fig 6a
```

```
perturb 2 global 0,1e-4,0 0,0,0
opd 5 Tout % Fig 6b
old CassWithExitPupil modify ngridpts=64 q stop obj 0,0,0
fex 5 yes
intensity 6 % Fig 10a
perturb 2 global 0,1e-4,0 0,0,0
fex 5 yes
intensity 6 % Fig 10b
```

A.1.4 CassWithExitPupil.in

```
% Cassegrain telescope example
% This version has an exit pupil
ChfRayDir= 0d0 0d0 1d0 ChfRayPos= 0d0 0d0 -5d0 zSource= 1.d+21
IndRef= 1d0
Extinc= 0d0 Wavelen= 1.d-06
Flux= 1d0 GridType= Circular Aperture= 4d0 Obscratn= 0d0 nGridpts= 15
xGrid= -1d0 0d0 0d0 yGrid= 0d0 -1d0 0d0 nElt= 6
iElt= 1
EltName= SecMirObs Element= Obscuring Surface= Conic
KrElt= -1.d+20 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 -4.2d+00 RptElt= 0d0 0d0 -4.2d+00 IndRef= 1d0
Extinc= 1d22
nObs= 3
ObsType= Circle
ObsVec= 5.d-01 0d0 0d0 ObsType= Rectangle
ObsVec= -0.003125 0.003125 -3 3
ObsType= Rectangle
ObsVec= -3 3 -0.003125 0.003125
xObs= 1d0 0d0 0d0 ApType= Circular ApVec= 2d0 0d0 0d0
zElt= 1.d+20 PropType= Geometric nECoord= -6
iElt= 2
EltName= Primary Element= Reflector Surface= Conic
KrElt= -1.08d+01
KcElt= -1d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= Circular
ApVec= 2.1d0 0d0 0d0 zElt= 5.4d+00
PropType= Geometric nECoord= -6
iElt= 3
EltName= Secondary Element= Reflector Surface= Conic
KrElt= -3.526787501D+00 KcElt= -2.670556022D+00
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -4.061145902D+00
RptElt= 0d0 0d0 -5.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.338854098D+00
PropType= Geometric nECoord= -6
iElt= 4
EltName= Return1 Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.5d+00 RptElt= 0d0 0d0 1.5d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
iElt= 5
EltName= ExitPupil Element= Return Surface= Conic
KrElt= -5.560145902D+00
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -4.060145902D+00 RptElt= 0d0 0d0 -4.060145902D+00
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 5.560145902D+00
PropType= FarField nECoord= -6
iElt= 6
EltName= Detector Element= FocalPlane Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.5d+00 RptElt= 0d0 0d0 1.5d+00 IndRef= 1d0
```

```
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
nOutCord= 5
Tout= 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0
0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.2 Diffractive Elements Examples

A.2.1 GratingExample.jou

```
% This macro generates example figures
% for Section 4 of the MACOS User Manual old gratingExample
mod OrderHOE(1)=0 q draw;;;;
mod OrderHOE(1)=1 q
draw;;;; % Fig ??
mod OrderHOE(1)=2 q draw;;;;
mod OrderHOE(1)=-1 q draw;;;;
mod OrderHOE(1)=-2 q
draw;;;; % Fig ??
```

A.2.2 GratingExample.in

```
ChfRayDir= 0d0 0d0 1d0 ChfRayPos= 0d0 0d0 -6d0 zSource= 1d22
IndRef= 1d0
Extinc= 0d0 Wavelen= 6.d-07
Flux= 1d0 GridType= Circular Aperture= 1d0 Obscratn= 0d0 nGridpts= 65
xGrid= 1d0 0d0 0d0 yGrid= 0d0 1d0 0d0 nElt= 2
iElt= 1
EltName= Grating Element= Grating Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0
```

IndRef= 1d0

Extinc= 1d22

h1HOE= 1d0 0d0 0d0

OrderHOE= 1d0

RuleWidth= 18.97366596d-7 nObs= 0

ApType= None

zElt= 1d22 PropType= Geometric nECoord= -6

---

iElt= 2

EltName= FP Element= FocalPlane Surface= Flat

KrElt= 1d22

KcElt= 0d0

psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 -3d0 RptElt= 0d0 0d0
0d0 IndRef= 1d0

Extinc= 0d0

nObs= 0

IndRef= 1d0

Extinc= 1d22

h1HOE= 1d0 0d0 0d0

OrderHOE= 1d0

RuleWidth= 18.97366596d-7 nObs= 0

ApType= None

zElt= 1d22 PropType= Geometric nECoord= -6

ApType= None

zElt= 1d22 PropType= Geometric nECoord= -6

nOutCord= 5 0d0 0d0 0d0

Tout= 1d0 0d0 0d0 0d0

0d0 1d0 0d0 0d0 0d0 0d0 0d0

0d0 0d0 0d0 1d0 0d0 0d0 0d0

0d0 0d0 0d0 0d0 1d0 0d0 0d0

0d0 0d0 0d0 0d0 0d0 0d0 1d0

A.2.3 HOEExample.jou

```
% This macro generates example figures
% for Section 4 of the MACOS User Manual
loa HOEExample mod ngridpts=15 q map
ray 76
stop elt 1 0,0 fex;;;
int;;
draw;;;; % Fig ??
mod Element(1)=reflector q ray 76
fex;;; int;;
```

draw;;;; % Fig ??

A.2.4 HOEExample.in

ChfRayDir= 0d0 0d0 1d0 ChfRayPos=
0d0 0d0 -4d0 zSource= 1d22

IndRef= 1d0

Extinc= 0d0 Wavelen= 6.d-07

Flux= 1d0 GridType= Circular Aperture=
5.d-01 Obscratn= 0d0 nGridpts= 64

xGrid= 1d0 0d0 0d0 yGrid= 0d0 1d0 0d0
nElt= 2

iElt= 1

draw;;;;	% Fig ??
EltName= HOEonMirror Element= HOE Surface= Conic	
KrElt= -6d0 KcElt= -1d0	
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0	
Extinc= 1d22	
h1HOE= 0d0 0d0 -1d22 h2HOE= 1d0 0d0 -3d0	
OrderHOE= 1d0 WaveHOE= 6.d-07	
nObs= 0	
ApType= None	
zElt= 1d22 PropType= Geometric nECoord= -6	
iElt= 2	
EltName= FP Element= FocalPlane Surface= Flat	
KrElt= -1d22 KcElt= 0d0	
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 -3d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0	
Extinc= 0d0	
nObs= 0	
ApType= None	
zElt= 1d22 PropType= Geometric nECoord= -6	
nOutCord= 5	0d0 0d0 0d0
Tout= 1d0 0d0 0d0 0d0	
0d0 1d0 0d0 0d0	0d0 0d0 0d0
0d0 0d0 0d0 1d0	0d0 0d0 0d0
0d0 0d0 0d0 0d0	1d0 0d0 0d0
0d0 0d0 0d0 0d0	0d0 0d0 1d0

A.3 Non-Sequential Surface Examples

A.3.1 CornerCube.jou

```
old CornerCube draw;;;xz
mod ChfRayPos(1)=0.762 ChfRayPos(2)=0.762 Aperture=0.125 nGridpts=11
q draw;;;yz
```

A.3.2 CornerCube.in

```
ChfRayDir= 0d0 0d0 -1d0 ChfRayPos= 1.d-05 1.d-05 1.d+01 zSource= 1.d22
IndRef= 1d0
Extinc= 0d0 Wavelen= 1.059d-04
Flux= 1d0 GridType= Circular Aperture= 2.54d+00 Obscratn= 0d0 nGridpts= 65
xGrid= 0d0 1d0 0d0 yGrid= -1d0 0d0 0d0 nElt= 5
iElt= 1
EltName= Ref Srf Element= Reference Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 3.5d+00 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= NFP1surf nECoord= 0
iElt= 2
EltName= CC1 Element= NSReflector Surface= Conic
KrElt= 1d22
KcElt= 0d0
psiElt= 8.164994677D-01 0d0 5.773461867D-01
VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 3
EltName= CC2 Element= NSReflector Surface= Conic
KrElt= 1d22
KcElt= 0d0
psiElt= -4.082497338D-01 7.071092812D-01 5.773461867D-01
VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 4
EltName= CC3 Element= NSReflector Surface= Conic
KrElt= 1d22
KcElt= 0d0
psiElt= -4.082497338D-01 -7.071092812D-01 5.773461867D-01
VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 5
EltName= FP Element= FocalPlane Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 3.5d+00 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.d+03 PropType= Geometric nECoord= 0
nOutCord= 5
Tout= 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0
0d0 1d0 0d0 0d0
0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.3.3 Luneberg.jou

```
% Luneberg.jou: a MACOS example macro. This macro
% demonstrates use of non-sequential refracting
% elements, polarization, and the gain command.
old Luneberg draw 0 16 XZ
beam cos 8.7500000000D-01 1.5
mod nGridPts=128 q int 16
```

```
mod nGridPts=64 q gain 17
col
gain 17 129
pol 1,0 0,0
mod nGridPts=128 q int 16
mod nGridPts=64 q gain 17
col
gain 17 129
```

A.3.4 Luneberg.in

```
% Luneberg.in: a MACOS example .in-file. This
% macro demonstrates use of non-sequential
% refractors, polarization, and the gain command.
ChfRayDir= 0d0 0d0 1d0
ChfRayPos= 0d0 0d0 -2.121332605D-01
zSource= -1.d-02 IndRef= 1d0
Extinc= 0d0 Wavelen= 2.15d-02
Flux= 1d0 GridType= Circular Aperture= 1.75d+00 Obscratn= 0d0 nGridpts= 33
xGrid= 1d0 0d0 0d0 yGrid= 0d0 1d0 0d0 nElt= 17
iElt= 1
EltName= Shell1 Element= NSRefractor Surface= Conic
KrElt= -2.0353d-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 -2.0353d-01
RptElt= 0d0 0d0 0d0 IndRef= 1.056692495D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 2
EltName= Shell2 Element= NSRefractor Surface= Conic
KrElt= -1.887578424D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.887578424D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.084805420D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 3
EltName= Shell3 Element= NSRefractor Surface= Conic
KrElt= -1.872644104D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.872644104D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.147803142D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 4
EltName= Shell4 Element= NSRefractor Surface= Conic
KrElt= -1.771655689D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.771655689D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.182543332D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 5
EltName= Shell5 Element= NSRefractor Surface= Conic
KrElt= -1.529604025D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.529604025D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.235415610D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 6
EltName= Shell6 Element= NSRefractor Surface= Conic
```

```
KrElt= -1.387929631D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.387929631D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.265593240D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 7
EltName= Shell7 Element= NSRefractor Surface= Conic
KrElt= -1.297999186D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.297999186D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.283513736D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 8
EltName= Shell8 Element= NSRefractor Surface= Conic
KrElt= -1.087287483D-01
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -1.087287483D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.312911218D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 9
EltName= Shell9 Element= NSRefractor Surface= Conic
KrElt= -9.780754704D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -9.780754704D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.340947910D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 10
EltName= Shell10 Element= NSRefractor Surface= Conic
KrElt= -8.777974770D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -8.777974770D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.338212289D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 11
EltName= Shell11 Element= NSRefractor Surface= Conic
KrElt= -7.943573990D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -7.943573990D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.371777076D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 12
EltName= Shell12 Element= NSRefractor Surface= Conic
KrElt= -6.183854602D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -6.183854602D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.386006296D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 13 EltName= Shell13 Element= NSRefractor Surface= Conic
```

```
KrElt= -4.663931773D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -4.663931773D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.402911090D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 14
EltName= Shell14 Element= NSRefractor Surface= Conic
KrElt= -2.956340242D-02
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -2.956340242D-02
RptElt= 0d0 0d0 0d0 IndRef= 1.412d+00 Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 15
EltName= Aperture Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 2.221359167D-01
RptElt= 0d0 0d0 0d0 IndRef= 1.0488d+00
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.d+01 PropType= Geometric nECoord= -6
iElt= 16
EltName= ExitPupil Element= Return
```

	Conic
	-1d22
	0d0
	0d0 0d0 -1d0 0d0 0d0 0d0 0d0 0d0 0d0 1 .0488d+00
	1d22
Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	0 None 1.d+01 FarField -6

iElt=	17
EltName= Element= Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	FP F ocalPlane Flat -1d22

	Conic	
	-1d22	
	0d0	
	0d0 0d0 -1d0 0d0 0d0 0d0 0d0 0d0 0d0 1 .0488d+00	
	1d22	
Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	0 None 1.d+01 FarField -6	
	0d0	
	0d0 0d0 -1d0 0d0 0d0 1d3 0d0 0d0 0d0 1d0	
	1d22	
	0	
	None 0d0	
	Geometric	
	-6	
nOutCord=	5	0d0 0d0 0d0 0d0
Tout=	1d0 0d0 0d0	
	0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0	0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 1d0
	0d0 0d0 0d0	

A.4 Adaptive Optics Example

A.4.1 AOExample.jou

```
% MACOS Adaptive Optics Example Macro
% First show system response without atmosphere:
old AOexample; % Load prescription
draw 0 26 xz % Draw full system
draw 6 26 xy % Draw AO optics only
spo 1 b y % Pupil spot diagram
opd 16 y % OPD in exit pupil shows deformable
% mirror test pattern
spo 26 Tout y % Hartman sensor spots on detector
% Next show system response with atmosphere but no
% deformable mirror:
modify Surface(7)=Flat % Flatten deformable mirror q
atmos 1 20 % Add atmospheric phase disturbance
2.854331e-5 0 atm 12.5 % at entrance pupil
opd 16 y % OPD in exit pupil
spo 26 Tout y % Hartman sensor spots on detector
```

A.4.2 AOExample.in

```
% MACOS Adaptive Optics Example Prescription
```

ChfRayDir=	0d0 0d0 1d0
ChfRayPos=	0d0 0d0 -2d+02
zSource=	1d22
IndRef=	1d0
Extinc=	0d0
Wavelen=	2.854330709D-05
Flux=	1d0
GridType=	Circular
Aperture=	1.991873225632d02
Obscratn=	40.5d0
nGridpts=	128
xGrid=	0d0 -1d0 0d0
yGrid=	-1d0 0d0 0d0
nElt=	26
iElt=	1
EltName=	Pupil
Element=	Reference
Surface=	Conic
KrElt=	-1d22
KcElt=	0d0
psiElt=	0d0 0d0 -1d0
VptElt=	0d0 0d0 -6d0
RptElt=	0d0 0d0 0d0
IndRef=	1d0
Extinc=	0d0
nObs=	0

ChfRayDir=	0d0 0d0 1d0
ApType=	None
zElt=	1d22
PropType=	Geometric
nECoord=	-6
iElt=	2
EltName=	PM
Element=	Reflector
Surface=	Conic
KrElt=	-1.3357d+03
KcElt=	-1d0
psiElt=	0d0 0d0 -1d0
VptElt=	0d0 0d0 0d0
RptElt=	0d0 0d0 0d0
IndRef=	1d0
Extinc=	0d0
nObs=	0
ApType=	None
zElt=	6.6785d+02
PropType=	Geometric
nECoord=	-6
iElt=	3
EltName=	SM

```
Element= Reflector Surface= Conic
KrElt= -3.184999999D+02 KcElt= -2.352756999D+00
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -5.423064572D+02 RptElt= 0d0 0d0 -5.423064572D+02
IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1.256970591D+02
PropType= Geometric nECoord= -6
iElt= 4
EltName= FM1 Element= Reflector Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 7.071067812D-01 0d0 7.071067812D-01
VptElt= 0d0 0d0 7.5375d+01 RptElt= 0d0 0d0 7.5375d+01 IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 5
EltName= OAP1 Element= Reflector Surface= Conic
KrElt= -1.364d+02
KcElt= -1d0
psiElt= 9.668919598D-01 -2.551860851D-01 0d0 VptElt= -4.140665047D+01 1.740405528D+01 7.5375d+01
RptElt= -4.140665047D+01 1.740405528D+01 7.5375d+01
IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None zElt= 6.82d+01
PropType= Geometric nECoord= -6
iElt= 6
```

```
EltName= FSM Element= Reflector Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 9.999929341D-01 3.759223637D-03 0d0 VptElt= 1.318856255D+01 -1.530759269D+01 7.5375d+01
RptElt= 1.318856255D+01 -1.530759269D+01 7.5375d+01
IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric
nECoord= -6
iElt= 7
EltName= DM Element= Reflector Surface= UserDefined
KrElt= -1d22
```

KcElt= 0d0

UDSrfType= 1

UDSrfFile= AOmirror.test % 1 or 5

pMon= -2.249426003D+00	-1.9 51098758D+01	7.5375d+01
xMon= -4.697040833D-01	8.8 28239202D-01	0d0
yMon= 0d0 0d0 1d0	4.6 97040833D-01	0d0
zMon= 8.828239202D-01		
lMon= 0d0	4.6 97040833D-01	0d0
psiElt= 8.828239202D-01		
VptElt= -2.250697813D+00	-1.9 50691308D+01	7.5375d+01
RptElt= -2.250697813D+00	-1.9 50691308D+01	7.5375d+01
IndRef= 1d0		
Extinc= 0d0		
nObs= 0		
ApType= None		
zElt= 1d22 PropType= Geometric nECoord= -6		
iElt= 8	9.9 59082335D-01	0d0
EltName= FM2 Element= Reflector Surface= Flat		
KrElt= -1d22 KcElt= 0d0		
psiElt= 9.037029675D-02		
VptElt= 2.331918268D+01	2.5 76404533D+00	7.5375d+01
RptElt= 2.331918268D+01	2.5 76404533D+00	7.5375d+01

KcElt= 0d0

UDSrfType= 1

UDSrfFile= AOmirror.test % 1 or 5

pMon= -2.249426003D+00 -1.9 51098758D+01 7.5375d+01

IndRef= 1d0

Extinc= 0d0

nObs= 0

ApType= None

zElt= 1d22 PropType= Geometric nECoord= -6

iElt= 9 7.7 91729129D-01 0d0

EltName= OAP2 Element= Reflector Surface= Conic

KrElt= -1.364d+02

KcElt= -1d0

psiElt= -6.268090394D-01

VptElt= 2.538458250D+01 -2.8 22361844D+01 7.5375d+01

RptElt= 2.538458250D+01 -2.8 22361844D+01 7.5375d+01

IndRef= 1d0

Extinc= 0d0

nObs= 0

ApType= None zElt= 6.82d+01

PropType= Geometric

nECoord= -6

	10		
	SSM1		
	Reflector Flat		
	-1d22		
iElt= EltName= Element= Surface= KrElt= KcElt=	0d0		
psiElt=	-9.937 679192D-01	1.1146 89322D-01	0d0
VptElt=	5.421 565944D+00	8.1171 54092D+00	7.5375d+01
RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	5.421 565944D+00	8.1171 54092D+00	7.5375d+01
	1d0 0d0 0		
	None 1d22		
	Geometric		
	-6		
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt=	11	7.6719 02813D-02	0d0
	SSM2		
	Reflector Flat		
	-1d22		
	0d0		
	-9.970 527522D-01		
VptElt=	1.733 428143D+01	1.3321 69152D+01	7.5375d+01
RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	1.733 428143D+01	1.3321 69152D+01	7.5375d+01
	1d0 0d0 0		
	None 1d22		
	Geometric		
	-6		

	10		
	SSM1		
	Reflector Flat		
iElt= EltName= Element= Surface= KrElt= KcElt=	-1d22		
	0d0		
psiElt=	-9.937 679192D-01	1.1146 89322D-01	0d0
iElt=	12		
EltName= Element= Surface= KrElt= KcElt=	FS	2.8234 14568D-01	0d0
	Reflector Flat		
psiElt=	-1d22		
	0d0		
	-9.593 139745D-01		
VptElt=	4.387 255193D+00	2.1538 12421D+01	7.5375d+01
RptElt= IndRef= Extinc= nObs=	4.387 255193D+00	2.1538 12421D+01	7.5375d+01
ApType= zElt= PropType= nECoord=	1d0 0d0 0		
	None 1d22		
	Geometric		
	-6		
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt=	13	-6.9812 60298D-03	0d0
	WFSRL1A		
	Refractor Conic		
	-1.550 872636D+00		
	0d0		
	9.999		

	10		
	SSM1		
	Reflector Flat		
iElt= EltName=	-1d22		
Element= Surface=	0d0		
KrElt= KcElt=			
psiElt=	-9.937679192D-01	1.114689322D-01	0d0
	756307D-01		
VptElt=	8.496015971D+00	2.150943918D+01	7.5375d+01
RptElt= IndRef=	8.496015971D+00	2.150943918D+01	7.5375d+01
Extinc= nObs=			
	1.512425970D+00		
	0d0 0		

	None 5.288796394D+00		
ApType=	Geometric		
zElt= P			
ropType=	-6		
nECoord=			
iElt=	14	6.981260298D-03	0d0
EltName=			
Element=	WFSRL1B		
Surface=	Refractor Conic		
KrElt=			
KcElt=	-4.419109771D+00		
psiElt=			
	0d0		
	-9.999756307D-01		
VptElt=	8.996003786D+00	2.150594855D+01	7.5375d+01
RptElt=	8.996003786D+00	2.150594855D+01	7.5375d+01
IndRef=			
Extinc=	1.789618121D+00		
nObs=			
ApType=	0d0 0		
zElt= P			

	None 5.288796394D+00		
ApType=	Geometric		
zElt= P			
ropType=	-6		
nECoord=			
ropType=	None 3.411018511D+00		
nECoord=	Geometric		
	-6		
iElt=	15	-6.981260298D-03	0d0
EltName=			
Element=	WFSRL1C		
Surface=	Refractor Conic		
KrElt=			
KcElt=	-2.771554824D+01		
psiElt=	0d0		
	9.999756307D-01		
VptElt=	9.495991602D+00	2.150245792D+01	7.5375d+01
RptElt=	9.495991602D+00	2.150245792D+01	7.5375d+01
IndRef=			
Extinc=	1d0 0d0 0		
nObs=			
ApType=	None 2.198859945D+01		
zElt= P			
ropType=	Geometric		
nECoord=	-6		
iElt=	16	-6.981321726D-03	3.414481680D-15
EltName=			
Element=	LensArrayReference Reference		
Surface=			
KrElt=	Conic		
KcElt=			
psiElt=	-1d22		
	0d0		
	9.999756303D-01		
VptElt=	1.272900024D+01	2.147992068D+01	7.537500000D+01
RptElt=	1.272900024D+01	2.147992068D+01	7.537500000D+01
IndRef=			
Extinc=	1d0 0d0 0		
nObs=			

None 5.288796394D+00

ApType= Geometric
zElt= P
ropType= -6
nECoord=

ApType= None 1d22
zElt= P
ropType= Geometric
nECoord= -6

iElt= 17
EltName=
Element= LensArray1A

LensArray

```
Surface= Conic
KrElt= -0.47244094488189d+00
KcElt= 0d0
psiElt= 9.999756303D-01 -6.981321725D-03 0d0
VptElt= 12.7291897290d0 21.4720113661d0 75.3671259843d0 RptElt= 1.272924470D+01 2.147988519D+01
7.5375d+01
LensArrayType= 2
LensArrayWidth= 0.01574803149606
pMon= 1.272924470D+01 2.147988519D+01 7.5375d+01
xMon= 0d0 0d0 -1d0
yMon= -6.981321725D-03 -9.999756303D-01 0d0 zMon= -9.999756303D-01 6.981321725D-03 0d0
IndRef= 1.512425970D+00
Extinc= 0d0
nObs= 0
ApType= None zElt= 5.17d+00
PropType= Geometric nECoord= -6
iElt= 18
EltName= LensArray1B Element= Refractor Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= -9.999756307D-01 6.981260298D-03 0d0 VptElt= 1.297923860D+01 2.147813988D+01 7.5375d+01
RptElt= 1.297923860D+01 2.147813988D+01 7.5375d+01
IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 19
EltName= FieldLensA Element= Refractor Surface= Conic
KrElt= -2.122047244D+00
KcElt= 0d0
psiElt= 9.999756307D-01 -6.981260298D-03 0d0 VptElt= 1.358583579D+01 2.147390496D+01 7.5375d+01
RptElt= 1.358583579D+01 2.147390496D+01 7.5375d+01 IndRef= 1.677871170D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1.359427521D+00
PropType= Geometric nECoord= -6
iElt= 20
EltName= FieldLensB Element= Refractor Surface= Conic
KrElt= -3.224409449D+00
KcElt= 0d0
psiElt= 9.999756307D-01 -6.981260298D-03 0d0 VptElt= 1.374331196D+01 2.147280555D+01 7.5375d+01
RptElt= 1.374331196D+01 2.147280555D+01 7.5375d+01
IndRef= 1d0
```

Extinc= 0d0

---

nObs= 0
ApType=

Extinc=	0d0		
zElt=	None 1.359427521D+00		
PropType=	Geometric		
nECoord=	-6		
iElt=	21	-6.981260298D-03	0d0
EltName=	WFSLAR1A		
Element=	Refractor Conic		
Surface=	-8.055118110D-01		
KrElt= KcElt=	0d0		
psiElt=	9.999756307D-01		
VptElt=	1.880962705D+01	2.143743542D+01	7.5375d+01
RptElt=	1.880962705D+01	2.143743542D+01	7.5375d+01
IndRef=	1.611736488D+00		
Extinc=	0d0 0		
nObs=	None 1.359427521D+00		
ApType=	Geometric		
zElt=	-6		
PropType=	22	6.981260298D-03	0d0
nECoord=	WFSLAR1B		
iElt=	Refractor Conic		
EltName=	-5.090551181D-01		
Element=	0d0		
Surface=	-9.999756307D-01		
KrElt= KcElt=	1.896631614D+01	2.143634151D+01	7.5375d+01
psiElt=	1.896631614D+01	2.143634151D+01	7.5375d+01
VptElt=	1.677871170D+00		
RptElt=	0d0 0		
IndRef=	None 6.038543863D-01		
Extinc=			
nObs=			
ApType=			
zElt=			
PropType=			

Extinc=	0d0		
nECoord=	Geometric		
	-6		
iElt=	23	6.981260298D-03	0d0
EltName= Element=	WFSLAR1C		
Surface= KrElt= KcElt=	Refractor Conic		
psiElt=	-2.980708661D+00		
	0d0		
	-9.999756307D-01		
VptElt=	1.901946445D+01	2.143597046D+01	7.5375d+01
RptElt=	1.901946445D+01	2.143597046D+01	7.5375d+01
IndRef= Extinc=	1d0 0d0 0		
nObs= ApType=	None 2.027987141D+00		
zElt= PropType=	Geometric		
nECoord=	-6		
iElt=	24		
	CCDWinA		
	Refractor		
EltName= Element= Surface=	Conic		
KrElt= KcElt=	-1d22	6.9812 60298D-03	0d0
psiElt=	0d0		
	-9.999 756307D-01		
VptElt=	2.027 861775D+01	2.1427 17977D+01	7.5375d+01
RptElt= IndRef= Extinc= nObs=	2.027 861775D+01	2.1427 17977D+01	7.5375d+01
ApType= zElt= PropType= nECoord=	1.512 425970D+00		

	CCDWinA Refractor		
EltName= Element= Surface=	Conic		
	0d0 0 None 1d22 Geometric -6		
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt=	25 CCDWinB Refractor Conic -1d22 0d0 -9.999 756307D-01	6.9812 60298D-03	0d0
VptElt=	2.040 361470D+01	2.1426 30711D+01	7.5375d+01
RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	2.040 361470D+01 1d0 0d0 0 None 1d22 Geometric -6	2.1426 30711D+01	7.5375d+01
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt=	26 CCD FocalPlane Flat -1.000 000000D+22 0.000 000000D+00	6.8571 53362D-03	0d0

	CCDWinA Refractor		
EltName= Element= Surface=	Conic		
	-9.999 764894D-01		
VptElt=	2.049 655742D+01	2.1425 65820D+01	7.537500000D+01
RptElt= IndRef= Extinc= nObs=	2.049 655742D+01	2.1425 65820D+01	7.537500000D+01
ApType= zElt= PropType= nECoord=	1.000 000000D+00		
	0.000 000000D+00		
	0		
	None 1.000 000000D+22		
	Geometric		
	-6		
nOutCord=	5	-9.9997 56307D-01	0d0 0d0 0d0 0d0 0d0
Tout=	-6.981 260298D-03		

```
0d0 0d0 -1d0 0d0 0d0 0d0 0d0 0d0
0d0 0d0 0d0 -6.981260298D-03 -9.999756307D-01 0d0 0d0
0d0 0d0 0d0 0d0 0d0 -1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.4.3 AOmirror.test

```
5.5000000e+01 4e-05
7.6000000e+01 -4e-05
7.7000000e+01 -4e-05
9.5000000e+01 -4e-05
9.6000000e+01 -4e-05
```

1.0200000e+02	3e-05
1.0300000e+02	3e-05
1.0400000e+02	3e-05
1.0500000e+02	3e-05
1.0600000e+02	3e-05
1.0700000e+02	3e-05
1.2300000e+02	4e-05

1.0200000e+02	3e-05
1.2400000e+02	4e-05
1.2500000e+02	4e-05
1.2600000e+02	4e-05
1.2700000e+02	4e-05
1.2800000e+02	4e-05
2.0700000e+02	5e-06
2.2800000e+02	5e-06
2.0800000e+02	1e-05
2.2900000e+02	1e-05
2.4900000e+02	1e-05
2.6800000e+02	1e-05
2.0900000e+02	1.5e-05
2.3000000e+02	1.5e-05
2.5000000e+02	1.5e-05
2.6900000e+02	1.5e-05
2.8700000e+02	1.5e-05
2.1000000e+02	2e-05
2.3100000e+02	2e-05
2.5100000e+02	2e-05
2.7000000e+02	2e-05
2.8800000e+02	2e-05
3.0400000e+02	2e-05
2.1100000e+02	2.5e-05
2.3200000e+02	2.5e-05
2.5200000e+02	2.5e-05
2.7100000e+02	2.5e-05
2.8900000e+02	2.5e-05
3.0500000e+02	2.5e-05
3.1900000e+02	2.5e-05
2.1200000e+02	3e-05
2.3300000e+02	3e-05
2.5300000e+02	3e-05
2.7200000e+02	3e-05
2.9000000e+02	3e-05
3.0600000e+02	3e-05
3.2000000e+02	3e-05
3.3200000e+02	3e-05
2.1300000e+02	3.5e-05

1.0200000e+02	3e-05
2.3400000e+02	3.5e-05
2.5400000e+02	3.5e-05
2.7300000e+02	3.5e-05
2.9100000e+02	3.5e-05
3.0700000e+02	3.5e-05
3.2100000e+02	3.5e-05
3.3300000e+02	3.5e-05
2.1400000e+02	4e-05
2.3500000e+02	4e-05
2.5500000e+02	4e-05
2.7400000e+02	4e-05
2.9200000e+02	4e-05
3.0800000e+02	4e-05
3.2200000e+02	4e-05

3.3400000e+02	4e-05
3.4300000e+02	4e-05
2.4200000e+02	4e-05
2.4300000e+02	4e-05
2.4400000e+02	4e-05
2.4500000e+02	4e-05
2.2000000e+02	4e-05
2.2100000e+02	4e-05
2.2200000e+02	4e-05
2.2300000e+02	4e-05
2.2400000e+02	4e-05
1.9900000e+02	4e-05
2.0000000e+02	4e-05
2.0100000e+02	4e-05
2.0200000e+02	4e-05
2.2000000e+02	4e-05
1.7800000e+02	4e-05
1.7900000e+02	4e-05
1.8000000e+02	4e-05
1.8100000e+02	4e-05
1.8200000e+02	4e-05
1.7500000e+02	4e-05

A.5 Segmented Telescope Example

A.5.1 SegDemo.jou

```
% SegmentDemo.jou -- a MACOS demonstration macro that illustrates the gener-ation
% of a linear model of a segmented mirror system. The linear model is used to
% generate an image and OPD numbers. These are compared with the same num-bers
% computed using the full nonlinear model. They agree very closely.
% Run this macro with macos256
old SegmentDemo; % Load prescription file
%
draw 0 10 xz; % Sketch optics layout
%
map 0 % Displays the ray mapping in the entrance pupil; also
% shows the segment mapping
%
intensity 10 % Generate simulated image at the focal plane -- no
% aberrations
%
log 10 % Display simulated unaberrated image with log10 stretch
%
%
perturb 3 % Perturb the 3rd element using the full nonlinear model NO % Use global coordinates
0,1e-4,0 % Rotational perturbation: rotate 100 urad about y 0,0,0 % No translational perturbation
%
opd 9 % Display wavefront error using linear model
%
log 10 % Display image (log10 stretch) using full nonlinear
% model
%
old SegmentDemo; % Reload prescription file
%
build 9 % Compute linear optical model
%
lperturb 3 % Perturb the 3rd element using the linear model NO % Use global coordinates
0,1e-4,0 % Rotational perturbation: rotate 100 urad about y 0,0,0 % No translational perturbation
%
lopd % Display wavefront error using linear model
%
llog % Display image (log10 stretch) using linear model
```

A.5.2 SegDemo.in

```
% SegmentDemo.in -- prescription file for SegmentDemo.jou, a MACOS demon-stration
% macro.
```

ChfRayDir=	0d0 0d0 -1d0
ChfRayPos=	0d0 0d0 1.5d+00
zSource=	1.d+19
IndRef=	1d0
Extinc=	0d0
Wavelen=	1.d-04
Flux=	1d0
GridType=	Pie
Aperture=	3.65d+00
Obscratn=	3.d-01
nGridpts=	25
xGrid=	1d0 0d0 0d0
yGrid=	0d0 1d0 0d0
nElt=	10
nSeg=	7

ChfRayDir=	0d0 0d0 -1d0	
width=	1.2d+00	
gap=	0d0	
SegXgrid=	1d0 0d0 0d0	
SegCoord=	0 0	0
	1 -1	-2
	2 1	-1
	1 2	1
	-1 1	2
	-2 -1	1
	-1 -2	-1

```
iElt= 1
EltName= S1 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 1.46d+00
PropType= Geometric
nECoord= -6
iElt= 2
EltName= S2 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0
RptElt= 6.124997330D-01 -1.060881274D+00 2.569563356D-01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 1.46d+00
PropType= Geometric nECoord= 6
TElt= 4.610697370D-01 8.660255296D-01 -1.934282287D-01 0d0 0d0 0d0
-7.985966745D-01 4.999997821D-01 3.350277145D-01 0d0 0d0 0d0 3.868566261D-01 -1.885624059D-18
9.221398760D-01 0d0 0d0 0d0
0d0 0d0 0d0 4.610697370D-01 8.660255296D-01 -1.934282287D-01
0d0 0d0 0d0 -7.985966745D-01 4.999997821D-01 3.350277145D-01
0d0 0d0 0d0 3.868566261D-01 -1.885624059D-18 9.221398760D-01
iElt= 3
EltName= S3 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0
RptElt= 1.225d+00 -3.699397078D-07 2.569563356D-01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 1.46d+00
PropType= Geometric nECoord= 6
TElt= 9.221398760D-01 3.019915982D-07 -3.868566261D-01 0d0 0d0 0d0
-2.784784949D-07 1d0 1.168274508D-07 0d0 0d0 0d0 3.868566261D-01 3.238991466D-24 9.221398760D-01 0d0
0d0 0d0
0d0 0d0 0d0 9.221398760D-01 3.019915982D-07 -3.868566261D-01
0d0 0d0 0d0 -2.784784949D-07 1d0 1.168274508D-07
0d0 0d0 0d0 3.868566261D-01 3.238991466D-24 9.221398760D-01
iElt= 4
EltName= S4 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0
RptElt= 6.125003738D-01 1.060880904D+00 2.569563356D-01
IndRef= 1d0
Extinc= 1d22
nObs= 0
```

```
ApType= None zElt= 1.46d+00
PropType= Geometric nECoord= 6
```

TElt=	4.610702194D-01 -8.660252276D-01 -1.934284311D-01	0 d 0	0 d 0	0 d 0
	7.985963960D-01 5.000003051D-01 -3.350275976D-01	0 d 0	0 d 0	0 d 0
	3.868566261D-01 2.270467800D-17 9.221398760D-01	0 d 0	0 d 0	0 d 0

```
0d0 0d0 0d0 4.610702194D-01 -8.660252276D-01 -1.934284311D-01
0d0 0d0 0d0 7.985963960D-01 5.000003051D-01 -3.350275976D-01
0d0 0d0 0d0 3.868566261D-01 2.270467800D-17 9.221398760D-01
iElt= 5
EltName= S5 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0
RptElt= -6.124995728D-01 1.060881366D+00 2.569563356D-01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 1.46d+00
PropType= Geometric nECoord= 6
TElt= -4.610696164D-01 -8.660256051D-01 1.934281781D-01 0d0 0d0 0d0 7.985967441D-01 -4.999996513D-01
-3.350277437D-01 0d0 0d0 0d0
3.868566261D-01 -2.095137844D-19 9.221398760D-01 0d0 0d0 0d0
0d0 0d0 0d0 -4.610696164D-01 -8.660256051D-01 1.934281781D-01
0d0 0d0 0d0 7.985967441D-01 -4.999996513D-01 -3.350277437D-01
0d0 0d0 0d0 3.868566261D-01 -2.095137844D-19 9.221398760D-01
iElt= 6
EltName= S6 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 0d0
RptElt= -1.225d+00 5.549095617D-07 2.569563356D-01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 1.46d+00
PropType= Geometric nECoord= 6
TElt= -9.221398760D-01 -4.529873973D-07 3.868566261D-01 0d0 0d0 0d0 4.177177424D-07 -1d0 -
1.752411762D-07 0d0 0d0 0d0
3.868566261D-01 -1.071601397D-23 9.221398760D-01 0d0 0d0 0d0
0d0 0d0 0d0 -9.221398760D-01 -4.529873973D-07 3.868566261D-01
0d0 0d0 0d0 4.177177424D-07 -1d0 -1.752411762D-07
0d0 0d0 0d0 3.868566261D-01 -1.071601397D-23 9.221398760D-01
iElt= 7
EltName= S7 Element= Segment Surface= Conic
KrElt= -2.92d+00
KcElt= -1d0
```

0d0 0d0 1d0 0d0 0d0 0d0

-6.125005340D-01 -1.060880811D+00

1d0 1d22

psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=

0

None 1.46d+00

2.56956 3356D-01

Geometric 6

1.93428 4817D-01

TElt=

-4.610703399D-01 8.660251521D-01

0d0 0d0 0d0

-7.985963264D-01 -5.000004359D-01

3.868566261D-01 -1.153428518D-17

3.35027 5684D-01

0d0 0d0 0d0

9.22139 8760D-01

0d0 0d0 0d0

0d0 0d0 0d0 -4 .610703399D-01 8 .660251521D-01 1 .934284817D-01

0d0 0d0 0d0 -7.985963264D-01 -5.000004359D-01

3. 350275684 D-01

0d0 0d0 0d0 3.868566261D-01 -1.153428518D-17

4. 221398760 D-01

iElt= EltName= Element= Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=

8

SecMir Reflector Conic

-2.229620353D-01

-1.174529970D+00

0d0 0d0 1d0

0d0 0d0 1.353d+00

0d0 0d0 1.353d+00

1d0 1d22

0

	0d0 0d0 1d0 0d0 0d0 0d0	
	-6.125005340D-01 -1.060880811D+00	
	1d0 1d22	
psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	0 None 1.46d+00 Geometric 6 -4.610703399D-01	2.56956 3356D-01 1.93428 4817D-01
TElt=	8.660251521D-01	0d0 0d0 0d0

	None 1.07d-01
	Geometric
	-6
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType= zElt= PropType= nECoord=	9 RefSurf Reference Conic -2.6d+00 0d0 0d0 0d0 -1d0 0d0 0d0 1.291d+00 0d0 0d0 1.291d+00 1d0 1d22 0 None 2.6d+00 FarField -6
iElt= EltName= Element= Surface= KrElt= KcElt= psiElt= VptElt= RptElt= IndRef= Extinc= nObs= ApType=	10 FclPlane FocalPlane Flat 0d0 -1.d+38

0d0 0d0 1d0 0d0 0d0 0d0

-6.125005340D-01 -
1.060880811D+00

1d0 1d22

psiElt= VptElt=
RptElt= IndRef=
Extinc= nObs=
ApType= zElt=
PropType=
nECoord=

0

None 1.46d+00

Geometric 6

-4.610703399D-01

TElt=

8.660251521D-01

2.56956
3356D-01

1.93428
4817D-01
0d0 0d0 0d0

0d0 0d0 -1d0

0d0 0d0 -1.309d+00

0d0 0d0 -1.309d+00

1d0 1d22

0

None


```
zElt= 1.d+19 PropType= Geometric nECoord= -6
nOutCord= 5
Tout= 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0
0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.6 Near-Field Propagation Example

A.6.1 coroExample.jou

```
% MACOS demonstration macro
% This system is a simple coronograph. The demo
% generates a bright on-axis (occulted) image, then
% adds a dim, nearby but slightly off-axis image.
% Run this with macos512!
```

old coroExample;

draw 0 16 XZ

% sketch system

---

stretch sqrt

undo occulting mask

%

mod ngridpts=128

nObs(6)=0

q

pupil function

%

```
old coroExample;
draw 0 16 XZ                                    %   sketch system
int 1
int 6                                               unocculted PSF
                                                %
pix 6 64 ;                                          blowup of PSF
                                                %
mod ngridpts=128                                    redo occulting mask
                                                %
nObs(6)=1
q                                                   blowup of occulted PSF
                                                %
pix 6 64 ;
stretch linear                                      pupil
                                                %
int 10
int 11                                              pupil at Lyot stop
                                                %
stretch sqrt int 16                                 on-axis PSF at detector
                                                %
win Tout 1.392542d-6 0,0 0,0                        set window for composite image
                                                %
compose 16 64 1.392542E-06                          build composite image
                                                %
add yes                                             blowup of on-axis PSF
                                                %
stop elt 11 0,0                                     set system stop at Lyot stop
                                                %
ffp 6 2d-5,2d-5 yes                                 move new image off occulting mask
                                                %
ors 5 yes                                           reset return surface
                                                %
ors 7 yes                                           reset return surface
                                                %
fex 15 yes                                          reset return surface
                                                %
mod flux=0.01                                       100x dimmer object
                                                %
q stretch log10
int 6 % new dim image at occulting mask stretch sqrt
add yes % add dim image to composite image
```

A.6.2 coroExample.in

```
ChfRayDir= 0d0 0d0 1d0 ChfRayPos= 0d0 0d0 -2.d+01 zSource= 1.d+30
IndRef= 1d0
Extinc= 0d0 Wavelen= 1.d-06
Flux= 1d0 GridType= Circular Aperture= 4d0 Obscratn= 0d0 nGridpts= 512
xGrid= 1d0 0d0 0d0 yGrid= 0d0 1d0 0d0 nElt= 16
iElt= 1
EltName= SMObs_1 Element= Obscuring Surface= Conic
KrElt= -1.d+30 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 -5.4d+00 RptElt= 0d0 0d0 -5.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 3
ObsType= Circle
ObsVec= 5d-01 0d0 0d0 ObsType= Rectangle
ObsVec= -5d-02 5d-02 -2d0 2d0 ObsType= Rectangle
ObsVec= -2d0 2d0 -5d-02 5d-02 xObs= 1d0 0d0 0d0
ApType= Circular ApVec= 2d0 0d0 0d0 zElt= 1.d+30
PropType= Geometric nECoord= -6
iElt= 2
EltName= PM_1 Element= Reflector Surface= Conic
KrElt= -1.08d+01
KcElt= -1d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 0d0 RptElt= 0d0 0d0 0d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 5.4d+00
PropType= Geometric nECoord= -6
iElt= 3
EltName= SM_1 Element= Reflector
Surface= Conic
KrElt= -3.526787501D+00 KcElt= -2.670556022D+00
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -4.061145902D+00
RptElt= 0d0 0d0 -5.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.338854098D+00
PropType= Geometric nECoord= -6
iElt= 4
EltName= ret1_1 Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.5d+00 RptElt= 0d0 0d0 1.5d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
iElt= 5
EltName= ret2_1 Element= Return Surface= Conic
KrElt= -6.329334641D+00
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 -4.829334641D+00 RptElt= 0d0 0d0 -4.829334641D+00
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 6.329334641D+00
PropType= NF1 nECoord= -6
iElt= 6
EltName= CoroMask Element= Obscuring Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.5d+00 RptElt= 0d0 0d0 1.5d+00 IndRef= 1d0
Extinc= 1d22
nObs= 1
ObsType= Circle
ObsVec= 1.5d-05 0d0 0d0 xObs= 1d0 0d0 0d0
ApType= Circular ApVec= 2d0 0d0 0d0 zElt= 1.d+30
PropType= NF2 nECoord= -6
iElt= 7
EltName= ret2_2 Element= Return Surface= Conic
KrElt= -6.329334641D+00
KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 7.829334641D+00 RptElt= 0d0 0d0 7.829334641D+00
```

```
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= -6.329334641D+00
PropType= Geometric nECoord= -6
iElt= 8
EltName= ret1_2 Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 1.5d+00 RptElt= 0d0 0d0 1.5d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
iElt= 9
EltName= SM_2 Element= Reflector Surface= Conic
KrElt= -3.526787501D+00 KcElt= -2.670556022D+00
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 7.061145902D+00
RptElt= 0d0 0d0 8.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.338854098D+00
PropType= Geometric nECoord= -6
iElt= 10
EltName= PM_2 Element= Reflector
Surface= Conic KrElt= -1.08d+01
KcElt= -1d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 3d0 RptElt= 0d0 0d0 3d0 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 5.4d+00
PropType= Geometric nECoord= -6
iElt= 11
EltName= LyotStop Element= Obscuring Surface= Conic
KrElt= -1.d+30 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 8.4d+00 RptElt= 0d0 0d0 8.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 3
ObsType= Circle
ObsVec= 5.d-01 0d0 0d0 ObsType= Rectangle
ObsVec= -2.5d-01 2.5d-01 -2d0 2d0
ObsType= Rectangle
ObsVec= -2d0 2d0 -2.5d-01 2.5d-01
xObs= 1d0 0d0 0d0 ApType= Circular ApVec= 2d0 0d0 0d0
zElt= 1.d+30 PropType= Geometric nECoord= -6
iElt= 12
EltName= PM_3 Element= Reflector Surface= Conic
KrElt= -1.08d+01
KcElt= -1d0
psiElt= 0d0 0d0 -1d0 VptElt= 0d0 0d0 1.38d+01 RptElt= 0d0 0d0 1.38d+01 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None zElt= 5.4d+00
PropType= Geometric nECoord= -6
iElt= 13
EltName= SM_3 Element= Reflector
Surface= Conic
KrElt= -3.526787501D+00 KcElt= -2.670556022D+00
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 9.738854098D+00
RptElt= 0d0 0d0 8.4d+00 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1.338854098D+00
PropType= Geometric nECoord= -6
iElt= 14
EltName= ret1_3 Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.53d+01 RptElt= 0d0 0d0 1.53d+01 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
```

```
iElt= 15
EltName= ret2_3 Element= Return Surface= Conic
KrElt= -6.329334641D+00
KcElt= 0d0
psiElt= 0d0 0d0 1d0
VptElt= 0d0 0d0 8.970665359D+00 RptElt= 0d0 0d0 8.970665359D+00
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 6.329334641D+00
PropType= FarField nECoord= -6
iElt= 16
EltName= foc_pln Element= FocalPlane Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 1d0 VptElt= 0d0 0d0 1.53d+01 RptElt= 0d0 0d0 1.53d+01 IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 0d0 PropType= Geometric nECoord= -6
nOutCord= 5
Tout= 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0
0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.7 Image Simulation Example

A.7.1 ImageDemo.jou

1. **ImageDemo.in**

```
ChfRayDir= 0d0 0d0 -1d0 ChfRayPos= 0d0 0d0 6.d+02 zSource= 1d22
IndRef= 1d0
Extinc= 0d0 Wavelen= 7.d-04
Flux= 1.093342085D+00
GridType= Circular Aperture= 3.8d+02 Obscratn= 0d0 nGridpts= 33
xGrid= -1d0 2.033814221D-09 0d0 yGrid= -2.033814221D-09 -1d0 0d0
nElt= 13
iElt= 1
EltName= SEC_OBS Element= Obscuring Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 4.002557400D+02 RptElt= 0d0 0d0 4.002557400D+02
IndRef= 1d0
Extinc= 1d22
nObs= 2
ObsType= Circle
ObsVec= 4.67036d+01 0d0 0d0
ObsType= Rectangle
ObsVec= 0d0 1.75d+02 2.39d+01 2.69d+01 xObs= 9.896513868D-01 1.434926220D-01 0d0
ApType= Circular
ApVec= 1.75d+02 0d0 0d0 zElt= 1d22
PropType= Annulus nECoord= -6
iElt= 2
EltName= SPIDER_2 Element= Obscuring Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 4.002557399D+02 RptElt= 0d0 0d0 4.002557400D+02
IndRef= 1d0
Extinc= 1d22
nObs= 1
ObsType= Rectangle
ObsVec= 0d0 1.75d+02 2.39d+01 2.69d+01 xObs= -6.190939493D-01 7.853169309D-01 0d0
ApType= Circular
ApVec= 1.75d+02 0d0 0d0 zElt= 1d22
PropType= Annulus nECoord= -6
iElt= 3
EltName= SPIDER_3 Element= Obscuring Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 4.002557398D+02 RptElt= 0d0 0d0 4.002557400D+02
IndRef= 1d0
Extinc= 1d22
nObs= 1
ObsType= Rectangle
ObsVec= 0d0 1.75d+02 2.39d+01 2.69d+01 xObs= -3.705574375D-01 -9.288095529D-01 0d0
```

```
ApType= Circular
ApVec= 1.75d+02 0d0 0d0 zElt= 1d22
PropType= Annulus nECoord= -6
```

iElt=	4
tName=	PRI_MIR
El ement=	Reflector
Su rface=	Zernike

KrElt=	-9.337886403D+02	
KcElt=	-1.009724001D+00	
p siElt=	0d0 0d0 1d0	
V ptElt=	0d0 0d0 0d0	
R ptElt=	0d0 0d0 0d0	
I ndRef=	1d0	
E xtinc=	1d22	
Zer nCoef=	0d0 0d0 0d0 -1.415805330D-04 -3.404653098D-05	5.4 12244128D-06

```
1.464334396D-05 -5.046596565D-05 1.587508743D-05 -3.536721061D-05
```

-1.951889059D-06 -2.971194787D-05 1.561541879D-05 -8.027617601D-06 8.191919142D-06 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 pMon= 0d0 0d0 0d0 xMon= 1d0 0d0 0d0 yMon= 0d0 1d0 0d0 zMon= 0d0 0d0 1d0 lMon= 1.75d+02 nObs= 0 ApType= None zElt= 1d22

	Geometric
P ropType=	
nECoord=	-6

iElt=	5	4.002602400D+02
EltName=		
Element=	SEC_MIR	
Surface=	Reflector Conic	
KrElt=	-1.565935400D+02	
KcElt=	-1.932283999D+00	
psiElt=	0d0 0d0 1d0	
VptElt=	-7.887151838D-04 -6.647047308D-05	
RptElt=	-7.887151838D-04 -6.647047308D-05	4.002602400D+02
IndRef=	1d0 1d22	
Extinc=		
nObs=	0	
ApType=		
zElt= P		

	Geometric
P ropType= nECoord=	-6
ropType= nECoord=	None 1d22
	Geometric
	-6
iElt=	6
EltName= Element=	COR_S1
Surface= KrElt=	Refractor Conic
KcElt= psiElt=	-4.241546000D+01
VptElt= RptElt=	0d0
IndRef= Extinc=	0d0 0d0 1d0
nObs= ApType=	0d0 0d0 -3.988054000D+01
zElt= P ropType=	0d0 0d0 -4.260596000D+01
nECoord=	1.61783d+00
	0d0 0
	None 1d22
	Geometric
	-6
iElt=	7
EltName= Element=	COR_S2
Surface= KrElt=	Refractor Conic
KcElt= psiElt=	-3.15595d+01
VptElt= RptElt=	0d0
IndRef= Extinc=	0d0 0d0 -1d0
nObs= ApType=	0d0 0d0 -4.138168000D+01
zElt= P ropType=	0d0 0d0 -4.260596000D+01
nECoord=	1.657863000D+00

	Geometric
P ropType=	
nECoord=	-6
	0d0 0
	None 1d22
	Geometric
	-6
iElt=	8
EltName=	
Element=	COR_S3
Surface=	
KrElt=	Refractor Conic
	-5.533237600D+02

```
KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -4.533138000D+01 RptElt= 0d0 0d0 -4.260596000D+01
IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 9
EltName= FLT_S1 Element= Refractor Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -4.763008000D+01
RptElt= 0d0 0d0 -4.9d+01 IndRef= 1.535018000D+00
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 10
EltName= FLT_S2 Element= Refractor Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -5.163058000D+01
RptElt= 0d0 0d0 -4.9d+01 IndRef= 1d0
Extinc= 0d0
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 11
EltName= IMG_RET Element= Return Surface= Conic
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 0d0 0d0 -5.175758000D+01 RptElt= 0d0 0d0 -5.175758000D+01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
iElt= 12
EltName= EXIT_RET Element= Return Surface= Conic
KrElt= -6.874662920D+01
KcElt= 0d0
psiElt= 1.095041158D-04 -5.786697499D-06 -9.999999940D-01 VptElt= -7.904014798D-05 -6.661258889D-06
1.698904900D+01 RptElt= -7.904014798D-05 -6.661258889D-06 1.698904900D+01
IndRef= 1d0
Extinc= 1d22
nObs= 0
```

```
ApType= None
zElt= 6.874662920D+01
PropType= FarField nECoord= -6
iElt= 13
EltName= IMG_PL Element= FocalPlane Surface= Flat
KrElt= -1d22 KcElt= 0d0
psiElt= 0d0 0d0 -1d0
VptElt= 7.421487214D-01 -4.232318881D+00 -5.175758000D+01 RptElt= 0d0 0d0 -5.175758000D+01
IndRef= 1d0
Extinc= 1d22
nObs= 0
ApType= None
zElt= 1d22 PropType= Geometric nECoord= -6
nOutCord= 5
Tout= 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0
0d0 1d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 0d0 1d0
```

A.8 S-MACOS Example Files

A.8.1 smacosvars.cmn

```
C********************************************************************** C Begin file smacosvars.cmn
C This file defines call-line variables and common blocks that give C user programs direct access to
MACOS variables. The form of the C SMACOS call line is:
```

```
C       CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
C     &  OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat,
C     &  PixArray,Cmatrix)
```

```
C The param.cmn file defines array sizes for SMACOS call-line and
C internal MACOS variables. The param.cmn file should be copied from C param128.cmn, param256.cmn,
etc. -- whichever size of SMACOS you C are using:
INCLUDE 'param.cmn'
C These are the SMACOS call-line variables:
CHARACTER*132 command CHARACTER*32 CARG(9) REAL*8 DARG(9) INTEGER IARG(9) LOGICAL LARG
REAL*4 RARG(9), OutRaySpot(mRay, 4), NomRayDir(2, mRay), & RMSWFE, NomOPDMat(mpts, mpts),
PertOPDMat(mpts, mpts), & PixArray(mPix, mPix), Cmatrix(7, mCm, bRay)
C These are other SMACOS common blocks, that you may use or comment C out as you wish. The ifEcho
flag, if set to .TRUE., provides for C full printout of SMACOS operations. If set to .FALSE., reduced
C printout is provided.
LOGICAL ifEcho COMMON /SCIO/ifEcho
INTEGER npts,NoiseSeed(2) REAL*4 CntrSpot(3)
COMMON /UserCommon/npts,NoiseSeed,CntrSpot
C These includes provide access to internal MACOS variables. They can C be commented out if you do
not plan to use them:
INCLUDE 'elt.cmn' INCLUDE 'src.cmn'
C These includes provide access to internal MACOS scratch arrays. They C should be commented out if
you do not plan to use them:
LOGICAL L1(md2)
INTEGER DrawEltVec(mDrawElt,mDrawRay) REAL*4 R1(mdttl,mdttl),RV1(md2),RV2(md2) REAL*8
D1(mdttl,mdttl),D2(mdttl,mdttl) REAL*8 DV1(md2),DV2(md2)
COMMON /SCRATCH/ D1,D2
EQUIVALENCE (D1(1,1),RV1(1)),(D1(1,1),DV1(1)),(D2(1,1),DV2(1)), & (D1(1,1),R1(1,1)),(D2(1,1),RV2(1))
c END of smacosvars.cmn C**********************************************************************
```

A.8.2 smacosExample.f

```
C********************************************************************** C Begin file smacosExample.f
IMPLICIT NONE
C SMACOS variable declarations INCLUDE 'smacosvars.cmn'
C These are local variables:
INTEGER REAL*4 REAL*8
C Load prescription file
command='OLD' CARG(1)='cassExitPupil'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Set plot type
command='GRAY'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Set system stop
command='STOP' CARG(1)='ELT' IARG(1)=3 DARG(1)=0d0 DARG(2)=0d0
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
```

```
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Set exit pupil
command='FEX' IARG(1)=5 CARG(1)='YES'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Plot OPD at exit pupil after resetting ray grid size
nGridPts=128 command='RESET'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
command='OPD' IARG(1)=5
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Plot detector intensity after resetting ray grid size
nGridPts=96 command='RESET'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
command='INTENSITY' IARG(1)=6
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Plot detector using a log10 stretch
command='STRETCH' CARG(1)='LOG10'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
command='INTENSITY' IARG(1)=6
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
C Quit!
command='QUIT'
CALL SMACOS(command,CARG,DARG,IARG,LARG,RARG,
& OutRaySpot,NomRayDir,RMSWFE,NomOPDMat,PertOPDMat, & PixArray,Cmatrix)
STOP END
C End file smacosExample.f C**************************************************************************
```

APPENDIX C: Complete Command Reference

The following is the complete command list as printed by the HELP command, regenerated from macos_f90/macos_help.inc (single source of truth). Casing convention: <UPPER prefix><lower tail> — the uppercase part is the minimum-match abbreviation tested by the dispatcher. Tags: [Rx] needs a loaded prescription; [BLD] needs a built linear model; [DIFF] needs a propagated wavefront. For a full per-command catalog — dialogs, behavior notes, and the mmacos/pymacos programming interfaces — see the companion **MACOS Command Reference** (macosCmdRef.pdf; source in docs/macos-manual/cmdref/).

MACOS command summary

Casing: UPPER part is the minimum-match abbreviation.

Tags: [Rx]=needs OLD/NEW, [BLD]=needs BUILD,

```
      [DIFF]=diffraction output (propagation runs
             automatically inside the command)
```

C.1 SESSION & FILES

```
Quit, EXIT, BYE - exit MACOS
HELP        - this command list
REset       - reset options & defaults to startup
STatus      - what is loaded / built / propagated
SUMmarize   - pretty-print the optical system
ELTS        - list elements (type, surface)
EXEcute     - run a .jou journal file
JOUrnal     - generate a .jou journal file
ECHo <text> - print <text> to stdout (annotate
              console / journal output)
MREset      - reset MACOS model size (128..8192)
! <cmd>     - run a shell command
PWD, CD     - working directory shell helpers
RX, LS, LL  - list .in files / current directory
VI, EMAcs   - open file in editor
```

C.2 PRESCRIPTION I/O

```
NEW         - start a new optical system
OLD <file>  - load a .in prescription
FID <id>    - load by file id (from RX listing)
VALidate <file> - syntax-check a .in prescription
                  without loading it
SAVe        - write current system to .in
EXPort      - write select results to file
SHOw <elt>  - print all data for one element [Rx]
MODify <elt>- interactively edit one element  [Rx]
```

C.3 SOURCE & WAVELENGTH

```
CHIefray    - chief ray as the FEX/SXP pupil-sphere
              axis (the default)
WLENS       - list Rx optimization wavelengths
SWL         - select wavelength by index
MULtispec   - multi-color image stacking
NFIlt, RFIlt, SFIlt - load / read / save
              MULtispec filter data
ATMosphere  - apply atmospheric phase screen
SETC        - set design-optimization params
SAOpt       - source-adjustment mode
```

C.4 RAY TRACING

```
RAY <i>     - trace ray i; print state at each
              surface                       [Rx]
SEGRAYTrace <i> - trace a ray through segment i s
              center (uses RptElt)          [Rx]
PRAy, RRAy  - find pupil / marginal rays
TPR         - trace the PRAy/RRAy rays
```

```
MAP          - print ray-to-segment map
COOrd <elt> - beam coordinates at element    [Rx]
ACOor        - all-element coords (xyzLocal.txt)
SYSprop      - first-order system properties  [Rx]
EFL          - system effective focal length /
               f-ratio / plate scale          [Rx]
BWK          - chief-ray walk vs nominal      [Rx]
FEXit        - find exit pupil
SXP          - set exit pupil at an element [Rx]
XPS          - exit-pupil surface fit over the
               full ray grid              [Rx]
FSR          - find source ray nearest an
               aperture position         [Rx]
CENter       - center beam in system stop
STOp         - define system stop element
CENTRoid     - beam-centroid FEX/SXP axis (opt-in;
               the chief ray is the default)
FFP <x,y>    - find field point that places chief
               ray at element coordinate
PFP <x,y>    - find field point that places chief
               ray at pixel coordinate
FDP          - find detector plane orientation
SPCenter     - spot-diagram centering option
```

C.5 BEHAVIOR TOGGLES

```
LNEg/NOLNeg - allow/disallow negative pathlengths
POLarized   - polarization tracing on
NOPolarization - polarization off
OBS          - spot-diagram obscuration option
REGrid/NOREGrid - resample wavefront at element
                  during propagation
ORS          - optimize reference surface
SRS          - slave one ref surface to another
NONe         - turn off plotting (no PGPLOT calls)
```

C.6 SURFACE DATA

```
SINt         - set up interpolated-surface data
UDSinit      - set up user-defined-surface data
```

C.7 PERTURBATION

```
PERturb <elt>   - apply 6-DOF perturbation     [Rx]
GPERturb <grp>  - perturb an element group     [Rx]
PREad <file>    - apply perturbations from file
```

C.8 LINEAR MODEL

```
BUild        - build linear model from chief-ray
               ray partials                      [Rx]
DMBuild      - build with deformable mirror     [Rx]
PARtials <i> - print partial derivs, ray i    [BLD]
LPErturb     - linear-model perturb             [BLD]
LPRead       - linear perturb from file         [BLD]
LREset       - zero linear perturbations        [BLD]
LSPot, LOPd - linear-model spot / OPD           [BLD]
LPIxilate    - linear-model pixelated image     [BLD]
```

C.9 DIFFRACTION

```
PROpagate    - note: NOT a command; propagation
               runs automatically inside any
               [DIFF] output command
BEAm         - set beam type (Gauss, plane, ...)
VECtor, SCAlar - vector vs scalar diffraction
                 (with POLarized)
COMpose      - start a multi-object/color image
ADD, DAdd    - add to / display composed image
CADD         - add current intensity to composed
NOIse, SEEd - image noise + RNG seed
STRetch      - LIN / LOG / SQRT display stretch
```

C.10 OUTPUTS (RAY-TRACE & DIFFRACTION)

```
OPD         - wavefront error map             [Rx]
OPDREF      - OPD reference: CHIEF or MEAN
              (the chief ray is the default)
SPOt        - spot diagram                    [Rx]
ZABerr      - Zernike OPD aberration          [Rx]
ZCOef       - Zernike coefficient extraction  [Rx]
LOS         - line-of-sight output
METcalc     - metrology beam output
INTensity   - wavefront intensity           [DIFF]
PIXel       - pixelated intensity output    [DIFF]
AMPlitude   - amplitude output              [DIFF]
PHAse, PH   - phase output                  [DIFF]
REAl        - real & imaginary wavefront    [DIFF]
LOG         - log10 intensity               [DIFF]
MTF, CMTF   - modulation transfer function  [DIFF]
IMG         - simulate imaging of the ObjFile
              object map            [DIFF]
```

C.11 PLOT STYLE

```
GRay        - gray-scale surface plot
WIRe        - wireframe plot
SLIce       - slice surface plot
COLumn      - column line plot
CONtour     - contour surface plot
IMGmode     - toggle image polarity
              (NEG=astro, POS=conventional)
CIR, GIR    - color / gray image rendering
PGP         - PGPLOT panel layout (1,2,3,4,9)
```

C.12 FILE OUTPUT

```
TEXt        - ASCII print output
BINary      - binary image file
FITs, WFIts - FITS / wide-FITS image file
MAT         - MATLAB .mat file
GETMatlabmatrix - (currently disabled: handler
              is a no-op stub)
                  (smacos only)
```

C.13 WINDOW & POST-PROCESSING

```
PLOcate     - set plot pixel-array location
NOPLOC      - clear plot location
PWIn        - launch external he.py plot GUI
SZCo        - select ZCOef Zernike basis
              (Cir / Ann / Hex)
GBS         - geometric beam footprint at an
              element                     [Rx]
BLUr, GBLur - blur / Gaussian blur
GAIn        - apply gain map to intensity
ODRaw, PGD  - outline drawing / PGPLOT diagnostics
ROW         - extract / plot row from image
```

C.14 SYSTEM OPTIMIZATION

```
AVAR <elt>  - add a variable element
MVAR <elt>  - modify a variable element
DVAR <elt>  - delete a variable element
VARS        - list all variable elements
AFOV <fov>  - add a field of view
DFOV <fov>  - delete a field of view
FOVS        - list all fields of view
CALib       - run system optimization
SFOV        - select field of view
```

C.15 MISC / DEBUG

```
SRAy, SRT, SRTrace - older single-ray variants
                     (use RAY)
DOPD, DOPL, ZRM     - debug / diagnostic prints
JWST_v3d, Vis3d     - 3D rendering demo
```

APPENDIX D: Working with AI Agents

This appendix is written for an AI agent that is asked to run, extend or document MACOS, and for the person who sets one to work. It says where the authoritative facts are, how to drive the program so that its answers can be trusted, which conventions bite, and the rules this project holds its own agents to. Nothing here is specific to one agent product.

D.1 Where the facts live

- **The Command Reference** (`docs/macos-manual/cmdref/`, rendered as macosCmdRef) is the authority on every command: its minimum abbreviation, its syntax, its prompts and their defaults, its binding in MATLAB and Python, and the journal in the test suite that exercises it. When this manual's prose and a transcript disagree, the transcript made on the current build wins, and the Command Reference is updated from it.
- **This manual** explains the models: the prescription language of Section 4 (Table 13 is the design-optimization keyword reference), the ray-trace, diffraction, imaging and linear-model commands of Sections 5 to 8, SMACOS in Section 9, worked examples in Section 10, polarization in Section 11. The example prescriptions of Appendix A are in `docs/macos-manual/examples/` and load as printed.
- **The notes beside the code.** `CLAUDE.md` at the top of each repository, and in `macos_f90/`, `macos_f90/giza/`, `macos_f90/slsqp/` and `MACOS_resources/pymacos/`, record the engine's conventions and the defects that were closed, each with the test that pins it; `macos_f90/notes/` holds the engine record by topic. They are maintained with the code and are the first thing to read before changing it.
- **The test suite is the specification.** `cli_tests/` runs the interactive program on every prescription in the repositories (the load gate) and one journal per command (the command gate), on both compilers; `MACOS_resources/mmacos/tests/` and `pymacos/tests/` are the binding suites. A behaviour that no test pins is not yet a promise.

D.2 Driving the program

- Run the interactive program on a pseudo-terminal with `TERM=dumb`; under a real terminal type the line editor turns on bracketed paste and prefixes every line with an escape sequence. Send one line per prompt. A prompt ends in `MACOS>`, in `]:` (a default in brackets) or in `?`; the next line answers it, and a line that happens to be a command name is read as the answer, not as a command.
- The first prompt asks the model size (the largest ray grid the session will hold: 128, 256, 512, 1024, 2048). A prescription whose `nGridpts` exceeds it is clamped with a message; a diffraction grid is four times the ray grid, and the coronagraph example of Section 7 wants 512.
- A journal (`cli_tests/cmd/*.jou`) is the reproducible form of a run: the lines sent, with `# EXPECT:` regular expressions the transcript must match and `# EXPECT-NOT:` ones it must not, written from a transcript on the current build, never from prose. `cli_tests/README.md` is the recipe, including the shell-escape and plot-device quirks.
- Every number an agent reports comes from a run on the build it names, with the prescription and the commands that produced it. The engine prints its OPD reference (chief ray or aperture mean), its ray-loss counts by cause and element, and its propagation summaries; read them.
- SAVE writes the prescription as loaded and modified; a saved prescription reloads to the same trace, and a second SAVE is byte-identical. Use the round trip to check that a change to a prescription means what was intended.

D.3 Conventions that bite

- `KrElt=` carries the sign of the vertex radius with a mirror's concave side facing the incoming beam negative; `KcElt=` is the conic constant; `psiElt=` points against the incoming beam on a reflector and along it on a focal plane or reference surface. The output-coordinate block (`nOutCord=`, `Tout=`) ends a prescription; when it is absent the program proposes the beam frame at the last element and asks.
- A ray trace never changes the prescription; a trace reads it and writes trace state. The design-optimization keywords of Table 13 are prescription state, written by SAVE and announced at load.
- Session options (POLARIZATION, VECTOR, CENTROID, LNEG, CIR, the OPD reference) survive loads; the bindings report them at every load.
- The OPD map's zero is not "no ray": a valid ray at the chief ray's path reads exactly 0. Validity is the mask the engine keeps (`opd_mask` in the bindings), not a test on the value.
- A numerical zero (a collimated system's RMS near 1e-12, an on-axis coordinate near 1e-11) has no shared digits across machines or architectures. Pin a real-signal value to five significant figures; bound a numerical zero in magnitude; never pin its digits.
- Masks for diffraction are `Element= Obscuring`; a Reference surface with obscurations clips rays only. Reference and Return surfaces are passive: they are read, never traced through.

D.4 Working rules

1. Every fix ships with a test, and the test has a leg that fails on the build before the fix. A test that cannot fail measures nothing.
2. Measure before you guard. Before adding a check, find the value's other writers; a refusal added at parse time once broke two hundred tests because the clamp already existed downstream.
3. Guards warn and count; they do not refuse behaviour a long-standing tool tolerated. A new refusal needs the maintainer's decision.
4. A change that can move a trace runs the whole corpus on the old and the new build before it is committed, and every prescription that moved is named and explained.
5. Read the frame before calling a mismatch: angles and lengths depend on the coordinate frame they are quoted in.
6. Look at the raw per-ray data before blaming a solver: a histogram of the per-ray OPD finds a wrong conic sheet that an outlier cut hides.
7. An "exact" replica of a kernel that passes a test can hide a broken kernel; diff the engine's own intermediate quantity instead.
8. Write the resolution of an oddity where the next reader will find it, at the time it is resolved.
9. Report outcomes faithfully: if a test fails, say so with its output; if a step was skipped, say that; a number comes from a run, not from memory.